EKNATHALABS · INTERVIEW PREP

Networking Fundamentals

Platform, SRE & DevOps interview guide

Questions	**170 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/networking_interview_guide.html**
Printed	**2026-10-21**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

certs.eknathalabs.com

Q1

Explain the OSI model — which layers matter most for a Platform/DevOps Engineer?

Fundamentals

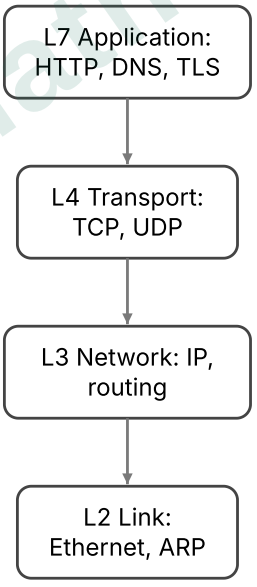
🎙 Spoken answer · ~86 s

The OSI model has seven layers: physical, data link, network, transport, session, presentation, and application. As a platform engineer I operate mostly at layers three to seven, but knowing all of them helps me localise a fault. Layer one is cables, NICs, and signals, rarely my problem in the cloud. Layer two is MAC addresses, switches, VLANs, and ARP. Layer three is IP addressing and routing, where ping, routes, and security groups matter. Layer four is TCP and UDP, ports, and connection state, where firewalls, load balancers, and kube-proxy operate. Layers five to seven are sessions, encoding, TLS, and application protocols like HTTP and DNS. The model is useful as a troubleshooting ladder: if I can't ping an address, I look at layers one to three; if I can reach the host but the port is refused or times out, layer four; if TCP connects but TLS fails, certificates and protocol versions; if TLS works but the API returns errors, the application. It also maps to tools: an L4 load balancer routes on IP and port, an L7 one inspects HTTP. In practice the TCP/IP model is what's implemented, but OSI gives shared vocabulary, so I can say this is an L7 problem and be understood.

Likely follow-ups

- How do you use the layers as a troubleshooting ladder?
- Where do load balancers and firewalls sit in the model?

🗺 OSI layers that matter



The OSI model has 7 layers. As a Platform Engineer you operate primarily at layers 3–7, but understanding all 7 helps isolate where a problem lives.

LAYER REFERENCE

```
L1 Physical   — cables, NICs, signals (rarely your problem in cloud)
```

```
L2 Data Link  — MAC addresses, switches, VLANs, ARP
```

certs.eknathalabs.com

```
L3 Network    – IP addresses, routing, ICMP (ping, traceroute)
```

```
L4 Transport  – TCP/UDP, ports, sessions, flow control
```

```
L5 Session    – connection management (mostly abstracted)
```

```
L6 Presentation – TLS/SSL encryption, encoding
```

```
L7 Application – HTTP, DNS, gRPC, SMTP
```

Troubleshooting strategy: start at L3 (can you ping?), move to L4 (can you reach the port?), then L7 (does the application respond correctly?). This narrows the problem space systematically. In cloud networking: firewall rules = L3/L4, Ingress/load balancer = L4/L7, TLS certificates = L6/L7.

Interview tip: When asked to troubleshoot "a service is unreachable," walk through the OSI layers explicitly. It shows structured thinking, not just trial-and-error command firing.

📚 **Official docs**

- Cloudflare Learning: OSI model

Q2 **What is the difference between TCP and UDP — when do you choose each?** Fundamentals

🎙 **Spoken answer · ~80 s**

TCP is connection-oriented and reliable: it does a three-way handshake, guarantees ordered delivery with acknowledgements and retransmission, and provides flow and congestion control. UDP is connectionless: no handshake, no acknowledgements, no ordering, just fire and forget, so it has lower latency and overhead but no delivery guarantees. I choose TCP when correctness matters: HTTP APIs, databases, file transfers, SSH, anything where lost or reordered data would break the application. I choose UDP when speed and timeliness matter more than perfection, or when the application handles reliability itself: DNS queries, which are tiny request-response exchanges, real-time voice and video, gaming, metrics and logging with tolerable loss, and QUIC, which builds reliable streams on top of UDP to avoid TCP head-of-line blocking. Operationally, ss shows TCP and UDP listeners separately, and the failure modes differ: TCP problems show as retransmissions, resets, and timeouts; UDP loss is silent, so I need application-level monitoring. Firewalls and load balancers must be configured for the right protocol, and UDP is harder to load balance and troubleshoot, since there's no connection state to inspect. UDP can also be abused for amplification attacks, so I'm careful exposing UDP services publicly.

Likely follow-ups

- Why does DNS mostly use UDP?
- How does QUIC get reliability over UDP?

TCP (connection-oriented, reliable) vs UDP (connectionless, fast) — the choice depends on whether you need delivery guarantees.

```
TCP: 3-way handshake → SYN, SYN-ACK, ACK → guaranteed ordered delivery
```

certs.eknathalabs.com

```
UDP: fire and forget → no handshake, no ACK, no ordering
```

```
ss -tlnp        # show TCP listeners
```

```
ss -ulnp        # show UDP listeners
```

```
netstat -an | grep ESTABLISHED | wc -l  # count active TCP connections
```

TCP use cases: HTTP/HTTPS, SSH, databases, any data where loss is unacceptable. UDP use cases: DNS (speed matters, client retries on loss), QUIC/HTTP3, video streaming, VoIP, gaming (latency beats reliability). In Kubernetes: CoreDNS listens on both TCP and UDP 53. Container health checks over TCP are more reliable than UDP.

HTTP/3 (QUIC): Built on UDP with its own reliability layer — combines UDP speed with connection migration (IP changes don't drop the connection). GCP's load balancers support HTTP/3. Modern platform engineers should understand this shift away from TCP for web traffic.

📚 **Official docs**

- RFC 768: UDP

Q3 **Explain the TCP 3-way handshake and TIME_WAIT state — why does it matter in production?** Fundamentals

🎙 **Spoken answer · ~85 s**

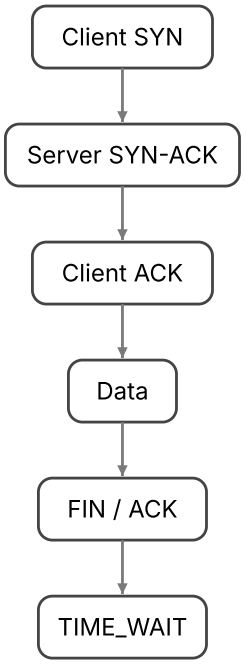
The TCP three-way handshake establishes a connection: the client sends a SYN, the server replies with SYN-ACK carrying its own sequence number, and the client sends an ACK; then data flows. When a connection closes, the side that closes first ends up in TIME_WAIT for about sixty seconds. It exists so that delayed packets from the old connection don't corrupt a new connection using the same address and port tuple, and so the final ACK can be retransmitted if lost. It matters in production because a client making many short connections accumulates TIME_WAIT sockets, and since each outbound connection to the same destination needs a unique source port, it can run out of ephemeral ports, giving cannot assign requested address errors. I count them with ss, check the port range, and look at tcp_tw_reuse. The right fix is to reuse connections with keep-alive and connection pooling, and widen the port range or spread across more source or destination addresses; enabling tw_reuse for outbound helps. The old tw_recycle was removed because it breaks clients behind NAT. Also relevant: SYN floods target the handshake, mitigated with SYN cookies and backlog tuning, and handshake latency adds round trips for every new connection, a reason to keep connections alive.

Likely follow-ups

- Why does TIME_WAIT exist?
- What is the right fix for ephemeral port exhaustion?

certs.eknathalabs.com

🗺 **TCP handshake and close**




```
Client → SYN       → Server   (I want to connect)
```

```
Client ← SYN-ACK   ← Server   (OK, here's my sequence number)
```

```
Client → ACK       → Server   (Connection established)
```

```
ss -tan | grep TIME_WAIT | wc -l  # count TIME_WAIT sockets
```

```
cat /proc/sys/net/ipv4/tcp_tw_reuse  # allow TIME_WAIT socket reuse
```

TIME_WAIT: after a connection closes, the socket stays in TIME_WAIT for 2×MSL (typically 60–120s) to handle delayed packets. Problem: high-throughput services (load balancers, APIs) can exhaust ephemeral ports if connections cycle too fast. Fix: enable `tcp_tw_reuse=1`, use connection pooling (keep connections alive rather than opening/closing per request), or tune `ip_local_port_range`.

Port exhaustion pattern: An app opening a new TCP connection per database query will eventually exhaust all ~28K ephemeral ports. This manifests as "connection refused" or "cannot assign requested address" errors. The fix is connection pooling, not more ports.

📚 **Official docs**

- RFC 9293: TCP

certs.eknathalabs.com

Q4 **What is ARP and how does it work — what problems can it cause?** Fundamentals

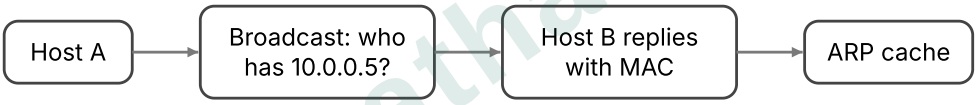
🎙 Spoken answer · ~83 s

ARP, the Address Resolution Protocol, maps IP addresses to MAC addresses within a local network segment, at layer two. When a host wants to send to an IP on the same subnet, it broadcasts who has this IP; the owner replies with its MAC, and the sender caches the mapping. For off-subnet destinations it ARPs for the default gateway. I inspect the cache with ip neigh and send probes with arping. Problems ARP can cause: stale or wrong entries after an IP moves to a new machine, causing traffic to go to the old MAC until the cache expires, which matters in failover with virtual IPs, solved with a gratuitous ARP announcement; duplicate IP addresses, where two hosts answer, giving intermittent connectivity; ARP spoofing or poisoning, an attack that redirects traffic through an attacker, mitigated by dynamic ARP inspection and port security; ARP storms or table exhaustion in large flat networks, which is one reason to keep layer-two domains small; and incomplete entries when the target is unreachable. In the cloud, ARP is handled by the virtual network, but it can still appear in on-premises, VM, and container bridge troubleshooting. Flushing the cache can clear a stale entry, used carefully.

Likely follow-ups

- Why is a gratuitous ARP used in failover?
- How can you detect a duplicate IP address?

🗺 ARP resolution



ARP (Address Resolution Protocol) maps IP addresses to MAC addresses within a local network segment. It operates at L2.

```
arp -n              # show ARP cache (IP → MAC mappings)
```

```
ip neigh show       # modern equivalent — neighbor table
```

```
arping -I eth0 <ip> # send ARP request to specific IP
```

```
ip neigh flush all  # clear ARP cache (use carefully)
```

ARP issues: ARP spoofing (attacker poisons cache to intercept traffic — MITM attack), stale ARP entries causing connectivity to a replaced VM/container, gratuitous ARP needed after IP migration (live migration, failover) to update all hosts. In Kubernetes: when a pod moves to a different node, the new node sends a gratuitous ARP to update the network switch's MAC table.

ARP spoofing in cloud: Cloud providers disable ARP spoofing at the hypervisor level — VMs cannot send crafted ARP packets. This is why you cannot do L2 network tricks between cloud VMs that you can do on bare metal.

📚 Official docs

- RFC 826: ARP

certs.eknathalabs.com

Q5 **What is NAT and how does it work — what are the different types?** Fundamentals

🎙 Spoken answer · ~87 s

NAT, Network Address Translation, rewrites addresses in packets so private hosts can share public addresses. Source NAT changes the source address, used for outbound internet access from private networks, like a home router or a cloud NAT gateway. Destination NAT changes the destination address, used for port forwarding and by load balancers, and Kubernetes services use DNAT to reach pod IPs. Masquerade is a dynamic form of SNAT that uses the outgoing interface's current address, handy when the public IP changes. Port address translation, PAT, maps many private addresses to one public address by using different source ports, which is what most NAT does. NAT is stateful: the device keeps a table of translations, so return traffic is mapped back. Operational consequences: the NAT table and port space can be exhausted, causing connection failures under heavy outbound traffic to the same destination; the source IP seen by a remote partner is the NAT address, so allow-lists must include it, and egress IPs should be stable, which cloud NAT gateways with elastic addresses provide; NAT breaks protocols that embed addresses, and complicates inbound connections; it adds a hop to troubleshoot, so conntrack tools help. IPv6 largely removes the need for NAT, though cloud designs still use NAT64 and egress controls.

Likely follow-ups

- Why does NAT port exhaustion happen?
- Why do partners need to allow-list your NAT IP?

🗺 Source NAT



NAT (Network Address Translation) maps between private and public IP addresses, enabling many hosts to share one public IP.

```
SNAT (Source NAT): changes source IP — used for outbound internet (Cloud NAT, home router)
```

```
DNAT (Destination NAT): changes dest IP — used for port forwarding, load balancers
```

```
Masquerade: dynamic SNAT for changing public IPs (iptables MASQUERADE target)
```

```
iptables -t nat -L -n -v  # show NAT rules on Linux
```

In Kubernetes: kube-proxy implements DNAT rules (iptables/IPVS) for Services — when a pod calls a ClusterIP, DNAT rewrites the destination to one of the backend pod IPs. This is how Kubernetes Services work under the hood. Cloud NAT on GCP uses SNAT so private VMs can reach the internet with a shared public IP pool.

Connection tracking: NAT relies on conntrack (connection tracking) to remember active sessions and reverse-translate response packets. High-throughput services can exhaust the conntrack table. Monitor with `conntrack -L | wc -l` vs `nf_conntrack_max`.

certs.eknathalabs.com

📚 **Official docs**

- RFC 3022: Traditional NAT

Q6 **Explain how DNS resolution works end-to-end — from browser to IP address.** DNS

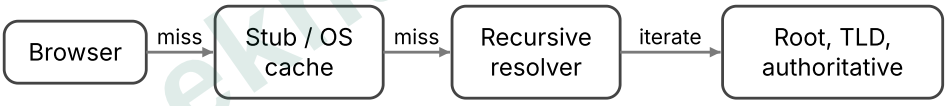
🎙 **Spoken answer · ~75 s**

When a client needs to resolve a name, it first checks local caches: the application, the OS resolver cache, and the hosts file. If there's no answer, it asks its configured recursive resolver, such as the ISP's, a corporate server, or the cloud provider's. The recursive resolver checks its own cache, and on a miss it walks the hierarchy. It asks a root server, which refers it to the TLD servers, like dot com; the TLD servers refer it to the domain's authoritative name servers; and the authoritative server returns the actual record. The resolver caches the result for the record's TTL and returns it to the client. Along the way there can be CNAME chains, which add more lookups, and the transport is usually UDP on port 53, falling back to TCP for large responses. For production, TTL is the key: low TTLs let you move traffic quickly but increase query load, and because every layer caches, changes are never instant. In Kubernetes, CoreDNS adds another layer, with search domains and ndots affecting how many queries a lookup triggers.

Likely follow-ups

- Why can ndots cause slow DNS in Kubernetes?
- How would you debug an intermittent DNS failure?

🗺 **DNS resolution**




```
1. Browser checks local cache (TTL not expired)
```

```
2. OS checks /etc/hosts, then queries /etc/resolv.conf nameserver
```

```
3. Recursive resolver checks its cache
```

```
4. If miss: asks root nameservers → TLD servers (.com) → authoritative NS
```

```
5. Authoritative NS returns A/AAAA record → cached at TTL
```

```
dig +trace example.com  # trace full resolution path
```

Key record types: A (IPv4), AAAA (IPv6), CNAME (alias to another name), MX (mail), NS (nameserver), TXT (SPF, DKIM, domain verification), SRV (service discovery), PTR (reverse DNS). TTL controls caching duration — low TTL (60s) for records that change often (blue-green deployments), high TTL (3600s) for stable records to reduce query load.

Kubernetes DNS: CoreDNS serves the cluster domain `svc.cluster.local`. Each Service gets `<name>.<namespace>.svc.cluster.local`. Pods use the search domain list — `ndots:5` means 5+ dot names are tried as absolute

certs.eknathalabs.com

first, causing multiple DNS lookups per query. A major source of latency in microservices.

📚 **Official docs**

- RFC 1035: DNS implementation and specification
- Cloudflare Learning: What is DNS?

Q7 **DNS resolution is intermittently failing — how do you systematically debug it?** DNS

🎙 **Spoken answer · ~83 s**

For intermittent DNS failures I debug systematically along the resolver path. First, dig the name and read the status and flags and the query time, to see whether it's a SERVFAIL, NXDOMAIN, timeout, or just slow. Then bypass the local resolver by querying a known public resolver, to see whether the problem is my resolver or the authoritative data; dig with trace walks the delegation chain from the root to find a broken or lame delegation. Check resolv.conf for nameservers, search domains, and options; with systemd-resolved, resolvectl status shows the actual state. Intermittent patterns suggest one bad server among several, so I query each nameserver directly several times. Check whether the failures correlate with UDP versus TCP fallback for large responses, with DNSSEC validation, with rate limiting, or with load on the resolver. In Kubernetes, check CoreDNS pod health and scaling, the conntrack UDP race, and ndots expansion generating many queries. Packet capture on port fifty-three shows unanswered queries and retries. Fixes: remove or repair the bad nameserver, add a local caching resolver, tune ndots and timeouts, scale the resolver, fix the zone or delegation, and make applications cache results. Monitor DNS latency and failure rate as a dedicated indicator.

Likely follow-ups

- Why query each nameserver individually?
- What does dig +trace show?

```
dig <hostname>              # full DNS response with flags
```

```
dig @8.8.8.8 <hostname>    # bypass local resolver — test against Google
```

```
dig +trace <hostname>       # trace full delegation chain
```

```
nslookup <hostname>         # quick check with default resolver
```

```
cat /etc/resolv.conf        # confirm nameserver + search domains
```

```
resolvectl status           # systemd-resolved status (modern Linux)
```

```
tcpdump -i any port 53      # capture DNS traffic
```

Intermittent failures suggest: resolver flapping (check if multiple nameservers in resolv.conf — some may be unreachable), TTL too low causing thundering herd on expiry, SERVFAIL from upstream (authoritative server overloaded), or UDP packet loss (DNS falls back to TCP for large responses, but many resolvers don't handle this). Test with both UDP and TCP: `dig +tcp @resolver hostname`.

certs.eknathalabs.com

NXDOMAIN vs SERVFAIL: NXDOMAIN = record doesn't exist (config issue). SERVFAIL = resolver couldn't get an answer (infrastructure issue). They require different fixes — don't confuse them.

📚 **Official docs**

- RFC 1035: DNS implementation and specification
- Cloudflare Learning: What is DNS?

Q8 **What is DNS TTL — how do you manage TTL for zero-downtime deployments?** DNS

🎙 **Spoken answer · ~85 s**

DNS TTL is how long resolvers may cache a record. A long TTL reduces load and latency but slows propagation of changes; a short TTL makes changes take effect quickly but increases query load and dependence on DNS availability. For zero-downtime changes, I plan the TTL. Several hours, at least one old TTL, before the planned change, I lower the TTL to something like sixty seconds. I wait for the old, high-TTL cached entries to expire across the internet. Then I make the DNS change, and verify propagation by querying several public resolvers, and test the new endpoint. Keep the old endpoint running until traffic has drained, because some resolvers and clients ignore TTL, cache longer, or keep long-lived connections. After the change is confirmed stable, I restore a higher TTL. Caveats: some clients, including JVMs by default in some configurations, cache DNS indefinitely or for a long period, so applications need sensible settings; negative caching of missing records can delay new records; and during the change window I should monitor traffic to both old and new. For failover records, keep the TTL low permanently, with health checks. For migrations, an alternative is to avoid DNS cutover altogether by switching at the load balancer.

Likely follow-ups

- Why lower the TTL well before the change?
- Why might some clients ignore the TTL?

```
dig <hostname> | grep -E "ANSWER|TTL"  # check current TTL
```

```
# Pre-deployment TTL strategy:
```

```
1. Lower TTL to 60s several hours before planned change
```

```
2. Wait for existing high-TTL cache to expire across the internet
```

```
3. Make the DNS change
```

```
4. Verify propagation with: dig @1.1.1.1, @8.8.8.8, @9.9.9.9
```

```
5. Restore high TTL after confirming stability
```

TTL is the number of seconds resolvers cache the record. Changing DNS with a 3600s TTL means some clients see the old IP for up to 1 hour after change. But you can't instantly reduce TTL mid-incident — the old TTL is already cached. The only solution is pre-planned TTL reduction. Cloud load balancers abstract this with virtual IPs that don't change — you update backends, not

certs.eknathalabs.com

DNS.

Production standard: Keep external DNS TTL at 300s (5min) for production services — fast enough for incident recovery, low enough DNS query overhead. Never set critical service TTL above 3600s unless the IP is guaranteed permanent.

📚 **Official docs**

- RFC 1035: DNS implementation and specification

Q9 **What is the difference between authoritative and recursive DNS — how does split-horizon DNS work?** DNS

🎙 **Spoken answer · ~83 s**

Authoritative DNS servers hold the actual zone data and give definitive answers for their domains; they don't go looking elsewhere. Recursive resolvers work on behalf of clients: they receive a query, walk the hierarchy from the root to the TLD to the authoritative server, and cache the results to speed up later answers. I can test an authoritative server directly with dig and the no-recursion flag, and find a domain's authoritative servers with an NS query. Split-horizon DNS, or split-brain DNS, serves different answers for the same name depending on who asks, typically internal clients get private IP addresses and external clients get public ones. This is common with a private hosted zone alongside a public zone, so internal services resolve to internal endpoints while the public name points to a load balancer. It's useful for security, performance, and avoiding hairpin routing. Pitfalls: inconsistent records between the views cause hard-to-debug it works from here but not there problems; VPN clients and containers may use the wrong resolver; caching and search domains complicate it; and zone changes must be made in both views. To troubleshoot, I query from each network location and compare, and check which resolver and view is answering.

Likely follow-ups

- How do you find which nameservers are authoritative for a domain?
- What problems does split-horizon DNS introduce?

🗺 **Authoritative vs recursive**

Authoritative	Recursive
Holds the zone records	Walks the tree and caches
Answers for its domains only	Answers for clients

certs.eknathalabs.com

```
Authoritative: holds the actual zone data — gives definitive answers for its domain
```

```
Recursive: queries on behalf of clients — asks authoritative servers, caches results
```

```
dig <hostname> +norec  # query without recursion (test authoritative directly)
```

```
dig ns <domain>        # find authoritative nameservers for a domain
```

Split-horizon DNS: the same hostname resolves to different IPs depending on where the query comes from. Internal queries (from VPC/office) get private IPs; external queries get public IPs. Used to route internal traffic directly to private endpoints (avoiding NAT hairpin) while external users reach public load balancers. In GCP: Cloud DNS private zones visible only within specified VPC networks implement split-horizon.

Kubernetes + split-horizon: CoreDNS (internal) resolves `api.internal.company.com` to a ClusterIP. External DNS (Cloud DNS) resolves the same name to a public load balancer IP. Same hostname, different answers based on network context.

📚 **Official docs**

- RFC 1035: DNS implementation and specification
- Cloudflare Learning: What is DNS?
- OpenStack documentation

Q10 **What does it mean when a connection is "refused" vs "timed out" — how do you distinguish them?** TCP/UDP

🎙 **Spoken answer · ~74 s**

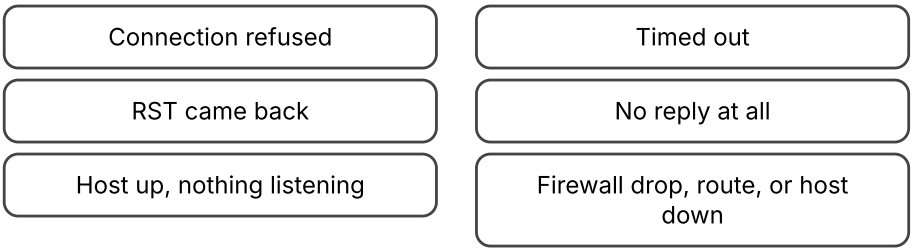
They point to different layers. Connection refused means the packet reached the destination host and it actively rejected it by sending a TCP reset, typically because nothing is listening on that port, or a firewall rule is set to reject. That tells me the network path works and the problem is the service: it's down, listening on a different port or interface, or bound to localhost only. Connection timed out means I got no response at all, so the SYN was silently dropped or never answered. Typical causes are a firewall or security group dropping packets, a routing problem, the host being down or unreachable, or an overloaded server with a full accept queue. To tell them apart and debug, I use curl or nc with verbose output, then check from the server with ss to see whether something is listening on the right address and port. For timeouts I work along the path with traceroute, security groups, network ACLs, and route tables, and use tcpdump to see whether the SYN arrives and whether a SYN-ACK goes back.

Likely follow-ups

- Why would a security group cause a timeout rather than a refusal?
- What does a full accept queue look like from the client side?

certs.eknathalabs.com

🗺 **Refused vs timed out**




```
Connection refused: RST packet received — port is closed or no listener
```

```
Connection timed out: no response — firewall dropping, host unreachable, or wrong IP
```

```
curl -v --connect-timeout 5 http://<host>:<port>  # verbose output shows which error
```

```
nc -zv <host> <port>   # TCP connect test — immediate refused or hangs on timeout
```

```
telnet <host> <port>   # same, more familiar to some
```

Refused = the host is reachable but no process is listening on that port (or the OS actively rejected it). Timeout = packet is lost or dropped — firewall black-holing traffic, wrong IP, routing issue, or the service is so overloaded it can't accept new connections. Timeout takes longer to manifest (depends on OS timeout, typically 15-120s). Refused is near-instant.

Debug path: Refused → check if service is running and bound to the correct port/interface. Timeout → check firewall rules, routing (traceroute), and whether the IP is reachable at all (ping first).

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q11 **Explain TCP connection states — what does a large number of CLOSE_WAIT or TIME_WAIT indicate?**

TCP/UDP

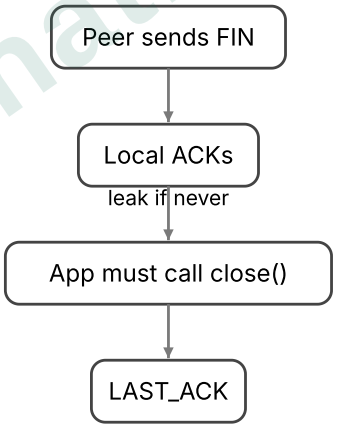
🎙 Spoken answer · ~85 s

I count sockets by state with ss and read what's abnormal. TIME_WAIT is the state after a graceful close by the side that closed first; many of them are normal in high-throughput services and indicate high connection churn. The risk is source port exhaustion for clients making many short connections to one destination; the fix is connection pooling and keep-alive, a wider port range, and tcp_tw_reuse for outbound. CLOSE_WAIT is different and usually a bug: the remote side closed the connection, and my application hasn't closed its end. The socket stays in CLOSE_WAIT until the application calls close, so a large and growing number means a connection or file descriptor leak in the application, such as not closing responses, missing finally blocks, or a pool that isn't releasing connections. It eventually causes too many open files. The fix is in the code: ensure sockets are closed in all paths, and I identify the owning process with ss and lsof. Other states worth knowing: SYN_RECV in large numbers suggests a SYN flood or backlog problem; FIN_WAIT2 piling up means the peer isn't closing; ESTABLISHED in large numbers means real concurrency or idle leaked connections. I monitor state counts as metrics to detect leaks early.

Likely follow-ups

- Why does CLOSE_WAIT indicate an application bug?
- What do many SYN_RECV sockets suggest?

🗺 TCP state: CLOSE_WAIT




```
ss -tan | awk '{print $1}' | sort | uniq -c | sort -rn  # count by state
```

```
ss -tan state close-wait  # show CLOSE_WAIT sockets
```

```
ss -tan state time-wait   # show TIME_WAIT sockets
```

TIME_WAIT: socket waiting after graceful close — normal in high-throughput services. Many TIME_WAIT sockets indicate high connection churn. Fix: connection pooling, persistent HTTP connections, or tune kernel params. CLOSE_WAIT: local side received FIN from remote but hasn't closed yet — indicates an application bug. The app received the peer's close signal but didn't call close() on its socket. This is always an application-layer problem, not a network or OS problem.

certs.eknathalabs.com

CLOSE_WAIT leak: A growing number of CLOSE_WAIT sockets means your application has a socket leak. Connections pile up until the system runs out of file descriptors. Restart the app as immediate fix; fix the bug long-term. Look for unclosed HTTP connections, missing finally blocks around socket.close().

📚 **Official docs**

- RFC 9293: TCP

Q12 **What is TCP flow control and congestion control — why do they matter for cloud services?** TCP/UDP

🎙 **Spoken answer · ~84 s**

Flow control and congestion control are two different mechanisms in TCP. Flow control protects the receiver: it advertises a window size in the TCP header telling the sender how much data it can accept; if the application reads slowly, the window shrinks, possibly to zero, and the sender pauses. Congestion control protects the network: the sender infers congestion from packet loss or rising round-trip time and adjusts its congestion window, starting slowly and growing, then backing off on loss. Algorithms include CUBIC, the Linux default, which is loss-based, and BBR, which models bandwidth and round-trip time and performs better on lossy or long-distance links. They matter for cloud services because throughput on a connection is limited by the window divided by round-trip time, so high-latency paths between regions need larger buffers and window scaling; loss triggers slowdowns that can dramatically reduce throughput; new connections start in slow start, which is another reason to reuse connections; and zero-window events indicate a slow consumer, an application problem, not a network one. I inspect with ss TCP info to see the congestion window, RTT, and retransmits, and with netstat statistics. Tuning options include buffer sizes, choosing BBR for wide-area transfers, and enabling window scaling.

Likely follow-ups

- What limits TCP throughput on a long-distance link?
- How do you spot a slow consumer using TCP information?

```
Flow control: receiver tells sender "slow down" via window size in TCP header
```

```
Congestion control: sender detects network congestion via packet loss/RTT and reduces rate
```

```
ss -tin <connection>   # show TCP info including cwnd, rtt, retransmits
```

```
netstat -s | grep -i retransmit  # global retransmit stats
```

Algorithms: CUBIC (default Linux), BBR (optimized for high-bandwidth long-latency links — used by GCP). BBR is better for cloud networking: maintains higher throughput and lower latency than CUBIC on lossy or bufferbloated paths. TCP window size limits throughput over high-latency links: max throughput = window_size / RTT. A 64KB window on a 100ms RTT link caps at ~640KB/s regardless of bandwidth.

Large file transfers failing: If TCP throughput is low despite good bandwidth, check RTT (`ping`) and TCP window scaling (`cat /proc/sys/net/ipv4/tcp_window_scaling`). Enable BBR with `net.ipv4.tcp_congestion_control=bbr` for cross-region cloud traffic.

certs.eknathalabs.com

📚 **Official docs**

- RFC 9293: TCP
- RFC 5681: TCP congestion control

Q13 **Explain subnetting — how do you calculate CIDR ranges and usable IPs?** Routing & subnets

🎙 **Spoken answer · ~83 s**

Subnetting divides an IP range into smaller networks, expressed in CIDR notation where the number after the slash is the count of network bits. The number of addresses is two to the power of thirty-two minus the prefix length. A slash twenty-four has two hundred fifty-six addresses, with two hundred fifty-four usable in traditional networking, as the network and broadcast addresses are reserved; cloud providers reserve a few more, for example five per subnet in AWS. A slash twenty-five has one hundred twenty-eight; slash sixteen has sixty-five thousand five hundred thirty-six, a common VPC size; and slash twenty has four thousand ninety-six, a common subnet size. Each bit added to the prefix halves the range. To check whether two addresses are in the same subnet, apply the mask and compare the network portions. In practice I plan ranges so that VPCs, subnets, and Kubernetes pod and service ranges don't overlap across environments, because overlaps block peering and VPN connections; leave room for growth; allocate by tier and availability zone; and document in an IP address management system. Tools like ipcalc or Python's ipaddress module do the arithmetic. Kubernetes with pod-per-IP networking can consume addresses quickly, so size pod subnets generously.

Likely follow-ups

- Why are cloud subnets' usable addresses fewer than 2^n minus 2?
- Why avoid overlapping CIDR ranges across environments?

```
ipcalc 10.0.0.0/24   # subnet calculator
```

```
/24 = 256 IPs, 254 usable (network + broadcast reserved)
```

```
/25 = 128 IPs, 126 usable  (split /24 in half)
```

```
/16 = 65,536 IPs (common VPC range)
```

```
/20 = 4,096 IPs (common subnet range)
```

```
python3 -c "import ipaddress; print(list(ipaddress.IPv4Network('10.0.0.0/28').hosts()))"
```

CIDR formula: hosts = 2^(32-prefix) - 2. Key ranges: /8 = class A (16M IPs), /16 = class B (65K IPs), /24 = class C (256 IPs). Private ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. For GKE, reserve /20 minimum for nodes, /14 for pods (GKE allocates /24 per node by default), /20 for services. Always plan subnet size for future growth — you cannot easily resize a VPC subnet in GCP.

Interview calculation: "How many hosts in /28?" → 2^(32-28) - 2 = 16 - 2 = 14 usable hosts. Know /8 through /30 cold — interviewers often ask these mentally.

certs.eknathalabs.com

📚 **Official docs**

- RFC 4632: CIDR

Q14 **How does IP routing work — what happens when a packet is sent to a different subnet?**

Routing & subnets

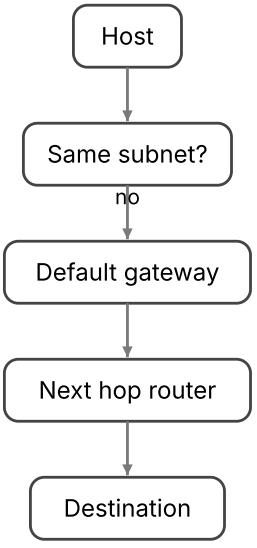
🎙 **Spoken answer · ~86 s**

IP routing is hop by hop. When a host sends a packet, it compares the destination with its own subnet. If it's on the same subnet, it delivers directly: ARP for the destination's MAC and send the frame. If not, it sends the packet to its default gateway, again using ARP for the gateway's MAC while keeping the destination IP unchanged. The gateway looks up the destination in its routing table, choosing the most specific matching route, the longest prefix match, and forwards to the next hop, decrementing the TTL, and rewriting the layer-two header at each hop. This repeats until the packet reaches a router directly connected to the destination subnet. The routing table is populated by connected networks, static routes, and dynamic protocols such as BGP and OSPF. I inspect with ip route, and ip route get shows exactly which route, interface, and source address would be used for a destination. Troubleshooting points: a missing or wrong default route, asymmetric routing where the return path differs and a stateful firewall drops the reply, overlapping routes, and the TTL expiry shown by traceroute. In the cloud, route tables attached to subnets decide whether traffic goes to an internet gateway, NAT gateway, peering, or a transit gateway.

Likely follow-ups

- What is longest prefix match?
- What is asymmetric routing and why does it break stateful firewalls?

🗺 **Routing a packet**




```
ip route show           # routing table
```

```
ip route get <dest-ip>  # which route would be used for this destination
```

certs.eknathalabs.com

```
route -n                # classic routing table view
```

When a host sends a packet: check if destination is on the same subnet (direct delivery via ARP). If not, forward to the default gateway. The gateway checks its routing table, matches the longest prefix, and forwards to the next hop — this repeats until the packet reaches its destination. Routing table entries: destination CIDR, next-hop IP, interface, metric (lower = preferred).

Cloud routing: In GCP/AWS, virtual routers are managed — you don't see a physical router, but you configure route tables (VPC routes). Custom static routes and Cloud Router (BGP) for dynamic routing. Every subnet implicitly has a route to the VPC and a default route to the internet (if configured).

📚 **Official docs**

- Linux man-pages: ip(7)

Q15 **What is BGP — how is it used in cloud networking?** Routing & subnets

🎙 **Spoken answer · ~80 s**

BGP, the Border Gateway Protocol, is the path-vector routing protocol that exchanges routes between autonomous systems and runs the internet. It runs over TCP port one seven nine. iBGP runs within one autonomous system; eBGP runs between different ones, used for internet routing and cloud connectivity. Routers advertise prefixes along with attributes such as the AS path, local preference, and MED, and choose the best path by policy and path attributes, rather than pure shortest distance. In cloud networking it's used for hybrid connectivity: Direct Connect, ExpressRoute, and Cloud Interconnect use BGP sessions to exchange on-premises and VPC routes dynamically; site-to-site VPNs often use BGP for dynamic failover between tunnels; transit gateways and cloud routers use it; and Kubernetes CNIs like Calico use BGP to advertise pod CIDRs to the network. Benefits: automatic failover when a link goes down, and route filtering for control. Operational care: filter what you advertise and accept, since a misconfiguration can leak routes or blackhole traffic, as happened in well-known internet outages; set prefix limits; use route priorities and AS path prepending to steer traffic; monitor session state and received prefixes; and use BFD for fast failure detection.

Likely follow-ups

- What does the AS path attribute do?
- How is BGP used for hybrid cloud failover?

certs.eknathalabs.com

```
BGP (Border Gateway Protocol): L4 (TCP 179), path-vector protocol for inter-AS routing
```

```
iBGP: within same AS — used inside large networks
```

```
eBGP: between different ASes — used for internet routing and cloud connectivity
```

```
gcloud compute routers get-status <router> --region=<r>  # BGP session status on GCP
```

BGP uses AS path, local preference, MED, and community attributes to select routes. In cloud: GCP Cloud Router uses BGP to exchange routes with on-premises over VPN or Interconnect — your on-prem router advertises its prefixes, Cloud Router advertises VPC subnets. BGP session flapping causes route withdrawals leading to connectivity drops. Monitor BGP session

state and route count as key SLIs for hybrid connectivity.

Why BGP matters for Platform Engineers: Even if you don't configure BGP directly, understanding it helps diagnose why on-prem → GCP routes disappear during maintenance windows, why traffic takes unexpected paths, and why adding a new VPC subnet requires a BGP re-advertisement.

📚 **Official docs**

- RFC 4271: BGP-4
- Cloudflare Learning: What is BGP?

Q16 **Traceroute shows packets stopping at a certain hop — how do you interpret and act on this?**

Routing & subnets

🎙 **Spoken answer · ~87 s**

Traceroute lists the routers along a path by sending packets with increasing TTLs and noting which router replies with time exceeded. Hops showing stars don't necessarily mean a problem: the router may filter or rate-limit ICMP time exceeded responses, or deprioritise them, while still forwarding traffic fine. The key question is what happens after the stars: if later hops respond, the earlier hop is just hiding; if the trace stops there and never reaches the destination, something may be dropping traffic. I interpret by comparing: use TCP-based traceroute to the real port, which behaves like the application's traffic and gets past ICMP and UDP filters, and use mtr for continuous per-hop loss and latency. Loss that starts at a hop and persists to the end is real; loss at a single middle hop that disappears later is rate limiting. A trace that stops at a particular hop consistently, and the application also fails, points to a firewall, route, or outage at or after that hop. Action: gather evidence, run it from both directions since return paths may differ, identify the network owner, whether my cloud VPC, my provider, or the destination, and engage them with the mtr output. Also check security groups, routes, and ACLs on my side first.

Likely follow-ups

- Why use TCP traceroute rather than the default?
- How do you read mtr loss correctly?

```
traceroute <dest>           # Linux (UDP by default)
```

```
traceroute -T -p 443 <dest> # TCP traceroute on port 443 (bypasses ICMP blocks)
```

```
mtr <dest>                  # continuous traceroute with packet loss stats
```

```
traceroute -I <dest>        # ICMP-based (like Windows tracert)
```

Interpreting results: hops showing * * * can mean (1) router is filtering ICMP TTL-exceeded replies (normal — not necessarily a problem), (2) packet is truly being dropped. Key: if hops after the * show responses, the intermediate router just blocks ICMP — traffic is flowing. If all subsequent hops also show *, the packet is dropped at that point. Use TCP traceroute on port 443 to bypass ICMP filtering and trace the actual application path.

Cloud traceroute: Many cloud hops show * because cloud providers filter ICMP. Google's backbone is often opaque (you see your

certs.eknathalabs.com

VM, then suddenly the destination). This is normal — use Connectivity Tests in GCP instead of traceroute for cloud path analysis.

📚 **Official docs**

- RFC 792: ICMP
- traceroute(8) / mtr

Q17 **Explain the HTTPS handshake — what happens between TCP connect and the first HTTP request?** HTTP/TLS

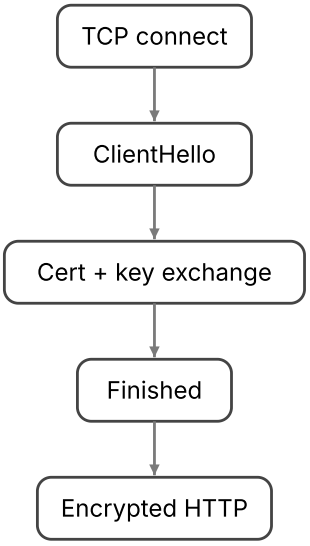
🎙 **Spoken answer · ~80 s**

After the TCP handshake, the HTTPS handshake negotiates encryption. In TLS 1.2 the sequence is: the client sends ClientHello with supported protocol versions, cipher suites, and the server name via SNI; the server replies with ServerHello choosing the parameters, and sends its certificate; the client verifies the certificate chain against trusted CAs, checks expiry, and that the hostname matches; then a key exchange, usually ephemeral Diffie-Hellman, which gives forward secrecy; both sides derive symmetric session keys and exchange finished messages; then application data flows encrypted. TLS 1.3 streamlines this to one round trip, and supports zero-RTT resumption for repeat connections. SNI lets one IP serve many certificates. ALPN negotiates HTTP/2 or HTTP/1.1 during the handshake. Why it matters operationally: the handshake adds latency, a few round trips, which is a reason for connection reuse and session resumption; certificate problems, expired, wrong hostname, missing intermediates, or an untrusted CA, are a common cause of failures; protocol or cipher mismatches cause handshake failures; and TLS termination at a load balancer versus end-to-end changes where certificates must be managed. I debug with openssl s_client, which shows the chain and negotiated parameters, and curl verbose.

Likely follow-ups

- What does SNI do?
- How does TLS 1.3 reduce handshake latency?

🗺 **HTTPS setup**




```
1. TCP 3-way handshake (SYN/SYN-ACK/ACK)
```

certs.eknathalabs.com

```
2. TLS ClientHello (supported cipher suites, TLS version, SNI)
```

```
3. TLS ServerHello (chosen cipher, server certificate)
```

```
4. Certificate verification (CA chain, expiry, hostname match)
```

```
5. Key exchange (ECDHE for perfect forward secrecy)
```

```
6. Symmetric key derived → encrypted channel established
```

```
7. First HTTP request sent
```

```
openssl s_client -connect <host>:443 -servername <host>  # inspect TLS handshake
```

TLS 1.3 reduces this to 1 RTT (vs TLS 1.2's 2 RTT). SNI (Server Name Indication) in ClientHello allows virtual hosting — multiple HTTPS sites on one IP. This is how cloud load balancers serve multiple TLS certificates on a single IP. ALPN negotiates the application protocol (http/1.1 vs h2 vs h3) during TLS handshake.

> **TLS debugging:** `openssl s_client` is the most powerful TLS debugging tool. It shows the full certificate chain, cipher negotiated, and TLS version. Always check this before assuming an HTTPS problem is a network problem.

> 📚 **Official docs**
>
> - RFC 9293: TCP
> - RFC 8446: TLS 1.3
> - Cloudflare Learning: TLS handshake

certs.eknathalabs.com

Q18 **What are HTTP status codes — which ones indicate network vs application vs server issues?** HTTP/TLS

🎙 Spoken answer · ~86 s

HTTP status codes are grouped by class. Two hundred codes mean success: 200 OK, 201 Created, 204 No Content. Three hundred codes are redirects: 301 permanent, 302 temporary, 304 not modified for caching. Four hundred codes are client errors: 400 bad request, 401 unauthenticated, 403 forbidden, 404 not found, and 429 rate limited. Five hundred codes are server-side: 500 internal error, 502 bad gateway, 503 service unavailable, 504 gateway timeout. For triage, the code and who generated it point to the layer. A 502 from a proxy or load balancer means it got an invalid response or the connection to the backend failed, so the backend crashed, closed the connection, or the keepalive timeouts are mismatched. A 503 usually means no healthy backends, overload, or deliberate load shedding. A 504 means the backend didn't respond within the proxy's timeout, so it's slow. A 500 is typically an application bug or failing dependency. 4xx surges can mean a bad client release, auth system trouble, or an attack. Also check which component emitted the code, by looking at response headers and access logs at each layer. Curl with headers shows quickly. For monitoring, alert on 5xx rates and latency, not 4xx alone, but watch for sudden 4xx changes.

Likely follow-ups

- What does a 502 versus a 504 tell you?
- Why check which component generated the error?

```
2xx — Success: 200 OK, 201 Created, 204 No Content
```

```
3xx — Redirect: 301 Permanent, 302 Temporary, 304 Not Modified
```

```
4xx — Client error: 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Rate Limited
```

```
5xx — Server error: 500 Internal, 502 Bad Gateway, 503 Unavailable, 504 Gateway Timeout
```

```
curl -sI <url> | head -5   # check status code + headers quickly
```

Network-related codes: 502 Bad Gateway = load balancer received an invalid/no response from the upstream (backend crashed, connection reset). 503 Service Unavailable = load balancer has no healthy backends. 504 Gateway Timeout = backend was reachable but didn't respond within the timeout window. 502/503/504 all originate at the load balancer, not the application — check backend health first.

GKE context: 502 on GKE Ingress = GCP LB health check is failing (pod not passing health check endpoint), 503 = all pods are unhealthy or not yet ready (just deployed), 504 = pod is receiving traffic but responding too slowly (increase LB backend timeout or fix slow queries).

📚 Official docs

- RFC 9110: HTTP Semantics (status codes, methods)

certs.eknathalabs.com

Q19 **What is HTTP/2 — how does it differ from HTTP/1.1 and why does it matter?** HTTP/TLS

🎙 Spoken answer · ~80 s

HTTP/1.1 uses one request at a time per connection, since pipelining is unreliable in practice, so browsers open several parallel connections; it suffers head-of-line blocking, and sends headers as uncompressed text on every request. HTTP/2 multiplexes many concurrent streams over a single TCP connection, compresses headers with HPACK, supports server push and stream prioritisation, and uses a binary framing layer. Benefits: fewer connections and handshakes, better use of bandwidth, lower latency for pages with many resources, and it's negotiated via ALPN in the TLS handshake, so it effectively requires HTTPS in browsers. Caveats: it still runs on TCP, so a single lost packet blocks all streams, TCP-level head-of-line blocking, which HTTP/3 solves by using QUIC over UDP. Operationally: load balancers and proxies must support it; gRPC requires HTTP/2, and long-lived multiplexed connections can cause uneven load distribution at layer four, since all requests from one client go to one backend, so use layer-seven balancing; connection-level limits, such as maximum concurrent streams, matter. Old optimisations like domain sharding and sprite sheets become unnecessary or counterproductive. I check with curl, which shows the protocol used, and with openssl s_client for the ALPN result.

Likely follow-ups

- Why does gRPC need layer-7 load balancing?
- What problem does HTTP/3 solve that HTTP/2 doesn't?

```
curl --http2 -I <url>          # force HTTP/2
```

```
curl -v <url> 2>&1 | grep "Using HTTP"  # confirm protocol used
```

```
openssl s_client -connect <host>:443 -alpn h2  # negotiate HTTP/2 via ALPN
```

HTTP/1.1: one request per connection (pipelining unreliable in practice), head-of-line blocking, header sent as text on every request. HTTP/2: multiplexed streams over single TCP connection (multiple requests in flight simultaneously), header compression (HPACK), server push, binary framing. Key for microservices: gRPC runs on HTTP/2 — multiplexing allows many concurrent RPCs over one connection, dramatically reducing connection overhead in service meshes.

Load balancer config: GCP HTTP(S) LB supports HTTP/2 between client and LB. Between LB and backends, ensure your backend supports HTTP/2 or configure LB to use HTTP/1.1 to backends. Mismatch causes protocol errors. Kubernetes Services serving gRPC must use HTTP/2 — configure Ingress with `grpc` backend protocol.

📚 Official docs

- RFC 9113: HTTP/2

certs.eknathalabs.com

Q20 **TLS certificate errors in production — how do you diagnose and resolve them?** HTTP/TLS

🎙 Spoken answer · ~84 s

For TLS certificate errors, I first inspect the actual certificate presented using openssl s_client with the server name, and view the chain, validity dates, subject alternative names, and issuer. Common causes: expired certificate, the most frequent, especially when auto-renewal silently failed; hostname mismatch, where the name isn't in the SAN list; an incomplete chain, a missing intermediate certificate, which works in browsers that fetch intermediates but fails in programmatic clients; an untrusted or private CA not in the client's trust store; a wrong certificate served because SNI isn't sent or the load balancer's default certificate is used; clock skew on the client making a valid certificate look expired; revoked certificates; and protocol or cipher mismatch. Intermittent errors suggest several backends with different certificates, so I test each IP. Curl with the insecure flag helps separate certificate problems from network problems, but only for diagnosis. Resolution: renew or reissue, fix the chain by deploying the full chain file, update trust stores, correct the SAN list, and reload the service so it picks up the new certificate. Prevention is automation and monitoring: ACME with cert-manager or managed certificates, expiry alerts at thirty, fourteen, and seven days, and synthetic checks that verify the full chain.

Likely follow-ups

- Why do some clients fail on a missing intermediate that browsers tolerate?
- How do you detect expiring certificates before they break something?

```
openssl s_client -connect <host>:443 -servername <host>  # full TLS inspection
```

```
openssl x509 -in cert.pem -noout -text | grep -E "Subject|Issuer|Not After|SAN"
```

```
echo | openssl s_client -connect <host>:443 2>/dev/null | openssl x509 -noout -dates
```

```
curl -vk <url>  # ignore cert errors to test if it's just a cert issue vs network
```

Common TLS errors: certificate expired (check Not After date), hostname mismatch (SAN/CN doesn't match the hostname — check wildcard coverage), untrusted CA (self-signed or private CA not in trust store), certificate chain incomplete (intermediate CA missing — openssl shows "unable to verify the first certificate"). In Kubernetes: cert-manager automates Let's Encrypt certificate issuance and rotation. Alerting on cert expiry is critical — never discover expiry via user complaints.

Internal service TLS: Service-to-service mTLS (mutual TLS) is enforced by service meshes (Istio, Linkerd). If mTLS is enabled and a service doesn't present a valid client cert, the connection is rejected with a TLS error — not a 403 HTTP error. Look at TLS layer, not HTTP layer.

📚 Official docs

- Cloudflare Learning: TLS handshake

certs.eknathalabs.com

Q21 **What is a firewall and how do stateful vs stateless firewalls differ?** Network security

🎙 Spoken answer · ~84 s

A stateless firewall evaluates each packet independently against rules on source, destination, port, and protocol; it doesn't remember connections, so to allow a reply you need an explicit rule for the return traffic, including ephemeral port ranges. A stateful firewall tracks connection state, so when an outbound connection is allowed, the return traffic is automatically permitted, and unsolicited inbound traffic is dropped. Linux iptables with conntrack is stateful, with a rule accepting established and related packets; cloud security groups are stateful, whereas network ACLs, such as AWS NACLs, are stateless and need both directions. Stateful firewalls are simpler to write and safer by default, but they consume memory for the connection table, which can be exhausted under floods, and can cause trouble with asymmetric routing, where the reply returns via a different path and the firewall has no state for it. Stateless filters are fast and suit high-throughput edge cases or coarse subnet-level controls. In practice I combine: security groups for per-instance stateful rules, NACLs as a coarse extra layer, and for deeper inspection, application-aware or next-generation firewalls and WAFs. Troubleshooting tip: a connection that works one way but not the other often points to a stateless rule missing return ports.

Likely follow-ups

- Why do NACLs need rules for return traffic?
- How can asymmetric routing break a stateful firewall?

```
Stateless: matches each packet independently (src IP, dst IP, port, protocol)
```

```
Stateful: tracks connection state — allows return traffic automatically
```

```
iptables -L -n -v          # stateful (conntrack) firewall rules
```

```
iptables -t filter -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

Stateful firewalls (iptables with conntrack, GCP Firewall, AWS Security Groups) track TCP sessions — if you allow outbound TCP 443, return traffic is automatically permitted. Stateless firewalls (AWS Network ACLs, some hardware firewalls) require explicit allow rules for both directions. GCP Firewall is stateful — you only need an ingress allow rule, not an egress rule for established connections. NACLs in AWS require both inbound and outbound rules.

GCP specifics: GCP Firewall rules are stateful and applied per VM via network tags or SA bindings. Default VPC allows all internal traffic and all outbound. Custom VPCs have a default-deny for all traffic — you must explicitly allow everything including intra-VPC.

📚 Official docs

- Amazon VPC: Network ACLs

certs.eknathalabs.com

Q22 **What is a DDoS attack — how do you protect cloud services against it?** Network security

🎙 Spoken answer · ~87 s

DDoS attacks aim to make a service unavailable by overwhelming it, and they come in three types. Volumetric attacks flood bandwidth, such as UDP or ICMP floods and amplification through DNS or NTP. Protocol attacks exhaust state in network devices and servers, like SYN floods that fill connection tables. Application-layer attacks exhaust CPU or backend resources with seemingly legitimate requests, like HTTP floods and slowloris. Protection works in layers. At the edge, use the provider's DDoS protection, such as AWS Shield, Cloud Armor, or a CDN, which absorb volumetric attacks by their sheer capacity, with anycast distributing traffic. Use a WAF with rate limiting, bot detection, geo and IP reputation rules, and challenge pages for application-layer floods. On the origin side: don't expose origin IPs directly, restrict them to the CDN or load balancer ranges; enable SYN cookies and tune backlogs; use autoscaling carefully with cost limits; cache aggressively so floods hit the cache, not the backend; apply per-client rate limits and timeouts; and design expensive endpoints to be authenticated or queued. Prepare a runbook: how to identify the attack pattern, who to call at the provider, how to enable stricter rules, and how to communicate. Test with game days and monitor traffic baselines to detect anomalies early.

Likely follow-ups

- Why hide the origin IP behind a CDN?
- How do application-layer attacks differ from volumetric ones?

```
Volumetric: flood bandwidth (UDP flood, ICMP flood)
```

```
Protocol: exhaust state (SYN flood — fill TCP connection table)
```

```
Application layer: exhaust CPU (HTTP flood, slow loris)
```

```
gcloud compute security-policies create <name>  # Cloud Armor WAF/DDoS policy
```

```
gcloud compute security-policies rules create 1000 --security-policy=<name> --
expression="request.headers['User-Agent'].matches('bad-bot')" --action=deny-403
```

GCP protection layers: Global HTTP(S) LB absorbs volumetric attacks (Google's global network scrubs traffic before it reaches your backends), Cloud Armor for WAF rules and L7 DDoS protection (rate limiting, geo-blocking, OWASP rules), Google Cloud Armor Adaptive Protection for ML-based attack detection. For SYN floods: GCP's load balancers handle SYN cookies automatically — your backend VMs never see the flood.

Rate limiting: Implement at multiple layers: Cloud Armor (LB level), application-level rate limiting (per-user, per-IP), and API gateway throttling. Defense in depth. A single rate limit at one layer can be bypassed by distributing requests across sources.

📚 Official docs

- Cloudflare Learning: DDoS attacks

certs.eknathalabs.com

Q23 **What is mTLS (mutual TLS) — how is it implemented in Kubernetes?** Network security

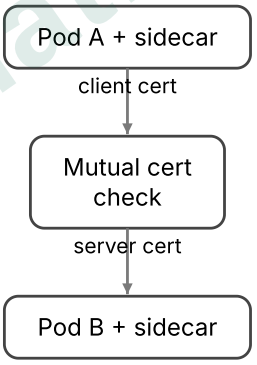
🎙 Spoken answer · ~87 s

In regular TLS only the server presents a certificate, and the client verifies the server's identity. In mutual TLS both sides present certificates and verify each other, so the server also cryptographically knows who the client is. This gives strong service-to-service authentication and encryption in transit without passwords or tokens, and a basis for zero-trust: a request is trusted because of its verified identity, not its network location. In Kubernetes, mTLS is usually implemented by a service mesh such as Istio or Linkerd: a sidecar proxy, or in newer designs a node-level proxy, is injected next to each workload and handles the TLS; the application talks plain HTTP to its local proxy, and proxies establish mTLS between themselves. The mesh control plane acts as a certificate authority, issuing short-lived certificates to each workload based on its service account identity and rotating them automatically. Policies, like Istio's PeerAuthentication, set strict mode, which requires mTLS, or permissive mode, useful during migration, and authorisation policies specify which identities may call which services. Operational concerns: certificate rotation and CA root management, the overhead of proxies, debugging encrypted traffic, interoperability with non-mesh workloads, and gradual rollout from permissive to strict. Without a mesh, cert-manager and application-level TLS can do it, with more work.

Likely follow-ups

- What identity does a mesh certificate carry?
- Why start with permissive mode when rolling out mTLS?

🗺 mTLS in Kubernetes




```
Regular TLS: only server presents a certificate (client verifies server identity)
```

```
mTLS: both client AND server present certificates (mutual verification)
```

```
kubectl get peerauthentication -A    # Istio mTLS policy
```

```
kubectl get destinationrule -A       # Istio client-side TLS config
```

In Kubernetes: Istio and Linkerd inject sidecar proxies that handle mTLS transparently — application code doesn't change. The service mesh CA issues short-lived certificates to each pod (using SPIFFE/SPIRE standard). Benefits: all service-to-service traffic is encrypted and authenticated, zero-trust within the cluster (even internal traffic is verified). Enable with `PeerAuthentication: STRICT` in Istio per namespace.

certs.eknathalabs.com

Zero trust networking: mTLS + NetworkPolicies + RBAC = zero trust inside a Kubernetes cluster. Even if an attacker gets into the cluster, they can't speak to services without a valid certificate, and NetworkPolicies block unexpected traffic paths.

📚 **Official docs**

- Cloudflare Learning: mutual TLS

Q24 **What is a network proxy — how do forward and reverse proxies differ?** Network security

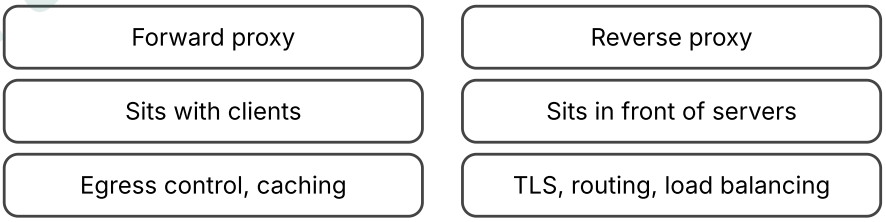
🎙 **Spoken answer · ~88 s**

A forward proxy sits in front of clients and sends their requests to the internet: the clients are hidden from the servers. It's used for egress filtering, policy enforcement, caching, and auditing of outbound traffic, and clients are configured to use it, such as through proxy environment variables. A reverse proxy sits in front of servers: clients talk to it as if it were the origin, and it forwards requests to backends, hiding the backends from the clients. Typical uses of reverse proxies: TLS termination, load balancing, caching, compression, routing by host or path, authentication, rate limiting, and WAF functions; nginx, Envoy, HAProxy, and cloud load balancers play this role, as do ingress controllers in Kubernetes. The distinction is about who it represents: forward proxies represent the client, reverse proxies represent the server. Operational points: proxies change what the backend sees, so I need X-Forwarded-For or the PROXY protocol to preserve client IPs; timeouts and keepalives must be coordinated between the proxy and backends to avoid 502 errors; proxy environment variables, including no-proxy lists, are a frequent source of why it works from my laptop but not from the server issues; and a proxy is also a single point of failure and a choke point that needs redundancy and monitoring.

Likely follow-ups

- Which proxy type hides the client and which hides the server?
- Why do proxies need X-Forwarded-For?

🗺 **Forward vs reverse proxy**




```
Forward proxy: client → proxy → internet (hides client, used for egress filtering)
```

```
Reverse proxy: client → proxy → backend servers (hides backend, used for LB/TLS
termination)
```

```
curl -x http://<proxy>:3128 <url>   # use forward proxy
```

```
env | grep -i proxy                  # check proxy env vars
```

certs.eknathalabs.com

Reverse proxy use cases: TLS termination (nginx/Envoy handles TLS, backends get plain HTTP), load balancing (distribute requests across backends), caching (serve cached responses), rate limiting, and authentication (add auth layer in front of unprotected service). In Kubernetes: every Ingress controller IS a reverse proxy (nginx, Envoy/Contour, Traefik). Service mesh sidecars (Envoy) are transparent reverse + forward proxies per pod.

Envoy proxy: Envoy is the de facto proxy for cloud-native networking. It powers Istio sidecars, GCP's load balancers (Envoy-based), Cloud Service Mesh, and many API gateways. Understanding Envoy's listener → filter chain → cluster → endpoint model helps debug service mesh issues at the proxy level.

📚 **Official docs**

- Cloudflare Learning: What is a reverse proxy?

Q25 **How does a load balancer distribute traffic — what are the different algorithms?** Load balancing

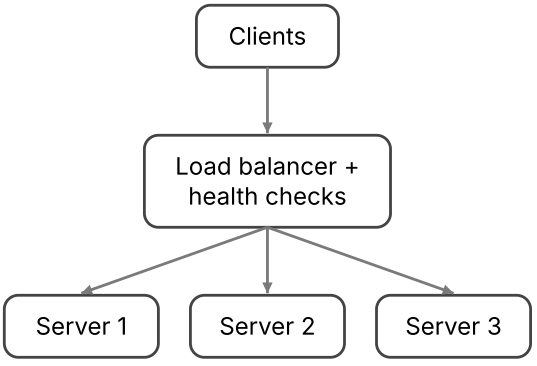
🎙 **Spoken answer · ~84 s**

Load balancers distribute requests across backends using different algorithms. Round robin goes through backends in sequence and works well when requests are similar and backends are uniform. Least connections sends to the backend with the fewest active connections, better when request cost varies or connections are long-lived. Weighted variants give proportional shares, useful for mixed instance sizes and canary releases, like ninety percent to the stable version and ten to the new one. IP hash maps a client's address to a backend, giving a form of affinity without cookies. Consistent hashing on a key such as a user or cache key keeps the same requests going to the same backend and minimises disruption when backends change, which is valuable for caches. Least response time or power-of-two-choices pick better-performing backends, and the latter is simple and effective at avoiding hot spots. Beyond the algorithm, health checks matter most: unhealthy backends must be removed quickly, and slow start avoids overwhelming new ones. Other considerations: connection draining on deploys, layer four versus layer seven, how stickiness interacts with balance, and how the load balancer itself scales and is made redundant. I pick the algorithm based on request characteristics and verify with metrics on per-backend load.

Likely follow-ups

- When is least connections better than round robin?
- Why is consistent hashing good for caches?

🗺 **Load balancer**



certs.eknathalabs.com

```
Round Robin: requests distributed sequentially across backends (default, good for uniform load)
```

```
Least Connections: route to backend with fewest active connections (good for variable request cost)
```

```
IP Hash: hash source IP to backend (session affinity without cookies)
```

```
Weighted: assign proportional traffic share (canary: 90% v1, 10% v2)
```

```
Random with 2 choices (P2C): pick 2 random, route to less loaded (best for microservices)
```

GCP HTTP(S) LB uses a combination of consistent hashing and session affinity. For gRPC/HTTP2 with long-lived connections, round-robin per-connection is wrong — you need per-request load balancing. Envoy in service mesh does per-request load balancing over HTTP/2 multiplexed connections, which is why service mesh L7 load balancing outperforms K8s kube-proxy (L4) for gRPC workloads.

Health checks: Load balancers continuously health check backends. If a backend fails health check, it's removed from the pool. Always configure separate health check endpoints (`/healthz`) that return 200 quickly without side effects — don't use your main API endpoint.

📚 **Official docs**

- Cloudflare Learning: Load balancing

Q26 **What is session affinity (sticky sessions) — when should you use it and what are the risks?** Load balancing

🎙 **Spoken answer · ~86 s**

Session affinity, or sticky sessions, routes requests from the same client to the same backend, using client IP hash, a cookie inserted with the backend ID, or a header. It's used when the application keeps session state in memory on the server, so moving the user to another backend would lose the session, or to improve cache locality, or for protocols like WebSocket where the connection naturally sticks. The risks: uneven load, since heavy users or clients behind one NAT IP all hit one backend, creating hotspots; failover pain, since if the sticky backend dies, those sessions are lost; harder deployments and scaling, because draining a backend means waiting for sessions, and new backends receive no traffic from existing users; and it hides poor design. The better approach is to make the application stateless: store sessions in a shared store like Redis or a database, or use signed tokens such as JWTs, so any backend can serve any request, which gives better resilience, even load distribution, and simpler scaling. If I must use affinity, I prefer cookie-based with a limited lifetime over IP hash, set sensible TTLs, plan for backend loss, and monitor per-backend load. For WebSockets and long-lived streaming, affinity is inherent, so handle reconnection gracefully.

Likely follow-ups

- Why is IP-hash affinity unreliable behind a NAT?
- How would you remove the need for sticky sessions?

certs.eknathalabs.com

```
Session affinity: route requests from the same client always to the same backend
```

```
Methods: client IP hash, cookie-based (insert cookie with backend ID), HTTP header
```

```
# GCP HTTP LB: configure affinity via backend service
```

```
gcloud compute backend-services update <name> --session-affinity=GENERATED_COOKIE --
affinity-cookie-ttl=3600
```

Use sticky sessions when: application state is stored in-process on the backend (legacy apps, websockets). Risks: hot spots (one backend gets all traffic from heavy users), failover breaks sessions (sticky session target dies — all its users get disrupted), and prevents horizontal scaling from distributing load evenly. Prefer stateless backends with shared session storage (Redis, Memorystore) — this eliminates the need for session affinity entirely.

Kubernetes websockets: WebSocket connections are long-lived — once established, they stay on one pod. This is inherently sticky. Ensure the pod it's connected to is not evicted/rescheduled without graceful connection migration, or use a message broker (Redis Pub/Sub) for shared connection state.

📚 **Official docs**

- Cloudflare Learning: Load balancing

Q27 **What is the difference between L4 and L7 load balancing — when do you need each?** Load balancing

🎙 **Spoken answer · ~85 s**

Layer four load balancers route based on IP address and port, without looking inside the traffic. They're fast, with low latency and high throughput, protocol-agnostic, can preserve the client's source IP, and handle any TCP or UDP protocol including databases and TLS passthrough; examples are kube-proxy's iptables or IPVS rules, AWS NLB, and HAProxy in TCP mode. Layer seven load balancers terminate and parse the application protocol, usually HTTP, so they can route by host, path, headers, or cookies, rewrite requests, terminate TLS, apply authentication, rate limiting, and retries, do canary splitting, gather detailed request metrics, and integrate with a WAF; examples are nginx or Envoy ingress, AWS ALB, and the GCP HTTP load balancer. Trade-offs: layer seven costs more CPU and adds latency, and the client IP is conveyed via headers; layer four is simple and scalable but blind to content, and with HTTP/2 or gRPC long-lived connections it balances connections, not requests, leading to uneven load. I choose layer seven for web and API traffic needing content-based routing and observability, and layer four for non-HTTP protocols, extreme throughput, or when I want end-to-end TLS untouched. Often both are combined: a network load balancer in front of layer-seven proxies for scale and static IPs.

Likely follow-ups

- Why can an L4 balancer distribute gRPC unevenly?
- When would you put an L4 balancer in front of L7 proxies?

```
L4 (Transport): routes by IP + port only — fast, no content inspection
```

```
L4 examples: kube-proxy (iptables DNAT), TCP Network LB, AWS NLB
```

certs.eknathalabs.com

```
L7 (Application): inspects HTTP headers, URL, cookies — intelligent routing
```

```
L7 examples: nginx Ingress, Envoy, GCP HTTP(S) LB, AWS ALB
```

L4 advantages: lower latency (no HTTP parsing), preserves original client IP (no need for X-Forwarded-For), handles any TCP/UDP protocol. L7 advantages: URL-based routing (route /api to one service, /static to another), header-based canary (route 10% by header), TLS termination, HTTP/2 multiplexing, WebSocket upgrade handling, request/response modification. Use L4 for non-HTTP (databases, SMTP, custom protocols). Use L7 for HTTP APIs — the routing intelligence is worth the small overhead.

Kubernetes services: ClusterIP/NodePort/LoadBalancer are L4 — they route by port. Ingress is L7 — routes by hostname and URL path. For gRPC load balancing, you need L7 (H2 stream-level balancing) — L4 kube-proxy only balances at the TCP connection level, which defeats multiplexing.

📚 **Official docs**

- Cloudflare Learning: OSI model
- Cloudflare Learning: Load balancing

Q28 **What is a VPC — how does cloud VPC networking differ from traditional networking?** Cloud networking

🎙 **Spoken answer · ~83 s**

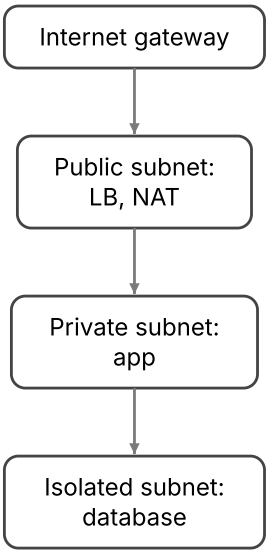
A VPC, virtual private cloud, is a logically isolated, software-defined network in a cloud account where I define IP ranges, subnets, route tables, gateways, and security controls. Traditional networking is built from physical switches, VLANs, hardware routers, and spanning tree, with changes needing physical or device-level configuration. A VPC is created by API in seconds and managed as code. Differences: there's no real layer two in the usual sense; the provider's fabric handles ARP and switching, so broadcast-dependent designs and tricks like gratuitous ARP failover work differently; routing is defined by route tables attached to subnets, deciding whether traffic goes to an internet gateway, NAT gateway, peering, VPN, or transit gateway; security is applied through stateful security groups per instance and stateless network ACLs per subnet; subnets are tied to availability zones, so multi-AZ designs use multiple subnets; IP addresses are partly reserved by the provider; and bandwidth, packet-per-second, and connection limits are per instance type. Design considerations: non-overlapping CIDR ranges for peering and hybrid connectivity, public versus private subnets, egress control, private endpoints to cloud services, flow logs for visibility, and segmentation by environment or account. Kubernetes adds pod networking on top, which consumes VPC addresses with some CNIs.

Likely follow-ups

- Why must VPC CIDR ranges not overlap?
- How do security groups differ from network ACLs?

certs.eknathalabs.com

🗺 **VPC anatomy**




```
gcloud compute networks create <name> --subnet-mode=custom  # create VPC
```

```
gcloud compute networks subnets create <name> --network=<vpc> --region=<r> --
range=10.0.0.0/24
```

```
gcloud compute networks describe <name>  # VPC config
```

Traditional networking: physical switches, VLANs, hardware routers, spanning tree. Cloud VPC: software-defined, no physical topology, global (GCP VPCs span all regions — single VPC with subnets in us-central1 and europe-west1), firewall rules enforced at the VM level (not at a perimeter device), and routes are virtual. Key differences: no broadcast/multicast (ARP is handled by the hypervisor), no spanning tree, no VLANs (use subnets for segmentation), no L2 adjacency between subnets.

GCP VPC is global: Unlike AWS where VPCs are regional, a GCP VPC spans all regions. You create subnets per region within the same VPC. VMs in different regions within the same VPC can communicate using internal IPs without VPC peering — a major architectural simplification.

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q29 What is an Overlay network — how does it work in Kubernetes (VXLAN, CNI)?

Cloud networking

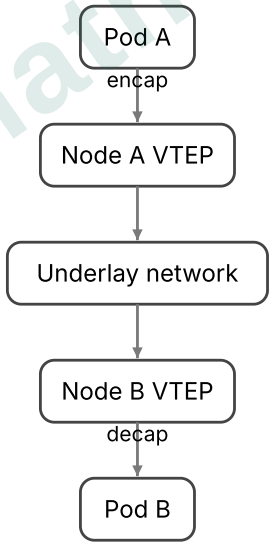
🎙 Spoken answer · ~85 s

An overlay network is a virtual network tunnelled over an underlying physical or cloud network, called the underlay. VXLAN is a common technology: it encapsulates layer-two Ethernet frames inside UDP packets, so pods on different nodes appear to be on the same layer-two segment even though the nodes are only connected by layer three. In Kubernetes, the CNI plugin implements pod networking: each pod gets an IP, and cross-node pod traffic is encapsulated by the overlay, as Flannel, Calico in VXLAN or IP-in-IP mode, Weave, and Cilium can do. The alternative is a non-overlay approach, routing pod CIDRs natively via BGP or cloud route tables, or using VPC-native addressing, which avoids encapsulation overhead. Trade-offs of overlays: they work on any network without cooperation from the underlay and simplify setup, but add header overhead, so the MTU of the pod network must be reduced or packets get fragmented or dropped, a classic cause of mysterious hangs on large transfers; they add CPU cost for encapsulation; and they make troubleshooting harder, since underlay tools see only UDP between nodes. I identify the CNI by looking at kube-system pods, inspect overlay interfaces with ip link, check MTU consistency, and use tcpdump on the node to see encapsulated traffic.

Likely follow-ups

- Why must you lower the MTU on an overlay network?
- What is the difference between overlay and native routing CNIs?

🗺 Overlay (VXLAN)




```
Overlay: virtual L2 network tunneled over L3 underlay (physical network)
```

```
VXLAN: encapsulates L2 frames in UDP packets — allows L2 across L3 boundaries
```

```
kubectl get pods -n kube-system | grep -E "calico|flannel|cilium|weave"  # identify CNI
```

```
ip link show | grep -E "vxlan|cni|flannel"  # show overlay interfaces
```

certs.eknathalabs.com

In Kubernetes: each CNI plugin (Calico, Cilium, Flannel) implements pod networking. Flannel uses VXLAN overlay — every pod IP is routable within the cluster via tunnels, regardless of the underlying node network. Calico can use BGP (no encapsulation overhead) if the underlay supports it. On GKE with VPC-native clusters, pod IPs are actual VPC IPs — no overlay needed, pods are directly routable from anywhere in the VPC.

GKE VPC-native: GKE VPC-native clusters allocate a /24 pod CIDR per node from a reserved VPC range. No overlay — pods are first-class VPC citizens with their own IPs. This enables direct VPC connectivity to pods and is more efficient than overlay networks.

📚 **Official docs**

- Kubernetes: Cluster networking model
- RFC 7348: VXLAN
- Kustomize reference

Q30 **What is a Service Mesh — how does it handle networking in microservices?** Cloud networking

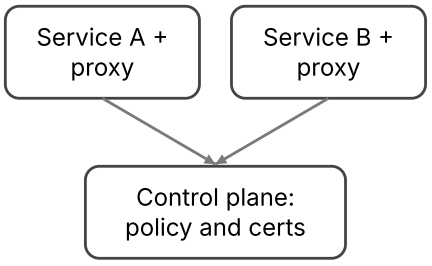
🎙 **Spoken answer · ~84 s**

A service mesh moves service-to-service networking concerns out of application code into the infrastructure. A sidecar proxy, usually Envoy, is injected into every pod, and all traffic goes through it; the application talks to localhost, and the proxy handles the rest, while a control plane such as Istiod configures the proxies. Capabilities: mutual TLS between services with automatic certificate rotation; traffic management, such as retries, timeouts, circuit breaking, load balancing, canary and weighted routing, and fault injection; authorisation policies; and observability, with consistent metrics, access logs, and distributed traces for every call without code changes. The benefit is uniform policy and visibility across polyglot services. The costs: added latency and resource overhead per pod, a control plane to operate and upgrade, more moving parts to debug, such as a misconfigured sidecar causing mysterious 503s, and complexity of configuration. Newer designs, like ambient mode or eBPF-based meshes, reduce the sidecar overhead. I'd adopt a mesh when there are many services needing mTLS, consistent traffic control, and observability, and avoid it for a handful of services where simpler tools suffice. For debugging, I use proxy status, config analysis, sidecar logs, and metrics, and check that sidecars are in sync with the control plane.

Likely follow-ups

- What problems does a mesh solve that libraries don't?
- What are the drawbacks of sidecar proxies?

🗺 **Service mesh**



certs.eknathalabs.com

```
kubectl get pods -n istio-system          # Istio control plane
```

```
istioctl proxy-status                     # sidecar sync status
```

```
istioctl analyze -n <ns>                 # config validation
```

```
kubectl logs <pod> -c istio-proxy         # sidecar proxy logs
```

Service mesh injects a sidecar proxy (Envoy) into every pod. All traffic goes through the sidecar — the app talks to localhost, the sidecar handles actual network calls. This transparently adds: mTLS, observability (traces, metrics, access logs per request), retry policies, circuit breaking, traffic shifting (canary), and timeout enforcement. The control plane (Istiod) distributes config to all sidecars.

Sidecar overhead: Every pod has an additional Envoy container consuming ~50-100MB RAM and adding ~1-2ms latency per hop. For very latency-sensitive or memory-constrained workloads, this matters. Cilium eBPF-based service mesh avoids sidecars entirely by implementing L7 policy in the kernel.

📚 **Official docs**

- Istio documentation

Q31 **A microservice is intermittently returning errors — walk through your systematic troubleshooting approach.** Troubleshooting

🎙 **Spoken answer · ~85 s**

For intermittent errors in a microservice I follow the signal methodically, not random command firing. First quantify: what percentage of requests fail, on which endpoints, and since when? Second correlate: with deploys, config changes, traffic spikes, resource saturation, or upstream and downstream incidents. Third isolate where in the call chain the failure originates, using distributed traces and per-hop metrics to see which service returns the first error. Then slice the failures by every dimension: pod, node, zone, client version, and request type, since intermittent often means one bad instance or path, for example one unhealthy pod still receiving traffic. Check logs of the failing pods around the error times, restart counts, readiness status, and resource limits for OOM kills or throttling. Check the network path: DNS, connection reuse, timeouts and retries, and conntrack. Check dependencies: database connection limits, rate limits, slow queries. Look for patterns in time, such as every five minutes pointing to a cron job, garbage collection, or a cache expiry. Once I have a hypothesis, test it safely, for example by removing the suspect pod from rotation. Fix the cause, and add the alert or test that would have caught it. Keep the team updated on findings and status during the investigation.

Likely follow-ups

- Why slice failures by pod and zone?
- How do you use time patterns to find a cause?

Intermittent errors require a methodical approach — not random command firing. Follow the signal, not the noise.

```
1. Quantify: what % of requests are failing? Which endpoints? Since when?
```

certs.eknathalabs.com

```
2. Correlate: CPU spikes? Deploy events? Upstream changes?
```

```
3. Isolate: which component in the call chain is failing?
```

```
kubectl logs <pod> -f --since=5m | grep -iE "error|warn|timeout"
```

```
kubectl top pods -n <ns>                  # resource pressure?
```

```
kubectl get events -n <ns> --sort-by=.lastTimestamp
```

```
curl -w "%{http_code} %{time_total}
" -o /dev/null -s <url>  # measure latency
```

Intermittent often means: a subset of pods have an issue (check per-pod metrics), external dependency is flapping (DNS, database, third-party API), resource exhaustion under load (connection pool, file descriptors, memory), or a race condition triggered by specific request patterns. Distributed traces (Cloud Trace, Jaeger) show exactly which service in the call chain added the latency or returned the error.

The 5 whys for networking: Error → which service? → which dependency? → which resource? → which config? → which change? Each why narrows from symptom to root cause.

📚 Official docs

- Brendan Gregg: the USE Method
- Google SRE Book: Effective Troubleshooting

Q32 **High network latency is reported between two services — how do you diagnose it?** Troubleshooting

🎙 Spoken answer · ~80 s

To diagnose high latency between two services I first establish a baseline and then find where the time goes. ping gives round-trip time and loss, and mtr shows per-hop latency and loss over many cycles. For application-level breakdown, curl with timing variables separates DNS lookup, TCP connect, TLS handshake, time to first byte, and total, which immediately shows whether time is in name resolution, connection setup, or the server's processing. ss with TCP info for active connections shows round-trip time, retransmissions, and congestion window. If connection setup is slow, look at DNS, SYN retransmits, and TLS; if the first byte is slow, the server is slow, so use tracing to look inside; if transfer is slow, look at bandwidth, window sizes, and loss. Compare same-zone versus cross-zone or cross-region paths, since physical distance sets a floor on latency. Check for packet loss and retransmissions, which cause large tail latency through retransmission timeouts; look at CPU saturation or throttling on either end, queueing in proxies and sidecars, noisy neighbours, and connection pool waits. Use percentiles, not averages. Fixes include co-locating services, reusing connections, tuning timeouts, caching, and eliminating unnecessary hops, validated with before-and-after measurements.

Likely follow-ups

- How does curl timing separate DNS, connect, and server time?
- Why can a few retransmissions dominate tail latency?

certs.eknathalabs.com

```
ping -c 100 <host> | tail -5    # baseline RTT and packet loss
```

```
mtr --report <host>             # per-hop latency + loss (run for 60s)
```

```
curl -w "namelookup: %{time_namelookup}
connect: %{time_connect}
starttransfer: %{time_starttransfer}
total: %{time_total}
" -o /dev/null -s <url>
```

```
ss -tin dst <ip>                # TCP info for active connection: RTT, retransmits
```

```
iperf3 -c <host> -t 30          # throughput test
```

Latency sources: DNS resolution (check time_namelookup — should be <10ms within cluster), TCP handshake (round-trip to the server — baseline RTT), TLS handshake (1-2 RTT for TLS 1.2, 1 RTT for TLS 1.3), server processing time (time_starttransfer - time_connect), and response transfer time. Isolate which phase is slow using curl's timing breakdown.

In-cluster latency: Pod-to-pod latency within a zone should be <1ms. If you're seeing 50ms+, check: NetworkPolicies adding iptables overhead, service mesh sidecar chains, cross-zone traffic (costs money + adds latency), or CPU throttling on the destination pod causing slow processing.

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

Q33 **Packet loss is occurring in the network — how do you detect and locate it?** Troubleshooting

🎙 **Spoken answer · ~87 s**

To detect packet loss, I start with ping at volume to get a loss percentage, and mtr with many cycles to see per-hop loss, remembering the rule that loss starting at a hop and persisting through to the destination is real, while loss only at one middle hop is likely ICMP rate limiting. On the hosts, look at interface statistics for errors and drops, with ip minus s link and ethtool statistics for NIC-level drop counters, TCP retransmission counters from netstat or nstat, and softnet statistics for kernel backlog drops. TCP retransmits in ss indicate loss on a specific connection. Locating it: capture on both ends to see where packets disappear, check each segment: the NIC and driver, ring buffer overflows, the virtual switch or cloud network, firewalls and conntrack tables dropping, overloaded middleboxes, and congested links. Common causes: duplex mismatch or bad cables on physical networks, microbursts overflowing buffers, an overloaded firewall or NAT, MTU issues, cloud instance network limits being hit, which providers expose as allowance exceeded counters, and wireless interference. Fixes depend on the segment: replace hardware, increase ring buffers, adjust queue sizes, move to larger instances, spread traffic, or engage the provider with evidence. Monitor retransmit rates and drops continuously to catch it early.

Likely follow-ups

- How do you tell NIC-level drops from network-level loss?
- What are cloud instance network allowance counters?

cors.eknathalabs.com

```
ping -c 1000 <host> | grep "packet loss"  # detect packet loss
```

```
mtr --report --report-cycles 100 <host>   # per-hop loss (find the problematic hop)
```

```
netstat -s | grep -i "retransmit|error|drop"  # interface-level drops
```

```
ip -s link show eth0              # interface rx/tx errors and drops
```

```
ethtool -S eth0 | grep -i drop   # NIC-level drop counters
```

Packet loss sources: NIC buffer overflow (increase ring buffer with ethtool), kernel buffer overflow (increase net.core.rmem_max, wmem_max), CPU overload causing interrupt processing delays, network congestion (QoS / buffer bloat), or deliberate firewall drops. Distinguish random loss (congestion) from burst loss (buffer overflow) with mtr's loss pattern. In cloud VMs: packet loss is usually due to bandwidth throttling — VMs have per-VM network egress limits.

TCP vs UDP loss: TCP hides packet loss via retransmission — you see latency spikes, not errors. UDP exposes loss directly. If TCP looks fine but UDP shows loss, the network has loss that TCP is masking with retransmits. Check `netstat -s | grep retransmit` to catch TCP-hidden loss.

📚 **Official docs**

- GNU grep manual

Q34 **A service works from one location but not another — how do you troubleshoot path-dependent connectivity?** Troubleshooting

🎙 **Spoken answer · ~86 s**

When a service works from one location but not another, I look at what differs along the path. I run the same tests from both places: curl verbose to see where it fails, whether in DNS, connecting, TLS, or the response; dig from each to compare DNS answers, since split-horizon, geo-routing, or resolver caching may return different IPs; and traceroute, preferably TCP on the service port, to compare the network paths. Bypass DNS by connecting to a known IP with the right Host header, to separate name resolution from connectivity. Then the usual path-dependent causes: DNS returning different addresses, one of which is bad; firewall, security group, or allow-list differences, since the failing location's source IP may not be permitted; different egress NAT addresses; routing or peering differences, BGP changes; proxies configured in one location only; MTU differences, such as a VPN path; CDN or edge POP problems affecting one region; TLS inspection by a corporate middlebox; and clock or trust store differences. Capture traffic at the failing location to see whether packets leave and what returns. Once I find the difference, fix it at the right layer, and record the knowledge. Also build synthetic monitoring from multiple locations to detect this class of problem proactively.

Likely follow-ups

- Why bypass DNS when comparing locations?
- What could differ in the source IP between the two locations?

```
curl -v <url>                    # from both working and failing location
```

certs.eknathalabs.com

```
traceroute -T -p 443 <host>    # compare network paths from both locations
```

```
dig <host>                      # compare DNS resolution from both locations
```

```
curl -H "Host: <expected-host>" http://<ip>  # bypass DNS — test by IP
```

Path-dependent issues: DNS resolves to different IPs (split-horizon, geo-routing), firewall rules blocking a specific source IP/range, routing asymmetry (request goes one path, reply goes another — stateful firewall breaks this), or geographic load balancer routing to different backends with different configs. First confirm DNS gives the same IP from both locations. Then test by IP (bypassing DNS) to isolate DNS vs routing.

Cloud Connectivity Tests: For GCP, use Network Intelligence Center → Connectivity Tests to simulate the packet path from source to destination including firewall rule analysis. Far faster than manual traceroute analysis for cloud-to-cloud paths.

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

Q35 **How do you use tcpdump and Wireshark to debug network issues?** Troubleshooting

🎙 **Spoken answer · ~88 s**

tcpdump is the command-line capture tool, and Wireshark is the graphical analyser. With tcpdump I choose the interface, or any, and use BPF filters to limit the capture to what I care about: a host, a port, a protocol, or TCP flags, for example only reset packets, or DNS on port fifty-three. I use minus n to avoid name resolution, write to a pcap file for later analysis, and limit size and duration or use rotation so I don't fill the disk. Then I open the file in Wireshark, or use tshark on the server, for deeper analysis: follow a TCP stream, view the handshake, see retransmissions, duplicate ACKs, zero windows, and the timing between request and response, and decode protocols including DNS and HTTP, and TLS handshake details though not encrypted payloads. I capture on both client and server to localise where packets are lost or delayed. In Kubernetes, I use an ephemeral debug container or a sniffing plugin to capture within a pod's network namespace. Reading patterns: SYN without SYN-ACK means a block or drop; SYN followed by RST means nothing is listening; repeated retransmissions mean loss; long gaps mean server delay. Be mindful of privacy and sensitive data in captures, handle them as such, and delete them when done.

Likely follow-ups

- How do you capture traffic inside a Kubernetes pod?
- What does a zero-window message tell you?

```
tcpdump -i eth0 -n port 80                    # capture HTTP traffic
```

```
tcpdump -i any -n host <ip> and port 443      # filter by host and port
```

```
tcpdump -i eth0 -w /tmp/capture.pcap          # write to file for Wireshark
```

certs.eknathalabs.com

```
tcpdump -i eth0 'tcp[tcpflags] & tcp-rst != 0'    # capture RST packets only
```

```
tcpdump -i any port 53 -n                         # capture DNS queries
```

```
kubectl sniff <pod> -n <ns> -f "port 80" -o cap.pcap  # capture from K8s pod
```

tcpdump is the primary packet capture tool for Linux. Key filters: `host <ip>` (either direction), `src/dst host <ip>` (directional), `port <n>`, `proto tcp/udp`. RST packets indicate rejected connections. Missing ACKs indicate loss. Retransmissions indicate loss or congestion. Load the pcap in Wireshark (Statistics → TCP Stream Graph for visual latency analysis) or use `tshark` for CLI analysis.

Kubernetes capture: Use `kubectl-sniff` plugin or `kubectl debug` with a debug container sharing the pod's network namespace to capture traffic from running pods without restart. This is essential for debugging service mesh encrypted traffic at the pod's network interface (before Envoy encrypts it).

📚 **Official docs**

- pcap-filter(7)
- tcpdump manual
- Wireshark documentation

Q36 **How do you debug high CPU on a network device or high kernel networking overhead?** Troubleshooting

🎙 **Spoken answer · ~84 s**

High kernel networking overhead shows up as CPU spent in interrupt handling instead of in applications. In top with the per-CPU view, the si column, software interrupts, is the one to watch; mpstat per CPU shows whether interrupt load is concentrated on one core, which is the typical problem when a single NIC queue is serviced by one CPU. The softnet statistics file shows per-CPU packet processing, including drops when the backlog is full and time squeezes. sar for network devices shows throughput and packets per second, since packet rate, not bandwidth, is often what overwhelms a CPU. perf can show kernel networking functions consuming time. Causes: very high packet rates with small packets; a single-queue NIC or poor interrupt distribution; a lack of receive-side scaling; excessive iptables or conntrack processing; encapsulation overhead from overlays or tunnels; or a DDoS. Fixes: enable and tune multiple NIC queues with RSS and spread interrupts across cores, using IRQ affinity or irqbalance; enable offloads such as GRO, GSO, and TSO; use larger packets where possible, jumbo frames in controlled networks; reduce firewall rule counts or use nftables sets and eBPF; scale out; and in the cloud use enhanced networking instances. Verify with before-and-after packet-per-second tests.

Likely follow-ups

- Why is packets-per-second often more important than bandwidth for CPU load?
- What does IRQ affinity do?

```
top -1                        # per-CPU usage — check si (softirq) column
```

```
mpstat -P ALL 1 5             # per-CPU interrupt distribution
```

certs.eknathalabs.com

```
cat /proc/net/softnet_stat     # softirq drops per CPU
```

```
sar -n DEV 1 10               # network interface throughput stats
```

```
perf top -e net:*              # kernel networking events (requires perf)
```

High `si` (software interrupt) CPU usage indicates network interrupt processing. This happens when a single CPU core handles all NIC interrupts (IRQ affinity not configured). Fix: enable RSS (Receive Side Scaling) to distribute interrupts across CPUs, or configure IRQ affinity manually. iptables with many rules also causes high `si` — each packet traverses all rules. Migrating to IPVS mode in kube-proxy or Cilium eBPF eliminates most iptables overhead.

Kubernetes scale: At 1000+ services, iptables kube-proxy mode has O(n) rule complexity — every new Service adds rules every pod must traverse. IPVS mode uses hash tables (O(1) lookup). Cilium replaces iptables entirely with eBPF. At scale, the networking datapath matters significantly for host CPU usage.

📚 **Official docs**

- Linux kernel docs: networking scaling

Q37 **What is ICMP and how is it used for network diagnostics?** Fundamentals

🎙 **Spoken answer · ~84 s**

ICMP, the Internet Control Message Protocol, is the network layer's control and error-reporting protocol. Its main message types: echo request and reply, used by ping to check reachability and measure round-trip time; destination unreachable, with codes for network, host, protocol, or port unreachable and, importantly, fragmentation needed, which drives path MTU discovery; time exceeded, sent when a TTL expires, which is how traceroute works; and redirects. For diagnostics: ping tests reachability and latency; ping with a larger size and the do-not-fragment flag finds the path MTU, which is how I diagnose tunnel and VPN black holes; traceroute and mtr use TTL-expired responses to map the path. Cautions: many networks filter ICMP, so a failed ping doesn't prove a host is down, and conversely a successful ping doesn't prove the service works, so test the actual TCP port. Blocking all ICMP is a mistake, because the fragmentation-needed messages are essential for path MTU discovery, and blocking them causes hangs on large transfers. Security groups and firewalls should allow the necessary ICMP types. ICMP can also be abused for floods and tunnelling, so rate limit it instead of removing it entirely. In IPv6, ICMPv6 is even more critical since it carries neighbour discovery.

Likely follow-ups

- Why must fragmentation-needed ICMP messages be allowed?
- Why doesn't a failed ping prove a host is down?

```
ping -c 5 <host>              # ICMP echo request/reply — reachability + RTT
```

```
ping -s 1400 <host>           # test with larger packet (detect MTU issues)
```

```
ping -M do -s 1472 <host>     # don't fragment — MTU path discovery
```

certs.eknathalabs.com

ICMP types: echo/echo-reply (ping), destination unreachable (port/host unreachable), time exceeded (TTL expired — used by traceroute), redirect (routing suggestion). Ping tests L3 reachability only — a successful ping doesn't mean the application port is open. "ICMP blocked but TCP works" is common in cloud environments where ICMP is filtered. Never conclude "host is down" from ping failure alone in cloud environments.

MTU and ICMP: If large pings fail but small ones succeed, you have an MTU (Maximum Transmission Unit) mismatch. This causes TCP sessions to hang after the initial handshake (which uses small packets) but fail for data transfers (which use large packets). Fix: reduce MTU on the interface or enable path MTU discovery.

📚 **Official docs**

- RFC 792: ICMP

Q38 **What is MTU and why do MTU mismatches cause mysterious connectivity issues?** Fundamentals

🎙 **Spoken answer · ~74 s**

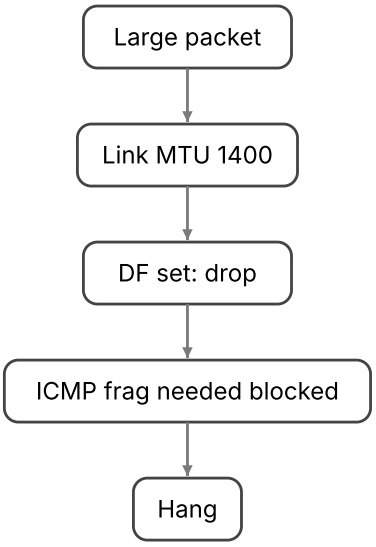
MTU is the largest packet a link can carry without fragmentation, typically 1500 bytes on Ethernet. Mismatches cause mysterious problems because small packets work while large ones fail. The classic symptoms are that ping works, a TCP connection establishes, but the TLS handshake hangs or large transfers stall, since larger packets are the first to exceed the path's MTU. Normally Path MTU Discovery handles it: a router that can't forward a big packet with the don't-fragment bit set sends back an ICMP fragmentation-needed message, and the sender reduces its packet size. But if a firewall blocks ICMP, the sender never learns, and you get a black hole. Overlay networks make it worse: VXLAN or IPsec tunnels add headers, so the effective MTU inside is lower, around 1450 for VXLAN, and a pod network configured at 1500 will have trouble. To diagnose, I ping with the don't-fragment flag at varying payload sizes to find the real path MTU. Fixes are consistent MTU on the interfaces and CNI, allowing the relevant ICMP messages, or clamping TCP MSS on the gateway.

Likely follow-ups

- Why does the TLS handshake specifically hang?
- How does MSS clamping work?

certs.eknathalabs.com

🗺 **MTU mismatch**




```
ip link show eth0 | grep mtu   # current MTU (default 1500 for Ethernet)
```

```
ping -M do -s 1472 <host>     # test with 1472 bytes + 28 bytes header = 1500 MTU
```

```
ping -M do -s 1400 <host>     # test with smaller packet (try if 1472 fails)
```

```
ip link set eth0 mtu 1450      # reduce MTU (VPN/overlay use case)
```

MTU is the maximum packet size a network can carry. Standard Ethernet MTU is 1500 bytes. VPNs, VXLAN overlays, and GRE tunnels add encapsulation headers (50-100 bytes), reducing effective MTU to 1400-1450 bytes. If a large packet exceeds the MTU and the DF (Don't Fragment) bit is set, a router sends ICMP "fragmentation needed" back. If ICMP is blocked (common in cloud), this message never arrives — TCP stalls mysteriously. Fix: reduce MTU on the source, or ensure ICMP type 3 (destination unreachable / fragmentation needed) is not blocked.

Kubernetes + MTU: CNI plugins often set pod MTU to 1450 to account for VXLAN overhead. If MTU is misconfigured (pod MTU set to 1500 but VXLAN adds 50 bytes), large pod-to-pod transfers silently fail. Always verify CNI MTU config matches the underlying network MTU minus overlay overhead.

📚 **Official docs**

- RFC 1191: Path MTU discovery
- Cloudflare Learning: What is MTU?

certs.eknathalabs.com

Q39 **What is DNS negative caching and how can it cause production incidents?** DNS

🎙 Spoken answer · ~87 s

Negative caching means resolvers cache the fact that a name doesn't exist, an NXDOMAIN response, for a period set by the SOA record's minimum value. That can cause incidents: if a service looks up a record before it has been created, say during a deployment ordering mistake or when a new service name is queried before the DNS record is added, the resolver caches the failure; then even after the record is created, clients keep getting NXDOMAIN until the negative TTL expires, so the new service seems unreachable for minutes or hours. It can also affect Kubernetes, where CoreDNS has its own cache settings and applications may cache failures. Diagnosis: dig for the name shows NXDOMAIN and the SOA record's TTL; querying the authoritative server directly shows the record exists while a cached resolver doesn't see it. Mitigation: flush the resolver caches affected, or wait; and prevent by creating DNS records before anything queries them, ordering deployment steps, setting a low SOA minimum on zones that change often, and configuring sensible negative TTLs in CoreDNS and applications. Also, applications shouldn't cache lookup failures for long, and retry logic should have backoff. It's one of those problems where DNS looks correct from the authoritative side yet clients still fail.

Likely follow-ups

- How do you prove the record exists but a cache is stale?
- What setting controls the negative TTL?

```
dig <nonexistent.domain>          # returns NXDOMAIN with SOA TTL
```

```
dig SOA <domain> | grep MINIMUM  # SOA minimum = negative cache TTL
```

```
# In Kubernetes CoreDNS: check negative_ttl in Corefile
```

```
kubectl describe cm coredns -n kube-system | grep cache
```

Negative caching: resolvers cache NXDOMAIN responses for the SOA minimum TTL. If a new DNS record is created but old caches hold the NXDOMAIN, clients continue to get "not found" for up to the negative TTL duration. Scenario: you create a new Service, its DNS propagates, but pods that recently got NXDOMAIN have it cached and won't retry for 30s (CoreDNS default negative TTL). Fix: reduce negative TTL in CoreDNS, or restart affected pods to flush their local resolver cache.

Application DNS caching: JVM caches DNS by default (ttl = -1 in older JVMs = cache forever). A JVM app that resolved a service IP will keep using that IP even after DNS changes. Set `networkaddress.cache.ttl=30` in JVM security properties. This catches many teams off-guard during rolling deployments.

📚 Official docs

- RFC 2308: DNS negative caching

certs.eknathalabs.com

Q40 **What are socket buffers and how does buffer bloat affect performance?** TCP/UDP

🎙 Spoken answer · ~83 s

Socket buffers are kernel memory used to hold data waiting to be sent or received on a connection. The receive buffer holds data that has arrived but the application hasn't read yet, and the send buffer holds data waiting for transmission or acknowledgement. Their size limits TCP throughput: the amount of data in flight can't exceed the window, which is bounded by buffer size, so on high-bandwidth, high-latency paths, the bandwidth-delay product determines the buffers needed. Too small, and throughput is limited; Linux auto-tunes within the minimum, default, and maximum values in tcp_rmem and tcp_wmem, bounded by the core maximums, which I raise for long fat networks. Too large can cause problems: memory consumption across many connections, and bufferbloat, where excessive buffering in network devices or hosts creates high latency under load, since packets wait in long queues; loss-based congestion control doesn't back off until buffers fill. Mitigations: active queue management such as fq_codel or cake, BBR congestion control, and right-sizing buffers. I check actual usage with ss showing memory, and the relevant sysctls. Symptoms to recognise: low throughput on high-latency links despite free bandwidth indicates small windows; high latency under load while bandwidth is saturated indicates bufferbloat.

Likely follow-ups

- What is the bandwidth-delay product?
- How does bufferbloat harm latency?

```
cat /proc/sys/net/core/rmem_default   # default recv buffer (bytes)
```

```
cat /proc/sys/net/core/rmem_max       # max recv buffer
```

```
cat /proc/sys/net/ipv4/tcp_rmem       # TCP recv buffer: min default max
```

```
sysctl -w net.core.rmem_max=134217728 # tune for high-bandwidth, high-latency links
```

```
ss -tm   # show actual buffer usage per socket
```

Send/receive buffers hold data in-flight. Too small: throughput is limited (can't fill the pipe on high-latency links). Too large: buffer bloat — data queues up in buffers, increasing latency without improving throughput. The bandwidth-delay product (BDP) formula: optimal buffer = bandwidth × RTT. For a 10Gbps link with 100ms RTT, BDP = 125MB — default Linux buffers (256KB) are too small. Increase for cross-region cloud transfers.

Kubernetes pod file descriptors: Each TCP connection consumes a file descriptor. Default `ulimit -n` is often 1024 or 65536. High-connection pods (proxies, databases) can exhaust this limit, causing "too many open files" errors. Set `ulimit -n 1000000` in container startup or adjust via pod securityContext.

📚 Official docs

- Linux man-pages: socket(7)
- PostgreSQL: Routine vacuuming

certs.eknathalabs.com

Q41 **What is ECMP (Equal-Cost Multi-Path) routing — how does it affect cloud networking?**

Routing & subnets

🎙 Spoken answer · ~87 s

ECMP, Equal-Cost Multi-Path routing, lets a router use several routes of equal cost to the same destination at once, distributing traffic across the paths instead of picking just one. Distribution is typically by hashing the flow, using source and destination IP, ports, and protocol, so all packets of one connection take the same path, avoiding reordering, while different connections spread across links. Benefits: more aggregate bandwidth, and automatic failover when a path fails, as the hash is redistributed across remaining paths. In cloud networking it appears in several places: multiple VPN tunnels or Direct Connect links advertising the same prefixes via BGP, transit gateways spreading traffic over multiple VPN connections to scale throughput past a single tunnel's limit, and the internal fabrics of cloud and data-centre networks, which use leaf-spine designs with ECMP. Caveats: a single flow is limited to a single path's bandwidth, so one big transfer doesn't benefit; hash collisions can cause uneven load, with elephant flows overloading a link; path asymmetry can confuse stateful firewalls and NAT if return traffic takes a different path; and changing the number of paths can reshuffle flows and reset connections, which resilient hashing mitigates. I verify with the routing table showing multiple next hops and test with multiple parallel flows.

Likely follow-ups

- Why does ECMP hash by flow rather than per packet?
- Why might a single large transfer not benefit from ECMP?

```
ECMP: multiple equal-cost routes to same destination — packets distributed across paths
```

```
ip route show | grep "nexthop"  # show multipath routes
```

```
# GCP: automatically uses ECMP for multi-NIC instances and multi-gateway VPNs
```

ECMP distributes traffic across multiple paths using a hash of {src IP, dst IP, src port, dst port, protocol}. Benefits: higher aggregate bandwidth, redundancy (if one path fails, hash rebalances). Gotcha: TCP connections are per-flow sticky (same 5-tuple always takes same path), so ECMP doesn't help for a single large TCP connection (use multipath TCP for that). In Kubernetes: kube-proxy and IPVS use ECMP-like per-connection hashing for Service load balancing.

GCP VPN ECMP: If you have two Cloud VPN tunnels to the same destination, GCP automatically does ECMP — traffic is load-balanced across both tunnels. You get ~2× the bandwidth of a single VPN tunnel. Use Active/Active VPN for throughput; Active/Passive for simple failover.

📚 Official docs

- Cloudflare Learning: What is a VPN?

certs.eknathalabs.com

Q42 **What is a CDN — how does it work and when should you use it?** HTTP/TLS

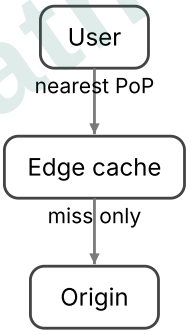
🎙 Spoken answer · ~88 s

A CDN, content delivery network, caches content at edge points of presence around the world and serves users from the nearest location, reducing latency, offloading the origin, and absorbing traffic spikes and some attacks. It works by routing the user, via DNS or anycast, to a nearby edge; on a cache hit the edge serves directly, on a miss it fetches from the origin and caches according to headers. Cache behaviour is controlled by Cache-Control headers, with max-age and public to allow caching, surrogate-control for CDN-specific TTLs, and the Vary header to cache variants, for example per encoding; cache keys can include query strings, headers, or cookies. When to use: static assets like images, scripts, and video, definitely; also cacheable API responses, and as a protective layer with TLS termination, DDoS mitigation, and WAF for dynamic traffic. Pitfalls: stale content requiring invalidation or versioned filenames; accidentally caching personalised or private content because of missing headers, which is a security risk; low cache hit ratio from poor cache keys; origin shielding to avoid thundering herds at the origin; and debugging that requires looking at CDN headers and logs. Measure the hit ratio, origin load, and latency by region, and secure the origin so that traffic can only come through the CDN.

Likely follow-ups

- How do you avoid caching personalised responses by mistake?
- What is origin shielding?

🗺 CDN




```
CDN: Cache static content at edge PoPs worldwide — serve from nearest location
```

```
Cache-Control: max-age=86400, public   # HTTP header to make content CDN-cacheable
```

```
Surrogate-Control: max-age=3600        # CDN-specific cache TTL (overrides Cache-Control)
```

```
Vary: Accept-Encoding                  # cache different variants per encoding
```

```
gcloud compute backend-buckets update <name> --enable-cdn  # enable Cloud CDN
```

CDN reduces latency (serve from edge PoP near user instead of origin datacenter), reduces origin bandwidth and load, and improves availability (CDN serves cached content even if origin is down). Use for: static assets (JS, CSS, images), cacheable API responses, video streaming. Don't use for: authenticated content (unless varies correctly), highly personalized responses, real-time data. GCP Cloud CDN integrates with Cloud Storage and HTTP(S) Load Balancer — enable with one flag.

certs.eknathalabs.com

Cache key: CDN caches by URL + selected headers (Vary). A cache miss goes to origin; a cache hit serves directly from edge. Cache hit rate is the key metric. Low hit rate means content is either not cacheable, has too-short TTL, or the cache key is too specific (vary on too many headers).

📚 **Official docs**

- Cloudflare Learning: What is a CDN?

Q43 **What is eBPF — how is it changing cloud-native networking?** Cloud networking

🎙 **Spoken answer · ~85 s**

eBPF, extended Berkeley Packet Filter, lets you run small, sandboxed programs inside the Linux kernel, verified for safety and JIT-compiled, without changing kernel source or loading modules. Programs attach to hooks such as network interfaces, system calls, and tracepoints, giving fast, programmable, low-overhead access to packets and events. It's changing cloud-native networking in several ways. For networking itself, Cilium uses eBPF to implement pod networking, service load balancing, and network policy, replacing iptables, which scales poorly because rules are evaluated linearly and updating large rule sets is slow; eBPF uses hash maps for constant-time lookups, so it handles thousands of services better, and it can bypass parts of the network stack for lower latency. For observability, eBPF gives deep visibility into connections, latency, DNS, and HTTP without sidecars or code changes, which tools like Hubble, Pixie, and Parca use. For security, it enables runtime enforcement and detection of suspicious syscalls and network behaviour, as in Tetragon and Falco. It also enables service mesh functionality without sidecars. Caveats: it needs relatively modern kernels, the tooling is complex, debugging eBPF programs requires specialised skills, and privileged access is needed. I inspect with bpftool, and adopt it where the scale, observability, or performance benefits justify it.

Likely follow-ups

- Why does eBPF scale better than iptables for services?
- What can eBPF observability do without sidecars?

```
eBPF: extended Berkeley Packet Filter — run sandboxed programs in Linux kernel
```

```
bpftool prog list    # list loaded eBPF programs
```

```
bpftool net show     # eBPF programs attached to network interfaces
```

```
# Cilium uses eBPF to replace iptables entirely:
```

```
cilium status        # cilium eBPF datapath status
```

eBPF allows safe, JIT-compiled programs to run in the kernel without kernel modules. In networking: Cilium uses eBPF to implement Kubernetes Services, NetworkPolicies, and L7 observability without iptables — dramatically lower CPU overhead and O(1) rule lookup at any scale. Pixie uses eBPF for zero-instrumentation observability (automatic tracing without code changes). Falco uses eBPF for runtime security. This is the future of cloud-native networking infrastructure.

certs.eknathalabs.com

Principal-level signal: Mentioning eBPF awareness — even conceptually — signals you follow infrastructure trends. Saying "we use Cilium in our cluster instead of kube-proxy for better performance at scale" is the kind of Principal-level architectural decision that differentiates candidates.

📚 **Official docs**

- Linux kernel docs: BPF
- ebpf.io: What is eBPF?
- Cilium: Introduction

Q44 **What is Zero Trust Networking — how do you implement it in practice?** Network security

🎙 **Spoken answer · ~87 s**

Zero trust means never trust, always verify: no implicit trust based on network location, so every request is authenticated, authorised, and encrypted regardless of where it comes from. The traditional model trusted everything inside the perimeter, so one compromised host could move laterally. In practice I implement it in layers. Identity: strong authentication for users with SSO and MFA, and for workloads using cryptographic identities such as SPIFFE or cloud workload identity, instead of IP addresses. Encryption: mTLS between services, often via a service mesh, so both sides prove who they are. Authorisation: least-privilege policies that say which identity may call which service and method, enforced at the proxy or application, and default-deny network policies in Kubernetes so pods can only talk to what they need. Device and context: check device posture and request context for access to internal tools through an identity-aware proxy instead of a VPN that grants broad network access. Segmentation: micro-segmentation to limit blast radius. Visibility: log and monitor all access decisions, flow logs, and anomalies. Rollout is gradual: start with inventory and visibility, then default-deny in audit mode, then enforce service by service. It isn't a product but a strategy, and the trade-offs are operational complexity, certificate management, and the effort of defining policies.

Likely follow-ups

- How does zero trust change the role of the VPN?
- How would you roll out default-deny network policies safely?

Zero Trust: never trust, always verify. No implicit trust based on network location. Every request is authenticated and authorized regardless of where it comes from.

```
Traditional: trust inside the perimeter, distrust outside
```

```
Zero Trust: authenticate everything, encrypt everything, least privilege everywhere
```

```
# Kubernetes zero trust layers:
```

```
1. mTLS (Istio) — every service-to-service call is mutually authenticated
```

```
2. NetworkPolicies — restrict which pods can talk to which
```

```
3. RBAC — restrict which services can call which APIs
```

certs.eknathalabs.com

```
4. Service accounts with minimal permissions — no broad access
```

GCP BeyondCorp Enterprise implements zero trust for user access — users authenticate via identity-aware proxy (IAP) using device context + identity, no VPN needed. For Kubernetes: service mesh mTLS + NetworkPolicies + OPA/Kyverno policies = zero trust within the cluster. The perimeter assumption is obsolete — assume breach and design for lateral movement resistance.

Interview relevance: Security-focused employers will appreciate zero trust framing. Every answer about network segmentation, access control, or service-to-service communication should reference zero trust principles — it aligns with their product philosophy and shows security-first platform thinking.

📚 **Official docs**

- NIST Zero Trust Architecture (SP 800-207)

Q45 **Walk through a complete methodology for troubleshooting "the application is slow" in a cloud-native environment.** Troubleshooting

🎙 **Spoken answer · ~85 s**

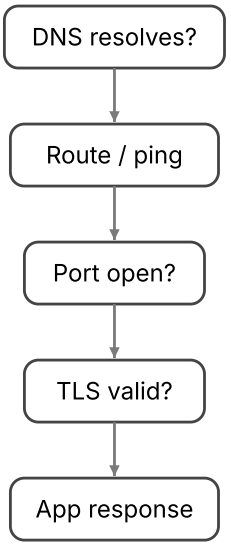
My methodology for the application is slow in a cloud-native environment has phases. Observe: quantify slow. Which percentile, P50 or P99? Which endpoints? Compared with what baseline? What is the error rate? Scope: is it all users or some, one region, one service, one tenant, and did it start after a deploy or change? Isolate: use distributed tracing to find which hop or span owns the time, and per-service golden signals, latency, traffic, errors, and saturation, to see which component is degraded. Drill into the layer: for the network, DNS, connection setup, TLS, retransmits; for compute, CPU throttling, memory pressure, garbage collection, thread and connection pools; for storage and databases, slow queries, locks, IOPS limits; for dependencies, third-party latency and rate limits; for Kubernetes, pod scheduling, restarts, noisy neighbours, HPA lag, and node pressure. Hypothesise and test one thing at a time, using the least invasive checks. Mitigate if possible while investigating: roll back a recent change, scale out, shed load, or fail over. Verify the fix against the original metric, not just that something changed. Communicate throughout, and finish with a postmortem and prevention: SLOs, better instrumentation, and load tests. The point is structure and evidence, not a list of random commands.

Likely follow-ups

- What do you check first when only P99 is bad?
- How do traces help isolate the slow hop?

certs.eknathalabs.com

🗺 **Troubleshooting ladder**



This is the Principal-level systems thinking question. The interviewer wants to see structure, not a list of random commands.

STRUCTURED METHODOLOGY

```
Phase 1 — Observe: quantify "slow" (P50/P99 latency? error rate? which endpoints?)
```

```
Phase 2 — Scope: is it all users? One region? One service? After which deploy?
```

```
Phase 3 — Isolate: L3 reachable? L4 port open? L7 response correct? Which hop is slow?
```

```
Phase 4 — Drill down: traces → logs → metrics → code
```

```
Phase 5 — Fix + verify: make one change, measure impact, confirm improvement
```

```
curl -w "dns:%{time_namelookup} tcp:%{time_connect} tls:%{time_appconnect} ttfb:%
{time_starttransfer} total:%{time_total}
" -o /dev/null -s <url>
```

```
kubectl top pods -n <ns> --sort-by=cpu   # resource saturation?
```

```
gcloud trace list --project=<project>   # distributed traces for call chain latency
```

The breakdown: DNS lookup slow → resolver issue. TCP connect slow → routing, firewall, server overload. TLS slow → cert negotiation, OCSP. TTFB (Time To First Byte) slow → server processing, database queries, upstream dependencies. Transfer slow → throughput/MTU issue. Each phase pinpoints the layer. Escalate with data: "P99 latency is 2s, TTFB accounts for 1.8s, traced to the database query in service X taking 1.6s on cache miss."

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

Platform Engineer framing: Don't just describe commands — describe how you'd present findings to the team, how you'd avoid recurrence (add this metric to dashboards, add SLO on this endpoint, add a test that catches this regression). That's the difference between

certs.eknathalabs.com

a Senior who fixes issues and a Principal who prevents them.

Q46 **What is MTU, MSS, and Path MTU Discovery — and how does an MTU mismatch cause a 'black hole'?** Fundamentals

🎙 Spoken answer · ~86 s

MTU is the largest payload a link can carry in one frame, typically fifteen hundred bytes on Ethernet. MSS is the largest TCP data segment, equal to the MTU minus the IP and TCP headers, usually fourteen hundred sixty. Path MTU Discovery finds the smallest MTU along the whole path: the sender sets the don't-fragment bit, and if a router can't forward a packet that large, it drops it and sends back an ICMP fragmentation-needed message, so the sender lowers its packet size. Tunnels, VPNs, and overlays like VXLAN add headers and reduce the effective MTU. A black hole occurs when the ICMP messages are blocked by a firewall: the sender never learns, keeps sending large packets that are silently dropped, while small packets and the handshake work. Symptoms: connections establish, small requests succeed, but large transfers or TLS handshakes with big certificates hang. I diagnose with ping using the do-not-fragment flag and varying sizes to find the working maximum, and packet captures showing retransmissions of large segments. Fixes: allow the needed ICMP, set a lower MTU on the tunnel or interface, clamp the MSS on the tunnel endpoint so TCP never sends oversized segments, and ensure consistent MTU along the path, including inside Kubernetes overlays.

Likely follow-ups

- How does MSS clamping avoid the black hole?
- How do you find the real path MTU with ping?

MTU is the largest L2 frame payload a link carries (typically 1500 bytes on Ethernet). MSS is the largest TCP *segment* the data payload can be. PMTUD finds the smallest MTU along a path.

THE RELATIONSHIP

```
MSS = MTU - IP header (20) - TCP header (20) = 1500 - 40 = 1460 (typical)
# Overlay/VPN/tunnels REDUCE effective MTU (VXLAN ~50 bytes, IPsec more)
```

THE PMTUD BLACK HOLE

When a packet exceeds a link's MTU and has the Don't-Fragment bit set, the router drops it and sends back an ICMP "fragmentation needed" (Type 3 Code 4). If a firewall **blocks that ICMP**, the sender never learns to shrink — large packets vanish silently. Small packets (handshake, pings) work; large transfers (a big POST, a TLS cert) hang. That asymmetry is the fingerprint.

```
ping -M do -s 1472 <host>     # 1472 + 28 = 1500; bump down until it passes
tracepath <host>               # shows PMTU along the path
ip link show eth0              # check the interface MTU
```

Production gotcha: Classic in overlay networks (Kubernetes CNI, VXLAN) and VPNs. Symptoms: SSH connects but hangs on output, TLS handshake stalls, some HTTP works and some doesn't. Fix: allow ICMP type 3, lower the MTU, or clamp MSS.

Interview tip: The senior fix is **MSS clamping** (`iptables ... --clamp-mss-to-pmtu` or router config) so TCP negotiates a safe segment size up front instead of relying on ICMP that firewalls often drop. Mentioning the "small works, large hangs" tell shows

certs.eknathalabs.com

you've debugged this for real.

📚 **Official docs**

- RFC 1191: Path MTU discovery
- Cloudflare Learning: What is MTU?

Q47 **Explain NAT, SNAT, DNAT, and PAT. Where does each happen in cloud networking?** Fundamentals

🎙 **Spoken answer · ~87 s**

NAT rewrites IP addresses, and often ports, as packets cross a boundary. SNAT, source NAT, rewrites the source address for outbound traffic from private to public. DNAT, destination NAT, rewrites the destination for inbound traffic, mapping a public address to a private backend, as load balancers and port forwarding do. PAT, port address translation, lets many private hosts share one public address by assigning different source ports; it's what most people mean by NAT. In cloud networking: a NAT gateway provides SNAT and PAT for private subnets to reach the internet outbound, without allowing inbound connections; an internet gateway performs one-to-one NAT between an instance's private and public or elastic address; load balancers and Kubernetes services perform DNAT to reach backends and pods; and kube-proxy or the CNI uses SNAT masquerade for pod egress. NAT is stateful, tracking translations, so the state table and port space can be exhausted, producing connection failures under heavy outbound traffic to one destination; the remote side sees the NAT address, so partners allow-list that IP and egress addresses should be stable; and logs at the destination show the NAT address, not the originating workload, so correlation needs NAT or flow logs. Understanding where each happens helps debug source IP and connectivity issues.

Likely follow-ups

- Why does a private subnet need a NAT gateway?
- How do you identify which workload used a NAT address?

NAT rewrites IP addresses (and ports) as packets cross a boundary. The variants differ by which side and what gets rewritten.

THE VARIANTS

```
SNAT (Source NAT)      : rewrite the SOURCE addr — outbound (private -> public)
DNAT (Destination NAT) : rewrite the DEST addr — inbound (public -> private backend)
PAT  (Port Address Tr.): many private IPs share one public IP via unique source ports
                         (a.k.a. "NAT overload" / masquerade)
```

WHERE IN THE CLOUD

```
NAT Gateway     : SNAT/PAT for private subnets reaching the internet (outbound)
Load balancer   : DNAT inbound traffic to backend pool members
Internet GW     : 1:1 NAT for public IPs (EIP <-> private IP)
K8s kube-proxy  : DNAT Service VIP -> pod IP; SNAT for node-to-pod return paths
```

Gotcha: SNAT/PAT hides the original client source IP from the backend — you see the NAT/LB IP instead. Preserve the real client IP via X-Forwarded-For (L7), PROXY protocol (L4), or `externalTrafficPolicy: Local` in Kubernetes.

certs.eknathalabs.com

Interview tip: A cloud NAT Gateway does SNAT+PAT so many private instances egress through a few public IPs — and at scale you can exhaust **source ports per destination** (~64k per dest IP:port tuple), causing connection failures. Knowing the source-IP-preservation problem and SNAT port exhaustion signals real cloud-network experience.

📚 **Official docs**

- RFC 3022: Traditional NAT

Q48 **How does ARP work, and what problems do gratuitous ARP and ARP caching cause in production?** Fundamentals

🎙 **Spoken answer · ~86 s**

ARP maps an IP address to a MAC address within a layer-two broadcast domain, the glue between layer-three routing and layer-two delivery. A host wanting to send to an address on its subnet broadcasts who has this IP; the owner replies with its MAC; the sender caches the mapping in its ARP table with a timeout. Off-subnet traffic goes via ARP for the gateway. Gratuitous ARP is an unsolicited announcement that an IP is now at a given MAC, used when a failover moves a virtual IP to a standby, so neighbours update their caches immediately instead of waiting for entries to expire. Problems: ARP caching means stale entries after a failover or IP reassignment send traffic to the old MAC until expiry, causing a short outage unless a gratuitous ARP is sent; duplicate IPs cause intermittent connectivity as two hosts answer; ARP spoofing lets an attacker redirect traffic, mitigated by dynamic ARP inspection; and large flat layer-two networks generate ARP broadcast storms and table pressure, which is why networks are segmented. In the cloud, the virtual network handles ARP, and floating IP movement works through API calls instead of gratuitous ARP. I inspect with ip neigh, probe with arping, and flush a suspect entry carefully.

Likely follow-ups

- Why does a failover send a gratuitous ARP?
- How does ARP caching cause a short outage after a VIP moves?

ARP maps an IP to a MAC address within a layer-2 broadcast domain — the glue between L3 routing and L2 delivery.

THE FLOW

```
Host wants to send to 10.0.0.5 on its subnet:
1. Broadcast: "who has 10.0.0.5? tell 10.0.0.1"  (ARP request)
2. 10.0.0.5 replies with its MAC                 (ARP reply)
3. Sender caches IP->MAC in its ARP table (with a timeout)
ip neigh show          # view the ARP/neighbor cache
```

GRATUITOUS ARP (FAILOVER)

An unsolicited ARP announcing "this IP is now at MY MAC" — used in VIP failover (keepalived, VRRP, cloud floating IPs) so switches and peers update their tables and traffic moves to the new active node.

Production problems: Stale ARP caches after a failover (peers keep sending to the dead node's MAC until the entry ages out) cause a brief outage. ARP spoofing is a security risk (attacker claims a gateway's IP). Duplicate IPs cause ARP flapping. On large flat L2 segments, ARP broadcast traffic itself becomes noise.

certs.eknathalabs.com

> **Interview tip:** Tie gratuitous ARP to VIP failover — it's how keepalived/VRRP redirects a floating IP. The classic incident is a failover where traffic doesn't move because a peer or switch cached the old MAC; gratuitous ARP (or a lower ARP timeout) is what fixes the convergence. Cloud platforms abstract this with API-driven IP reassignment instead of L2 ARP.

📚 **Official docs**

- RFC 826: ARP

Q49 **Map L2/L3/L7 concepts to cloud load balancers. What operates at each layer and why does it matter?** Fundamentals

🎙 **Spoken answer · ~85 s**

Cloud networking maps onto OSI layers, and the layer of a component tells you what it can and can't see. Layer two, frames and MAC addresses: VLANs and the switching fabric under the VPC, which the cloud mostly abstracts. Layer three, IP and routing: VPC route tables, subnets, NAT gateways, internet gateways, transit gateways, and network ACLs and security groups matching on IP. Layer four, TCP and UDP: network load balancers, like AWS NLB, kube-proxy, and security group port rules; they see addresses and ports but not content. Layer seven, application: application load balancers, ingress controllers, API gateways, WAFs, and service meshes, which parse HTTP and can route by host, path, header, or cookie, terminate TLS, authenticate, and retry. It matters because it determines capability and failure modes. An L4 balancer can't route by URL and preserves the client IP; an L7 balancer can, but needs headers to convey the client IP and adds processing latency. A WAF only sees what an L7 component exposes, encrypted traffic passing through at L4 is invisible, and a health check at L4 only verifies the port while L7 can verify an actual endpoint. When troubleshooting, naming the layer narrows the search and tells me which tools apply.

Likely follow-ups

- What can an L4 load balancer not do that an L7 can?
- Why does the layer of a health check matter?

Cloud networking maps directly onto OSI layers; knowing the layer of each component tells you what it can and can't see.

THE MAPPING

```
L2 (frames/MAC)  : VLANs, the VPC's underlying switching (mostly abstracted in cloud)
L3 (IP/routing)  : VPC route tables, subnets, NAT GW, Network LB (NLB) — IP-level
L4 (TCP/UDP)     : Network/L4 LB — balances by IP:port, no app awareness, fast
L7 (HTTP/app)    : Application LB (ALB) — sees URLs, headers, cookies, host
```

WHY THE LAYER MATTERS

```
L4 LB : ultra-fast, protocol-agnostic, preserves source IP easily, can't route by path
        -> use for TCP/UDP, gRPC raw, extreme throughput, static IP needs
L7 LB : path/host routing, TLS termination, header manipulation, WAF integration
        -> use for HTTP(S) microservices, content-based routing
```

> **Gotcha:** An L4 LB can't do path-based routing or read headers — it only sees IP:port. An L7 LB terminates the connection (two TCP sessions: client<->LB and LB<->backend), which changes source IP visibility and TLS handling. Pick the layer to match what you need to route on.

certs.eknathalabs.com

Interview tip: The crisp framing: choose the lowest layer that meets your routing needs. L4 for raw speed/protocol-agnostic/source-IP preservation; L7 when you must route on HTTP attributes or terminate TLS centrally. Naming AWS NLB (L4) vs ALB (L7), or GCP's equivalents, grounds it in real cloud products.

📚 **Official docs**

- Cloudflare Learning: Load balancing
- Cloud Load Balancing

Q50 **Compare IPv4 vs IPv6 in practice. What is dual-stack and the Happy Eyeballs algorithm?** Fundamentals

🎙 **Spoken answer · ~82 s**

IPv6 solves IPv4 address exhaustion with a 128-bit address space, so every device can have a globally routable address and NAT isn't needed. Practical differences: IPv4 has 32-bit addresses, about four billion, and relies on NAT and ARP; IPv6 uses neighbour discovery instead of ARP, supports SLAAC autoconfiguration, link-local addresses on every interface, and has a simpler header, with no router fragmentation. ICMPv6 is essential and must not be blocked. Adoption is gradual, so most networks run dual-stack: hosts have both IPv4 and IPv6 addresses and use whichever works. The Happy Eyeballs algorithm makes dual-stack robust: the client starts connecting over IPv6 and, after a very short delay of a few hundred milliseconds, also tries IPv4, using whichever connection succeeds first, so a broken IPv6 path doesn't cause long delays. Operational points: firewalls, security groups, and monitoring must be configured for both protocols, since forgotten IPv6 rules can expose services; DNS needs AAAA records, and a wrong AAAA record causes failures that only some clients see; applications must handle IPv6 literals in logs and allow-lists; and IPv6-only with NAT64 and DNS64 is used for large environments such as Kubernetes clusters that run out of IPv4 addresses.

Likely follow-ups

- What does Happy Eyeballs protect users from?
- Why is blocking ICMPv6 harmful?

IPv6 solves IPv4 address exhaustion with a vastly larger space and removes the need for NAT, but adoption is gradual via dual-stack.

KEY PRACTICAL DIFFERENCES

```
IPv4 : 32-bit (~4.3B addrs), needs NAT, ARP for L2 resolution
IPv6 : 128-bit (huge), no NAT needed (global addrs), uses NDP not ARP,
       SLAAC autoconfig, link-local addresses (fe80::/10)
```

DUAL-STACK + HAPPY EYEBALLS

Dual-stack hosts have both IPv4 and IPv6. **Happy Eyeballs** (RFC 8305) is the client algorithm that races both: it fires the AAAA (IPv6) and A (IPv4) lookups, attempts connections in parallel with a slight head start for IPv6, and uses whichever connects first — so a broken IPv6 path doesn't stall the user.

```
# Without Happy Eyeballs, a broken IPv6 route = long timeout before IPv4 fallback.
# With it, the client falls back near-instantly -> resilient UX during v6 issues.
```

certs.eknathalabs.com

Gotcha: Misconfigured IPv6 (AAAA record exists but the path is broken) causes intermittent slowness on dual-stack clients that *lack* Happy Eyeballs, or where the fallback timer is long. Test both stacks; don't publish a AAAA record until the v6 path actually works end to end.

Interview tip: Happy Eyeballs is the detail that signals depth — it's why "my IPv6 is broken but the internet still works." In cloud, dual-stack VPCs help with IPv4 exhaustion (a real cost/scarcity issue); mention that IPv6 eliminates NAT, simplifying some architectures while complicating firewall rules.

📚 **Official docs**

- RFC 3022: Traditional NAT

Q51

What is connection tracking (conntrack) and how does table exhaustion take down a busy host?

Fundamentals

🎙 **Spoken answer · ~81 s**

Conntrack is the kernel's table of every connection passing through a host that does stateful filtering or NAT, so return traffic can be matched and NAT mappings kept consistent. Each entry records the flow's tuple, protocol, and state. It's finite: the table size is set by nf_conntrack_max, and when it fills up, the kernel logs table full, dropping packet, and new connections fail while existing ones continue. That takes down busy hosts such as NAT gateways, Kubernetes nodes running kube-proxy, load balancers, and API servers handling many short connections. I check the current count against the maximum, list entries with the conntrack tool, and look at timeouts. Causes: genuine high concurrency or churn, very long established-state timeouts keeping dead entries, UDP flows like DNS creating many short entries, connection leaks, and attacks. Fixes: raise the maximum and hash table size, remembering memory cost; lower timeouts, especially the established TCP timeout; reduce connection churn with keepalive and pooling; exempt traffic that needs no tracking with raw table NOTRACK rules; and spread load across more nodes. In Kubernetes, node-local DNS caching cuts DNS entries. Monitor the usage percentage and alert early, since symptoms look like random timeouts.

Likely follow-ups

- Why do short-lived UDP flows stress conntrack?
- How do you exempt traffic from connection tracking?

Conntrack is the kernel's stateful table of every connection passing through a NAT/firewall host, so return traffic is matched and NAT is consistent. It's finite.

WHY IT EXISTS

```
# For stateful firewalling + NAT, the kernel remembers each flow:
#   (src ip:port, dst ip:port, proto, state)
conntrack -L | wc -l                    # current entries
cat /proc/sys/net/netfilter/nf_conntrack_max     # the cap
cat /proc/sys/net/netfilter/nf_conntrack_count    # current count
```

certs.eknathalabs.com

EXHAUSTION FAILURE

When the table fills, the kernel drops new connections and logs `nf_conntrack: table full, dropping packet`. Symptoms: sudden connection failures/timeouts on a busy gateway, NAT instance, or Kubernetes node — even though CPU/mem look fine.

```
dmesg | grep conntrack                    # "table full" messages
# Mitigate: raise nf_conntrack_max, tune timeouts (esp. TIME_WAIT/UDP),
# or bypass conntrack (NOTRACK) for high-volume flows that don't need it
```

Production gotcha: A Kubernetes node doing lots of SNAT, or a NAT gateway under load, silently drops connections at the conntrack ceiling. UDP floods and short-lived connections fill it fast. The error is in `dmesg`, not your app logs — so it looks like a mysterious network blip.

Interview tip: This is a favorite "intermittent failures under load" root cause. The tells: failures correlate with connection volume, CPU is fine, and `dmesg` shows table-full. Fixes: raise the max, shorten timeouts, or use `NOTRACK`. eBPF datapaths (Cilium) can reduce conntrack pressure by handling some flows without it.

📚 **Official docs**

- Netfilter documentation

Q52 **Stateful vs stateless firewall — explain via AWS Security Groups vs Network ACLs.** Network security

🎙 **Spoken answer · ~85 s**

The stateful versus stateless distinction determines whether I must explicitly allow return traffic, and it's exactly the security group versus network ACL difference in AWS. Security groups are stateful: they track connections, so if I allow an outbound request, the response is automatically allowed, and I only write rules in the direction I initiate. They are allow-only, with no explicit deny, apply at the instance or network interface level, and can reference other security groups, which is powerful for tiered architectures. Network ACLs are stateless: they apply at the subnet level, evaluate each packet independently, process numbered rules in order, can include explicit allow and deny, and need rules for both directions including ephemeral port ranges for return traffic. Typical use: security groups as the primary, fine-grained control, and NACLs as a coarse extra layer, for example to block a malicious address range at the subnet boundary. Troubleshooting implications: if a connection works one way but replies are lost, check the NACL return rules; if a subnet-wide block is needed quickly, a NACL deny is the tool; and flow logs show which layer rejected traffic. Mistakes to avoid are forgetting ephemeral ports in NACLs, rule ordering errors, and over-permissive 0.0.0.0/0 rules in security groups.

Likely follow-ups

- Why do NACLs need ephemeral port rules?
- When would you use a NACL deny rule?

The stateful/stateless distinction determines whether you must explicitly allow return traffic — and it's exactly the SG-vs-NACL difference in AWS.

STATEFUL (SECURITY GROUPS)

certs.eknathalabs.com

```
- Tracks connections: if you ALLOW outbound, the RETURN traffic is auto-allowed
- Only need rules in the direction you initiate
- Allow-only (no explicit deny); evaluated as a whole
- Attached to ENIs/instances
```

STATELESS (NETWORK ACLS)

```
- No connection memory: must ALLOW both inbound AND the return path explicitly
- Must open ephemeral port range (1024-65535) for return traffic!
- Has explicit allow AND deny; numbered, evaluated in order
- Attached to subnets
```

Classic mistake: Adding a NACL inbound allow but forgetting the outbound ephemeral-port range for replies (or vice versa) — traffic goes one way and the response is dropped. With Security Groups you never hit this because the return is stateful. The #1 NACL bug is the missing ephemeral return range.

Interview tip: Crisp summary: Security Group = stateful (return traffic implicit), NACL = stateless (must allow return + ephemeral ports explicitly). SGs are allow-only at the instance; NACLs add deny rules at the subnet for broad blocks (e.g. blacklisting an IP). Use SGs for app rules, NACLs for coarse subnet-level guardrails.

📚 **Official docs**

- Amazon VPC: Security groups
- Amazon VPC: Network ACLs

Q53 **Walk through recursive DNS resolution end to end, from a cold cache to an answer.** DNS

🎙 **Spoken answer · ~83 s**

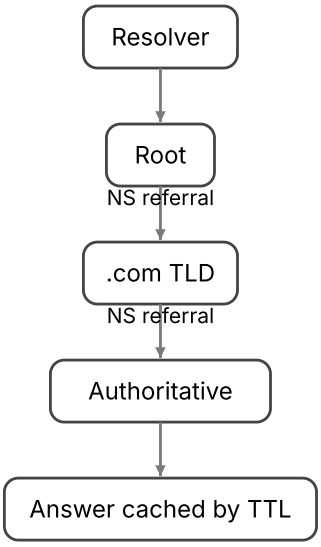
Recursive resolution is how a name becomes an IP when nothing is cached. Take www.example.com. First the client's stub resolver asks its recursive resolver, such as a corporate or public one. If it has no cache entry, the recursive resolver asks a root server, which doesn't know the answer but refers it to the servers for the dot-com TLD. It asks a TLD server about example.com, which refers it to the domain's authoritative nameservers, found from NS records and glue records where necessary to avoid circular lookups. Then it asks the authoritative server for www.example.com and gets the answer, possibly following a CNAME chain with further lookups. The resolver caches each step according to its TTL, including the NS delegations, so later queries skip the upper levels, and returns the answer to the client, which caches it too. Cached layers include the application, the OS, and the recursive resolver. Operational relevance: a cold cache query can take several round trips, which is why resolvers matter for latency; failures at any step show up as SERVFAIL or timeouts; lame delegations and broken NS records cause intermittent errors; and dig with trace performs this walk to show where it breaks.

Likely follow-ups

- What are glue records for?
- What does dig +trace show you?

certs.eknathalabs.com

🗺 **Cold-cache DNS**



Recursive resolution is how a name becomes an IP when nothing is cached, walking the DNS hierarchy from root to authoritative.

THE WALK (COLD CACHE, RESOLVING WWW.EXAMPLE.COM)

```
1. Stub resolver (client) -> recursive resolver (e.g. 8.8.8.8 / corporate)
2. Recursive -> ROOT servers: "who handles .com?"      -> referral to TLD servers
3. Recursive -> .com TLD servers: "who handles example.com?" -> referral to authoritative
NS
4. Recursive -> example.com authoritative NS: "A record for www?" -> the IP
5. Recursive caches each answer per its TTL, returns IP to the client
```

ITERATIVE VS RECURSIVE

The *recursive* resolver does the legwork on the client's behalf; each server it queries answers *iteratively* (gives a referral, not the final answer) until the authoritative server returns the record.

```
dig +trace www.example.com     # shows the full root -> TLD -> authoritative walk
dig www.example.com @8.8.8.8   # ask a specific recursive resolver
```

Interview tip: The key terms: stub resolver (client), recursive resolver (does the walk + caches), root -> TLD -> authoritative hierarchy, and TTL-based caching at each hop. `dig +trace` is the tool that makes the walk visible — knowing it signals you've actually debugged resolution, not just memorized the steps.

📚 **Official docs**

- RFC 1035: DNS implementation and specification
- Cloudflare Learning: What is DNS?

certs.eknathalabs.com

Q54 Explain the main DNS record types and the CNAME-at-apex problem (and its fixes).

DNS

🎙 Spoken answer · ~88 s

DNS record types each have a purpose. A maps a name to an IPv4 address and AAAA to IPv6. CNAME makes one name an alias of another name. MX lists mail servers. TXT holds arbitrary text, used for SPF, DKIM, DMARC, and domain verification. NS delegates a zone to nameservers. SOA holds zone metadata including the negative caching TTL. PTR does reverse lookups, SRV gives service host and port, and CAA restricts which CAs may issue certificates. The CNAME-at-apex problem: DNS rules say a CNAME can't coexist with other records at the same name, and the zone apex, like example.com, must have SOA and NS records, so you can't put a CNAME there. That's painful when you want the root domain to point at a cloud load balancer or CDN hostname, which gives a name instead of a fixed IP. Fixes: provider-specific alias or ANAME records, such as Route 53 alias records, which resolve to the target's IPs at the DNS level; CNAME flattening offered by some DNS providers, which resolves the target and serves A records; redirecting the apex to www at the web layer; or using static IPs, such as an anycast address, in an A record. Alias records also avoid an extra lookup and can follow health checks.

Likely follow-ups

- Why can't a CNAME exist at the zone apex?
- How do alias records differ from CNAMEs?

Each record type serves a purpose; the apex-CNAME restriction trips up nearly everyone pointing a root domain at a cloud endpoint.

COMMON RECORDS

```
A     : name -> IPv4         AAAA : name -> IPv6
CNAME : name -> another name (alias)   MX : mail servers
TXT   : arbitrary text (SPF, DKIM, domain verification)
NS    : delegates a zone to nameservers   SOA : zone authority/metadata
SRV   : service location (host+port)   PTR : reverse (IP -> name)
```

THE CNAME-AT-APEX PROBLEM

You cannot put a CNAME on the zone apex (`example.com` itself) — the DNS spec forbids a CNAME coexisting with the required SOA/NS records at the apex. So you can't `example.com CNAME my-lb.cloud.com`.

```
# Fixes for pointing the apex at a cloud LB whose IP changes:
- Route 53 ALIAS / Azure A-alias / Cloud DNS: provider-specific apex "alias" records
- ANAME / flattened CNAME (some DNS providers resolve it server-side to an A record)
- (Avoid) hardcoding the LB's IP in an A record — it can change
```

Gotcha: Hardcoding an A record to a cloud load balancer's current IP breaks when the provider rotates that IP. Always use the provider's ALIAS/ANAME at the apex so it tracks the endpoint automatically.

Interview tip: Knowing why apex CNAMEs are illegal (apex must hold SOA/NS, CNAME can't coexist) and the cloud-specific ALIAS fix (Route 53 alias, etc.) is a very common real-world DNS task. Subdomains (`www`) can use CNAME freely — it's only the apex that's special.

certs.eknathalabs.com

📚 **Official docs**

- RFC 1035: DNS implementation and specification

Q55 **How is DNS used for load balancing and failover? What are the limits of DNS-based LB?** DNS

🎙 **Spoken answer · ~87 s**

DNS can distribute and fail over traffic by returning different answers, but it's a coarse tool limited by caching. Techniques: round-robin DNS returns multiple A records and clients pick one, giving a rough spread. Weighted records send a percentage to one set of endpoints and the remainder to another, useful for canaries and migrations. Latency-based and geo routing return the nearest or best region's endpoint. Failover routing uses health checks to withdraw unhealthy endpoints, and multi-value answers return only healthy IPs. Limits: caching, since resolvers, operating systems, and applications cache answers for the TTL, and some ignore low TTLs, so failover takes minutes, not seconds, and not all clients switch; there's no awareness of actual load or connection state, so distribution is uneven when a few large resolvers represent many users; health checks only see what the DNS provider's checkers see; clients may pin to one IP with long-lived connections; and the response doesn't reflect capacity. So DNS is good for global traffic steering between regions, and failover with acceptable delay, but not for fine-grained load balancing. For that, use load balancers or anycast, which react in seconds. Common design is DNS to choose a region, and load balancers within the region, with low TTLs on failover records.

Likely follow-ups

- Why is DNS failover slower than load balancer failover?
- When would you use weighted DNS records?

DNS can distribute traffic by returning different/multiple answers, but it's a coarse tool limited by caching.

TECHNIQUES

```
Round-robin DNS : return multiple A records, clients pick (rough spread)
Weighted        : send X% to one set of IPs, Y% to another (canary/migration)
Latency/Geo     : return the nearest region's endpoint (GeoDNS / latency routing)
Failover        : health-check endpoints; stop returning the unhealthy one
```

THE FUNDAMENTAL LIMITS

```
- TTL caching: clients/resolvers cache answers; a failover only takes effect
  after the TTL expires (and some ignore low TTLs) -> slow, uneven cutover
- No real-time load awareness: DNS can't see backend load/health per request
- Client behavior varies: some pin the first IP, ignoring round-robin
```

Gotcha: Setting a 60s TTL doesn't guarantee 60s failover — misbehaving resolvers and OS/browser caches can hold answers far longer. DNS failover is best-effort and slow; never rely on it alone for fast HA.

Interview tip: Position DNS LB as coarse, global traffic steering (region selection, failover, weighted migrations) layered *above* a real LB (L4/L7) that does precise, health-aware balancing within a region. The TTL/caching limitation is the key insight — that's why DNS failover is measured in minutes, and why you pair GeoDNS with anycast or in-region LBs for fast local failover.

certs.eknathalabs.com

📚 **Official docs**

- Cloudflare Learning: Load balancing

Q56 **What is split-horizon DNS and when do you need it?** DNS

🎙 **Spoken answer · ~89 s**

Split-horizon DNS returns different answers for the same name depending on who is asking, typically internal versus external clients. For example, api.example.com resolves to a private IP for clients inside the VPC or corporate network, giving a direct path, and to a public load balancer address for clients on the internet. It's implemented with separate internal and external zones, such as a private hosted zone attached to a VPC alongside a public zone, or with views in BIND. You need it when internal traffic should stay on the private network for security, cost, or latency, when internal services shouldn't expose their private addresses publicly, to avoid hairpinning through NAT, and when the same hostname must work consistently for TLS certificates in both contexts. Pitfalls: the two views drift, so a record added to one but not the other causes it works from here but not there problems; clients using the wrong resolver, such as VPN users, containers, or public DNS configured on a laptop, see the wrong view; caching and search domains complicate debugging; and changes need to be made in both zones. To troubleshoot I query from each network location and compare, check which resolver answered, and manage both zones as code to keep them consistent. Document the design clearly for the team.

Likely follow-ups

- How do you keep internal and external zones consistent?
- What goes wrong for VPN users with split-horizon DNS?

Split-horizon (split-brain) DNS returns different answers for the same name depending on *who is asking* — typically internal vs external clients.

HOW IT WORKS

```
api.example.com:
  - internal clients (VPC/corp)  -> 10.0.5.20  (private IP, direct path)
  - external clients (internet)  -> 203.0.113.10 (public LB)
# Same hostname, different view based on source/resolver
```

WHEN YOU NEED IT

```
- Internal services reachable privately (no hairpin through the public internet)
- Same app name works inside and outside the network seamlessly
- Cloud: a private hosted zone (Route 53 private zone / Cloud DNS private)
  overrides the public zone for VPC-internal resolvers
```

Gotcha: Forgetting split-horizon makes internal clients route to the public IP and "hairpin" out to the internet and back — extra latency, NAT/data-transfer cost, and sometimes a security-group/path failure. Conversely, a private zone that shadows a public name can confuse debugging ("works from my laptop, fails from the pod").

Interview tip: In cloud, this is the **private hosted zone** attached to a VPC — internal resolvers get private IPs, the public zone serves everyone else. It avoids hairpinning internal traffic out to the public LB. Mention that mismatched split-horizon views are a common cause of "resolves differently depending on where I run it."

certs.eknathalabs.com

📚 **Official docs**

- Cloudflare Learning: What is DNS?
- OpenStack documentation

Q57 **Explain DNS caching layers and negative caching. Why is a stale or cached failure so persistent?** DNS

🎙 **Spoken answer · ~89 s**

DNS answers are cached at many layers, each with its own TTL behaviour, which is why both good and bad answers can stick around longer than expected. The layers: the application or runtime, such as a JVM which historically cached forever, and connection pools holding resolved addresses; the OS resolver cache, such as systemd-resolved or nscd; local forwarders and node-level caches like node-local DNS in Kubernetes; the recursive resolver at the ISP or cloud; and sometimes browsers. Negative caching stores the failure itself: an NXDOMAIN or SERVFAIL result is cached for the SOA minimum TTL or a configured value. So if a record is queried before it exists, the resolver remembers does not exist, and even after the record is created, clients keep getting the failure until the negative TTL expires. That's why a failure can be so persistent and inconsistent: different layers expire at different times, so some clients work and others don't. Diagnosis: query the authoritative server directly to confirm the record exists, query each resolver layer to find the stale one, and check TTLs in the answers. Remedies: flush the caches that hold the failure, wait out the TTL, set low negative TTLs, create records before anything queries them, and configure application DNS caching sensibly, for example setting JVM TTLs.

Likely follow-ups

- Why do some clients work while others fail after a DNS fix?
- How can a JVM's DNS caching cause an outage?

DNS answers are cached at many layers, each with its own TTL handling — which is why both good and bad answers can stick around far longer than expected.

THE CACHING LAYERS

```
Application/runtime : e.g. JVM caches DNS (historically forever!), connection pools
OS resolver cache   : systemd-resolved / nscd / Windows DNS Client
Recursive resolver  : the big shared cache (8.8.8.8, corporate, NodeLocal DNSCache)
Authoritative TTL   : the value the zone owner sets per record
```

NEGATIVE CACHING (CACHING THE FAILURE)

NXDOMAIN and SERVFAIL responses are cached too, governed by the SOA's minimum TTL. So after you create a record, a prior "does not exist" answer can linger — the record exists but clients still get NXDOMAIN until the negative cache expires.

JVM gotcha: Older JVMs cache successful lookups *forever* (`networkaddress.cache.ttl=-1`) — so a failover that changes an IP never takes effect for that JVM until restart. Set a finite `networkaddress.cache.ttl`. Connection pools that resolve once at startup have the same problem.

> **Interview tip:** Two senior points: (1) negative caching means "I just created the record but it still says NXDOMAIN" is expected until the SOA negative TTL expires; (2) application-level DNS caching (especially the JVM) defeats DNS-based failover — the app keeps hitting the dead IP. The fix is finite caching TTLs and re-resolving on connection errors.

certs.eknathalabs.com

📚 **Official docs**

- RFC 2308: DNS negative caching

Q58 **'DNS works for some names but not others.' Walk through how you isolate the cause.** Troubleshooting

🎙 **Spoken answer · ~87 s**

Selective DNS failure, where some names resolve and others don't, points to a specific zone, resolver, or record, not total DNS breakage, so I bisect. First compare a working and a failing name against the same resolver with dig, and read the status: NXDOMAIN means the record truly doesn't exist, SERVFAIL means the resolver couldn't get a valid answer, and a timeout means no response. Then compare resolvers: query the failing name against several resolvers and against the authoritative server directly. If the authoritative server answers but a resolver doesn't, the issue is in resolution: a stale negative cache, DNSSEC validation failing, or a forwarding rule. Use dig with trace to find a broken delegation, lame nameserver, or missing glue. Check whether the failing names share something: the same zone, domain, or suffix points to a delegation or forwarding rule, like a conditional forwarder for a private zone that doesn't exist anymore, or a search domain and ndots expansion producing wrong lookups. Check record types: an A works but AAAA fails, or large responses failing because UDP truncation and TCP fallback is blocked by a firewall. Also compare with the application's resolver path, using getent. Once located, fix the zone, forwarder, or firewall, and add monitoring on those names.

Likely follow-ups

- What does SERVFAIL tell you compared with NXDOMAIN?
- How can blocked TCP/53 cause selective failures?

Selective DNS failure points to a specific zone, resolver, or record rather than total DNS breakage. Bisect systematically.

1. COMPARE A WORKING VS FAILING NAME AGAINST THE SAME RESOLVER

```
dig good.example.com       # works
dig bad.example.com        # fails — note the status (NXDOMAIN? SERVFAIL? timeout?)
# NXDOMAIN = record truly missing;  SERVFAIL = resolver/authoritative error;
# timeout = network/resolver reachability
```

2. BYPASS CACHES / TRY DIFFERENT RESOLVERS

```
dig bad.example.com @8.8.8.8      # public resolver
dig bad.example.com @<internal-resolver>
dig +trace bad.example.com        # find WHERE the delegation/answer breaks
```

3. COMMON SELECTIVE-FAILURE CAUSES

```
- Split-horizon: internal zone missing the record the public zone has (or vice versa)
- One authoritative NS out of several is broken -> intermittent SERVFAIL
- search-domain / ndots: short name expands wrong (works as FQDN, fails as short)
- DNSSEC validation failure on one zone -> SERVFAIL only for that zone
- A specific subdomain delegated to a down nameserver
```

Interview tip: Lead with reading the response *status* — NXDOMAIN vs SERVFAIL vs timeout each points somewhere different. Then bisect by resolver (internal vs public) and use `dig +trace` to localize a broken delegation. Selective failure almost always

certs.eknathalabs.com

means one zone/resolver/record, not "DNS is down" — saying that shows structured thinking.

📚 **Official docs**

- Cloudflare Learning: What is DNS?

Q59 **What is DNSSEC and what does a SERVFAIL caused by validation failure look like?** DNS

🎙 **Spoken answer · ~83 s**

DNSSEC cryptographically signs DNS records so resolvers can verify that answers weren't forged, protecting against cache poisoning and spoofing. Each zone signs its records, producing RRSIG signatures, and publishes its public key as a DNSKEY; a chain of trust is built because the parent zone holds a DS record, a hash of the child's key, up to the signed root. A validating resolver checks signatures along the chain. The cost is operational fragility. If something is wrong, such as expired signatures, a key rollover mistake, or a DS record at the registrar that doesn't match the zone's key, validating resolvers refuse the answer and return SERVFAIL for the whole domain, while non-validating resolvers still work. So the outage looks like DNS is broken for some users but not others, depending on which resolver they use, which is confusing. To diagnose: dig with the checking-disabled flag; if it returns a normal answer with checking disabled but SERVFAIL otherwise, validation is the cause. Tools like DNSViz or delv show where the chain breaks. Prevention: automate signing and key rollover, monitor signature expiry, carefully coordinate DS updates, test changes before publishing, and use managed DNS that handles it. DNSSEC protects integrity, not confidentiality.

Likely follow-ups

- How does dig +cd help prove a DNSSEC failure?
- Why does a DS record mismatch break resolution?

DNSSEC cryptographically signs DNS records so resolvers can verify answers weren't forged (cache poisoning / spoofing). The cost is operational fragility.

HOW IT WORKS

```
- Each zone signs its records (RRSIG); publishes public key (DNSKEY)
- A chain of trust: parent zone holds a DS record hashing the child's key
- Validating resolver walks root -> TLD -> zone, checking signatures
- Tampered/unsigned-but-expected answer -> validation FAILS
```

THE FAILURE MODE

A DNSSEC validation failure returns **SERVFAIL** to validating resolvers (not NXDOMAIN) — the name may resolve fine on non-validating resolvers, so it's intermittent depending on which resolver a client uses.

```
dig +dnssec example.com          # shows RRSIG records if signed
dig +cd example.com              # "checking disabled" — bypasses validation;
                                 # if +cd WORKS but normal FAILS -> DNSSEC issue
```

Operational risk: Expired signatures (RRSIG has a validity window — forget to re-sign and the whole zone goes SERVFAIL), a broken chain of trust (DS record not updated after a key rollover), or a registrar that mishandles DS records. DNSSEC outages take a domain fully dark for validating clients.

certs.eknathalabs.com

Interview tip: The diagnostic gem: DNSSEC failures = SERVFAIL on validating resolvers, and `dig +cd` (checking disabled) succeeding while a normal query fails confirms it. Stress the operational fragility — expired RRSIGs and botched key rollovers cause real outages — which is why some orgs weigh DNSSEC's benefit against its blast radius.

📚 **Official docs**

- Cloudflare Learning: DNS security

Q60

Compare DNS over UDP/TCP, DNS over TLS (DoT), and DNS over HTTPS (DoH). Operational impact?

DNS

🎙 **Spoken answer · ~83 s**

Classic DNS is plaintext over UDP port fifty-three, with TCP fallback for large responses, such as DNSSEC or zone transfers, and when UDP truncates. DNS over TLS wraps queries in a TLS session on port eight five three, and DNS over HTTPS carries them inside HTTPS on port four four three. Both add encryption and authentication of the resolver, protecting privacy and integrity from on-path observers. The operational impact: with plaintext DNS, network teams can see, log, filter, and monitor queries, and enforce policies such as blocking malicious domains. DoT is easy to identify and manage by port. DoH blends with normal HTTPS traffic, so it bypasses network-level DNS controls, and if a browser or application uses its own DoH resolver, internal names may not resolve, split-horizon DNS breaks, and security tools lose visibility. In enterprises, this means deciding on a policy: provide an internal encrypted resolver, configure clients to use it, and block or control external DoH endpoints. Performance considerations: encrypted transports add handshake overhead, mitigated by connection reuse. For troubleshooting, packet captures no longer show queries in clear text, so logging at the resolver is needed. Also TCP fallback must be allowed through firewalls, or large responses fail.

Likely follow-ups

- Why does DoH complicate enterprise DNS controls?
- When does DNS use TCP?

Classic DNS is plaintext UDP (with TCP fallback); DoT and DoH add encryption, which changes privacy, security, and your ability to observe/control DNS.

THE TRANSPORTS

```
UDP/53  : default; fast, but truncated large answers fall back to TCP/53
TCP/53  : used for big responses (DNSSEC, zone transfers) + when UDP truncates
DoT     : DNS over TLS on port 853 — encrypted, on a dedicated port (visible as DNS)
DoH     : DNS over HTTPS on 443 — encrypted AND blended with web traffic (hard to see)
```

OPERATIONAL IMPACT

```
+ Privacy/integrity: queries can't be sniffed or trivially tampered
- Visibility loss: DoH on 443 bypasses your DNS-based monitoring/filtering
- Browsers shipping DoH can ignore your corporate/NodeLocal resolver entirely
- Harder to enforce split-horizon / internal resolution if clients use external DoH
```

Gotcha: A browser or app silently using public DoH can break internal name resolution (it skips your private resolver/split-horizon) and evade security DNS filtering — "works on my phone's data, fails on corp wifi" or vice versa. Enterprises often disable client DoH

certs.eknathalabs.com

or point it at an internal DoH endpoint.

Interview tip: The trade-off is privacy vs control/observability: DoT is encrypted but still identifiable on 853 (you can manage it); DoH on 443 is deliberately indistinguishable from HTTPS, which is great for user privacy but breaks enterprise DNS visibility, filtering, and split-horizon. Knowing why DoH specifically complicates ops is the senior point.

📚 **Official docs**

- RFC 768: UDP
- Cloudflare Learning: DNS security

Q61 **Walk through the TCP three-way handshake and connection teardown. What are TIME_WAIT and CLOSE_WAIT?** TCP/UDP

🎙 **Spoken answer · ~83 s**

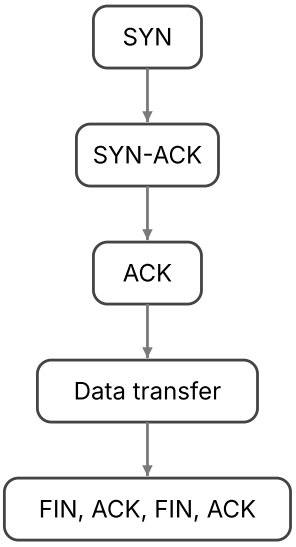
TCP opens a connection with a three-way handshake: SYN, SYN-ACK, ACK. It closes with a four-way exchange: each side sends a FIN and the other acknowledges, because each direction closes independently. The lingering states cause confusion. TIME_WAIT is held by the side that actively closes first, for around sixty seconds, so late packets from the old connection can't be mistaken for a new one and the last ACK can be retransmitted. Many TIME_WAIT sockets are normal on busy clients and proxies, but can exhaust ephemeral ports; the fix is connection reuse, a wider port range, and tcp_tw_reuse for outbound. CLOSE_WAIT is on the side that received a FIN: it's waiting for its own application to close the socket. If sockets pile up in CLOSE_WAIT, the application isn't closing connections after the peer hung up, which is a leak bug that eventually exhausts file descriptors. So TIME_WAIT is a kernel protocol mechanism, typically benign, while CLOSE_WAIT is application behaviour, typically a bug. I diagnose with ss counting states, and lsof to identify the process. In production, watch both counts as metrics. Also note that an abrupt close uses RST instead of FIN, which skips these states and indicates something different.

Likely follow-ups

- Which side enters TIME_WAIT?
- What causes CLOSE_WAIT to accumulate?

certs.eknathalabs.com

🗺 **TCP lifecycle**



TCP establishes a connection in three steps and tears it down in four. The lingering states (TIME_WAIT, CLOSE_WAIT) are common sources of confusion and production issues.

HANDSHAKE + TEARDOWN

```
OPEN:  SYN ->  | <- SYN-ACK | ACK ->          (3-way)
CLOSE: FIN -> | <- ACK | <- FIN | ACK ->        (4-way, each side closes)
```

TIME_WAIT (THE ACTIVE CLOSER)

The side that closes first sits in TIME_WAIT for ~2*MSL (often 60s) to absorb stray delayed packets and ensure the final ACK arrived. Many TIME_WAIT sockets is **normal** on a busy client/proxy, but can exhaust local ports if it initiates huge numbers of short connections.

CLOSE_WAIT (THE BUG SIGNAL)

```
# CLOSE_WAIT = the OTHER side sent FIN, but YOUR app hasn't called close().
# Piling-up CLOSE_WAIT sockets = an APPLICATION BUG (leaked sockets / fds).
ss -tan state close-wait | wc -l     # if this grows, your app isn't closing conns
```

Key distinction: Lots of TIME_WAIT is usually fine (it's the kernel doing its job); lots of CLOSE_WAIT is almost always your application failing to close sockets — leading to file-descriptor exhaustion and eventual "too many open files".

Interview tip: Nail the distinction: TIME_WAIT is on the active closer and self-clears (tune with reuse, not the dangerous `tcp_tw_recycle`); CLOSE_WAIT growth means the app isn't closing connections — a code bug, not a tuning knob. Saying that confidently separates people who've debugged this from those who haven't.

📚 **Official docs**

- RFC 9293: TCP

certs.eknathalabs.com

Q62 **What is ephemeral port exhaustion and how do you diagnose and fix it?** TCP/UDP

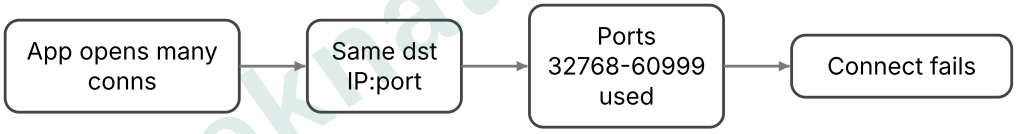
🎙 Spoken answer · ~87 s

Every outbound connection consumes a local source port from the ephemeral range, and a connection is uniquely identified by the four-tuple of source address, source port, destination address, and destination port. To one destination address and port, the client has only the ephemeral range to work with, around twenty-eight thousand ports by default. Because closed connections linger in TIME_WAIT, a client creating many short connections to the same destination can run out, producing errors like cannot assign requested address, or connect timeouts and failures that appear under load. Diagnosis: check the port range, count sockets by state with ss, particularly TIME_WAIT to that destination, and see which process makes the connections. Fixes in order of preference: reuse connections through keep-alive and pooling; fix connection leaks; widen the ephemeral port range; enable tcp_tw_reuse for outbound connections; spread across more destination addresses or ports; and add source IP addresses. With NAT gateways, the gateway has its own limit of ports per destination, so adding more NAT addresses or spreading destinations helps; cloud NAT gateways expose metrics for port allocation errors. In Kubernetes, many pods behind one node address share the node's port space when SNAT is used. Monitor socket counts and NAT port errors to catch this before users do.

Likely follow-ups

- Why does the limit apply per destination?
- How can NAT gateways exhibit the same problem?

🗺 Port exhaustion



Every outbound connection consumes a local source port from the ephemeral range. A host making many connections to the same destination can run out.

THE MECHANISM

```
# A connection is uniquely keyed by the 4-tuple:
#   (src IP, src PORT, dst IP, dst PORT)
# To ONE dst IP:port, you have only ~28k-64k source ports available.
cat /proc/sys/net/ipv4/ip_local_port_range   # the ephemeral range
```

SYMPTOMS + DIAGNOSIS

```
- "Cannot assign requested address" (EADDRNOTAVAIL) on connect()
- Bursts of failures when connecting to one backend/DB/proxy
ss -s                           # summary of socket states
ss -tan | grep TIME-WAIT | wc -l # often TIME_WAIT eating the ports
```

FIXES

certs.eknathalabs.com

```
- Connection POOLING / keep-alive (reuse connections — the real fix)
- Widen ip_local_port_range
- net.ipv4.tcp_tw_reuse=1 (safely reuse TIME_WAIT for new OUTbound conns)
- Spread load across more destination IPs/ports (more 4-tuples)
```

Gotcha: The limit is **per destination** 4-tuple, so a service hammering a single DB or downstream IP exhausts ports long before the host is "busy." Don't reach for the deprecated/dangerous `tcp_tw_recycle` (removed in modern kernels, breaks behind NAT).

Interview tip: The headline fix is connection pooling — reuse instead of churn. The 4-tuple insight (per-destination limit) explains why "the host isn't even busy" yet connections fail. Cite `tcp_tw_reuse` (safe, outbound) and explicitly warn against `tcp_tw_recycle` — that distinction is a senior tell.

📚 **Official docs**

- IANA: Service name and port number registry
- Terraform: Sensitive data in state

Q63 **Explain TCP keep-alive vs HTTP keep-alive, and how load balancer idle timeouts cause errors.** TCP/UDP

🎙 **Spoken answer · ~88 s**

These are two different keep-alive mechanisms at different layers, and confusing them causes mysterious connection resets behind load balancers. TCP keep-alive is a kernel feature: after a period of idleness, it sends small probes to detect whether the peer is still there. The default idle time is about two hours, too long to be useful without tuning, but with tuning it can keep connections alive through NATs and firewalls that drop idle flows, and detect dead peers. HTTP keep-alive means reusing one TCP connection for multiple HTTP requests, avoiding repeated handshakes; the server decides how long it keeps idle connections open. Problems arise from timeout mismatches: a load balancer has an idle timeout, for example sixty seconds; if the backend's keep-alive timeout is shorter, the backend closes a connection just as the load balancer sends a request on it, producing 502 errors or resets. The rule: the server behind a load balancer should have an idle timeout longer than the load balancer's, so the load balancer is always the one to close first. The same applies to client pools holding connections longer than the server keeps them. Fixes: align timeouts across the chain, set TCP keep-alive probes below the shortest idle timeout in the path, and retry idempotent requests on reset.

Likely follow-ups

- Why should the backend idle timeout exceed the load balancer's?
- What does TCP keep-alive protect against?

Two different "keep-alive" mechanisms at different layers — confusing them leads to mysterious connection-reset errors behind load balancers.

THE TWO KEEP-ALIVES

```
TCP keep-alive : kernel probes to detect a DEAD peer on an idle connection
                 (default ~2hrs — too slow to be useful by default)
HTTP keep-alive: REUSE one TCP connection for multiple HTTP requests
                 (avoids handshake per request) — an app/L7 concept
```

certs.eknathalabs.com

THE LB IDLE-TIMEOUT TRAP

```
# LB silently drops a connection idle longer than its idle timeout (e.g. 60s).
# Client thinks the (pooled, kept-alive) connection is still good, reuses it,
# and gets a RST / "connection reset by peer" or 502 on the next request.
```

This hits long-lived pools, databases behind an LB, and idle HTTP keep-alive connections.

The fix — order the timeouts: client/app idle timeout < LB idle timeout < backend idle timeout. If the client's keep-alive idle is shorter than the LB's, the client recycles the connection before the LB kills it, avoiding the reset. Also enable TCP keep-alive on long-lived/pooled connections so dead ones are detected.

Interview tip: The senior insight is the **timeout ordering**: most "random 502/connection reset behind the ALB" issues come from the app's connection-pool idle timeout being longer than the LB's idle timeout. Set the client idle below the LB idle. This is one of the most common real cloud bugs.

📚 **Official docs**

- Cloudflare Learning: Load balancing

Q64 **What is head-of-line blocking? How do HTTP/1.1, HTTP/2, and HTTP/3 (QUIC) address it?** TCP/UDP

🎙 **Spoken answer · ~86 s**

Head-of-line blocking is when one stuck item delays everything queued behind it, and it occurs at different layers. In HTTP/1.1 it's at the application layer: one request at a time per connection, so a slow response blocks the next request on that connection. Browsers work around it by opening about six parallel connections per host, and developers used tricks like domain sharding. HTTP/2 fixes the application layer by multiplexing many streams over one TCP connection, so a slow response doesn't block others at the HTTP level. But it moves the problem down: TCP delivers bytes in order, so a single lost packet stalls every stream on that connection until it's retransmitted, which is TCP-level head-of-line blocking, painful on lossy mobile networks. HTTP/3 runs over QUIC, which is built on UDP and implements streams itself, so packet loss in one stream only blocks that stream, not the others. QUIC also integrates TLS 1.3 for faster connection setup, supports connection migration across networks using connection IDs, and avoids ossification of TCP. Operationally, HTTP/3 needs UDP to be allowed through firewalls and load balancers, and clients fall back to HTTP/2 when it's blocked. Benefits show up most on lossy or high-latency networks, while on clean networks the difference is small.

Likely follow-ups

- Why does HTTP/2 still suffer head-of-line blocking?
- What does QUIC do differently with streams?

Head-of-line (HOL) blocking is when one stuck item delays everything queued behind it. It appears at both the HTTP and TCP layers, and each protocol generation tackles a different layer.

HTTP/1.1 — APPLICATION-LAYER HOL

One request per connection at a time; a slow response blocks reuse. Browsers work around it by opening ~6 parallel connections per host (wasteful).

certs.eknathalabs.com

HTTP/2 — MULTIPLEXING (FIXES APP-LAYER, NOT TCP)

```
# Many streams multiplexed over ONE TCP connection -> no app-layer HOL.
# BUT all streams share one TCP byte-stream: a single lost packet stalls
# ALL streams until retransmit -> TCP-layer HOL blocking remains.
```

HTTP/3 — QUIC OVER UDP (FIXES TCP-LAYER TOO)

```
# QUIC runs over UDP with independent streams: a lost packet only stalls
# ITS stream, not the others. Also: 0/1-RTT handshakes, connection migration
# (survives IP changes), TLS 1.3 built in.
```

Gotcha: HTTP/2's win is real for high-latency/many-object pages, but on a lossy network its single TCP connection can be *worse* than HTTP/1.1's parallel connections because one loss stalls every stream. That's exactly the gap HTTP/3 closes by moving to per-stream delivery over UDP.

Interview tip: The clean progression: HTTP/1.1 = app HOL (one-at-a-time), HTTP/2 = multiplex over one TCP (fixes app HOL, keeps TCP HOL), HTTP/3/QUIC = independent streams over UDP (fixes TCP HOL + faster handshakes + connection migration). Knowing that HTTP/2 still suffers TCP-layer HOL is the detail interviewers probe.

📚 **Official docs**

- RFC 9113: HTTP/2
- RFC 9114: HTTP/3

Q65 **Explain TCP congestion control and what BBR changes versus loss-based algorithms like CUBIC.** TCP/UDP

🎙 **Spoken answer · ~82 s**

Congestion control governs how fast TCP sends to avoid overwhelming the network. Loss-based algorithms such as Reno and CUBIC, long the default, ramp up the sending rate until they see packet loss, treat loss as the signal of congestion, then back off sharply, with slow start growing exponentially, then congestion avoidance growing more gently. The weakness: they only react after buffers overflow, which causes high queueing latency, bufferbloat, and they interpret random loss, as on wireless or long-distance paths, as congestion and throttle unnecessarily, hurting throughput. BBR, from Google, is model-based: it continuously estimates the path's bottleneck bandwidth and minimum round-trip time and paces sending to match the bandwidth-delay product, without filling queues. The result is higher throughput on lossy or long-haul links and lower latency under load. Trade-offs: BBR can be unfair to loss-based flows sharing a bottleneck, and early versions caused excess retransmissions, which BBRv2 addressed. Practically, I enable BBR on servers sending over wide-area or lossy paths, such as content delivery and cross-region transfers, along with the fq queueing discipline for pacing, and compare throughput and latency before and after. I inspect connection details with ss to see the algorithm, congestion window, and RTT.

Likely follow-ups

- Why does loss-based congestion control struggle on lossy links?
- What does BBR estimate to set its sending rate?

certs.eknathalabs.com

Congestion control governs how fast TCP sends to avoid overwhelming the network. The big shift is from loss-based to model-based algorithms.

LOSS-BASED (RENO/CUBIC — LONG THE DEFAULT)

```
# Ramp up until packet LOSS, then back off (loss = congestion signal).
# Slow start (exponential) -> congestion avoidance (linear) -> loss -> halve.
# Problem: treats ALL loss as congestion, and fills buffers (bufferbloat).
```

BBR (MODEL-BASED)

```
# Models the path's bottleneck BANDWIDTH and round-trip propagation time,
# paces sending to that estimate instead of waiting for loss.
# -> high throughput on lossy/long-haul links, less bufferbloat/latency.
sysctl net.ipv4.tcp_congestion_control     # see/set the algorithm
```

WHY IT MATTERS

On long-haul or lossy paths (mobile, cross-region, satellite), loss-based CUBIC under-performs because random (non-congestion) loss makes it back off needlessly. BBR keeps throughput high and latency low, which is why CDNs and cloud providers adopted it.

> **Interview tip:** The crisp contrast: CUBIC infers congestion from loss (and bloats buffers); BBR builds a bandwidth/RTT model and paces to it, winning on high-BDP and lossy links. Mention bufferbloat and that enabling BBR is a common, low-risk throughput/latency win for cross-region or edge traffic — a practical optimization, not just theory.

> 📚 **Official docs**
>
> - RFC 5681: TCP congestion control

certs.eknathalabs.com

Q66 **What are the SYN backlog and accept queue, and how do SYN cookies defend against SYN floods?** TCP/UDP

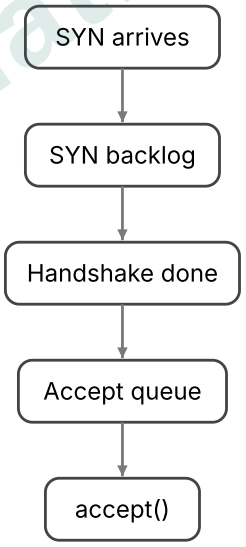
🎙 Spoken answer · ~83 s

Inbound TCP connections pass through two kernel queues. The SYN queue holds incomplete, half-open connections that have received a SYN and sent SYN-ACK, waiting for the final ACK; its size is influenced by tcp_max_syn_backlog. The accept queue holds fully established connections waiting for the application to call accept; its size is the smaller of the application's listen backlog and somaxconn. A SYN flood sends many SYNs without completing handshakes, filling the SYN queue so legitimate clients are dropped. SYN cookies defend against it: when the queue is full, the server encodes the connection state cryptographically into the sequence number of the SYN-ACK, and doesn't store anything; only when a valid ACK comes back does it reconstruct the connection. So attackers can't exhaust memory, at the cost of losing some TCP options. Separately, the accept queue overflowing under load, because the application is too slow to accept or the backlog is too small, causes dropped connections and client timeouts or resets, visible in netstat statistics as listen queue overflows and in ss as the receive queue on listening sockets. Fixes: raise somaxconn and the application's backlog, make the application accept faster, scale out, and enable SYN cookies. Monitor the overflow counters.

Likely follow-ups

- How do SYN cookies avoid storing state?
- How do you see an accept queue overflow?

🗺 SYN queues



Inbound TCP connections pass through two kernel queues. Understanding them explains both SYN-flood attacks and "connection dropped under load" issues.

THE TWO QUEUES

```
SYN queue (incomplete): half-open conns awaiting the final ACK (after SYN-ACK)
                        sized by net.ipv4.tcp_max_syn_backlog
Accept queue (complete): fully established, waiting for the app to accept()
                        sized by min(somaxconn, app's listen() backlog)
```

certs.eknathalabs.com

SYN FLOOD + SYN COOKIES

```
# Attack: flood SYNs, never send the final ACK -> SYN queue fills ->
#         legit connections dropped.
# Defense: net.ipv4.tcp_syncookies=1 -> when the SYN queue is full, the kernel
#   encodes connection state into the SYN-ACK sequence number instead of
#   storing it. Validates the returned ACK -> no queue state needed.
```

Accept-queue overflow: If the app accept()s too slowly (event-loop blocked, thread starvation), the accept queue fills and the kernel drops/resets new connections even though SYNs succeed. `ss -ltn` shows Recv-Q (current) vs Send-Q (max) on listening sockets — a full Recv-Q means the app isn't accepting fast enough.

Interview tip: Two takeaways: SYN cookies defend the SYN queue against floods without storing per-connection state (at the cost of some TCP options); and a backed-up **accept queue** (slow app, low `somaxconn`/listen backlog) drops connections under load. Reading `ss -ltn` Recv-Q/Send-Q on a listener is the diagnostic.

📚 **Official docs**

- RFC 9293: TCP
- Cloudflare Learning: DDoS attacks

Q67 **How does a host decide where to send a packet? Explain the routing table and longest-prefix match.**

Routing & subnets

🎙 **Spoken answer · ~90 s**

Every packet's next hop is chosen by consulting the routing table and picking the most specific matching route, known as longest-prefix match. The routing table lists destination prefixes with a next hop and interface. For example, a host might have a default route, zero-zero-zero-zero slash zero, via its gateway, a connected route for its own subnet, say ten dot zero dot zero dot zero slash twenty-four, directly on the interface, and a more specific route for another network via a VPN gateway. For a destination, the kernel finds all routes whose prefix contains the address and chooses the one with the longest prefix length, so a slash twenty-four beats a slash sixteen, which beats the default route. If multiple routes tie, metrics decide. I inspect with ip route and use ip route get with a destination to see exactly which route, interface, and source address would be used. The same logic applies in cloud route tables: the most specific route wins, with rules for ties between types. This explains many surprises: a more specific route from a peering or VPN silently overriding the internet path, overlapping CIDR ranges, blackhole routes after deleting a target, and asymmetric routing when return paths use different tables. When debugging connectivity, check routes on both ends and at every hop.

Likely follow-ups

- Why does a /24 route beat a /16 route?
- What does ip route get show?

Every packet's next hop is chosen by consulting the routing table and picking the most specific matching route — longest-prefix match.

THE ROUTING TABLE

certs.eknathalabs.com

```
ip route                        # show the routing table
ip route get 8.8.8.8            # show exactly which route a dest would use
# Example:
# default via 10.0.0.1 dev eth0          (0.0.0.0/0 — catch-all)
# 10.0.0.0/24 dev eth0 proto kernel      (directly connected subnet)
# 10.0.5.0/24 via 10.0.0.9 dev eth0      (more specific route)
```

LONGEST-PREFIX MATCH

If multiple routes match a destination, the one with the **longest prefix** (most specific, largest /N) wins. A packet to `10.0.5.20` matches both `10.0.0.0/24`... no — it matches `10.0.5.0/24` (more specific) over the `default` route. The default route (`/0`) is the last resort.

Gotcha: Adding a more-specific route silently overrides a broader one — a stray `/32` or a VPN pushing routes can hijack traffic you expected to go elsewhere. `ip route get <dst>` is the definitive answer to "which path will this actually take," rather than eyeballing the table.

Interview tip: The two facts: longest-prefix-match (most specific wins) and the default route as catch-all. The killer tool is `ip route get <ip>` — it resolves the exact route + source IP + interface the kernel will use, ending arguments about "why is traffic going out the wrong interface." Great for debugging asymmetric routing and VPN route leaks.

📚 **Official docs**

- Linux man-pages: ip(7)

Q68 **Explain CIDR and subnetting. How do you size and avoid overlap in VPC design?** Routing & subnets

🎙 **Spoken answer · ~84 s**

CIDR notation, slash N, says how many bits are the network prefix, and the rest are host addresses. A slash twenty-four has two hundred fifty-six addresses, two hundred fifty-four usable; slash sixteen has sixty-five thousand five hundred thirty-six; slash twenty-eight has sixteen; usable hosts equal two to the power of thirty-two minus N, minus reserved addresses, and cloud providers reserve a few extra per subnet. Subnet planning is a day-zero decision that's painful to undo. For VPC design I choose a range large enough for growth, including Kubernetes pods, which can consume many addresses with VPC-native networking, and divide it by tier, public, private, and data, and by availability zone, leaving unallocated space for future subnets. Overlap is the main danger: VPCs or networks with overlapping CIDRs can't be peered or connected over VPN or transit gateway without NAT workarounds, and on-premises ranges must be considered too. So I allocate from a central IP address management plan, using the RFC 1918 private ranges, avoid commonly used defaults like 10.0.0.0/16 across all accounts, document allocations, and enforce through infrastructure as code and automated checks. Also plan for IPv6 and secondary ranges. A little extra address space is cheap compared with renumbering later.

Likely follow-ups

- Why avoid default CIDR ranges across many VPCs?
- How much address space should Kubernetes pods get?

CIDR notation (`/N`) defines how many bits are the network prefix; the rest are host addresses. Subnet planning is a day-0 decision that's painful to undo.

certs.eknathalabs.com

CIDR MATH

```
/24 = 256 addresses (254 usable)      /16 = 65,536
/28 = 16 addresses (small subnet)     /8  = 16,777,216
# usable = 2^(32-N) minus reserved (network, broadcast, + cloud reserves a few)
# AWS reserves 5 IPs per subnet (.0 network, .1 router, .2 DNS, .3 future, .255 bcast)
```

VPC SIZING PRINCIPLES

```
- Pick a large VPC CIDR (e.g. 10.0.0.0/16) with room to grow
- Carve subnets per AZ + tier (public/private/data) — leave gaps for expansion
- NEVER overlap CIDRs between VPCs/on-prem you intend to peer/VPN
  (overlapping ranges can't be routed/peered without NAT gymnastics)
```

The cardinal sin: Using the same range (e.g. everyone defaults to 10.0.0.0/16) across VPCs or with on-prem. You can't VPC-peer or VPN overlapping CIDRs — routing is ambiguous. Plan a global IP address scheme up front; resizing or re-IPing later is a major project.

Interview tip: Two senior points: (1) plan non-overlapping CIDRs across all VPCs/on-prem from the start because peering/VPN require unique ranges; (2) cloud reserves addresses per subnet (AWS: 5), and Kubernetes pod CIDRs consume real VPC IPs — so size generously. Mention that overlap forces ugly NAT workarounds, which is why address planning is a foundational architecture task.

📚 **Official docs**

- RFC 4632: CIDR
- Amazon VPC: What is Amazon VPC?

certs.eknathalabs.com

Q69 **What is asymmetric routing and why does it break stateful firewalls and load balancers?**

Routing & subnets

🎙 Spoken answer · ~89 s

Asymmetric routing is when a request and its reply take different network paths. For plain routers that's fine, but it breaks anything that tracks connection state. Suppose a request goes through firewall A, but the reply returns through firewall B. Firewall B never saw the SYN, so it considers the reply an invalid, unsolicited packet and drops it; firewall A never sees the reply, so its state entry times out. The result is connections that hang or fail, often intermittently depending on which path is chosen, which makes it hard to diagnose. The same affects stateful load balancers, NAT devices, and security appliances, and cloud environments with multiple interfaces, transit gateways, or ECMP paths. Typical causes: multiple default routes or multi-homed hosts, ECMP hashing differently in each direction, partial BGP advertisements, VPN plus direct paths, and policy routing mistakes. Diagnosis: packet captures on both sides and at the firewalls showing the SYN passing one device and the SYN-ACK another, traceroute from both ends, and routing table inspection. Fixes: make the paths symmetric by adjusting routing, use source-based policy routing so replies leave the same interface, use stateless filtering or state synchronisation between firewall pairs, or in clouds use appliance mode on transit gateways to pin flows. Design so that state is on the path.

Likely follow-ups

- How would you prove the paths are asymmetric?
- What is appliance mode on a transit gateway for?

Asymmetric routing is when the request and its reply take different network paths. It's fine for plain routing but breaks anything that tracks connection state.

THE PROBLEM

```
# Request:  client -> path A -> firewall A -> server
# Reply:    server -> path B -> firewall B -> client
# Firewall B never saw the SYN, so the reply looks like an unsolicited packet
#   -> dropped by the stateful firewall (no matching conntrack entry).
```

WHERE IT BITES

```
- Multiple egress paths / multi-homed hosts / dual NAT gateways
- Stateful firewalls or LBs in each path that must see BOTH directions
- Linux rp_filter (reverse-path filtering) dropping packets arriving on an
  interface the route table wouldn't have used to reach the source
```

Gotcha: Stateless routing tolerates asymmetry; stateful devices (firewalls, NAT, L4 LBs, conntrack) require seeing both directions on the same device. Symptoms: connections that establish but hang, or work intermittently depending on path. `rp_filter` can silently drop the asymmetric replies on multi-NIC hosts.

Interview tip: The crux: state requires symmetry. Fix by forcing both directions through the same stateful device (policy/source-based routing, sticky paths), syncing firewall state, or relaxing `rp_filter` to `loose` mode on multi-homed hosts. Recognizing that "it establishes then hangs with multiple gateways" smells like asymmetric routing is a strong signal.

certs.eknathalabs.com

📚 **Official docs**

- Cloudflare Learning: Load balancing

Q70 **What is BGP, and how do BGP issues cause large internet/cloud outages?** Routing & subnets

🎙 **Spoken answer · ~84 s**

BGP is the protocol that routes between autonomous systems, the backbone routing of the internet. Autonomous systems advertise which prefixes they can reach, and neighbours choose paths using policy and AS-path length. It's the glue between ISPs, clouds, and enterprises. The problem is that BGP is trust-based: it accepts advertisements from neighbours, and without filtering, a misconfiguration or malicious announcement spreads quickly. Outages come from route leaks, where an AS accidentally advertises routes it shouldn't, attracting traffic it can't handle; hijacks, where someone announces another's prefix, or a more specific one, which wins by longest prefix match, diverting or blackholing traffic; withdrawals, where a network accidentally withdraws its own routes, as in the large outage where a company's prefixes vanished and even its DNS became unreachable, hindering recovery; and session or configuration failures in cloud interconnects. Defences: prefix filters and maximum-prefix limits, RPKI route origin validation, IRR-based filtering, authentication on sessions, and monitoring of your prefixes with alerting on unexpected origin changes. Design for resilience: multiple providers, redundant BGP sessions with BFD, avoid circular dependencies where your recovery tools depend on the failed network, and keep out-of-band access. In interviews, show that you know both the mechanism and the blast radius.

Likely follow-ups

- What does RPKI protect against?
- Why do more specific prefixes win in a hijack?

BGP is the protocol that routes between autonomous systems (the internet's backbone routing). It's trust-based, which makes misconfigurations and hijacks high-impact.

WHAT BGP DOES

```
- Autonomous Systems (ASes) advertise which IP prefixes they can reach
- Neighbors choose best paths by policy + AS-path length
- It's the glue between ISPs, clouds, and large networks
- Largely trust-based historically: you believe what peers advertise
```

HOW IT CAUSES OUTAGES

```
- Route leak / fat-finger: an AS wrongly advertises prefixes it shouldn't
  -> traffic blackholes or detours globally (e.g. major provider self-outages)
- BGP hijack: a malicious/erroneous AS claims someone else's prefixes
  -> traffic redirected/intercepted
- Withdrawing your own routes by mistake -> your services vanish from the internet
```

MITIGATION: RPKI

RPKI (Resource Public Key Infrastructure) cryptographically validates that an AS is authorized to announce a prefix (Route Origin Validation), reducing accidental leaks and hijacks. Adoption is growing but partial.

certs.eknathalabs.com

Interview tip: You won't run BGP as an app engineer, but understanding that a BGP misconfiguration can withdraw a provider's own routes (causing a global self-outage) or let someone hijack prefixes explains real headline incidents. Naming **RPKI / Route Origin Validation** as the defense shows you understand internet-scale routing risk — relevant when reasoning about provider dependencies and DR.

📚 **Official docs**

- RFC 4271: BGP-4
- Cloudflare Learning: What is BGP?

Q71 **Compare VPC peering, Transit Gateway, VPN, and Direct Connect. When use each?**

Cloud networking

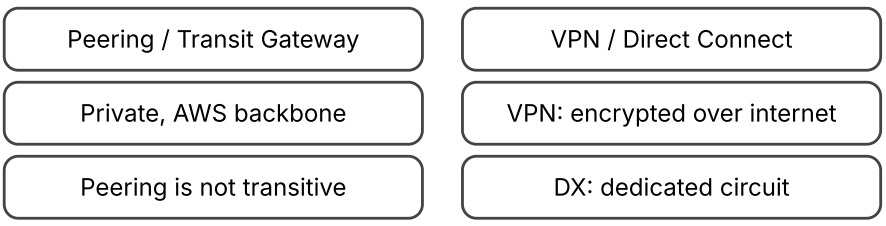
🎙 **Spoken answer · ~87 s**

These options connect networks, and they differ in topology, scale, bandwidth, and whether traffic crosses the public internet. VPC peering is a one-to-one private connection between two VPCs: simple and cheap, low latency, but not transitive, so A to B and B to C doesn't give A to C, and a mesh of peerings grows quadratically and becomes unmanageable. A transit gateway is a hub-and-spoke router connecting many VPCs and on-premises networks, with transitive routing and central route tables and segmentation, so it scales and simplifies, at a per-attachment and data processing cost. A site-to-site VPN uses IPsec tunnels over the public internet: quick to set up, encrypted, lower cost, but bandwidth per tunnel is limited and latency and jitter vary with the internet; BGP over multiple tunnels gives failover. Direct Connect, ExpressRoute, or Cloud Interconnect is a dedicated private link into the cloud: consistent low latency, higher bandwidth, and often lower data transfer costs, but needs lead time, a physical presence or partner, and redundancy design; it isn't encrypted by default, so add encryption if required. Choice: peering for a few VPCs, transit gateway for many, VPN for quick or backup connectivity, dedicated links for steady high-volume hybrid traffic, often with VPN as backup. Plan non-overlapping CIDRs.

Likely follow-ups

- Why isn't VPC peering transitive an issue at scale?
- Why combine Direct Connect with a VPN?

🗺 **Connecting networks**



These connect networks together; they differ in topology, scale, bandwidth, and whether traffic crosses the public internet.

THE OPTIONS

certs.eknathalabs.com

```
VPC Peering     : 1:1 private link between two VPCs; NOT transitive; simple/cheap
                  -> mesh of peerings gets unmanageable at scale (N^2)
Transit Gateway : hub-and-spoke router connecting MANY VPCs + on-prem; transitive
                  -> the scalable answer for many-VPC/multi-account networks
Site-to-Site VPN: encrypted tunnel over the PUBLIC internet to on-prem
                  -> quick, cheap, but internet-variable latency + bandwidth caps
Direct Connect  : DEDICATED private physical link to the cloud
                  -> consistent low latency, high bandwidth, no public internet
```

CHOOSING

```
Few VPCs, simple        -> VPC peering
Many VPCs / hub-spoke   -> Transit Gateway
On-prem, cheap/fast set  -> VPN
On-prem, performance/SLA -> Direct Connect (often + VPN as encrypted backup)
```

Gotcha: VPC peering is **non-transitive** — if A peers B and B peers C, A still can't reach C. People build sprawling peering meshes then hit this; Transit Gateway exists precisely to provide transitive hub-and-spoke routing. And remember: overlapping CIDRs can't peer at all.

Interview tip: The two senior points: peering is non-transitive (so it doesn't scale past a handful of VPCs — use Transit Gateway as the hub), and VPN vs Direct Connect is the internet-vs-dedicated trade-off (cost/speed-to-deploy vs consistent performance, often combined with VPN as encrypted backup over Direct Connect).

📚 **Official docs**

- Cloudflare Learning: What is a VPN?
- Amazon VPC: VPC peering
- AWS Transit Gateway

certs.eknathalabs.com

Q72 **What is policy-based / source-based routing and when do you need it?** Routing & subnets

🎙 Spoken answer · ~89 s

Normal routing decides by destination only. Policy-based routing, or source-based routing, lets me choose the route using other attributes such as the source address, firewall mark, incoming interface, or port, by using multiple routing tables and rules to select between them. On Linux: ip rule says traffic from a given source uses table one hundred, and that table has its own default route via a different gateway. I list with ip rule and view tables with ip route show table. When I need it: multi-homed hosts, with two interfaces on different networks, where replies must leave through the same interface the request arrived on, otherwise asymmetric routing and reverse-path filtering drop traffic; this is common with multiple ENIs on cloud instances, each in different subnets. Sending certain traffic, like backups or a specific tenant, over a particular link or through a VPN or inspection appliance; steering traffic by firewall mark, set by iptables, for fine control; and routing containers or namespaces differently. Pitfalls: complexity and hard-to-debug behaviour, because routing depends on rules, ordering, and priorities; rules and tables not persisting across reboots unless configured in the network manager; and interaction with rp_filter. Document it, test with ip route get including from the source address, and persist the configuration through the network configuration tooling.

Likely follow-ups

- Why do multi-homed hosts need source-based routing?
- How do you test which route a given source address would use?

Normal routing decides by *destination* only. Policy-based routing (PBR) lets you route by source, mark, port, or other attributes using multiple routing tables.

THE MECHANISM (LINUX)

```
ip rule add from 10.0.5.0/24 table 100   # traffic FROM this source uses table 100
ip route add default via 10.0.5.1 table 100
ip rule list                              # show policy rules (consulted in order)
# Selectors: from (source), fwmark, iif, dport, etc. -> pick a routing table
```

WHEN YOU NEED IT

```
- Multi-homed host: replies must egress the SAME interface they arrived on
  (fixes the asymmetric-routing/rp_filter drop problem)
- Send specific subnets/apps over a VPN or a specific gateway
- Separate management vs data-plane traffic onto different links
- Multiple ISPs/egress paths with per-source policy
```

Gotcha: Without source-based routing on a dual-NIC host, replies to traffic that arrived on eth1 may try to leave via eth0's default route — asymmetric routing, and `rp_filter` then drops them. PBR pins the reply to the correct interface/table.

Interview tip: The canonical use is the multi-homed / dual-gateway host where you must reply out the interface a request came in on — exactly the fix for the asymmetric-routing problem from the routing section. `ip rule` + a dedicated routing table per source is the implementation. Recognizing when destination-only routing isn't enough is the signal.

certs.eknathalabs.com

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

Q73 **How do you distinguish packet loss, latency, and jitter — and what does mtr tell you?** Troubleshooting

🎙 **Spoken answer · ~89 s**

Latency, packet loss, and jitter degrade connections differently, and conflating them sends debugging down the wrong path. Latency is the time for a packet to travel one way or round trip; high latency slows interactive requests and limits throughput per connection. Packet loss is the percentage of packets that never arrive; it triggers TCP retransmissions and congestion backoff, which reduces throughput sharply and adds tail latency, and breaks real-time media. Jitter is the variation in latency between packets; it harms voice, video, and any timing-sensitive traffic even if average latency is fine. mtr combines ping and traceroute and shows per-hop loss percentage, average, best, worst, and standard deviation of latency over many cycles. Reading it: real loss shows as loss that starts at a hop and continues to the final destination; loss at a single intermediate hop that disappears later is ICMP rate limiting, not a problem. A latency jump at a particular hop that persists to the end points to that link or region crossing; a high standard deviation indicates jitter. Use enough cycles for statistical value, and run in both directions. Then confirm with application-level measurements, since ICMP can be treated differently from real traffic, and use TCP-based probes if needed. The result guides who to contact and what to fix.

Likely follow-ups

- How do you recognise real loss versus ICMP rate limiting in mtr?
- Why does jitter matter even when average latency is fine?

These three degrade connections differently, and conflating them sends you down the wrong debugging path. `mtr` exposes all three per hop.

THE THREE

```
Latency : time for a packet to travel one way / round trip (RTT)
Packet loss : % of packets that never arrive (triggers TCP retransmits -> slow)
Jitter  : VARIATION in latency between packets (kills VoIP, video, real-time)
```

MTR — TRACEROUTE + PING PER HOP, CONTINUOUSLY

```
mtr -rwzbc 100 <host>      # 100 cycles, report mode, per-hop loss% + latency
# Read it: loss that STARTS at a hop and CONTINUES to the end = real loss there.
# Loss at ONE middle hop that DOESN'T continue = that router de-prioritizing ICMP
#   (cosmetic, not real loss) — a classic misread.
```

Gotcha: A single hop showing loss while later hops show none is almost always ICMP rate-limiting on that router, NOT a real problem — many people chase this phantom. Real loss **persists** from the offending hop through to the destination. Also: high RTT to a far region is physics (distance), not necessarily a fault.

Interview tip: Show you can read `mtr` correctly: loss must persist to the final hop to be real; transient mid-path loss is usually ICMP deprioritization. Match the symptom to the metric — retransmits/throughput issues = loss; slow-but-steady = latency; choppy real-time = jitter. That diagnostic precision is exactly what's being tested.

certs.eknathalabs.com

📚 **Official docs**

- traceroute(8) / mtr

Q74 **What is hairpinning / NAT loopback and why does an internal service fail to reach itself by public name?** Routing & subnets

🎙 **Spoken answer · ~88 s**

Hairpinning, or NAT loopback, is when traffic from inside a network goes out to a public address and must turn around and come back to a host on the same network. Many setups don't support it. The scenario: an internal client resolves api.example.com to the public load balancer address; the packet goes to the gateway or load balancer, which would need to forward it back into the internal network to a backend. If the device doesn't support hairpin NAT, the connection fails or hangs; or with partial support, the backend replies directly to the client, bypassing the translating device, so the client sees a reply from an unexpected source address and drops it, a form of asymmetric routing. Symptoms: the service works from the internet but not from inside, or an internal service can't reach itself by its public name. Fixes: use split-horizon DNS so internal clients resolve the internal address directly; enable NAT loopback or hairpin support on the gateway, with source NAT on the hairpinned traffic so replies return via the device; in clouds, use internal load balancers or private endpoints; and in Kubernetes, enable hairpin mode on the bridge for pods reaching their own service. Prefer the DNS approach as it avoids unnecessary traffic and keeps flows simple.

Likely follow-ups

- Why does the reply get dropped in a hairpin without SNAT?
- How does split-horizon DNS remove the need for hairpinning?

Hairpinning is when traffic from inside a network goes out to a public endpoint and must "turn around" to come back to a host on the same network. Many setups don't support it.

THE SCENARIO

```
# Internal client resolves api.example.com -> the PUBLIC LB IP
# Packet goes to the gateway/LB and must hairpin back to an internal backend
#   on the same network it came from.
# Many NAT gateways / LBs don't allow this loopback -> connection fails/times out.
```

WHY IT FAILS + FIXES

```
- NAT device won't route a packet back out the interface it arrived on
- Source/dest on the same subnet confuses the NAT/security path
FIXES:
- Split-horizon DNS: internal clients resolve to the PRIVATE IP (no hairpin)
- An internal load balancer for internal traffic
- Enable NAT loopback/hairpinning on the gateway (if supported)
```

Gotcha: The tell is "works from outside, fails from inside" (or from the same host/subnet) when using the public hostname. It's not a DNS or app bug — it's the network refusing the U-turn. This is extremely common with containers calling their own service's public URL.

certs.eknathalabs.com

Interview tip: The clean fix ties back to split-horizon DNS — internal clients should resolve the name to the private endpoint so traffic never has to hairpin. Recognizing "external works, internal-to-itself fails" as NAT loopback (not DNS) demonstrates real network debugging, and the private-IP/internal-LB fix is the production-grade answer.

📚 **Official docs**

- RFC 3022: Traditional NAT

Q75 **What does 'connection reset by peer' (RST) actually mean, and what are the common causes?** TCP/UDP

🎙 **Spoken answer · ~85 s**

An RST is TCP's abrupt way to say this connection is over now, as opposed to the graceful FIN. Applications see it as connection reset by peer, ECONNRESET, and in-flight data may be lost. Common causes: the peer sent it intentionally, for example connecting to a port with nothing listening gives an immediate RST, so the service is down or on a different port; the application crashed or was killed and the kernel reset open connections; a timeout mismatch where a load balancer or backend closed an idle connection and the client reused it, so the next request gets an RST; a firewall or middlebox injecting resets, or a stateful device that dropped its state after a NAT or conntrack timeout and now answers packets for that connection with RST; a listen queue overflow with reset behaviour configured; a protocol policy violation closing the connection; and abort-on-close with SO_LINGER set to zero. Diagnosis: packet capture to see who sent the RST and when, since the sender's address and TTL tell whether it's the real server or a device in between, then correlate with logs, deploys, and idle times. Fix the cause: align timeouts and keepalives, retry idempotent requests, and fix capacity or crashes.

Likely follow-ups

- How do you tell whether a middlebox sent the RST?
- Why do idle timeout mismatches produce resets?

An RST is TCP's abrupt "this connection is over now" — unlike a graceful FIN. It signals one side tore down or rejected the connection unexpectedly.

RST VS FIN

```
FIN : graceful close ("I'm done sending") — normal teardown
RST : abort NOW — no graceful close; in-flight data may be lost
# Seen by the app as ECONNRESET / "connection reset by peer"
```

COMMON CAUSES

```
- Sending to a socket the peer already closed (use-after-close)
- LB/firewall idle timeout killed the connection; you reused it (the LB trap)
- Backend crashed / OOMKilled / restarted mid-connection
- Connecting to a port with nothing listening (RST instead of timeout)
- A firewall configured to REJECT (RST) rather than DROP (silent)
- Backpempt: app calling close() with unread data (SO_LINGER) sends RST
```

RST vs timeout distinction: An RST is an *active* rejection (something replied "no") — port closed, app crashed, or firewall REJECT. A *timeout* is silence — wrong IP, DROP firewall, or network black hole. RST = someone said no; timeout = no one answered. That

certs.eknathalabs.com

distinction alone narrows the cause dramatically.

Interview tip: Two strong moves: (1) RST = active rejection vs timeout = silence — immediately splits "reachable but refusing" from "unreachable"; (2) the most common production RST is the LB idle-timeout + connection-pool reuse pattern from the keep-alive question. Capturing with `tcpdump` to see who sent the RST (and when) pinpoints it.

📚 **Official docs**

- RFC 9293: TCP

Q76 **Walk through the full lifecycle of an HTTPS request from URL to rendered response.** HTTP/TLS

🎙 **Spoken answer · ~86 s**

A single HTTPS request touches DNS, TCP, TLS, and HTTP, and knowing each stage lets me attribute latency. First, DNS resolves the hostname to an IP, possibly from cache at several layers or through a full recursive walk. Second, TCP connects with the three-way handshake to port four four three, costing one round trip. Third, TLS: the client and server negotiate the version and cipher, the server proves its identity with a certificate, which the client validates, and they exchange keys; TLS 1.2 takes two round trips, 1.3 one, and resumption can reduce it further. Fourth, the client sends the HTTP request, possibly through a CDN, load balancer, or WAF, which may terminate TLS and forward to a backend, adding hops. Fifth, the server processes the request, perhaps calling databases and other services, and sends back the response; the time to first byte captures network latency plus server processing. Sixth, the client receives the response, may fetch more resources, and the browser parses and renders. Each stage can be measured: curl's timing variables break out DNS lookup, connect, TLS handshake, start transfer, and total, and browser developer tools show the same as a waterfall. Optimisations are connection reuse, HTTP/2 or HTTP/3, caching, CDNs, and TLS session resumption.

Likely follow-ups

- How would you measure each stage with curl?
- How many round trips happen before the first byte of an uncached HTTPS request?

🗺 **URL to page**



A single HTTPS request touches DNS, TCP, TLS, and HTTP. Knowing each stage lets you attribute latency precisely.

THE STAGES

```
1. DNS    : resolve hostname -> IP (may be cached at several layers)
2. TCP    : 3-way handshake to IP:443 (1 RTT)
3. TLS    : handshake — negotiate version/cipher, validate cert, key exchange
            (TLS 1.2 = 2 RTT, TLS 1.3 = 1 RTT, 0-RTT on resume)
4. HTTP   : send request; server processes; sends response
5. Transfer + render
```

ATTRIBUTING LATENCY WITH CURL

certs.eknathalabs.com

```
curl -w "dns:%{time_namelookup} conn:%{time_connect} \
tls:%{time_appconnect} ttfb:%{time_starttransfer} total:%{time_total}\n" \
  -o /dev/null -s https://example.com
```

This breaks the request into DNS, TCP connect, TLS, time-to-first-byte, and total — so you know *which* stage is slow.

Reading it: High `time_namelookup` = DNS; gap between connect and appconnect = TLS; high `time_starttransfer` after TLS = slow server (TTFB); high total but fast TTFB = large body / slow transfer. Each points to a different fix.

Interview tip: The `curl -w` timing breakdown is the single most useful HTTP-debugging trick — it turns "the site is slow" into "DNS is slow" or "TLS is slow" or "the server's TTFB is slow." Citing the per-phase timing variables shows you debug latency empirically, not by guessing.

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

Q77 **Explain the TLS 1.3 handshake and how it improves on TLS 1.2. What is 0-RTT and its risk?** HTTP/TLS

🎙 **Spoken answer · ~89 s**

TLS 1.3 streamlined the handshake for speed and removed legacy insecure options. In TLS 1.2 a full handshake takes two round trips before application data: the hello exchange, then the key exchange and finished messages. In TLS 1.3 the client sends its key share in the ClientHello, so the server can respond with its hello, certificate, and finished in one flight, and the connection is encrypted after a single round trip. TLS 1.3 also encrypts more of the handshake, including the certificate, removes weak ciphers, static RSA key exchange, compression, and renegotiation, and mandates forward secrecy. Session resumption allows zero round-trip data, 0-RTT: on a repeat connection, the client can send application data in its very first flight using a pre-shared key from an earlier session. The risk is replay: since the early data isn't protected by a fresh handshake from the server, an attacker can capture and resend that data, and the server may process it twice. So 0-RTT should be limited to idempotent requests, like GETs, or protected with anti-replay mechanisms such as single-use tickets, and sensitive actions shouldn't be allowed in early data; many deployments disable it or restrict it. Operationally, check that load balancers, proxies, and clients support TLS 1.3, and monitor for handshake failures when disabling old versions.

Likely follow-ups

- Why is 0-RTT vulnerable to replay?
- What did TLS 1.3 remove from TLS 1.2?

TLS 1.3 streamlined the handshake for speed and removed legacy insecure options, cutting a round trip and enabling session resumption with zero round trips.

1.2 VS 1.3 HANDSHAKE

```
TLS 1.2 : 2 round trips (ClientHello/ServerHello, then key exchange) before data
TLS 1.3 : 1 round trip — client sends key share in ClientHello; server responds
          with everything needed -> app data after 1 RTT
          Also: removed old ciphers/RSA key exchange, forward secrecy mandatory
```

certs.eknathalabs.com

0-RTT RESUMPTION

```
# On a RESUMED connection, the client can send app data in the FIRST flight
# (0 round trips) using a pre-shared key from the prior session.
# Great for latency — but...
```

0-RTT replay risk: 0-RTT early data can be **replayed** by an attacker (it's sent before the handshake completes, so there's no freshness guarantee). Only use 0-RTT for idempotent, non-state-changing requests (e.g. a GET) — never for a POST that transfers money. Many servers restrict 0-RTT to safe methods for this reason.

Interview tip: Two points score well: TLS 1.3 cuts the handshake to 1 RTT (and 0-RTT on resume) while mandating forward secrecy and dropping weak ciphers; and 0-RTT's replay vulnerability means it's only safe for idempotent requests. Knowing the security caveat, not just the speed win, is the senior signal.

📚 **Official docs**

- RFC 8446: TLS 1.3

Q78 **How does TLS certificate validation work, and what is the 'missing intermediate certificate' trap?** HTTP/TLS

🎙 **Spoken answer · ~85 s**

A client validates a server's certificate by building a chain of trust from the server's leaf certificate, signed by an intermediate CA, signed by a root CA that the client already trusts in its trust store. The client checks each signature, the validity dates, that the hostname matches the subject alternative names, key usage, and possibly revocation status. The missing intermediate trap: the server must send the leaf certificate plus the intermediates, but not necessarily the root. If an administrator installs only the leaf, some clients still succeed because browsers cache intermediates or fetch them using the authority information access extension, so it works in my browser. But programmatic clients such as curl, Java, Python, and mobile libraries don't fetch missing intermediates and fail with errors like unable to get local issuer certificate. Diagnosis: run openssl s_client with the server name and look at the chain the server actually sends; online SSL checkers flag incomplete chains. Fix: configure the server or load balancer with the full chain bundle, the leaf followed by intermediates, in the right order. Other causes of validation failure: expired certificates, hostname mismatch, private CAs missing from the trust store, and clock skew. Automate renewal and test with a non-browser client.

Likely follow-ups

- Why does an incomplete chain work in some browsers?
- How do you check the chain a server actually sends?

A client validates a server's certificate by building a chain of trust from the server's cert up to a trusted root CA. A broken chain causes confusing "works in browser, fails elsewhere" errors.

THE CHAIN OF TRUST

```
Server (leaf) cert  -> signed by -> Intermediate CA -> signed by -> Root CA
# Client trusts the Root (in its trust store); must be able to build the full
# chain leaf -> intermediate(s) -> root to validate.
```

certs.eknathalabs.com

THE MISSING-INTERMEDIATE TRAP

```
# The SERVER must send leaf + intermediate(s). If it sends only the leaf:
# - Browsers often still work (they cache/fetch intermediates, AIA chasing)
# - curl, Java, Go, mobile apps FAIL: "unable to get local issuer certificate"
openssl s_client -connect host:443 -showcerts   # inspect the chain the server sends
```

Classic gotcha: "It works in my browser but my service/curl/Java client gets a cert error." The server is sending an incomplete chain (missing intermediate); browsers paper over it via AIA fetching or cached intermediates, but strict clients don't. Fix: configure the server to present the full chain (fullchain.pem), not just the leaf.

Interview tip: The tell "browser works, programmatic client fails" almost always = missing intermediate cert. `openssl s_client -showcerts` reveals exactly what the server sends. The fix is serving the full chain. This is one of the most common real TLS misconfigurations and recognizing the asymmetry instantly is a strong signal.

📚 **Official docs**

- Cloudflare Learning: TLS handshake

Q79 **Explain the difference between 502, 503, and 504 errors. How do you debug each?** HTTP/TLS

🎙 **Spoken answer · ~88 s**

These 5xx errors usually come from a gateway, proxy, or load balancer, and each points to a distinct backend condition. A 502 Bad Gateway means the proxy received an invalid response or couldn't get one: the backend crashed mid-response, closed the connection, reset it, or spoke the wrong protocol; common causes are application crashes, keep-alive timeout mismatches where the backend closes idle connections first, or a TLS mismatch to the backend. A 503 Service Unavailable means there is no healthy backend to send the request to, or the service is deliberately shedding load: all targets failing health checks, a deployment with no ready pods, overload, or a circuit breaker open. A 504 Gateway Timeout means the backend didn't respond within the proxy's timeout, so the backend is slow or hung: a slow query, a dependency stalled, thread exhaustion, or the proxy timeout being shorter than legitimate processing time. To debug, I check which component emitted the error through response headers and access logs; look at backend health status and target counts; read backend logs and crashes around the time; check resource saturation and latency; and compare proxy timeouts with backend behaviour. Fixes: stabilise the app, align keepalive and timeouts, fix health checks, add capacity, and set sensible retries. Alert on each separately.

Likely follow-ups

- Which of the three usually indicates no healthy targets?
- How do keep-alive mismatches cause 502s?

These 5xx errors come from a gateway/proxy/LB and each points to a distinct backend condition — knowing which narrows the cause immediately.

THE THREE

certs.eknathalabs.com

```
502 Bad Gateway     : proxy got an INVALID/garbage response from the backend
                      (backend crashed mid-response, protocol mismatch, RST)
503 Service Unavailable : no healthy backend to send to (all failing health checks,
                      overloaded, or scaled to zero)
504 Gateway Timeout : backend accepted but didn't respond in time
                      (slow backend, deadlock, downstream dependency hang)
```

DEBUGGING EACH

```
502 -> check backend logs for crashes/RST; protocol/keep-alive mismatch; OOMKills
503 -> check health checks + backend pool: are ANY targets healthy? scaled down?
504 -> check backend latency + its downstream calls (DB, API); raise/align timeouts
       or fix the slow dependency (don't just bump the timeout)
```

Gotcha: A 504 often masks a downstream problem — the backend is waiting on a slow DB/API. Bumping the gateway timeout hides the symptom; find the actual slow dependency. A 502 frequently traces to the LB-idle-timeout/keep-alive reset pattern (backend closed the pooled connection).

Interview tip: The crisp mapping: 502 = bad/garbage response (backend broke), 503 = no healthy backend (capacity/health), 504 = backend too slow (timeout). Tie 504 to chasing the downstream dependency rather than raising timeouts, and 502 to the connection-reset/keep-alive pattern — that connects this to the TCP section and shows systems thinking.

📚 **Official docs**

- RFC 9110: HTTP Semantics (status codes, methods)

Q80 **Explain HTTP idempotency and safe methods. How do idempotency keys make retries safe?** HTTP/TLS

🎙 **Spoken answer · ~87 s**

Idempotency means applying the same request multiple times has the same effect as applying it once, and it's the foundation of safe retries in distributed systems. Safe methods have no side effects: GET, HEAD, and OPTIONS. Idempotent methods include GET, PUT, and DELETE: repeating a PUT sets the same value, repeating a DELETE leaves the resource deleted. POST is not idempotent by default, as each call may create another resource or charge a card again. Retries happen constantly, due to timeouts, network failures, and client or proxy logic, and a timeout doesn't tell the client whether the server processed the request, so retrying a non-idempotent operation risks duplicates, like double charges. Idempotency keys solve this: the client generates a unique key per logical operation and sends it in a header; the server records the key with the result the first time, and on any repeat with the same key returns the stored result instead of executing again. Implementation details: store keys with the response, with an expiry; make the check-and-set atomic to handle concurrent duplicates; scope keys per client; and validate that a reused key comes with the same payload. Combine with retries using exponential backoff and jitter. Make message consumers idempotent too, since at-least-once delivery means duplicates.

Likely follow-ups

- Why is POST not idempotent by default?
- How must the server store an idempotency key to be safe under concurrency?

certs.eknathalabs.com

Idempotency — the same request applied multiple times has the same effect as once — is the foundation of safe retries in distributed systems.

METHOD SEMANTICS

```
Safe (no side effects)     : GET, HEAD, OPTIONS
Idempotent (repeat = same)  : GET, PUT, DELETE  (PUT sets a value; repeating = same)
NOT idempotent              : POST (each call may create a new resource)
```

WHY IT MATTERS FOR RETRIES

```
# A timeout is ambiguous: did the request succeed and the response get lost,
# or did it fail? Safe to retry an idempotent op; retrying a POST might
# double-charge / double-create.
```

IDEMPOTENCY KEYS

```
# Client sends a unique key with a POST:  Idempotency-Key: <uuid>
# Server records the key + result; a retry with the SAME key returns the
# stored result instead of re-executing -> POST becomes safe to retry.
```

Gotcha: Blind retries (LB/client auto-retry, mesh retry policies) on non-idempotent endpoints cause duplicates — double payments, duplicate orders. Retries are only safe on idempotent operations or POSTs protected by idempotency keys. This is why payment APIs require an idempotency key.

Interview tip: The senior framing: a timeout is ambiguous, so retries are mandatory for reliability but dangerous on non-idempotent ops. The fix is idempotency keys (server dedupes by key) — exactly how Stripe et al. make POST retries safe. Mention that mesh/LB auto-retry policies must be scoped to idempotent methods, or they create duplicates under load.

📚 **Official docs**

- RFC 9110: HTTP Semantics (status codes, methods)
- Microsoft: Cloud design pattern – Retry

certs.eknathalabs.com

Q81 Explain cookies, SameSite, and CORS. What problems does each solve?

Network security

🎙 Spoken answer · ~89 s

These browser mechanisms govern credentials and cross-origin behaviour, and they're a frequent source of works locally, breaks in production. Cookie attributes: Secure means the cookie is sent only over HTTPS; HttpOnly makes it unreadable by JavaScript, mitigating token theft via XSS; SameSite controls whether it is sent on cross-site requests: Strict never, Lax only on top-level navigations such as clicking a link, and None always, which requires Secure. SameSite mitigates cross-site request forgery. Domain and path scope where it's sent, and expiry controls lifetime. CORS, cross-origin resource sharing, addresses a different problem: the same-origin policy prevents a page from one origin reading responses from another. CORS lets a server opt in via headers such as Access-Control-Allow-Origin, allowed methods and headers, and whether credentials are allowed. For non-simple requests, the browser sends a preflight OPTIONS request first, and the server must answer correctly. Typical production bugs: a frontend on a different domain from the API failing with CORS errors because the allowed origin is wrong or the preflight isn't handled by the load balancer or gateway; credentials mode requiring a specific origin instead of a wildcard; cookies not sent because of SameSite or missing Secure; and cookies set on the wrong domain. CORS is enforced by browsers, not servers, so it isn't an authentication mechanism.

Likely follow-ups

- Why does a wildcard origin not work with credentials?
- What is a CORS preflight request?

These browser mechanisms govern how credentials and cross-origin requests behave — central to web security and a frequent source of "works locally, breaks in prod" bugs.

COOKIE ATTRIBUTES

```
Secure   : only sent over HTTPS
HttpOnly : not readable by JavaScript (mitigates XSS token theft)
SameSite : Strict / Lax / None — controls cross-site sending (CSRF defense)
           None requires Secure; Lax is the modern default
```

SAMESITE (CSRF DEFENSE)

`SameSite=Lax` (default) means cookies aren't sent on cross-site subrequests (e.g. a forged POST from another site), blunting CSRF. `None` allows cross-site (needed for third-party/embedded contexts) but must be `Secure`.

CORS (CROSS-ORIGIN RESOURCE SHARING)

```
# Browser enforces Same-Origin Policy; CORS is how a server OPTS IN to
# cross-origin JS requests via response headers:
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Credentials: true   # to send cookies cross-origin
# Preflight: browser sends OPTIONS first for non-simple requests
```

Gotcha: CORS is enforced by the **browser**, not the server — a CORS error means the server didn't return the right allow headers, not that the request was blocked server-side (curl/Postman ignore CORS, which is why "it works in Postman but fails in the browser"). And `Allow-Origin: *` can't be combined with credentialed requests.

Interview tip: Clarify that CORS is a browser-side relaxation of the same-origin policy (server opts in via headers; the preflight OPTIONS catches people out), and SameSite cookies are the modern CSRF defense. The "works in Postman, fails in browser" tell for

certs.eknathalabs.com

CORS, and HttpOnly+Secure+SameSite as the cookie hardening trio, are the practical points.

📚 **Official docs**

- Mozilla Observatory / HTTP security headers (MDN)

Q82 **How does HTTP caching work? Explain Cache-Control, ETag, and the validation flow.** HTTP/TLS

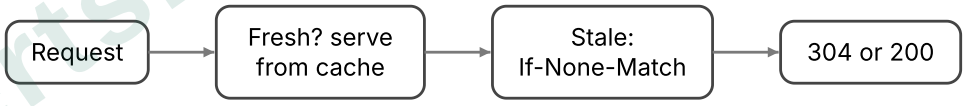
🎙 **Spoken answer · ~88 s**

HTTP caching avoids refetching unchanged content, using freshness directives and validators. Freshness is set by Cache-Control: max-age says the response is fresh for that many seconds, and can be served from cache without contacting the origin. no-cache is often misunderstood: it means the cache must revalidate with the origin before using the stored copy, not that it can't be stored; no-store means don't store at all, used for sensitive data. public and private say whether shared caches such as CDNs may store it, and s-maxage sets a shared-cache lifetime. Validators let a cache check whether a stale copy is still valid cheaply: the server sends an ETag, a version identifier, or Last-Modified; the client later sends If-None-Match or If-Modified-Since; and if unchanged, the server answers 304 Not Modified with no body, saving bandwidth. Misconfiguration causes problems: no caching headers leads to heuristic caching or none; long max-age on files that change causes stale content, so use content-hashed filenames with a very long lifetime for assets and short or revalidated caching for HTML entry points; missing Vary headers can serve the wrong variant, such as compressed content to a client that can't handle it; and caching personalised responses publicly is a security risk. I verify headers with curl and test the revalidation flow.

Likely follow-ups

- What is the difference between no-cache and no-store?
- How does an ETag validation save bandwidth?

🗺 **HTTP cache validation**



HTTP caching avoids re-fetching unchanged content, using freshness directives and validators. Misconfiguring it causes either stale content or no caching at all.

FRESHNESS (CACHE-CONTROL)

```
Cache-Control: max-age=3600          # fresh for 1 hour, serve from cache
Cache-Control: no-cache              # MUST revalidate before using (not "don't cache")
Cache-Control: no-store              # truly don't cache (sensitive data)
Cache-Control: private / public      # browser-only vs shared (CDN) caches
Cache-Control: immutable             # never revalidate (fingerprinted assets)
```

VALIDATION (ETAG / LAST-MODIFIED)

certs.eknathalabs.com

```
# When stale, the client revalidates instead of re-downloading:
Response:  ETag: "abc123"
Next req:  If-None-Match: "abc123"
Server:    304 Not Modified  (no body — cheap!)  OR  200 + new body
```

Common confusion: `no-cache` does NOT mean "don't cache" — it means "cache but always revalidate." Use `no-store` for truly sensitive data. The classic strategy: long `max-age + immutable` for fingerprinted static assets (`app.a1b2c3.js`), and `no-cache` for HTML that references them.

Interview tip: Two points: the freshness-then-validation model (max-age serves from cache; on expiry, ETag/304 avoids re-downloading unchanged bodies), and the `no-cache` vs `no-store` distinction that trips everyone up. The fingerprinted-assets-immutable + no-cache-HTML pattern is the production caching strategy worth naming.

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

Q83 **What is mutual TLS (mTLS) and where do you use it in production architectures?** Network security

🎙 **Spoken answer · ~85 s**

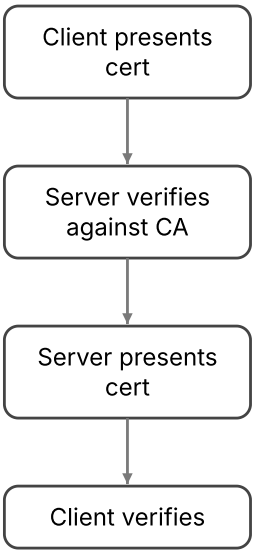
In standard TLS only the server proves its identity, with a certificate the client verifies. Mutual TLS adds client authentication: both sides present certificates and verify each other, establishing strong two-way identity and encrypted transport. It replaces or supplements passwords and API tokens with cryptographic identity. Where it's used in production: service-to-service communication inside microservice architectures, commonly via a service mesh that issues short-lived certificates to workloads and rotates them automatically, giving zero-trust internal networking; securing APIs between business partners or from devices, where each client has its own certificate; connections to databases, message brokers, and Kubernetes components, which use mTLS internally between the API server, kubelets, and etcd; and as the basis of identity-aware access. Benefits: strong authentication tied to workload identity, protection against impersonation and eavesdropping, and a good fit for authorisation policies based on identity. Costs: certificate lifecycle management is the hard part, covering issuance, rotation, revocation, and CA protection; debugging is harder because traffic is encrypted and failures show as handshake errors; clock skew and expired certificates cause outages; and clients must be able to hold and present certificates. Mitigation is automation with cert-manager, SPIFFE or mesh CAs, short lifetimes, and monitoring of expiry. Roll out gradually, using permissive mode first.

Likely follow-ups

- Why are short-lived certificates preferred for mTLS?
- What breaks first when mTLS certificates expire?

certs.eknathalabs.com

🗺 **mTLS**



In standard TLS only the server proves its identity. mTLS adds client authentication — both sides present and verify certificates — establishing strong two-way identity.

HOW IT DIFFERS FROM ONE-WAY TLS

```
One-way TLS : server presents cert; client verifies it (typical web HTTPS)
mTLS        : server AND client present certs; each verifies the other
              -> the connection itself authenticates both parties
```

WHERE IT'S USED

```
- Service-to-service auth in a mesh (Istio/Linkerd auto-mTLS, SPIFFE identity)
- Zero-trust internal networks: identity per workload, not network location
- Machine-to-machine / API clients where a shared secret is too weak
- IoT/device auth; high-security B2B APIs
```

OPERATIONAL COST

You need a certificate authority and, crucially, **rotation/lifecycle** — short-lived certs auto-issued and rotated (a mesh handles this; doing it by hand is painful).

Gotcha: The hard part of mTLS is certificate lifecycle, not the protocol — expired client certs cause outages, and manual rotation doesn't scale. This is why service meshes (which issue short-lived per-workload certs automatically) are the usual delivery vehicle for mTLS.

Interview tip: Frame mTLS as the core of zero-trust service identity — authenticate the workload via cert, not the network it sits on. The senior nuance is that the protocol is easy but cert lifecycle/rotation is the operational challenge, which is exactly why meshes (with SPIFFE identities + auto-rotation) exist. Ties directly to the service-mesh and zero-trust topics.

📚 **Official docs**

- Cloudflare Learning: mutual TLS

certs.eknathalabs.com

Q84 **What does a WAF do, and what are its limits? How does it relate to L7 DDoS protection?**

Network security

🎙 Spoken answer · ~84 s

A web application firewall inspects HTTP and HTTPS requests at layer seven to block application-layer attacks, but it's a filter, not a substitute for secure code. What it blocks: common injection patterns such as SQL injection, cross-site scripting, and command injection from the OWASP Top Ten; known bad bots and scanners; suspicious request shapes; and abusive request rates, with rules, managed rule sets, and custom rules. It also supports virtual patching, blocking exploit patterns for a known vulnerability while the real fix is prepared. Limits: it can be bypassed by obfuscation and novel attacks, it produces false positives that block legitimate users unless tuned, so deploy in detect mode first and then enforce; it can't understand business logic flaws like broken authorisation; it sees only what is decrypted at its position, so it must sit after TLS termination; and it adds latency. Relation to layer-seven DDoS protection: floods of seemingly legitimate requests are handled by rate limiting, bot management, and challenges, features often delivered in the same product, but the WAF isn't designed to absorb volumetric network floods, which need upstream scrubbing and anycast capacity. Best practice: layer it with secure coding, input validation, patching, and monitoring, and review its logs regularly.

Likely follow-ups

- Why deploy a WAF in detect mode first?
- What is virtual patching?

A Web Application Firewall inspects HTTP(S) requests at L7 to block application-layer attacks — but it's a filter, not a substitute for secure code.

WHAT A WAF BLOCKS

```
- Common injection patterns: SQLi, XSS, command injection (OWASP Top 10)
- Known bad bots / scanners; suspicious request shapes
- Rate-based rules; geo/IP reputation blocking
- Virtual patching: block a CVE's exploit pattern before you can patch the app
```

LIMITS

```
- Signature/rule-based -> bypassable with novel/encoded payloads
- False positives block legitimate traffic (tuning burden)
- Can't fix the underlying vulnerability — it's defense in depth, not a fix
- Sees only what it can decrypt (must terminate or inspect TLS)
```

RELATION TO L7 DDOS

WAFs often pair with L7 DDoS protection (rate limiting, bot management, challenge pages) since application-layer floods (expensive queries, login bursts) look like valid HTTP and can't be stopped by L3/4 volumetric defenses alone.

Interview tip: Position the WAF as defense-in-depth / virtual patching, not a security fix — secure code still matters, and rule-based WAFs are bypassable and noisy. The strong nuance: L7 DDoS (app-layer floods of valid-looking requests) needs WAF/rate-limiting/bot management, distinct from L3/4 volumetric DDoS handled upstream by the network/CDN — connecting to the next question.

certs.eknathalabs.com

📚 **Official docs**

- Cloudflare Learning: DDoS attacks

Q85 **Explain L3/L4 vs L7 DDoS attacks and the layered defenses against each.** Network security

🎙 **Spoken answer · ~85 s**

DDoS attacks target different layers, so the defences differ. Layer three and four attacks are volumetric or protocol-based: UDP and ICMP floods, amplification through DNS or NTP reflection, and SYN floods. The goal is to saturate bandwidth or exhaust connection state. Defences: upstream scrubbing and anycast capacity at the provider or CDN, since no single origin can absorb a large flood; SYN cookies; rate limiting and filtering of spoofed or unneeded protocols; and hiding the origin, so attackers can't bypass the edge. Layer seven attacks exhaust application resources with seemingly valid requests: HTTP floods, slow-connection attacks like slowloris, and expensive-endpoint abuse such as search or login. The goal is to exhaust CPU, threads, or the database. Defences: a WAF with rate limiting per client, bot detection and challenges, caching so floods hit the cache, timeouts and connection limits, protecting expensive endpoints with authentication or queues, and autoscaling with cost caps. Across both: baseline normal traffic to detect anomalies, have a runbook and contacts at the provider, design for graceful degradation, and test with game days. A layered approach is needed because a defence at one layer is blind to the other. Network scrubbing doesn't see application semantics, and a WAF can't stop a bandwidth flood.

Likely follow-ups

- Why can't a WAF stop a volumetric flood?
- What makes slowloris effective?

DDoS attacks target different layers, and the defenses differ accordingly — volumetric flooding versus application exhaustion.

L3/L4 (VOLUMETRIC / PROTOCOL)

```
- UDP/ICMP floods, amplification (DNS/NTP reflection), SYN floods
- Goal: saturate bandwidth or exhaust connection state
- Defense: upstream scrubbing/CDN absorbs volume, SYN cookies, anycast to
  spread load across many POPs, provider-level DDoS protection
```

L7 (APPLICATION)

```
- HTTP floods, slowloris (slow requests holding connections), hitting
  expensive endpoints (search, login, report generation)
- Looks like valid traffic -> can't filter by volume alone
- Defense: rate limiting, bot detection/challenges (CAPTCHA/JS), WAF rules,
  caching/CDN to absorb, autoscaling as a buffer (with cost guardrails)
```

WHY ANYCAST HELPS

Anycast advertises the same IP from many locations; volumetric attack traffic is distributed across global POPs instead of hammering one site, and a CDN absorbs/scrubs it close to the source.

Gotcha: Autoscaling to absorb an L7 flood can rescue availability but blow up your bill (a "denial of wallet" attack). Pair autoscaling with rate limits and cost guardrails so an attack doesn't simply convert downtime into a huge invoice.

certs.eknathalabs.com

Interview tip: The clean split: L3/4 = volume (absorb/scrub upstream with anycast + CDN + provider DDoS), L7 = valid-looking app requests (rate limit, bot management, WAF, caching). Naming anycast for distributing volumetric load and the "denial of wallet" autoscaling risk are both senior-level observations.

📚 **Official docs**

- Cloudflare Learning: OSI model
- Cloudflare Learning: DDoS attacks
- Cloudflare Learning: Load balancing

Q86 **What cloud-native network security controls would you layer for a defense-in-depth posture?** Cloud networking

🎙 **Spoken answer · ~83 s**

Cloud network security is layered, because no single control suffices; the controls stack from the edge down to the workload. At the edge: a CDN with DDoS protection and a WAF to absorb, scrub, and filter at the perimeter. At the network level: VPC segmentation into public, private, and data subnets, with network ACLs as a coarse subnet-level guard, and route tables that enforce what can reach where. At the instance level: security groups as stateful, least-privilege firewalls, referencing other groups for tiered access. For internal traffic: Kubernetes network policies or service mesh authorisation for micro-segmentation, default-deny between services, and mTLS for encryption and identity. For egress: NAT gateways with filtering, proxies or firewalls with domain allow-lists, and private endpoints for cloud services so traffic doesn't traverse the internet. For access: no public SSH, using session managers or identity-aware proxies, and VPN or zero-trust access for admins. For visibility: flow logs, DNS logs, WAF logs, and intrusion detection, centralised and monitored. For data: encryption in transit and at rest. For governance: infrastructure as code with policy checks, and guardrails that prevent public exposure. Each layer assumes the others may fail, so a breach in one does not become full compromise.

Likely follow-ups

- Why default-deny between services?
- How do private endpoints reduce exposure?

Cloud network security is layered — no single control suffices. You stack controls from the edge down to the workload.

THE LAYERS (EDGE TO WORKLOAD)

```
Edge      : CDN + DDoS protection + WAF (absorb/scrub/filter at the perimeter)
Network    : VPC segmentation (public/private/data subnets), NACLs (subnet guardrails)
Instance   : Security Groups (stateful, least-privilege per service)
Egress     : controlled outbound (NAT GW + egress filtering / no open 0.0.0.0/0 out)
Private     : VPC endpoints / PrivateLink so traffic to cloud services stays private
Workload    : mTLS / service mesh, NetworkPolicies (K8s), identity-based auth
```

KEY PRINCIPLES

```
- Least privilege both INBOUND and OUTBOUND (egress filtering stops exfil/beaconing)
- Private by default: no public IPs unless required; use private endpoints
- Segment so a breach in one tier can't reach the data tier
- Identity-based (zero-trust) over network-location trust
```

certs.eknathalabs.com

Commonly missed: Egress control. Most teams lock down inbound but leave outbound wide open (0.0.0.0/0) — which is exactly what malware uses to beacon out and exfiltrate. Default-deny egress with an allowlist is a high-value, often-skipped control.

Interview tip: Structure the answer edge-to-workload (CDN/WAF -> VPC segmentation -> SG/NACL -> egress control -> private endpoints -> mTLS/NetworkPolicy) and stress two things teams skip: egress filtering (stops exfiltration/beaconing) and private endpoints (PrivateLink keeps cloud-service traffic off the public internet). Defense-in-depth + least-privilege-both-directions is the framing.

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

Q87 **Explain zero-trust networking versus the traditional perimeter model.** Network security

🎙 **Spoken answer · ~84 s**

The perimeter model, castle and moat, builds a hard outer wall with a firewall and VPN, and trusts everything inside. Its problem is that one breach, a phished laptop or a compromised workload, gives broad implicit access, and attackers move laterally across the flat internal network. Cloud, remote work, and SaaS also dissolved the perimeter: users and workloads are everywhere, so there is no single inside. Zero trust assumes the network is hostile and verifies every request regardless of location. The principles: authenticate and authorise every request using strong identity of users and workloads, with MFA and device health; grant least privilege, scoped narrowly and ideally just in time; encrypt all traffic, including internal; segment the network so a compromise stays contained; and continuously monitor and log decisions, evaluating context such as device posture, location, and behaviour. In practice: identity-aware proxies replace broad VPN access for applications; service meshes provide mTLS and authorisation policies between services; network policies default-deny; and short-lived credentials replace static keys. It's a strategy, not a product, and adoption is incremental. Trade-offs: more operational complexity, certificate and identity management overhead, and the need for good policy definition and observability. It complements, rather than replaces, basic network hygiene.

Likely follow-ups

- Why does a flat internal network enable lateral movement?
- What replaces broad VPN access in a zero-trust design?

The perimeter model trusts anything inside the network; zero-trust trusts nothing by default and verifies every request regardless of location.

PERIMETER ('CASTLE AND MOAT')

```
# Hard outer wall (firewall/VPN); once inside, broad implicit trust.
# Problem: a single breach -> lateral movement across the flat internal network.
# VPN access = "you're inside" = trusted -> over-broad access.
```

ZERO-TRUST ('NEVER TRUST, ALWAYS VERIFY')

certs.eknathalabs.com

```
- Identity-based: authenticate + authorize EVERY request (user AND workload)
- Network location grants NO trust (inside == outside)
- Least-privilege, per-resource access; continuous verification
- Micro-segmentation: limit blast radius if one workload is compromised
- mTLS for service identity; short-lived credentials
```

IN PRACTICE

Identity-aware proxies (BeyondCorp-style) replace VPNs for user access; service mesh mTLS + NetworkPolicies enforce workload-to-workload zero-trust; secrets are short-lived and identity-bound.

Interview tip: The headline contrast: perimeter trusts location (and a breach moves laterally freely); zero-trust trusts verified identity per request, so a compromised workload can't roam. Concretely: replace VPN-grants-everything with identity-aware access, and enforce service identity via mTLS + micro-segmentation. This ties together mTLS, NetworkPolicy, and least-privilege egress from earlier answers.

📚 **Official docs**

- NIST Zero Trust Architecture (SP 800-207)

Q88 **What is a reverse proxy vs a forward proxy vs an API gateway? Give concrete uses.** HTTP/TLS

🎙 **Spoken answer · ~86 s**

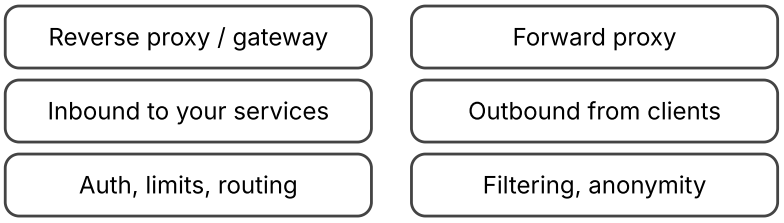
All three sit between clients and servers, but they face different directions and serve different roles. A forward proxy sits in front of clients and proxies their outbound requests: corporate egress proxies, content filtering, caching, auditing, and anonymity. A reverse proxy sits in front of servers and fronts inbound traffic: TLS termination, load balancing, caching, compression, routing by host or path, and hiding backend topology; nginx, HAProxy, and Envoy are common, and Kubernetes ingress controllers are reverse proxies. An API gateway is a specialised reverse proxy focused on API management: authentication and authorisation, rate limiting and quotas, request and response transformation, routing to microservices, API versioning, usage analytics, and developer portal features. Concrete uses: a forward proxy to restrict which sites servers can reach and log outbound calls; a reverse proxy to expose many internal services through one TLS endpoint and balance load; an API gateway to publish a public API with keys, quotas, and consistent security. The overlap is real, since many products do several jobs, so the distinction is about the role. Considerations: all are potential single points of failure and performance bottlenecks, need redundancy, and add a hop to troubleshoot, requiring client IP preservation via headers. Choose the lightest component that meets the need.

Likely follow-ups

- How does an API gateway differ from a plain reverse proxy?
- Why might servers need a forward proxy?

certs.eknathalabs.com

🗺 **Proxy roles**



All sit between clients and servers, but they face different directions and serve different roles.

THE THREE

```
Forward proxy : sits in front of CLIENTS, proxies their outbound requests
                (corporate egress proxy, content filtering, caching, anonymity)
Reverse proxy : sits in front of SERVERS, fronts inbound requests to backends
                (NGINX/Envoy: TLS termination, LB, caching, routing)
API gateway   : a specialized reverse proxy for APIs — adds auth, rate limiting,
                request transformation, versioning, metering, aggregation
```

CONCRETE USES

```
Forward : employees -> egress proxy -> internet (filter/log/cache outbound)
Reverse : internet -> NGINX -> app servers (terminate TLS, route /api vs /static)
Gateway : clients -> Kong/Apigee/cloud API GW -> microservices
          (JWT validation, per-key rate limits, usage plans, canary routing)
```

Distinction: Forward proxy protects/serves the *client* side (who's making requests); reverse proxy protects/serves the *server* side (what's receiving them). An API gateway is a reverse proxy with API-management features bolted on — don't conflate it with a plain LB.

Interview tip: The crisp mental model: forward proxy faces clients (egress control), reverse proxy faces servers (ingress: TLS/LB/routing), API gateway = reverse proxy + API concerns (auth, rate limit, metering, transformation). Knowing where the gateway adds value over a plain LB (auth, rate limiting, per-consumer plans) shows platform-level thinking.

📚 **Official docs**

- Cloudflare Learning: What is a reverse proxy?
- Amazon API Gateway
- NGINX documentation

oerts.eknathalabs.com

Q89 How do you securely expose an internal service to the public internet?

Network security

🎙 Spoken answer · ~87 s

Exposing an internal service safely is about minimising the attack surface and adding layers between the internet and the workload. The typical path is: internet to a CDN or edge with DDoS protection and a WAF, then a public load balancer as the single managed entry point terminating TLS, then the backend in a private subnet with no public IP, reachable only from the load balancer's security group. Further controls: authenticate at the edge or gateway, with OAuth, API keys, or mTLS depending on the clients; apply rate limiting and request size limits; expose only the specific paths and methods needed, not the whole service; use TLS with modern versions and managed certificates; restrict the origin so it accepts traffic only from the CDN or load balancer; harden the service itself, with patching, least-privilege identities, and input validation; and separate the exposed service from sensitive data stores with network segmentation. Observability matters: access logs, WAF logs, and alerts on anomalies. Consider alternatives that avoid public exposure entirely: private connectivity, VPN, private endpoints, or an identity-aware proxy for internal users and partners. Before going live, run security review and testing, such as a penetration test, and prepare incident response. Keep the exposure under change control and review it periodically.

Likely follow-ups

- Why should the backend have no public IP?
- How do you make sure traffic only reaches the origin via the CDN?

Exposing an internal service safely is about minimizing the attack surface and adding layers between the internet and the workload.

THE LAYERED EXPOSURE

```
Internet
  -> CDN/edge (DDoS protection, WAF, caching)
  -> Public load balancer (TLS termination, single managed entry point)
  -> private subnet backends (NO public IPs on the actual workloads)
# Only the LB is internet-facing; backends live in private subnets.
```

CONTROLS TO APPLY

```
- Terminate TLS at the edge/LB; valid certs; modern TLS only
- WAF + rate limiting in front; bot protection if needed
- Security Groups: backends accept ONLY from the LB's SG (not 0.0.0.0/0)
- AuthN/AuthZ at the gateway (don't rely on "internal = trusted")
- Least-privilege egress from the backends; private endpoints for cloud deps
- Logging/monitoring on the edge for abuse detection
```

Anti-pattern: Slapping a public IP directly on the workload, or opening the backend's security group to `0.0.0.0/0`. The workload should only be reachable *through* the LB, whose SG is the only allowed source — so even if someone learns the backend IP, they can't reach it directly.

Interview tip: The pattern: CDN/WAF -> public LB (TLS) -> private backends, with backend SGs that trust only the LB's SG, plus gateway auth (no implicit internal trust). The key control is that workloads have no public IP and only accept traffic from the LB — single, hardened, observable entry point. This is the production-grade exposure answer.

certs.eknathalabs.com

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

Q90 **Compare gRPC and REST over HTTP. What networking issues does gRPC introduce with load balancers?** HTTP/TLS

🎙 **Spoken answer · ~85 s**

gRPC uses HTTP/2 with binary Protobuf messages for efficient, strongly typed RPC, supports streaming in both directions, generates client and server code from contracts, and has low overhead, which makes it great for internal service-to-service calls. REST is typically JSON over HTTP/1.1 or HTTP/2: human-readable, widely supported by browsers and tools, easy to debug and cache, but larger and without a strict contract unless OpenAPI is used. The networking catch with gRPC is load balancing. gRPC clients keep one long-lived HTTP/2 connection and multiplex many requests over it, so a layer-four load balancer balances connections, not requests: all of a client's requests go to one backend, and when new backends are added or after a scale-out, existing connections don't rebalance, leaving hot and idle instances. Solutions: use a layer-seven load balancer or proxy that understands HTTP/2 and balances per request, such as Envoy, or an ingress that supports gRPC; use client-side load balancing with service discovery; use a service mesh; or set maximum connection age on servers so connections recycle and rebalance. Other issues: browsers can't speak native gRPC without gRPC-Web, health checking needs the gRPC health protocol, deadlines and retries need explicit configuration, and debugging binary traffic requires tooling such as grpcurl.

Likely follow-ups

- Why does an L4 balancer distribute gRPC unevenly?
- What does max connection age do?

gRPC uses HTTP/2 with binary Protobuf for efficient, strongly-typed RPC; REST is typically JSON over HTTP/1.1. The performance win comes with load-balancing gotchas.

GRPC VS REST

```
gRPC : HTTP/2, binary Protobuf, streaming (bi-directional), strict contracts,
       low overhead, great for internal service-to-service
REST : HTTP/1.1 + JSON, human-readable, ubiquitous tooling/caching,
       great for public APIs and browser clients
```

THE LOAD-BALANCING PROBLEM

```
# gRPC multiplexes many calls over ONE long-lived HTTP/2 connection.
# An L4 (connection-level) LB pins that connection to ONE backend ->
# all requests hit one pod; new/scaled pods get no traffic. Uneven load.
```

FIXES

```
- L7 LB that balances per-REQUEST (Envoy, gRPC-aware ingress, service mesh)
- Client-side LB: resolve all endpoints (headless service) + round_robin policy
- Connection max-age to force periodic re-resolution/rebalancing
```

certs.eknathalabs.com

Gotcha: Scaling up a gRPC service often doesn't relieve load — existing HTTP/2 connections never move to the new pods. You must force request-level balancing or connection recycling; adding replicas alone does nothing.

Interview tip: The headline gRPC networking issue: per-connection (L4) balancing breaks HTTP/2 multiplexing, so you need request-level L7 balancing, client-side LB, or connection max-age. This is the same multiplexing problem as the HTTP/2 HOL discussion and ties to load-balancer behavior — connecting the protocol and LB sections shows depth.

📚 **Official docs**

- Cloudflare Learning: Load balancing
- gRPC documentation

Q91 **Compare L4 and L7 load balancing in depth. What can each see and not see?** Load balancing

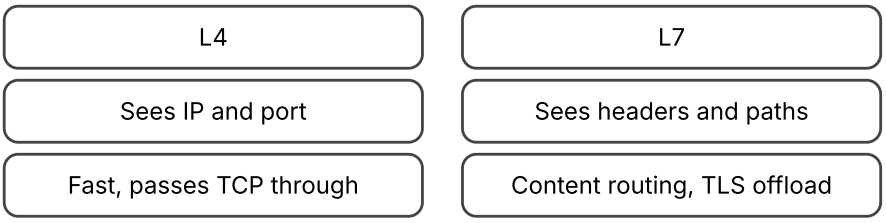
🎙 **Spoken answer · ~86 s**

The layer a load balancer operates at determines what it can route on, and what it costs. A layer-four balancer works on IP and port for TCP or UDP, without parsing the payload. It balances per connection, is very fast with low overhead and high throughput, can preserve the client's source IP, handles any protocol including databases and TLS passthrough, and is simple to scale. What it can't see: URLs, headers, cookies, or HTTP status codes, so it cannot route by path, do canary by header, or give request-level metrics. A layer-seven balancer terminates and parses the protocol, typically HTTP: it can route by host, path, header, or cookie, rewrite requests, terminate TLS, authenticate, apply WAF rules, retry on failure, split traffic by weight, and provide detailed request metrics, access logs, and per-request balancing, important for HTTP/2 and gRPC. The costs: more CPU and latency per request, the client IP is carried in headers, and it only understands the protocols it supports. Layer seven sees decrypted content only if it terminates TLS, which raises the question of where certificates live. I choose layer seven for web and API traffic needing content-based routing and observability, layer four for non-HTTP protocols, extreme throughput, static IPs, or end-to-end encryption.

Likely follow-ups

- What can an L7 balancer do that an L4 cannot?
- Why might you terminate TLS at the load balancer?

🗺 **L4 vs L7 balancing**



The layer a load balancer operates at determines what information it can route on and what it costs in performance and visibility.

L4 LOAD BALANCER

certs.eknathalabs.com

```
- Operates on IP:port (TCP/UDP); doesn't parse the payload
- Can balance by connection/4-tuple; very fast, low overhead, high throughput
- Preserves source IP more easily; protocol-agnostic (any TCP/UDP app)
- CANNOT: route by URL/header/cookie, terminate TLS, inspect HTTP
```

L7 LOAD BALANCER

```
- Parses HTTP: routes by path/host/header/method; terminates TLS
- Can do content-based routing, header rewrite, WAF, sticky by cookie
- Two connections (client<->LB, LB<->backend) -> source IP needs X-Forwarded-For
- More CPU/latency per request; the app-aware choice
```

CHOOSING

L4 for raw TCP/UDP, extreme throughput, non-HTTP protocols, static-IP needs. L7 when you must route on HTTP attributes, centralize TLS, or apply app-layer policy.

Gotcha: Because an L7 LB terminates the connection, the backend sees the LB's IP, not the client's — you must propagate the real client via `X-Forwarded-For` (and trust it carefully). An L4 LB in passthrough preserves source IP but can't do any HTTP-level routing.

Interview tip: The principle: pick the lowest layer that satisfies your routing needs. L4 = fast, protocol-agnostic, source-IP-friendly, no app awareness; L7 = app-aware routing + TLS termination at a CPU/latency cost and source-IP indirection. Real examples (AWS NLB vs ALB) anchor it.

📚 **Official docs**

- Cloudflare Learning: OSI model
- Cloudflare Learning: Load balancing

certs.eknathalabs.com

Q92 **Explain load balancing algorithms (round-robin, least-conn, consistent hashing). When use each?**

Load balancing

🎙 Spoken answer · ~88 s

The algorithm decides which backend receives each request, and the right choice depends on request uniformity and whether I need stickiness. Round robin rotates evenly and works well when requests and backends are uniform. Weighted round robin gives larger backends a larger share, and supports canary splits. Least connections sends to the backend with the fewest active connections, better when request cost varies or connections are long-lived, such as streaming or uploads. Least response time or power of two choices picks between two random backends the less loaded one, which avoids herding and works well in large fleets. IP hash maps a client to a backend, giving rough affinity without cookies, but unevenness with NATed clients. Consistent hashing on a key, such as a user ID or cache key, sends the same key to the same backend and minimises remapping when backends are added or removed, ideal for caches and sharded stateful services, though it can create hot spots unless virtual nodes or bounded loads are used. Random is simple and surprisingly good at scale. Whatever the algorithm, health checks, slow start for new instances, and connection draining matter as much. I verify the choice by measuring per-backend load, latency, and error rates, and adjust when the distribution is skewed.

Likely follow-ups

- Why is least connections better for variable request cost?
- Why does consistent hashing suit caches?

The algorithm decides which backend gets each request; the right choice depends on request uniformity and whether you need stickiness.

THE ALGORITHMS

```
Round-robin     : rotate evenly; great when requests + backends are uniform
Least-connections: send to the backend with fewest active conns;
                   better for variable/long-lived request durations
Weighted        : bias toward bigger/faster backends (heterogeneous fleet, canary)
IP hash / consistent hashing : map a key (client IP, session) to a backend
                   deterministically -> stickiness + cache locality
```

CONSISTENT HASHING — WHY SPECIAL

```
# Plain "hash mod N" reshuffles EVERYTHING when N changes (add/remove a backend).
# Consistent hashing remaps only ~1/N of keys on a change -> minimal disruption.
# Used for cache affinity, sharding, sticky routing that survives scaling.
```

Gotcha: Least-connections beats round-robin when request costs vary widely (round-robin can pile expensive requests on one node). But avoid stickiness/hashing unless you need it — it undermines even distribution and can hot-spot one backend (a heavy client pinned to one node).

Interview tip: Match algorithm to workload: round-robin for uniform short requests, least-conn for variable/long-lived ones, consistent hashing when you need stickiness or cache locality with minimal reshuffling on scale events. Explaining why consistent hashing (only 1/N keys move) beats hash-mod-N on resize is the standout detail.

certs.eknathalabs.com

📚 **Official docs**

- Cloudflare Learning: Load balancing
- Google SRE Book: Load Balancing in the Datacenter
- Microsoft: Sharding pattern

Q93 **How do health checks work, and how can a misconfigured health check cause a cascading outage?** Load balancing

🎙 **Spoken answer · ~81 s**

Load balancers route only to backends that pass health checks, so the check's design is safety-critical: a bad one can take down a healthy fleet. Active checks have the balancer probe an endpoint on each backend periodically; passive checks observe real traffic and mark a backend unhealthy after consecutive errors. Key settings are the interval, timeout, and the healthy and unhealthy thresholds. The classic cascade is a deep health check that tests dependencies like the database: when the database slows, every backend fails its check, the balancer removes all of them, and a partial degradation becomes a total outage. Other mistakes: a timeout shorter than normal latency under load, so overloaded but working instances are ejected and the rest get more load and fail too; thresholds so aggressive that transient blips eject healthy instances; and a shallow check that only tests the port, so broken instances stay in rotation. Best practice: separate liveness from readiness; keep the check cheap and local; handle dependency failure with graceful degradation instead of ejecting everything; use a panic mode or minimum healthy percentage, so the balancer keeps routing if too many targets fail; and test failure scenarios deliberately.

Likely follow-ups

- Why is a deep health check that tests the database dangerous?
- What is panic mode in a load balancer?

Load balancers route only to backends passing health checks. The check's design is safety-critical — a bad one can take down a healthy fleet.

ACTIVE VS PASSIVE

```
Active  : LB periodically probes /health on each backend
Passive : LB observes real traffic (consecutive errors -> mark unhealthy)
Key knobs: interval, timeout, healthy/unhealthy thresholds, the probe path
```

SHALLOW VS DEEP CHECKS

```
Shallow (/healthz returns 200): is the process up? — safe, isolated
Deep (/health checks DB + deps): is everything OK? — DANGEROUS as an LB check
```

THE CASCADE

If the health check is **deep** (verifies a shared dependency like the DB) and that dependency blips, *every* backend fails its check simultaneously, the LB marks the entire fleet unhealthy and has nowhere to route — turning a brief dependency hiccup into a total outage.

Rule: LB health checks should be shallow ("is this instance serving?"), NOT deep ("is the database up?"). A shared-dependency check shared across all instances is a single point of failure that fails them all at once. Keep dependency checks for readiness/alerting, not for

certs.eknathalabs.com

the load balancer's route decision.

Interview tip: This mirrors the Kubernetes liveness-probe trap: a health check that depends on a shared downstream takes out the whole fleet when that downstream blips. Keep LB checks shallow and instance-local; surface dependency health via separate signals. Also mention thresholds — too-aggressive checks flap healthy nodes out under transient load.

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

Q94 **What is session affinity (sticky sessions) and why is it considered an anti-pattern?** Load balancing

🎙 **Spoken answer · ~86 s**

Session affinity pins a client to the same backend across requests, using a cookie that names the backend or a hash of the source IP. It's sometimes necessary, but undermines what load balancing offers, which is why it's called an anti-pattern. The problems: uneven load, since heavy users or many clients behind one NAT address land on one backend; fragile failover, because when the pinned backend dies, those users lose their session state; hard deployments and scaling, since draining a backend means waiting for sessions to end and new instances get no traffic from existing users; and it hides state in server memory, a design smell. The better approach is to make the application stateless: keep session data in a shared store such as Redis or a database, or in signed tokens like JWTs, so any backend can serve any request. Then load balancing is even, instances are disposable, autoscaling works properly, and failures are invisible to users. When affinity is legitimately needed, for example WebSockets, caches with local state, or legacy applications, I use cookie-based affinity with a bounded lifetime instead of IP hash, plan for backend loss, keep sessions recoverable, and monitor per-backend load. Document it as technical debt with a plan to remove it.

Likely follow-ups

- What alternatives remove the need for sticky sessions?
- Why is IP-hash affinity unreliable?

Session affinity pins a client to the same backend across requests (via cookie or IP hash). It's sometimes necessary but undermines the benefits of load balancing.

HOW IT'S DONE

```
Cookie-based : LB sets a cookie naming the backend; routes that client back to it
IP-based     : hash source IP -> same backend (breaks behind NAT/shared IPs)
```

WHY IT'S AN ANTI-PATTERN

```
- Uneven load: long-lived sticky sessions hot-spot some backends
- Scaling is blunted: new backends get only NEW sessions, not existing load
- Resilience hit: a backend dying drops all its sticky sessions' state
- Deploys/rollouts disrupt pinned users
```

THE BETTER PATTERN

Make backends **stateless** — externalize session state to a shared store (Redis/Memcached) or use signed stateless tokens (JWT). Then any backend can serve any request; no stickiness needed.

certs.eknathalabs.com

Gotcha: IP-hash affinity breaks badly behind NAT/CGNAT — many users share one source IP and all pin to one backend (hot spot), while a single user behind round-robin NAT may bounce backends. Cookie affinity is more reliable but still carries the load-skew and resilience costs.

Interview tip: The senior answer: stickiness is a crutch for stateful backends — fix the root cause by externalizing state (Redis/JWT) so the tier is stateless and any node serves any request. If you must use affinity (e.g. legacy app, WebSocket), prefer cookie-based over IP-based and acknowledge the load-skew/resilience trade-offs.

📚 **Official docs**

- Cloudflare Learning: Load balancing
- Microsoft: Anti-patterns (performance antipatterns)

Q95 **How do you load balance WebSockets and other long-lived connections?** Load balancing

🎙 **Spoken answer · ~87 s**

WebSockets are long-lived, stateful, full-duplex connections, and they break assumptions load balancers make about short request and response cycles. The challenges: load is decided at connect time and never rebalances while connections persist for minutes or hours, so scale-out doesn't help existing connections; a backend restart or deploy drops all its sockets, causing reconnect storms; idle timeouts on balancers and proxies close quiet connections; and the upgrade handshake must be supported end to end. Solutions: use a balancer that supports the WebSocket upgrade and configure generous idle timeouts, with ping and pong heartbeats from the application to keep connections active and detect dead peers; use least connections instead of round robin so new connections go to the least loaded instance; implement client reconnection with exponential backoff and jitter to avoid thundering herds; drain gracefully during deploys by stopping new connections, signalling clients to reconnect elsewhere, and giving a termination grace period; keep connection state out of the process where possible, by using a shared pub/sub backplane such as Redis so a message can reach a client whichever backend holds its socket; and size capacity by concurrent connections and memory per connection, including file descriptor limits. Monitor connection counts per instance, and consider rebalancing by periodically closing old connections.

Likely follow-ups

- Why do reconnect storms happen after a deploy?
- How does a pub/sub backplane help WebSocket scaling?

WebSockets are long-lived, stateful, full-duplex connections — they break assumptions that load balancers make about short request/response cycles.

THE CHALLENGES

```
- Connections persist for minutes/hours -> load set at connect time, never rebalances
- A backend deploy/restart drops all its live sockets (clients must reconnect)
- LB idle timeouts can kill a quiet-but-active socket -> need keep-alive/ping-pong
- Scaling up doesn't move existing connections to new nodes
```

HANDLING THEM

certs.eknathalabs.com

```
- Ensure the LB supports WS upgrade (HTTP/1.1 Upgrade) and long idle timeouts
- App-level ping/pong heartbeats to keep the connection alive < LB idle timeout
- Least-connections balancing (not round-robin) since connections are long-lived
- Graceful drain on deploy: stop new connections, let existing ones finish/migrate
- Externalize state + use a pub/sub backplane (Redis) so any node can serve any client
```

Gotcha: Because connections are assigned at connect time and never move, a rolling deploy or scale-up leaves load badly skewed (old nodes full, new nodes empty) until clients reconnect. Plan reconnection (client backoff + jitter) so a mass reconnect after a deploy doesn't thundering-herd one node.

Interview tip: Key points: enable WS upgrade + long/aligned idle timeouts, heartbeat to beat the idle timeout, least-conn over round-robin, connection draining on deploy, and a pub/sub backplane for cross-node fan-out. The reconnect-thundering-herd risk (and client jitter to avoid it) is the detail that shows you've run real-time systems at scale.

📚 **Official docs**

- Cloudflare Learning: Load balancing

Q96 **What is connection draining (graceful deregistration) and why is it essential for zero-downtime deploys?** Load balancing

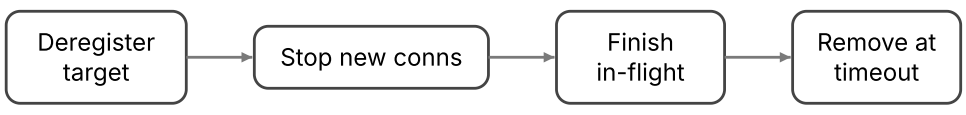
🎙 **Spoken answer · ~86 s**

Connection draining, or graceful deregistration, lets a backend finish its in-flight requests before it's removed from the pool, so deploys and scale-in don't drop requests. Without it, deregistering a backend instantly cuts off requests mid-response, and users see errors or resets on every deploy. With draining, the load balancer stops sending new requests to the backend but keeps existing connections open for a configured timeout, so in-flight work completes, and then removes it. For zero-downtime, it must work with the application: on SIGTERM the application stops accepting new connections, finishes in-flight requests, and exits within the grace period; readiness should flip to not-ready first so traffic stops; and the timeouts must be consistent, the load balancer's deregistration delay should be at least as long as the longest typical request, and the orchestrator's termination grace period longer than that. In Kubernetes, there's a race between endpoint removal and pod termination, so a preStop sleep gives time for the change to propagate. For long-lived connections like WebSockets, draining takes longer, so signal clients to reconnect. Also watch for keep-alive connections that the client keeps reusing: send connection close headers during drain. Test by deploying under load and watching for errors. Set realistic timeouts, since too long slows deployments.

Likely follow-ups

- Why does Kubernetes need a preStop delay?
- What must the deregistration delay be relative to request duration?

🗺 **Connection draining**



certs.eknathalabs.com

Connection draining lets a backend finish in-flight requests before it's removed from the pool, preventing dropped requests during deploys and scale-down.

THE PROBLEM WITHOUT IT

```
# Deregister a backend instantly -> in-flight requests are cut off mid-response
# -> users see errors/resets during every deploy or scale-in event.
```

HOW DRAINING WORKS

```
1. Mark the target "draining" -> LB stops sending NEW requests to it
2. Existing in-flight requests are allowed to COMPLETE (up to a drain timeout)
3. After they finish (or timeout), the target is fully deregistered/terminated
```

TIES TO THE PLATFORM

This is the LB side of graceful shutdown: it must pair with the app handling SIGTERM (stop accepting, finish in-flight) and, in Kubernetes, a `preStop` delay so endpoint removal propagates before the process exits.

Gotcha: The drain timeout must exceed your longest legitimate request, or long requests still get cut. And there's a race: the LB must remove the target from rotation *before* the process stops — without a small delay (preStop sleep / deregistration delay), the LB may still send a request to a shutting-down instance.

Interview tip: Connect it to the full zero-downtime chain: LB connection draining + app SIGTERM handling + (in K8s) preStop delay so deregistration propagates first. The two gotchas — drain timeout sized to longest request, and the dereg-before-stop race — are exactly what separates "I clicked deploy" from "I designed for zero downtime."

📚 **Official docs**

- Kubernetes: Safely drain a node

certs.eknathalabs.com

Q97 **Explain VPC anatomy: subnets, route tables, internet gateway, and NAT gateway.** Cloud networking

🎙 Spoken answer · ~85 s

A VPC is your isolated virtual network in the cloud, with its own address range, such as 10.0.0.0/16. Subnets are slices of that range, each tied to one availability zone; whether a subnet is public or private is decided by routing, not by a property. A route table says where traffic goes: the main components are the local route for VPC-internal traffic and a default route. An internet gateway provides two-way connectivity between the VPC and the internet; a public subnet has a route to it and instances with public addresses. A NAT gateway lives in a public subnet and gives private subnets outbound-only internet access, translating their private addresses to its own public address, without allowing unsolicited inbound connections. A typical layout has public subnets in each AZ for load balancers and NAT gateways, private subnets for application servers, and isolated data subnets with no internet route for databases. Security groups and network ACLs sit on top. Design points: one NAT gateway per AZ for resilience and to avoid cross-AZ charges; endpoints for cloud services so traffic doesn't traverse NAT; non-overlapping CIDRs for peering; and flow logs for visibility. Understanding the routing table explains most of why something can or can't reach the internet.

Likely follow-ups

- What makes a subnet public rather than private?
- Why use one NAT gateway per AZ?

A VPC is your isolated virtual network in the cloud. A handful of components determine what's public, what's private, and how traffic flows.

THE COMPONENTS

```
VPC          : the isolated network + its CIDR (e.g. 10.0.0.0/16)
Subnet       : a CIDR slice, tied to one AZ; "public" or "private" by ROUTING
Route table  : per-subnet rules deciding where traffic goes
Internet GW  : enables bidirectional internet for PUBLIC subnets
NAT Gateway  : lets PRIVATE subnets egress to the internet (outbound only)
```

WHAT MAKES A SUBNET PUBLIC VS PRIVATE

```
# It's the ROUTE TABLE, not a flag:
Public subnet  : route 0.0.0.0/0 -> Internet Gateway  (+ instances have public IPs)
Private subnet : route 0.0.0.0/0 -> NAT Gateway        (outbound only, no inbound)
# Typical 3-tier: public (LB) / private (app) / private (data)
```

Gotcha: A subnet is "public" only because its route table points 0.0.0.0/0 at the Internet Gateway AND the instance has a public IP — both are required. A common mistake: instance in a "public" subnet but no public IP (or vice versa), so connectivity silently fails. The NAT Gateway is one-directional: private instances reach out, but the internet can't reach in.

Interview tip: The insight that impresses: public-vs-private is defined by the route table (IGW vs NAT GW), not an attribute. Describe the standard 3-tier layout (public LB tier, private app tier, private data tier) and that NAT GW is egress-only. Mention NAT GW cost/data-transfer charges as a FinOps consideration — a frequent surprise on cloud bills.

certs.eknathalabs.com

📚 **Official docs**

- RFC 3022: Traditional NAT
- Amazon VPC: What is Amazon VPC?
- Amazon VPC: NAT gateways

Q98 **What are VPC endpoints / PrivateLink and why use them instead of going over the internet?** Cloud networking

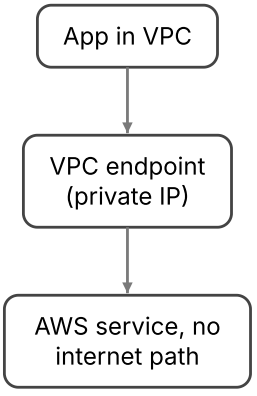
🎙 **Spoken answer · ~85 s**

VPC endpoints let resources reach cloud services, and partner or your own services, privately, without traversing the public internet or a NAT gateway. In AWS there are two types. Gateway endpoints, for S3 and DynamoDB, are a route-table entry pointing to the service; they have no hourly cost and keep traffic on the provider's network. Interface endpoints, PrivateLink, create an elastic network interface with a private IP in my subnet that fronts the service, so I reach it via a private address and private DNS name; they cost per hour and per gigabyte but work for hundreds of services and for my own services exposed to other VPCs or accounts without peering or overlapping CIDR concerns. Why use them: security, since traffic never leaves the private network and the VPC can have no internet route at all; endpoint policies restrict which resources or principals can be accessed, helping prevent data exfiltration; reduced NAT gateway data processing charges, which can be large for heavy S3 traffic; potentially lower latency and better reliability; and compliance requirements for private connectivity. Considerations: DNS settings must resolve the service name to the endpoint, endpoints are regional and per VPC, interface endpoints need security groups, and cross-region access needs more design.

Likely follow-ups

- How does a gateway endpoint differ from an interface endpoint?
- How do endpoint policies reduce data exfiltration risk?

🗺 **PrivateLink**



VPC endpoints let your resources reach cloud services (and third-party/partner services) privately, without traversing the public internet or a NAT gateway.

THE TWO TYPES (AWS TERMS)

certs.eknathalabs.com

```
Gateway endpoint   : for S3 / DynamoDB — a route-table entry, no cost, stays on-net
Interface endpoint : an ENI with a private IP in your subnet (PrivateLink) for most
                     services + your own/partner services exposed privately
```

WHY USE THEM

```
- Security: traffic to the cloud service never leaves the private network
- No NAT GW needed for that traffic -> avoids NAT data-processing cost
- Avoids the public internet entirely (compliance, reduced exposure)
- Can reach services even in fully private (no-IGW) subnets
```

Gotcha: Without endpoints, a private instance calling the cloud's API/S3 goes out through the NAT Gateway to the public endpoint — incurring NAT data-processing charges and leaving your network. For high-volume S3/API traffic this is both a cost and a security/compliance concern. Gateway endpoints (S3/DynamoDB) are free; use them.

Interview tip: The two wins are security (traffic stays private, never hits the internet) and cost (bypass the NAT Gateway's per-GB charge for cloud-service traffic). Knowing gateway endpoints (S3/DynamoDB, free) vs interface endpoints/PrivateLink (ENI-based, for most services and exposing your own services to other VPCs) is a strong cloud-networking detail.

📚 **Official docs**

- AWS PrivateLink

Q99

How do load balancing, autoscaling, and health checks work together — including the warm-up problem?

Load balancing

🎙 **Spoken answer · ~85 s**

Load balancing, autoscaling, and health checks form the elasticity loop. A metric such as CPU, request rate, or latency crosses a threshold, and the autoscaler adds instances. A new instance boots, starts the application, and registers with the load balancer; the health check passes, and then it begins receiving traffic. The balancer distributes load across healthy members, and health checks remove failing ones. The warm-up problem is that new instances are not immediately useful: they need boot time, image pull, application start, cache warm-up, JIT compilation, and connection pool filling. If the load balancer sends full traffic as soon as the health check passes, a cold instance can be overwhelmed, fail health checks, get removed, and cause thrashing. Mitigations: readiness checks that reflect true readiness; slow start settings that ramp traffic gradually to new targets; instance warm-up periods in the autoscaling policy so metrics from warming instances aren't counted; warm pools or pre-initialised capacity; faster images and startup; scaling on leading indicators like queue depth or request rate, with enough headroom; scheduled or predictive scaling for known peaks; and cooldowns to avoid flapping. Also protect backends with concurrency limits and load shedding. Test by simulating a surge and measuring time to effective capacity.

Likely follow-ups

- What does slow start do for new targets?
- Why shouldn't warming instances count in scaling metrics?

These three form the elasticity loop: health checks gate the pool, autoscaling adjusts capacity, and the LB distributes — but timing issues like cold starts can cause failures.

certs.eknathalabs.com

THE LOOP

```
1. Metric (CPU/RPS/latency) crosses threshold -> autoscaler adds instances
2. New instance boots, passes health check -> LB adds it to the pool
3. LB routes traffic; load drops -> autoscaler scales in (with draining)
```

THE WARM-UP / COLD-START PROBLEM

```
# A new instance passes a shallow health check but isn't truly "warm":
# - JIT not compiled, caches cold, connection pools empty
# -> first requests are slow or error; sending full traffic immediately hurts.
```

MITIGATIONS

```
- Slow-start / warm-up mode: ramp traffic to a new target gradually
- Readiness reflects real readiness (warm caches/pools) before taking full load
- Pre-scale ahead of predictable spikes; keep warm headroom
- Scale on a leading metric (RPS/queue depth) not lagging CPU
```

Gotcha: Reactive CPU-based autoscaling lags — by the time CPU spikes, you're already overloaded, and cold new instances make it worse before better. A traffic surge can outrun boot+warm time, causing a brief brownout. Scale on leading indicators and use slow-start so new nodes ramp in.

Interview tip: The standout is the warm-up problem: a node can be "healthy" yet cold, so flooding it with traffic causes errors — use LB slow-start/warm-up and readiness that reflects true warmth. Also note reactive CPU scaling lags; scale on leading signals (RPS/queue) and keep headroom. This shows you've operated elastic systems through real traffic spikes.

📚 **Official docs**

- Cloudflare Learning: Load balancing

certs.eknathalabs.com

Q100 **What is anycast and how do CDNs and global load balancers use it?** Cloud networking

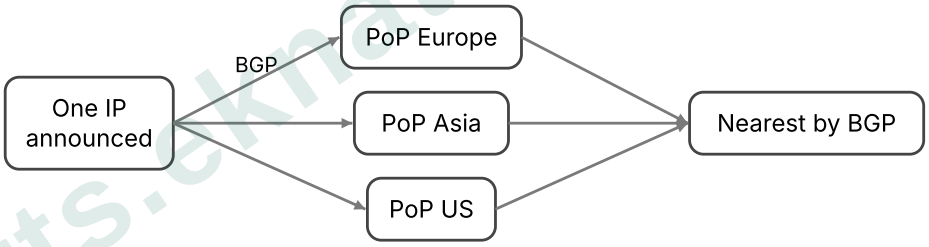
🎙 Spoken answer · ~88 s

Anycast advertises the same IP address from many locations, and the network, through BGP, routes each client's packets to the topologically nearest location. That gives low latency without client-side logic, and automatic failover, because if one location stops advertising, traffic shifts to the next nearest. It's the backbone of global CDNs and DNS: the root and public DNS resolvers use anycast, and CDN edges and global load balancers advertise one IP from hundreds of points of presence. Benefits: proximity, resilience, and DDoS resistance, since attack traffic is distributed across many locations instead of concentrating on one, so each absorbs a fraction. Caveats: routing follows BGP policy, not true latency, so the nearest by network path isn't always the fastest; flows can shift mid-connection when routes change, which is a problem for long-lived TCP connections unless the design copes, for example by short-lived requests or state at the edge; troubleshooting is harder because the same IP leads to different machines depending on where you are, so I need to identify which site answered; and capacity must be planned per site. Cloud global load balancers use anycast front ends, then route internally over the provider's backbone to the best healthy region. For unicast alternatives, GeoDNS achieves similar steering with DNS caching limits.

Likely follow-ups

- Why does anycast help against DDoS?
- What problems can anycast cause for long-lived TCP connections?

🗺 Anycast



Anycast advertises the *same* IP address from many locations; the network routes each client to the topologically nearest one. It's the backbone of global CDNs and DNS.

HOW IT WORKS

```
# Many POPs advertise the same IP via BGP from different locations.
# Each client's packets are routed (by BGP) to the NEAREST advertising POP.
# One IP, many physical endpoints -> automatic geographic proximity routing.
```

WHERE IT'S USED

```
- CDNs: serve content from the closest edge (low latency)
- Public DNS (8.8.8.8, 1.1.1.1): same IP answered by the nearest datacenter
- Global load balancers / DDoS absorption: spread load across POPs worldwide
- Automatic failover: a dead POP withdraws its BGP route -> clients reroute
```

certs.eknathalabs.com

ANYCAST VS GEODNS

GeoDNS returns different IPs based on the resolver's location (DNS-layer, subject to caching/TTL). Anycast uses one IP and lets routing pick the POP (network-layer, faster failover, no DNS caching issues).

Gotcha: Anycast routing can shift mid-session if BGP paths change (a POP withdrawing or a route flap), which is fine for stateless request/response (HTTP, DNS) but problematic for long-lived stateful connections that assume a fixed endpoint. It's ideal for short, stateless interactions.

Interview tip: Explain anycast as "one IP, nearest POP via BGP" and contrast with GeoDNS (DNS-layer, TTL-bound) — anycast gives faster, caching-free proximity routing and failover (dead POP withdraws its route). It underpins CDNs, public DNS, and volumetric DDoS absorption. The stateful-connection caveat is the nuance that signals depth.

📚 **Official docs**

- Cloudflare Learning: Load balancing
- Cloudflare Learning: Anycast

Q101 **How do you design multi-region networking for low latency and failover?** Cloud networking

🎙 **Spoken answer · ~87 s**

Multi-region architecture serves users from the nearest region and survives a regional failure, but introduces cross-region latency and data consistency challenges. For traffic steering, I use GeoDNS or latency-based routing, or an anycast global load balancer, with health checks per region so traffic shifts away from a failed region; low TTLs on failover records. For the application tier, run the stateless services in each region, deployed through the same pipeline, with capacity planned so a surviving region can absorb the failed region's load. The data layer is the hard part: choose a model, active-passive with asynchronous replication and promotion on failure, simpler but with a recovery time and some data loss; or active-active with multi-region databases, which needs conflict handling, regional write ownership, or strong-consistency systems that trade latency. Define RPO and RTO per dataset. Networking: private, encrypted connectivity between regions through the provider backbone, peering or transit gateways, non-overlapping CIDRs, and control over cross-region data transfer costs. Operations: failover runbooks that are tested through regular game days, since untested failover usually fails; configuration and secrets replicated; observability that spans regions; and avoiding hidden single-region dependencies, such as a control plane, a global service, or a database in one region. Weigh the cost and complexity against the availability requirement.

Likely follow-ups

- What is the hard part of multi-region design?
- Why must a surviving region have spare capacity?

Multi-region serves users from the nearest region and survives a regional failure, but introduces cross-region latency and data-consistency challenges.

TRAFFIC STEERING TO THE RIGHT REGION

```
- GeoDNS / latency-based routing OR anycast global LB -> nearest healthy region
- Health checks per region -> stop routing to a failed/degraded region
- CDN at the edge to serve cacheable content close to users
```

certs.eknathalabs.com

THE HARD PART: DATA

```
Active-passive : one primary region writes; others standby (async replicate)
                 -> simpler; failover promotes a standby (RTO/RPO trade-off)
Active-active  : all regions serve writes -> needs conflict resolution / global DB
                 -> low latency everywhere, hard consistency + replication lag
```

CROSS-REGION CONNECTIVITY

Connect regions privately (cloud backbone / inter-region peering / Transit Gateway peering) rather than over the public internet for security and consistent latency on replication traffic.

Gotcha: Cross-region latency is physics (e.g. tens of ms between continents) — you can't replicate it away. Synchronous cross-region writes add that latency to every request; most designs use async replication and accept some RPO, or keep writes in a home region. Don't pretend multi-region is free of consistency trade-offs.

Interview tip: Lead with the split: routing (GeoDNS/anycast + per-region health) is the easy half; **data** (active-passive vs active-active, replication lag, conflict resolution) is the hard half driven by RPO/RTO. Acknowledge cross-region latency as a physical constraint and connect regions over the private backbone. This honest, trade-off-aware framing is what senior interviews reward.

📚 **Official docs**

- Google SRE Book: Data Integrity

Q102 **Explain the shared responsibility model for networking in the cloud. What's yours vs the provider's?** Cloud networking

🎙 **Spoken answer · ~86 s**

The shared responsibility model splits cloud security and networking duties: the provider secures the infrastructure, and the customer secures what they build on it. The provider is responsible for security of the cloud: the physical network and data centres, hardware, the hypervisor, the backbone between regions and zones, and the isolation primitives of the virtual network. The customer is responsible for security in the cloud: designing the VPC and subnets, routes, security groups and network ACLs, which ports and sources are open, exposure of services to the internet, TLS configuration and certificate management, firewall and WAF rules, identity and access policies, patching guest operating systems and applications, encryption choices and key management, logging and monitoring, and the architecture's resilience across zones and regions. The line shifts with the service model: with infrastructure as a service the customer handles more, with managed services the provider handles patching of the underlying platform, but configuration, access, and data protection stay with the customer. Typical failures are on the customer side: an open security group, a public bucket, overly broad IAM, or unmonitored logs. In practice I treat the provider's guarantees as a foundation and use guardrails, policy as code, configuration scanning, and flow logs to keep my side correct.

Likely follow-ups

- Which side is responsible for an open security group?
- How does the split change for managed services?

Cloud security and networking responsibilities are split: the provider secures the infrastructure; you secure what you build on it. Networking spans both sides.

certs.eknathalabs.com

PROVIDER'S RESPONSIBILITY ('OF THE CLOUD')

```
- Physical network, hardware, hypervisor, backbone between regions/AZs
- The VPC isolation primitive, the LB/NAT GW service availability
- DDoS protection of the underlying infrastructure
- Patching the managed network services themselves
```

YOUR RESPONSIBILITY ('IN THE CLOUD')

```
- VPC/subnet design, route tables, public-vs-private placement
- Security Groups / NACLs / firewall rules (least privilege)
- TLS config, certs, what you expose publicly
- Egress controls, NetworkPolicies, application-layer security
- Correct use of private endpoints, encryption in transit
```

Gotcha: The provider gives you secure *primitives*, but a wide-open Security Group, a public S3 bucket, or 0.0.0.0/0 egress is entirely **your** fault — most cloud breaches are customer misconfiguration, not provider failure. "It's in the cloud" doesn't mean "it's secured for me."

Interview tip: The crisp line: provider secures the network fabric and service availability; you secure configuration — VPC design, SG/NACL rules, exposure, TLS, egress. Stressing that the vast majority of cloud incidents are customer misconfiguration (open SGs, public buckets) shows mature ownership and sets up why guardrails/policy-as-code matter.

📚 **Official docs**

- Kubernetes: Security overview (4C)
- AWS Shared Responsibility Model

certs.eknathalabs.com

Q103 **How does service discovery work in dynamic environments, and what role does DNS play?**

Cloud networking

🎙 Spoken answer · ~90 s

In dynamic environments with autoscaling and containers, instances come and go with changing IP addresses, so clients can't hardcode them. Service discovery provides current, healthy endpoints. Patterns: client-side discovery, where the client queries a registry such as Consul or Eureka and picks an instance itself, which gives control but puts logic in every client; and server-side discovery, where the client calls a stable endpoint, a load balancer or proxy, that consults the registry, which keeps clients simple. DNS plays a central role as the universal interface. In Kubernetes, every Service gets a DNS name, such as service dot namespace dot svc, resolved by CoreDNS to a stable cluster IP, or for headless services to the individual pod IPs, and the control plane keeps the endpoints updated with readiness. Cloud service discovery and Consul also expose DNS interfaces, with SRV records giving ports. Caveats of DNS-based discovery: caching and TTLs mean clients may use stale addresses, so TTLs must be low and clients must re-resolve; some runtimes cache forever; DNS returns no health or load information beyond what the registry filters; and large record sets or UDP truncation can be an issue. A service mesh adds richer discovery through its control plane. I choose based on scale and needs, and ensure health information drives the registry.

Likely follow-ups

- How do headless services differ from regular ones in Kubernetes?
- Why can DNS caching break service discovery?

In dynamic environments (autoscaling, containers) instances come and go with changing IPs, so clients can't hardcode addresses — service discovery provides current, healthy endpoints.

PATTERNS

```
Client-side  : client queries a registry (Consul, Eureka) -> picks an instance itself
Server-side  : client hits a stable endpoint (LB/VIP) that knows the backends
DNS-based    : a stable name resolves to current healthy IPs (K8s Services,
               Cloud Map, Consul DNS) -> the most common, transparent approach
```

DNS'S ROLE + ITS CAVEATS

```
# Kubernetes: svc.ns.svc.cluster.local -> current ready pod IPs (via EndpointSlices)
# Service mesh: control plane tracks endpoints, sidecars route to healthy ones
# CAVEAT: DNS TTL/caching means clients may use stale IPs after a change
#   -> low TTLs + re-resolution on connection failure
```

Gotcha: DNS-based discovery inherits DNS caching problems — a client (or JVM!) caching a resolved IP keeps hitting a dead instance after it's replaced. Health-aware discovery (mesh/registry) + finite TTLs + re-resolve-on-error avoid routing to gone endpoints. Pure round-robin DNS also lacks real-time health/load awareness.

Interview tip: Frame the three patterns (client-side registry, server-side LB/VIP, DNS-based) and note DNS is the most transparent but inherits caching/health-awareness limits — which is why meshes and registries add real-time health. The JVM/connection-pool stale-IP caching problem (from the DNS section) is the concrete failure mode to cite.

certs.eknathalabs.com

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

Q104 **How do you reduce cloud data transfer (egress) costs through network design?** Cloud networking

🎙 **Spoken answer · ~85 s**

Data transfer is a major and often surprising cloud cost, and network design can reduce it. The cost hierarchy, roughly from cheapest to most expensive: traffic within the same availability zone over private IPs is usually free or cheapest; cross-AZ traffic within a region carries a per-gigabyte charge in both directions, often overlooked; cross-region traffic costs more; and internet egress is the most expensive. NAT gateways add a data processing charge on top. Tactics: keep chatty services in the same AZ where resilience allows, using topology-aware routing in Kubernetes to prefer same-zone endpoints; use gateway and interface endpoints so traffic to S3 and other services bypasses NAT and the internet; put a CDN in front of content, since CDN egress is typically cheaper than origin egress and offloads the origin; compress responses and use efficient protocols and formats; cache aggressively; process data where it lives instead of moving it across regions; use private connectivity such as Direct Connect for heavy hybrid transfer, which has lower per-gigabyte rates; and replicate across regions only what's needed. Visibility comes first: use cost allocation tags, flow logs, and billing analysis to find the top talkers, then optimise those. Weigh savings against resilience, as removing cross-AZ traffic can reduce availability.

Likely follow-ups

- Why is cross-AZ traffic often overlooked?
- How do VPC endpoints reduce NAT costs?

Data transfer is a major, often-surprising cloud cost. Smart network design keeps traffic on cheaper paths and avoids needless egress.

THE COST HIERARCHY (ROUGHLY CHEAPEST -> MOST EXPENSIVE)

```
Same AZ (private IPs)          : usually free / cheapest
Cross-AZ (same region)         : a per-GB charge (often overlooked at scale)
Cross-region                   : more expensive
Egress to the internet         : the most expensive (and NAT GW adds processing $)
```

DESIGN LEVERS

```
- Keep chatty services in the SAME AZ (AZ-aware routing / topology-aware hints)
- VPC endpoints/PrivateLink for S3/API traffic -> skip NAT GW egress charges
- CDN to serve cacheable content (offload origin egress)
- Compress + batch; avoid pulling large data across regions repeatedly
- Co-locate compute with the data it processes
```

Gotcha: Cross-AZ traffic is NOT free — a microservices mesh chatting across AZs, or a DB replica read across AZs, racks up per-GB charges that hide in the bill. NAT Gateway data processing for traffic that could have used a (free) gateway endpoint is another silent cost. These dominate bills more than people expect.

Interview tip: The cost hierarchy (same-AZ cheap -> cross-AZ -> cross-region -> internet egress) plus concrete levers (AZ-affinity for chatty services, VPC endpoints to dodge NAT charges, CDN offload) is a strong FinOps-meets-networking answer. Calling out

certs.eknathalabs.com

that cross-AZ chatter and NAT data-processing are the hidden line items shows real cost ownership.

📚 **Official docs**

- RFC 3022: Traditional NAT
- AWS PrivateLink

Q105 **How do you implement rate limiting? Compare token bucket and leaky bucket, and where to enforce it.** Load balancing

🎙 **Spoken answer · ~88 s**

Rate limiting protects services from overload and abuse. The algorithm shapes how bursts are handled. A token bucket refills tokens at a steady rate, and each request spends one; it allows bursts up to the bucket size, then throttles to the refill rate, which suits APIs where short bursts are acceptable. A leaky bucket processes requests at a fixed rate, queueing or dropping the excess, which smooths traffic into a steady output and suits protecting a downstream system that can't take bursts. Other approaches are fixed windows, simple but allowing double bursts at window boundaries, sliding windows, more accurate, and concurrency limits. Where to enforce: at the edge, CDN or WAF, to stop abuse before it consumes bandwidth; at the API gateway or load balancer for per-client and per-route quotas, keyed by API key, user, or IP; and in the application or service for fine-grained limits and protection of expensive operations, plus client-side limits and backoff. In distributed systems, counters need shared state, such as Redis, or approximate local limits. Good practice: return 429 with a Retry-After header and rate limit headers; choose keys carefully, since IP-based limits hurt users behind NAT; set different limits per tier and endpoint; combine with load shedding and circuit breakers; and monitor the rejection rates.

Likely follow-ups

- How does a token bucket differ from a leaky bucket?
- Why return a Retry-After header?

Rate limiting protects services from overload and abuse. The algorithm shapes how bursts are handled, and the enforcement point determines coverage.

TOKEN BUCKET VS LEAKY BUCKET

```
Token bucket : tokens refill at a steady rate; each request spends one.
               Allows BURSTS up to the bucket size, then throttles to the refill rate.
Leaky bucket : requests queue and drain at a FIXED rate (smooths output).
               No bursts — steady outflow regardless of input spikes.
```

WHERE TO ENFORCE

```
Edge/CDN/WAF : stop abuse before it costs you compute (best for DDoS/abuse)
API gateway  : per-API-key / per-tenant quotas + usage plans
Service mesh  : per-service limits between internal services
Application   : fine-grained, business-logic-aware limits (per-user action)
```

DISTRIBUTED CHALLENGE

Across many instances you need **shared** counters (e.g. Redis) so the limit is global, not per-instance — otherwise N instances each allow the full rate (N x the intended limit).

certs.eknathalabs.com

Gotcha: Per-instance in-memory rate limits don't enforce a global limit — 10 instances each allowing 100 rps = 1000 rps total. Use a shared store (Redis) or a coordinated edge limiter. Also: return `429` with `Retry-After` so clients back off correctly instead of hammering.

Interview tip: Token bucket = allows bursts (most common for APIs), leaky bucket = smooths to a constant rate. Enforce as early as possible (edge/gateway) to protect compute, with shared-state counters for a true global limit across instances. Returning 429 + Retry-After and the per-instance-counter pitfall are the practical details interviewers look for.

📚 **Official docs**

- RFC 6585: HTTP 429 Too Many Requests
- Microsoft: Cloud design pattern – Throttling

Q106 **A service is unreachable. Walk through a systematic layer-by-layer debugging approach.**

Troubleshooting

🎙 **Spoken answer · ~87 s**

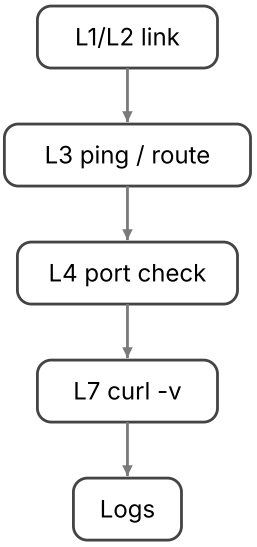
Can't reach the service is ambiguous, so I work up the stack methodically instead of guessing, and each layer has a definitive test. First DNS: does the name resolve, and to the right IP, with dig or getent. Second layer three: is the host reachable, with ping if ICMP is allowed, and which route does traffic take, with ip route get and traceroute. Third the port: can I open a TCP connection, with nc or curl, which separates refused from timeout; refused means the host is reachable but nothing is listening, timeout means packets are dropped somewhere. Fourth, on the server, is the process listening on the expected address and port, with ss, and not just on localhost. Fifth the firewalls: host rules, security groups, network ACLs, and any intermediate firewall, using flow logs to see rejects. Sixth TLS: does the handshake and certificate verify, with openssl s_client. Seventh the application: does it respond correctly, what do its logs say, is it overloaded or failing health checks, and what does the load balancer show about target health. I compare with a working client or location to find what differs, and capture packets if layers are ambiguous. Documenting the layer where it fails tells me which team owns the fix.

Likely follow-ups

- What separates refused from timeout?
- Where do flow logs help in this process?

certs.eknathalabs.com

🗺 **Layered debugging**



"Can't reach the service" is ambiguous — work up the stack (or down) methodically rather than guessing. Each layer has a definitive test.

LAYER-BY-LAYER

```
1. DNS   : does the name resolve, and to the RIGHT IP?    dig <name>
2. L3    : is the host reachable?                         ping <ip> (if ICMP allowed)
3. Route : which path does it take?                       ip route get <ip>; traceroute
4. L4    : is the PORT open / accepting?                  nc -vz <ip> <port>; curl
5. L7    : does the app respond correctly?                curl -v https://<host>/path
```

INTERPRETING RESULTS

```
- DNS wrong IP        -> stale record / split-horizon / cache
- ping ok, port closed -> firewall/SG, app not listening, wrong port
- connection REFUSED   -> reached host, nothing listening (RST) — app/port issue
- connection TIMEOUT   -> firewall DROP, wrong route, security group, black hole
- TLS error            -> cert/chain problem (openssl s_client)
```

Refused vs timeout is the key fork: REFUSED means you reached the host and it actively said no (port closed / app down) — an app/port problem. TIMEOUT means silence (firewall DROP, wrong route, security group blocking) — a network/filtering problem. This single distinction halves your search space immediately.

Interview tip: Show a repeatable method (DNS -> L3 -> route -> L4 -> L7) and lead with the refused-vs-timeout fork — it's the fastest way to split "app problem" from "network problem." Naming the one tool per layer (`dig`, `ip route get`, `nc -vz`, `curl -v`, `openssl s_client`) demonstrates hands-on fluency.

📚 **Official docs**

- Kubernetes: Debug Services

certs.eknathalabs.com

Q107 **How do you use tcpdump effectively to debug a network issue? Give a real workflow.**

Troubleshooting

🎙 Spoken answer · ~89 s

tcpdump captures packets to show what is actually on the wire, ground truth when logs and assumptions disagree. The skill is targeted capture and reading the result. I start by deciding the question: does traffic arrive, does the handshake complete, are there resets or retransmissions, does DNS get answered? Then capture on the right interface with a precise filter, a host and port, or TCP flags such as SYN and RST, with minus nn to skip name resolution. For intermittent issues, write to a file with size and count rotation so the disk doesn't fill, then read the file with tcpdump or Wireshark. Capture on both ends to see where packets disappear. Reading the flow: a SYN with no SYN-ACK means blocked or dropped; SYN followed by RST means nothing is listening or a firewall rejects; a complete handshake but no response to the request means the application is slow or hung; repeated retransmissions mean loss; a zero window means the receiver is slow; unexpected RSTs from a different TTL suggest a middlebox. Check timing deltas for latency. Considerations: captures can contain sensitive data, so handle them per policy; overhead on busy hosts; encrypted payloads aren't readable, but handshake and timing are. In Kubernetes use a debug container in the pod's network namespace.

Likely follow-ups

- What does SYN followed by RST indicate?
- How do you capture an intermittent problem safely?

tcpdump captures packets to see what's *actually* on the wire — ground truth when logs and assumptions disagree. The skill is targeted capture + reading the result.

TARGETED CAPTURE

```
tcpdump -i any -n host 10.0.0.5 and port 443     # filter to the flow you care about
tcpdump -i eth0 -nn 'tcp[tcpflags] & (tcp-syn|tcp-rst) != 0'   # just SYN/RST
tcpdump -i any -w capture.pcap host 10.0.0.5     # write to file -> open in Wireshark
tcpdump -A -s0 port 80                           # print ASCII payload (HTTP)
```

A REAL WORKFLOW (INTERMITTENT FAILURE)

```
1. Reproduce while capturing on BOTH ends (client + server) with a tight filter
2. Look for: SYN with no SYN-ACK (firewall/route?), SYN-ACK no ACK (asymmetric?),
   RST (who sent it, when?), retransmits (loss?), no packets at all (DNS/route?)
3. Correlate timestamps with the app error to pin the exact failing exchange
```

Reading tips: SYN but no SYN-ACK = traffic not reaching/accepted (firewall, wrong port, no listener). SYN-ACK but client never ACKs = return-path/asymmetric routing. Unexpected RST = find which side sent it. Lots of retransmits = packet loss. *No* packets leaving = it's DNS or routing, not the network path. Capture on both ends to see which side is missing what.

Interview tip: Emphasize **targeted** filters (host + port) — never capture everything on a busy host. The diagnostic logic (SYN/SYN-ACK/ACK/RST/retransmit patterns) is what turns a capture into an answer, and capturing on both ends localizes one-way drops. Mention writing a pcap for Wireshark for deeper analysis. This is a strong "I actually debug networks" signal.

certs.eknathalabs.com

📚 **Official docs**

- pcap-filter(7)
- tcpdump manual

Q108 **Intermittent connection timeouts under load, but fine when idle. What are the likely causes?** Troubleshooting

🎙 **Spoken answer · ~88 s**

Fine when idle but failing under load points to a resource that's exhausted only at scale: a finite limit, not a path problem. The prime suspects: the conntrack table full on a NAT gateway, node, or firewall, shown by kernel messages about table full and dropping packets; ephemeral port exhaustion when many connections go to one destination, especially with TIME_WAIT buildup; the listen backlog or accept queue overflowing, shown in the netstat listen overflow counters; file descriptor limits, giving too many open files; connection pool exhaustion in the application or database; NAT gateway port allocation limits; cloud instance network allowances being exceeded, such as packets per second or bandwidth, which providers expose as counters; CPU saturation or softirq overload dropping packets; and downstream rate limits. Diagnosis: capture the exact error and when it begins relative to load, which hints at the threshold; check the counters for each limit at that time; look at kernel logs, ss state counts, pool metrics, and cloud network metrics; and reproduce with a gradual load test while watching the suspects. Fixes follow the limit: raise the limit where sensible, reuse connections, add capacity or spread load, tune queues, or add backpressure. Add alerts for approaching limits, since by the time errors appear, users are affected.

Likely follow-ups

- Which counters reveal a listen queue overflow?
- How do cloud network allowance counters show a limit being hit?

"Fine when idle, fails under load" points to a resource that's exhausted only at scale — a finite limit, not a path problem.

PRIME SUSPECTS (LOAD-DEPENDENT LIMITS)

```
- conntrack table full (NAT/gateway/node) -> dmesg "table full, dropping packet"
- Ephemeral port exhaustion (many conns to one dest 4-tuple) -> EADDRNOTAVAIL
- Connection pool exhausted (app waits for a free conn -> timeout)
- Accept queue / backlog overflow (somaxconn) -> dropped SYNs under burst
- File descriptor limit hit (ulimit -n) -> "too many open files"
- NAT gateway / LB connection or bandwidth limits
- DB max_connections reached -> app blocks waiting
```

HOW TO CONFIRM

```
ss -s ; conntrack -C ; dmesg | grep -Ei 'conntrack|table full'
ss -ltn        # listener Recv-Q full = accept queue overflow
cat /proc/sys/fs/file-nr ; ulimit -n          # fd usage vs limit
# Correlate failure rate with concurrency/RPS -> confirms a saturation limit
```

Key signal: Failures scaling with load + healthy CPU/memory = a *countable resource limit* (ports, conntrack, fds, pool, accept queue, DB conns), not bandwidth or a broken path. The error is usually in `dmesg` or socket stats, not the app log.

certs.eknathalabs.com

Interview tip: The framing that scores: "fine idle, fails under load" = some finite resource exhausts at concurrency. Enumerate the usual suspects (conntrack, ephemeral ports, connection pool, accept queue/somaxconn, fd limit, DB connections) and how to confirm each. Connection pooling + raising the right limit is the fix — guessing "the network is slow" is the wrong instinct here.

📚 **Official docs**

- Linux man-pages: ulimit / getrlimit(2)
- PgBouncer

Q109 **What does 'netstat -s' / 'ss' tell you, and which counters reveal network problems?** Troubleshooting

🎙 **Spoken answer · ~88 s**

Kernel network counters, from netstat dash s, nstat, and ss, expose problems that are invisible in application logs. High-value counters: TCP retransmits indicate packet loss or congestion; listen queue overflows and SYNs dropped indicate the accept queue or backlog is too small or the application is too slow to accept, meaning connections are being refused or delayed under load; TCP timeouts and resets show connection failures; out-of-order packets and duplicate ACKs point to reordering or loss; receive buffer pruning and collapsed packets point to memory or buffer pressure from slow consumers; and TIME_WAIT and other state counts show churn or leaks. For interfaces, ip minus s link shows errors, drops, and overruns; ethtool statistics give NIC-level counters such as ring buffer drops; and proc net softnet stat shows kernel backlog drops per CPU. ss with extended information shows per-connection round-trip time, retransmissions, congestion window, and send and receive queues; a growing receive queue means the application isn't reading, and a growing send queue means the peer or network isn't draining. I look at rates over time, not absolute numbers since boot, by sampling twice or exporting to monitoring. Combined, these tell me whether a problem is loss, queueing, a slow application, or a resource limit, and where to look next.

Likely follow-ups

- What does a growing receive queue on a socket mean?
- Why look at counter rates instead of totals?

Kernel network counters (`netstat -s`, `nstat`, `ss`) expose problems invisible in app logs — retransmits, drops, queue overflows.

HIGH-VALUE COUNTERS

```
netstat -s | grep -i retransmit     # TCP retransmits -> packet loss / congestion
netstat -s | grep -i "listen queue" # SYNs dropped (listen overflow) -> backlog too small
netstat -s | grep -i "buffer"        # out-of-memory pruning -> socket buffer pressure
nstat -az | grep -i TcpExt           # rich TCP extension counters (resets, drops)
ss -tin                              # per-socket: rtt, cwnd, retrans, pacing
```

WHAT THEY REVEAL

```
Rising retransmits        -> loss or congestion on the path
ListenOverflows/Drops     -> accept queue too small / app accepting too slowly
SYN cookies sent          -> SYN queue overflow (load or SYN flood)
PruneCalled / collapsed   -> socket memory pressure (tune rmem/wmem)
TimeWait reuse/overflow   -> port pressure
```

certs.eknathalabs.com

Gotcha: These are cumulative since boot — take two snapshots and diff to see the *rate*, not the lifetime total. A big absolute retransmit number on a host up for a year may be nothing; a fast-climbing delta during an incident is the signal. `nstat` resets counters per run, which makes deltas easy.

Interview tip: Name the counters that matter — retransmits (loss), listen overflows (accept-queue/backlog), SYN cookies (SYN-queue pressure), buffer pruning (socket mem) — and stress reading the *delta* over time, not the total. `ss -tin` for per-socket RTT/cwnd/retrans is the modern, granular view. This shows you debug from kernel evidence, not vibes.

📚 **Official docs**

- GNU grep manual

Q110 **Users in one region report slowness while others are fine. How do you investigate?** Troubleshooting

🎙 **Spoken answer · ~86 s**

Region-specific slowness localises the problem to a path, point of presence, region, or routing decision affecting only those users, so I bisect by geography and layer. First determine scope: is it all users in that region or a subset, perhaps one ISP? A single ISP suggests a peering or transit issue; all users suggest a regional edge or origin problem. Test from that region using synthetic probes or volunteers: mtr and traceroute to the endpoint to see where latency or loss appears, reading mtr correctly. Break down the request timing: DNS resolution, connect, TLS, time to first byte; slow DNS may mean they're sent to a distant or overloaded edge; slow connect indicates network path latency; slow first byte with good connect points to the origin or cache misses. Check CDN behaviour in that region: which POP serves them, the cache hit ratio, and the origin fetch latency. Check for BGP or routing changes, and provider incidents. Look at real user monitoring segmented by region, country, and ISP. Check regional backends: is that region's replica, cache, or dependency slow? Fixes: reroute traffic to a different edge or region, fix cache configuration, add a regional origin or replica, or work with the CDN or ISP with evidence.

Likely follow-ups

- How does real user monitoring help here?
- What does slow DNS suggest about a region's problem?

Region-specific slowness localizes the problem to a path, POP, region, or routing decision affecting only those users — bisect by geography and layer.

NARROW IT DOWN

```
- Is it ALL users in that region or a subset (one ISP)? -> path vs regional issue
- Test from that region: mtr/traceroute to the endpoint (where does latency/loss start?)
- Compare DNS answers from that region (GeoDNS sending them to a far/unhealthy POP?)
- CDN: are they hitting a degraded/cold edge POP? check cache hit ratio + POP health
- Is the regional backend/replica itself degraded (DB read replica, cross-region hop)?
```

COMMON REGION-SPECIFIC CAUSES

certs.eknathalabs.com

```
- GeoDNS/anycast routing them to the wrong (distant or unhealthy) region/POP
- A bad transit/peering path for that region's ISP (mtr shows loss at a hop)
- CDN edge issue in that geography; cold cache after a purge
- Cross-region dependency: that region's app calling a DB/service far away
- Regional cloud incident (check the provider status page)
```

Gotcha: Don't assume it's your app — region-specific slowness is frequently *path* (ISP/transit/peering) or *routing* (GeoDNS/anycast steering to the wrong place), which `mtr` from the affected region reveals. Synthetic monitoring from multiple geographies catches this before users report it.

Interview tip: Structure it: confirm scope (whole region vs one ISP), then test from that geography with `mtr`/traceroute and compare GeoDNS/CDN routing. The senior insight is that the cause is often *outside your stack* — transit/peering paths or routing decisions — so multi-region synthetic monitoring and reading mtr from the affected region are how you prove it.

📚 **Official docs**

- traceroute(8) / mtr

Q111 **How do you debug a TLS/SSL handshake failure? What tools and what do the errors mean?**

Troubleshooting

🎙 **Spoken answer · ~86 s**

TLS failures produce varied and cryptic errors, and openssl s_client is the universal probe to see what the server presents and where the handshake breaks. I connect with the server name flag, because SNI matters: without it the server may present a default certificate. The output shows the certificate chain, the negotiated protocol and cipher, verification results, and the validity dates. What the errors mean: certificate has expired points to expiry or clock skew; unable to get local issuer certificate means the chain is incomplete or the CA isn't trusted by the client; hostname mismatch means the name isn't in the subject alternative names; handshake failure or no shared cipher means a protocol version or cipher mismatch, for example the server only allows TLS 1.3 and the client is old; a certificate unknown or bad certificate alert in mutual TLS means the server rejected the client's certificate; connection reset during the handshake can mean a firewall or middlebox, or a missing SNI. Intermittent failures suggest multiple backends with different certificates, so I test each IP. I can force versions to test compatibility, use the showcerts flag to see the chain, and curl verbose for a client-side view. For deeper analysis, a packet capture shows the TLS alert.

Likely follow-ups

- Why does SNI matter when probing with openssl?
- How do you test each backend IP behind a hostname?

TLS failures throw varied, cryptic errors. `openssl s_client` is the universal probe to see exactly what the server presents and where the handshake breaks.

THE PROBE

certs.eknathalabs.com

```
openssl s_client -connect host:443 -servername host   # SNI matters!
openssl s_client -connect host:443 -showcerts          # full chain server sends
openssl s_client -connect host:443 -tls1_2             # force a version to test support
curl -vI https://host                                  # curl's TLS verbose output
```

COMMON ERRORS -> MEANING

```
"unable to get local issuer certificate" -> missing INTERMEDIATE cert (server side)
"certificate has expired"                 -> expired cert (renew/automate!)
"hostname mismatch" / SNI                 -> cert CN/SAN doesn't match the name
                                             or no SNI sent (wrong vhost/cert returned)
"no shared cipher" / protocol             -> client/server TLS version/cipher mismatch
"self signed certificate"                 -> untrusted CA / cert not in trust store
```

SNI gotcha: Omitting `-servername` makes the server return its default cert, not the one for your hostname — leading to a false "wrong cert" diagnosis. Always pass SNI. And "works in browser, fails in client" is the missing-intermediate signature (browsers paper over it; strict clients don't).

Interview tip: `openssl s_client` (with `-servername` for SNI and `-showcerts` for the chain) is the answer — it shows the negotiated version, cipher, full chain, and exact failure. Mapping the top errors (missing intermediate, expired, hostname/SNI mismatch, no shared cipher) to causes, plus automating renewal to prevent expiry, demonstrates real TLS operations experience.

📚 **Official docs**

- RFC 8446: TLS 1.3
- Cloudflare Learning: TLS handshake

Q112 **What is SSRF and how does the cloud metadata endpoint make it dangerous? How is it mitigated?** Network security

🎙 **Spoken answer · ~83 s**

Server-side request forgery tricks a server into making requests on an attacker's behalf. If an application fetches a user-supplied URL, for example to load an image or call a webhook, an attacker can supply an internal target instead of an external one, and the server, with its network position and trust, performs the request. In the cloud this is especially dangerous because of the instance metadata endpoint, a link-local address that returns instance details and temporary IAM credentials for the attached role. An SSRF that reaches it can leak those credentials, letting the attacker act with the instance's permissions, which was behind several major breaches. Mitigations: for the metadata service, require IMDSv2 on AWS, which needs a session token via a PUT request with a header that simple SSRF can't forge, and limit hop count so containers can't reach it; apply least privilege to instance roles so stolen credentials are limited; and block metadata access for workloads that don't need it. For the application: validate and allow-list destinations, resolve DNS and check the resulting IP against private and link-local ranges, taking care with redirects and DNS rebinding, avoid returning raw responses to the user, and disable unneeded URL schemes.

Likely follow-ups

- Why does IMDSv2 mitigate SSRF?
- Why check the resolved IP address and not just the hostname?

certs.eknathalabs.com

Server-Side Request Forgery tricks a server into making requests on an attacker's behalf — and in the cloud it can reach the instance metadata endpoint to steal credentials.

THE ATTACK

```
# App fetches a user-supplied URL (e.g. "fetch this image/webhook").
# Attacker supplies an INTERNAL target instead:
http://169.254.169.254/latest/meta-data/iam/security-credentials/<role>
# The server fetches it from INSIDE the trust boundary and returns/leaks
# the instance's temporary IAM credentials -> cloud account compromise.
```

WHY METADATA IS THE PRIZE

The link-local metadata service (`169.254.169.254`) hands out the instance's role credentials to anything that can reach it locally — exactly what SSRF gives an attacker a path to.

MITIGATIONS

```
- IMDSv2: require a session token + hop limit (blocks simple SSRF GETs) — enforce it
- Validate/allowlist outbound URLs; block link-local + private ranges (RFC1918, 169.254)
- Egress controls / no direct metadata access from app containers
- Least-privilege instance roles (limit blast radius if creds leak)
```

Gotcha: The classic cloud breach chain is SSRF -> metadata -> IAM creds -> lateral movement. **IMDSv2** (token + low hop limit) defeats the naive version and should be *enforced* (not just available) — leaving IMDSv1 enabled keeps the door open. Block 169.254.0.0/16 and private ranges from any user-influenced fetch.

Interview tip: Connect SSRF to the cloud metadata endpoint as the high-value target (leaks IAM creds) — that chain is behind several famous breaches. Enforcing IMDSv2 (not just enabling it), URL allowlisting that blocks link-local/private ranges, and least-privilege roles are the layered defenses. Knowing 169.254.169.254 by heart signals real cloud-security awareness.

📚 **Official docs**

- OWASP Top Ten

certs.eknathalabs.com

Q113 **How do you debug 'connection refused' vs 'connection timeout' vs 'no route to host'?** Troubleshooting

🎙 Spoken answer · ~86 s

These three errors each point to a different layer, and distinguishing them narrows the cause immediately. Connection refused means I reached the host, but nothing is listening on that port, and the host actively sent a TCP reset; it's fast. Check that the service is running, listening on the right address and port, with ss, and that a local firewall isn't rejecting. Connection timeout means silence: no response at all, packets are being dropped or not delivered, usually a firewall, security group, network ACL, or route problem, or the host is down; it's slow, since the client waits for retries. Check security groups, flow logs for rejects, routes, and capture packets on both ends. No route to host means the network layer could not find a path or ARP resolution failed: a missing route, a wrong subnet or gateway, an unreachable host on the local segment, or an ICMP host unreachable message from a router; check the routing table, ARP or neighbour state, and the interface configuration. A quick table: refused means the host is up, service is wrong; timeout means something is dropping; no route means no path. Curl or nc show which one. Then fix in the identified layer, and verify from the client side.

Likely follow-ups

- Which of the three is fast and which is slow, and why?
- What would you check for a 'no route to host' error?

These three connection errors each point to a distinct layer — distinguishing them instantly narrows the cause.

THE THREE ERRORS

```
Connection REFUSED  : reached the host, but nothing listening on that port
                    (the host actively sent a TCP RST) -> ECONNREFUSED
Connection TIMEOUT  : silence — no response at all within the timeout
                    -> firewall DROP, wrong route, black hole, host down
No route to host     : the kernel has no route to the destination at all
                    (or an ICMP host-unreachable came back) -> EHOSTUNREACH
```

WHAT EACH MEANS + NEXT STEP

```
REFUSED -> network path is FINE; app isn't listening / crashed / wrong port
        -> check the service is up + bound to the right addr:port (ss -ltn)
TIMEOUT -> something SILENTLY drops packets: firewall/SG DROP rule, wrong
        route, or the host is down -> check SG/NACL, ip route get, security groups
NO ROUTE -> routing/interface problem: missing route, interface down, bad gateway
         -> ip route get <dst>; ip addr; check subnet/route table
```

Key insight: REFUSED is GOOD news for the network — you reached the host and it answered (just nothing on that port), so it's an app/port issue. TIMEOUT means you never got an answer — a filtering or routing problem. NO ROUTE is local routing. Reading the exact error word saves enormous time.

Interview tip: Memorize the mapping: REFUSED = reached host, no listener (app/port); TIMEOUT = silent drop (firewall/route/down); NO ROUTE = local routing/interface. The senior move is recognizing REFUSED actually *confirms* network reachability, redirecting you to the app — while TIMEOUT sends you to firewalls/security groups. This precision is exactly what's being tested.

certs.eknathalabs.com

📚 **Official docs**

- RFC 9293: TCP

Q114 **How do you implement micro-segmentation and egress filtering, and why does egress matter so much?**

Network security

🎙 **Spoken answer · ~88 s**

Micro-segmentation limits east-west traffic, the service-to-service flows inside the network, to only what's needed, and egress filtering controls outbound traffic. Together they contain a breach. For micro-segmentation: default-deny between services and explicitly allow required flows; in Kubernetes use NetworkPolicies, or the richer policies of Cilium and Calico, or service mesh authorisation based on workload identity; in VPCs use security groups referencing other groups, and separate subnets and accounts for sensitive tiers. Start in observe mode using flow logs to learn real traffic, then enforce incrementally. Egress filtering matters because compromised workloads depend on outbound connectivity: to download tools, communicate with command-and-control servers, and exfiltrate data. Controls: restrict outbound traffic to known destinations via allow-lists of domains or addresses through a proxy or firewall, DNS filtering, private endpoints instead of internet access for cloud services, and blocking direct internet access from sensitive subnets. Attackers often succeed in getting code execution; egress controls turn a breach into a limited event by preventing data from leaving and stopping lateral tools from phoning home. Operational points: maintain the allow-lists as code, handle updates and package repositories through mirrors, log denied traffic and alert on it, and test that policies work. The trade-off is operational friction when teams need new destinations, so make the request process fast.

Likely follow-ups

- Why is egress filtering valuable even if the attacker gets in?
- How do you roll out default-deny network policy safely?

Micro-segmentation limits east-west (service-to-service) traffic to only what's needed; egress filtering controls outbound traffic — together they contain a breach.

MICRO-SEGMENTATION (EAST-WEST)

```
- Default-deny between services; explicitly allow only required flows
- Kubernetes: NetworkPolicies (or Cilium identity-based / L7 policies)
- Cloud: tight Security Groups referencing other SGs (not CIDRs), per-tier
- Goal: a compromised service can't freely reach the rest of the network
```

EGRESS FILTERING (OUTBOUND)

```
- Default-deny egress; allowlist only required destinations (APIs, registries)
- Block arbitrary outbound to the internet (0.0.0.0/0)
- Force traffic through an egress proxy/firewall for logging + allowlisting
```

WHY EGRESS MATTERS SO MUCH

Most teams lock down *inbound* and ignore *outbound* — but outbound is how compromised workloads **beacon to C2 servers and exfiltrate data**. Default-deny egress with an allowlist directly stops the most damaging post-compromise actions.

certs.eknathalabs.com

Gotcha: Wide-open egress (the default in most setups) means a breached container can exfiltrate your data and phone home freely. Egress control is one of the highest-value, most-skipped controls — and don't forget to allow DNS (port 53) and required service endpoints, or you'll break the app while locking it down.

Interview tip: Frame micro-segmentation + egress filtering as breach containment: segmentation stops lateral movement, egress filtering stops exfiltration/beaconing. Stress that egress is the commonly-neglected half and the highest-leverage control against data theft. Tie to identity-based policy (Cilium) and SG-referencing-SG patterns to show implementation depth — connects to zero-trust.

📚 **Official docs**

- Kubernetes: Network Policies

Q115 **Compare VPN, bastion host, and modern alternatives (SSM/IAP) for secure access to private resources.** Network security

🎙 **Spoken answer · ~86 s**

Reaching private resources securely has evolved from network-level VPNs and SSH bastions toward identity-based, agent-mediated access with no open inbound ports. A VPN extends the private network to the client, so once connected the user has broad network access; it is over-broad, relies on the you're inside so you're trusted model, and creates a large lateral movement surface, and it needs client software and capacity management. A bastion host is a hardened jump server exposed to the internet that users SSH through to reach private instances; it limits the exposed surface to one host, but needs patching, key management, and access logging, and is a high-value target. Modern alternatives, AWS Systems Manager Session Manager, GCP Identity-Aware Proxy, and Azure Bastion, use an agent on the instance that connects outbound to the service, so there are no inbound ports or public IPs at all; access is controlled by IAM identity and policy, with MFA, short-lived sessions, and full session logging and auditing, and no SSH keys to distribute. For applications, identity-aware proxies give per-application access with device and user context. My preference is the identity-based approach with least privilege, just-in-time access, and audit logs, keeping a VPN or bastion only where needed, and a break-glass path for emergencies.

Likely follow-ups

- Why do SSM or IAP need no open inbound ports?
- What is the main weakness of a VPN compared with identity-aware access?

Reaching private resources securely has evolved from network-level VPNs and SSH bastions toward identity-based, agent-mediated access with no open inbound ports.

THE APPROACHES

```
VPN        : extends the private network to the client; broad network access once on
           -> "you're inside = trusted" — over-broad, lateral-movement risk
Bastion/jump host : a hardened SSH gateway; you hop through it to private hosts
           -> single audited entry, but it's an internet-facing box to secure/patch
Modern (SSM Session Manager / IAP / Teleport) : identity-based, agent/proxy-mediated
           -> NO inbound ports open, per-session auth + audit, no standing SSH
```

WHY THE MODERN APPROACH WINS

certs.eknathalabs.com

```
- No public SSH port / no bastion to attack (agent dials OUT to the control plane)
- Per-session, identity-based authorization (IAM/SSO) + full session logging
- No long-lived SSH keys to manage/leak; ephemeral credentials
- Fine-grained: who can reach what, when, with recording
```

Gotcha: A VPN grants network-level trust — once connected, the user often can reach far more than they need (violating least privilege), and a compromised VPN credential is a broad foothold. Bastions remove that but are still an internet-facing target with SSH keys to manage. Agent-based (SSM/IAP) eliminates both the open port and the standing keys.

Interview tip: The trajectory is VPN (network trust) -> bastion (single audited hop) -> identity-based agent access (SSM/IAP/Teleport: no inbound port, per-session IAM auth, full audit, no standing keys). Framing it as zero-trust applied to admin access — least privilege per session instead of network-level trust — is the senior answer and ties to the zero-trust topic.

📚 **Official docs**

- Cloudflare Learning: What is a VPN?

Q116 **Explain TLS termination, passthrough, and re-encryption at a load balancer. Trade-offs of each?**

Network security

🎙 **Spoken answer · ~89 s**

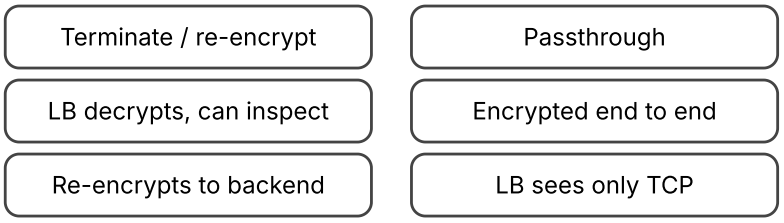
Where TLS is decrypted relative to the load balancer determines visibility, performance, and internal security. In termination, or offload, the balancer decrypts traffic and sends plaintext to the backends: the balancer can inspect, route on content, cache, and apply WAF rules, and backends are relieved of TLS work and certificate management. The trade-off is that traffic inside the network is unencrypted, which may not satisfy compliance or a zero-trust posture. In passthrough, the balancer forwards the encrypted bytes at layer four without decrypting: end-to-end encryption is preserved and certificates live only on backends, but the balancer can't see content, so no path routing, header manipulation, or WAF, and the client IP must be conveyed via the PROXY protocol. In re-encryption, or bridging, the balancer terminates the client's TLS, inspects and routes, then opens a new TLS connection to the backend: it gives both layer-seven features and encryption on the internal hop, at the cost of double the TLS processing, more certificates to manage, and added latency. I choose termination for simplicity and performance when the internal network is trusted, passthrough when end-to-end encryption or client certificates must reach the backend, and re-encryption when I need both inspection and internal encryption, as many compliance regimes and zero-trust designs require. Automate certificate lifecycle for all modes.

Likely follow-ups

- When is passthrough the right choice?
- What are the costs of re-encryption?

certs.eknathalabs.com

🗺 TLS at the LB



Where TLS is decrypted relative to the load balancer determines visibility, performance, and internal security.

THE THREE MODES

```
Termination (offload) : LB decrypts; LB<->backend is PLAINTEXT (HTTP)
                        -> LB can route/inspect/cache; backend offloaded from TLS
Passthrough           : LB forwards encrypted bytes; backend terminates TLS
                        -> end-to-end encryption, but LB can't see/route on HTTP (L4 only)
Re-encryption (bridging): LB decrypts (to inspect/route) then RE-encrypts to backend
                        -> L7 features AND encrypted internal hop (best of both, more CPU)
```

CHOOSING

```
Termination   : simple, fast, central cert mgmt; OK if the internal network is trusted
Passthrough   : compliance/end-to-end requirements, or backend must see the client cert
Re-encryption : zero-trust internal network + need L7 routing/WAF (common in regulated
envs)
```

Gotcha: Plain termination leaves LB->backend traffic in cleartext on the internal network — fine only if you truly trust that segment. In a zero-trust model, that internal hop should be encrypted too (re-encryption or mesh mTLS). Passthrough gives end-to-end encryption but forfeits all L7 routing/inspection at the LB (it's L4-only).

Interview tip: Map the trade-off to your trust model: termination (fast, central certs, plaintext internal) for trusted networks; re-encryption when you need both L7 features and an encrypted internal hop (zero-trust/regulated); passthrough for true end-to-end or client-cert-to-backend needs. Noting that termination's cleartext internal hop conflicts with zero-trust — and mesh mTLS as the alternative — shows architectural maturity.

📚 Official docs

- Cloudflare Learning: Load balancing

certs.eknathalabs.com

Q117 **An app's connection pool keeps exhausting. How do you diagnose pool vs network vs backend issues?** Troubleshooting

🎙 Spoken answer · ~82 s

Pool exhaustion, with errors like timeout acquiring connection, can come from the pool configuration, the network, or a slow backend, and distinguishing them prevents fixing the wrong layer. The three root causes. First, the pool is too small or leaking: connections aren't returned, for example a missing close in an error path, or the maximum size is too low for the concurrency; the signs are a pool at its maximum with connections held idle and the backend showing few active queries, and leaked connections visible in pool diagnostics that report where they were borrowed. Second, the backend is slow: connections are held longer because queries take longer, so the same pool serves fewer requests per second by Little's Law; the signs are high query latency, database active sessions, locks, and the pool full of connections in use that are busy. Third, the network: connection establishment is slow or connections drop, with retransmissions, resets from idle timeouts, DNS delays, or packet loss; signs include slow connect times and many reconnects. Diagnose by looking at pool metrics, the active, idle, and waiting counts, along with the time connections are held, the backend's own view of sessions, and network statistics.

Likely follow-ups

- How does Little's Law relate to pool size?
- Why can enlarging the pool make things worse?

Pool exhaustion ("timeout acquiring connection") can stem from the pool config, the network, or a slow backend — distinguishing them prevents fixing the wrong layer.

THE THREE ROOT CAUSES

```
Pool too small / leaked : connections not returned (leak) or maxPoolSize too low
                          for the concurrency -> requests queue for a free conn
Slow backend            : each query/call takes longer -> conns held longer ->
                          pool drains even at normal request rates
Network/LB issue        : conns silently dead (LB idle timeout killed them) but
                          still "in" the pool -> acquiring a dead conn stalls/fails
```

DIAGNOSIS

```
- Pool metrics: active vs idle vs waiting, acquire wait time, timeouts
- Is backend latency up? (slow query log, downstream p99) -> backend cause
- Are there RSTs/resets on reuse? (the LB idle-timeout pattern) -> network cause
- Does active-conn count grow without release? -> a LEAK (code not closing)
ss -tn dst <backend> | wc -l    # actual live conns vs what the pool thinks
```

Multiplication trap: If every app instance has a pool of N and you run M instances, the backend sees up to N*M connections — scaling out can exhaust the *backend's* max_connections even though each pool looks fine. Size pools with the total fan-in to the backend in mind, not per-instance.

Interview tip: Separate the three causes (undersized/leaked pool, slow backend holding conns longer, dead conns from LB idle timeout) and use pool metrics + backend latency + reset patterns to tell them apart. The connection-multiplication insight (N pools x M instances -> backend max_connections) is the senior detail — it explains why "we scaled up and the database fell over."

certs.eknathalabs.com

📚 **Official docs**

- PgBouncer

Q118 **What is ECMP and how can it cause confusing, inconsistent network behavior?** Troubleshooting

🎙 **Spoken answer · ~89 s**

ECMP, Equal-Cost Multi-Path routing, spreads traffic across multiple equal-cost paths for bandwidth and redundancy, but per-flow hashing can produce puzzling per-connection differences. The router hashes each flow, typically the five-tuple of source and destination address, ports, and protocol, and picks a path, so packets of one flow stay in order on one path while different flows spread out. Confusing behaviours: a single large transfer is limited to one path's bandwidth, so adding links doesn't speed it up; different connections between the same hosts take different paths, so one works and another fails or is slower if one path has a problem, causing intermittent, per-connection failures that vary with source port; traceroute can show inconsistent paths because probes hash to different next hops; hash collisions send multiple heavy flows to one link while others sit idle; asymmetric paths can break stateful firewalls, because the request and reply hash differently; and when a path is added or removed, the hash redistributes flows and may reset connections unless resilient hashing is used. To debug, vary the source port and test repeatedly to find the bad path, use tools such as paris-traceroute that keep the flow consistent, and check the hash inputs at each device. Mitigate with symmetric hashing, stateful device synchronisation, and spreading flows with more entropy.

Likely follow-ups

- Why does ECMP cause a failure that depends on source port?
- What is resilient hashing?

Equal-Cost Multi-Path routing spreads traffic across multiple equal-cost paths for bandwidth and redundancy — but per-flow hashing can produce puzzling per-connection differences.

HOW ECMP WORKS

```
# Multiple equal-cost routes to a destination; the router hashes each FLOW
# (typically the 5-tuple: src/dst IP, src/dst port, proto) to pick a path.
# Same flow -> same path (consistent ordering); different flows -> spread.
```

WHY IT CONFUSES DEBUGGING

```
- One connection is fine, another to the SAME host is slow/lossy
  (they hashed onto different paths, one of which is degraded)
- Intermittent issues that depend on the source port chosen for the flow
- A capture on one path misses traffic taking another path
- "It works sometimes" — really "it works on the healthy ECMP path"
```

Gotcha: Because ECMP pins a flow to a path by hash, a single bad link/path causes a *fraction* of connections to fail or slow down while others are perfectly fine — looking random, but it's deterministic per 5-tuple. Re-running with a different source port may hash to a good path and "fix" it, masking the real bad link.

Interview tip: Recognize the pattern: per-connection inconsistency to the same destination, where some flows are bad and others fine, often means ECMP hashing onto a degraded path. The tell that it changes with source port (different hash -> different path) confirms

certs.eknathalabs.com

it. This is an advanced troubleshooting insight that signals you've debugged real multi-path networks, not just single links.

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

Q119 **How do you design and verify network failover so it actually works when you need it?** Troubleshooting

🎙 **Spoken answer · ~85 s**

Failover that's never tested usually fails when it matters. Designing it means accounting for detection time, convergence, and state, then proving it with drills. Failover time has components: detection, how quickly the failure is noticed, which is the health-check interval times the threshold, or BFD for routing; convergence, how fast traffic actually moves, which depends on DNS TTLs and client caching, BGP convergence, load balancer updates, and connection draining; and recovery of state, such as database promotion and cache warm-up. Design: remove single points of failure at every layer, with redundant links, devices, and zones; use health checks that reflect real service capability; choose fast mechanisms such as load balancers or anycast over slow ones like long-TTL DNS where the objective requires; ensure surviving capacity can carry the load; handle stateful components with replication and tested promotion procedures; and avoid split-brain with quorum and fencing. Verification: run regular failover drills and game days, including killing a zone, a link, or a database primary in production-like conditions, measuring actual detection and recovery time against the RTO and RPO, and watching for hidden dependencies and client behaviour such as stale DNS or non-retrying clients. Automate the failover where possible and keep runbooks for manual steps.

Likely follow-ups

- What are the components of failover time?
- Why monitor redundancy itself?

Failover that's never tested usually fails when it matters. Designing it means accounting for detection time, convergence, and state — then proving it with drills.

THE COMPONENTS OF FAILOVER TIME

```
Detection  : how fast you notice failure (health-check interval x threshold)
Convergence: how fast traffic reroutes (DNS TTL, BGP, gratuitous ARP, LB dereg)
State      : what's lost on cutover (in-flight requests, sessions, sticky state)
RTO/RPO    : the actual downtime + data-loss you can tolerate
```

WHERE FAILOVER SILENTLY DOESN'T WORK

```
- DNS failover blocked by long client/JVM caching (TTL ignored) -> slow/no cutover
- Stale ARP/conntrack after VIP move -> peers keep hitting the dead node
- Standby was never actually in sync / never tested -> promotes to bad state
- Health check too lax (detects failure minutes late) or too aggressive (false failover)
```

VERIFY IT

certs.eknathalabs.com

```
- GAME DAYS: actually kill the primary / a zone and measure real RTO/RPO
- Chaos testing: inject failures continuously, not once a year
- Monitor failover events; alert if the standby is unhealthy/out of sync
```

Gotcha: The most common failure is assuming a configured failover works — DNS TTL caching, unsynced standbys, and stale ARP/conntrack routinely defeat it. Detection + convergence often add up to far more than the "60s TTL" you assumed. Untested failover is a liability, not a safety net.

Interview tip: Break failover into detection + convergence + state, and name the silent killers (DNS/JVM caching defeating TTL, stale ARP/conntrack, unsynced standby). The headline: **test it with game days / chaos engineering** and measure real RTO/RPO — "we have failover configured" is meaningless until you've pulled the plug and watched it recover. This maturity is exactly what senior/SRE interviews probe.

📚 **Official docs**

- RFC 826: ARP
- Google SRE Book: Data Integrity

Q120 **Design the network architecture for a highly available 3-tier web application across multiple AZs.** Cloud networking

🎙 **Spoken answer · ~84 s**

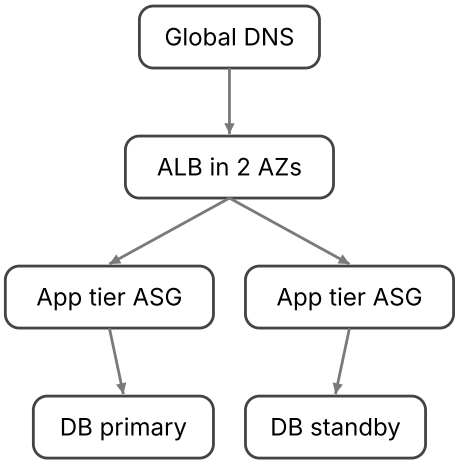
This is a capstone design question, and the goal is no single point of failure, secure tiering, and graceful handling of an AZ loss. Per region, span at least two, preferably three, availability zones. At the edge: DNS with health-checked routing, a CDN, and a WAF and DDoS protection. Tier one: public subnets in each AZ holding an internet-facing, multi-AZ layer-seven load balancer, plus one NAT gateway per AZ for outbound access. Tier two: private application subnets in each AZ, running stateless services in an auto scaling group or Kubernetes node groups spread across zones, behind the load balancer, with health checks and connection draining; capacity sized so remaining zones can absorb the loss of one. Tier three: isolated data subnets with no internet route, running a managed database with a multi-AZ standby and automatic failover, read replicas, and a replicated cache; backups to object storage, cross-region where required. Security: security groups chained by tier, so the load balancer reaches the application and only the application reaches the database; network ACLs as a coarse guard; private endpoints for cloud services; TLS everywhere; secrets management; flow logs and centralised logging. Operations: infrastructure as code, monitoring per AZ, and regular AZ failure tests.

Likely follow-ups

- Why one NAT gateway per AZ?
- How would you size the application tier for an AZ loss?

certs.eknathalabs.com

🗺 **HA 3-tier**



This is a capstone design question. The goal: no single point of failure, secure tiering, and graceful handling of an AZ loss.

THE ARCHITECTURE (PER REGION, SPANNING >=2 AZS)

```
Edge   : Route53/GeoDNS + CDN + WAF/DDoS protection
Tier 1 : PUBLIC subnets (one per AZ) -> internet-facing LB (multi-AZ, L7)
Tier 2 : PRIVATE subnets (one per AZ) -> app servers in an autoscaling group,
         spread across AZs; reachable ONLY from the LB's security group
Tier 3 : PRIVATE/data subnets (one per AZ) -> primary DB + standby in another AZ
         (synchronous replica), reachable only from the app tier's SG
Egress : NAT Gateway per AZ (not one shared -> avoid an AZ-failure SPOF)
Private : VPC endpoints for S3/API traffic (skip NAT, stay private)
```

HA PROPERTIES

```
- Multi-AZ everything: lose an AZ -> LB routes to surviving AZs, DB fails over
- Autoscaling replaces failed app instances; health checks gate the pool
- Least-privilege SGs per tier (LB->app->db chain, no public IPs on app/db)
- Connection draining + graceful shutdown for zero-downtime deploys
```

Common SPOF mistakes: a single NAT Gateway (its AZ dies -> all private egress dies — use one per AZ), a single-AZ database with no standby, or all replicas in one AZ. Also: a deep LB health check on a shared DB can fail the whole app tier at once. Spread across AZs and keep checks shallow.

Interview tip: Walk it tier by tier (edge -> public LB -> private app ASG -> private multi-AZ DB), stressing multi-AZ at every layer, least-privilege SG chaining (LB->app->DB, no public IPs below the LB), and per-AZ NAT gateways to avoid an egress SPOF. Then state how it survives an AZ loss. Calling out the classic SPOFs (single NAT GW, single-AZ DB) and tying in draining/graceful shutdown shows you can design AND operate it.

📚 **Official docs**

- Amazon RDS: Multi-AZ deployments

certs.eknathalabs.com

Q121 **How do switches learn MAC addresses, and what is the difference between a hub, a switch and a router?** Fundamentals

🎙 Spoken answer · ~64 s

A hub is a layer one repeater: whatever arrives on one port is copied to every other port, so everyone shares bandwidth and sees everyone's traffic. A switch works at layer two. When a frame arrives, it records the source MAC address against the incoming port in its address table, then looks up the destination MAC and forwards the frame only out of that port. If the destination is unknown or is a broadcast, the switch floods the frame to all ports in the same VLAN. Entries age out after a few minutes. A router works at layer three: it examines the destination IP address, consults its routing table and forwards the packet to the next hop, rewriting the layer two header, and it does not forward broadcasts, so it divides broadcast domains. Layer three switches combine both functions. In the cloud, virtual switches and routers are software, but the same logic applies.

Likely follow-ups

- What happens when the MAC table is full?
- What is a broadcast domain?

A hub repeats every frame to all ports. A switch builds a MAC address table from source addresses and forwards frames only to the right port, flooding unknown destinations. A router forwards packets between IP networks using routing tables and separates broadcast domains.

```
bridge fdb show | head
```

```
ip neigh show
```

```
arp -an
```

```
sudo tcpdump -eni eth0 -c 5 arp
```

Interview tip: Add that VLANs split one switch into several broadcast domains, and a router or L3 switch is needed to move between them.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q122 **What is the five-tuple, and why do sockets, ports and connection tracking all revolve around it?** Fundamentals

🎙 Spoken answer · ~62 s

Every TCP or UDP flow can be uniquely identified by five values: the protocol, the source address and port, and the destination address and port. A server listening on port four hundred forty-three can hold hundreds of thousands of simultaneous connections because each client contributes a different source address or port, making every tuple unique. Network devices use the tuple to keep state. A stateful firewall remembers that an outbound flow was allowed and permits the return packets, a NAT gateway maps the internal tuple to an external tuple, a load balancer hashes the tuple to choose a backend consistently, and the kernel's conntrack tracks it. This explains several common problems: a client connecting to the same destination from one source IP can run out of source ports, ECMP hashing can pin a heavy flow to one path, and NAT devices have limited port capacity per external address.

Likely follow-ups

- Why does ECMP pin one flow to one path?
- How does this relate to ephemeral port exhaustion?

A flow is identified by protocol, source IP, source port, destination IP and destination port. A listening socket accepts many connections because each has a unique five-tuple. NAT, firewalls, load balancers and conntrack key their state on it.

```
ss -tan state established
```

```
conntrack -L | head
```

```
ss -o state established '( sport = :443 )'
```

Interview tip: Port exhaustion is a five-tuple problem: a client has only about 28,000 ephemeral ports per source IP and destination pair.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q123 **What are private IP ranges, CGNAT and link-local addresses, and why do they matter when designing cloud and VPN networks?**

Fundamentals

🎙 Spoken answer · ~64 s

Private ranges are not routable on the public internet, so every organisation reuses them, which is fine until networks must connect. If two VPCs, a data centre and a partner all use ten dot zero dot zero dot zero slash sixteen, peering, VPN and transit gateways cannot route between them without NAT. So I keep a central IP address plan, allocate ranges per environment, region and account with room to grow, and check for overlap automatically in the pipeline. The shared address space one hundred dot sixty-four slash ten is used by service providers for carrier-grade NAT, and it is also handy as a secondary range for container pods, such as with EKS custom networking, because it rarely clashes with corporate ranges. Link-local addresses in one sixty-nine dot two fifty-four slash sixteen are not routed and are used for things like the instance metadata service, which is why securing access to that address matters.

Likely follow-ups

- Why use 100.64.0.0/10 for pods?
- How would you check for overlaps automatically?

RFC 1918 reserves 10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16 for private use. 100.64.0.0/10 is carrier-grade NAT shared space. 169.254.0.0/16 is link-local, used for metadata services. Overlap between environments breaks routing and peering.

```
ipcalc 10.0.0.0/16
```

```
ip route get 169.254.169.254
```

```
python3 -c "import ipaddress as i;
print(i.ip_network('10.1.0.0/16').overlaps(i.ip_network('10.1.128.0/17')))"
```

Interview tip: Plan one non-overlapping address plan for all clouds and on-premises from day one. Re-addressing later is painful.

📚 Official docs

- RFC 1918: Private address space
- Cloudflare Learning: What is a VPN?

certs.eknathalabs.com

Q124 **How do SPF, DKIM, DMARC and MX records work together to protect email, and how do you debug them with DNS?** DNS

🎙 Spoken answer · ~64 s

Email authentication is almost entirely published in DNS. MX records list the mail servers that accept mail for the domain, with priorities. An SPF record, a TXT record, lists which IP addresses or providers may send mail using the domain in the envelope sender, and receivers check the connecting IP against it. DKIM has the sending server sign selected headers and the body, adding a signature header, and the receiver fetches the public key from a TXT record under the selector and domain key name to verify it. DMARC builds on both: it requires that the domain in the visible From address aligns with an SPF or DKIM pass, and sets a policy of none, quarantine or reject, plus addresses for aggregate reports. When mail is rejected, I use dig to check each record, watch the SPF ten lookup limit, ensure DKIM keys are published correctly, and begin DMARC in monitoring mode before enforcing.

Likely follow-ups

- What is DMARC alignment?
- What happens when SPF exceeds the lookup limit?

MX records say where mail for a domain is delivered. SPF lists servers allowed to send for the domain. DKIM adds a cryptographic signature verified with a public key in DNS. DMARC tells receivers what to do when SPF and DKIM alignment fail, and where to send reports.

```
dig +short MX example.com
```

```
dig +short TXT example.com | grep spf
```

```
dig +short TXT selector1._domainkey.example.com
```

```
dig +short TXT _dmarc.example.com
```

Interview tip: SPF has a limit of ten DNS lookups. Too many include mechanisms silently break it.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q125 **What are delegation, NS records and glue records, and why can a missing or wrong glue record break a domain?** DNS

🎙 Spoken answer · ~67 s

DNS is a hierarchy, and a zone gets authority over a subdomain through delegation. The parent zone contains NS records naming the authoritative servers for the child. If the name server names are in another domain, a resolver can look those up separately. But if the name servers are inside the delegated zone, such as ns1 dot example dot com for example dot com, resolving the name server requires already knowing where example dot com is served, a circular dependency. Glue records break this loop: the parent, such as the dot com servers, includes the IP addresses of those name servers in its response. If glue is missing, stale or wrong, resolvers can't reach the zone, or reach the old servers after a migration. A lame delegation points to servers that are not authoritative for the zone. I diagnose with dig trace, which follows delegation from the root, and compare NS sets at the parent and child, which must match.

Likely follow-ups

- What is a lame delegation?
- Why must parent and child NS sets match?

A parent zone delegates a child zone using NS records. If the name servers are inside the delegated domain itself, the parent must also serve glue A or AAAA records, otherwise resolvers can't find the name servers, creating a circular dependency.

```
dig +trace example.com
```

```
dig NS example.com @a.gtld-servers.net
```

```
dig +norec A ns1.example.com @a.gtld-servers.net
```

Interview tip: When DNS fails only for new or recently moved domains, +trace usually reveals a broken delegation or lame server.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q126 **What is EDNS Client Subnet and how does geo-DNS decide which answer to return?** DNS

🎙 Spoken answer · ~61 s

Geographic and latency-based DNS routing choose an answer by estimating where the client is. But the authoritative server only sees the IP address of the recursive resolver that asked, which for a user of a public resolver or a corporate resolver in another country can be far from the user, leading to poor choices. EDNS Client Subnet is an extension where the resolver includes a truncated prefix of the client's address, typically a slash twenty-four, so the authoritative server can pick the closest edge or region. The trade-offs are privacy, since part of the client address is exposed, and cache fragmentation, because answers are cached per subnet. Some resolvers disable it. When troubleshooting why a user is sent to a distant region, I compare answers from different resolvers and test with dig's subnet option. Anycast CDNs reduce reliance on this by routing at the network layer.

Likely follow-ups

- What is the privacy concern with ECS?
- How does anycast avoid this problem?

Authoritative servers normally see only the resolver's IP, not the end user. EDNS Client Subnet lets the resolver pass a truncated client prefix so the authority can return a nearby endpoint. Without it, users of a distant public resolver get a poor geographic answer.

```
dig +subnet=203.0.113.0/24 www.example.com @8.8.8.8
```

```
dig +short www.example.com @1.1.1.1
```

```
dig +short www.example.com @8.8.8.8
```

Interview tip: Differences between resolvers' answers for a CDN hostname are a hint that geo or ECS-based routing is in play.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q127 **How do DNS amplification attacks work, and what do response rate limiting and closing open resolvers do?** DNS

🎙 Spoken answer · ~60 s

DNS uses UDP, which does not verify the source address, so an attacker can send queries with the victim's address forged as the source. The server sends its reply to the victim. Because certain queries, such as those for ANY or DNSSEC-signed records, produce responses many times larger than the request, a small amount of attacker bandwidth is multiplied into a large flood against the victim. To avoid being an amplifier, I make sure recursive resolvers answer only trusted clients and are not open to the internet, enable response rate limiting on authoritative servers so repeated identical responses to one address are throttled, minimise large responses by not answering ANY and keeping records tight, and where I control the network, apply ingress filtering so spoofed packets never leave. As a victim, I rely on upstream DDoS protection, anycast and rate limiting at the edge.

Likely follow-ups

- What is BCP38?
- Why is ANY queries special?

Attackers send small queries with a spoofed victim source IP to open resolvers or authoritative servers, which reply with much larger answers to the victim. Defences include not running open recursive resolvers, response rate limiting, BCP38 source filtering and anycast capacity.

```
dig ANY example.com @ns1.example.com +bufsize=4096
```

```
dig +short test.openresolver.com TXT
```

```
named-checkconf; grep -A4 'rate-limit' named.conf
```

Interview tip: Say amplification factor: a 60-byte query can trigger a response of several kilobytes.

📚 Official docs

- Cloudflare Learning: DDoS attacks
- RFC 6585: HTTP 429 Too Many Requests
- Microsoft: Cloud design pattern – Throttling

certs.eknathalabs.com

Q128 **How do TCP retransmissions, fast retransmit and SACK work, and how do you spot trouble in a capture?** TCP/UDP

🎙 Spoken answer · ~60 s

TCP guarantees delivery by numbering bytes and requiring acknowledgements. If the sender does not see an ACK within a retransmission timeout, derived from measured round-trip time, it resends the data and doubles the timeout each time, so an unlucky segment can stall a connection for seconds. To react faster, if the receiver gets out-of-order data it sends duplicate ACKs, and after three duplicates the sender retransmits the missing segment immediately, which is fast retransmit. Selective acknowledgements add detail in the ACK about which blocks have arrived, so the sender fills only the gaps instead of resending everything after the loss. In a capture, I filter for retransmissions, duplicate ACKs and zero windows. Isolated events are normal, but high rates show packet loss, congestion, a bad link, or an overloaded receiver. Comparing per-path counters tells me whether the problem is local or in the network.

Likely follow-ups

- What triggers a retransmission timeout?
- What does a zero window indicate?

If an ACK doesn't arrive before the retransmission timeout, the sender resends, with exponential backoff. Three duplicate ACKs trigger fast retransmit without waiting. SACK lets the receiver report which ranges arrived so only missing segments are resent.

```
ss -ti dst 10.0.0.5
```

```
nstat -az TcpRetransSegs TcpExtTCPFastRetrans
```

```
tshark -r cap.pcap -Y 'tcp.analysis.retransmission || tcp.analysis.fast_retransmission' |
head
```

```
netstat -s | grep -i retrans
```

Interview tip: A few retransmits are normal. A rising retransmit percentage on one path points to loss or congestion there.

📚 Official docs

- RFC 9293: TCP

certs.eknathalabs.com

Q129 **What are Nagle's algorithm and delayed ACKs, and why can they add 40 ms latency to small requests?** TCP/UDP

🎙 Spoken answer · ~61 s

Nagle's algorithm tries to avoid flooding the network with tiny segments. If there is unacknowledged data in flight, new small writes are buffered until an ACK returns or enough data accumulates to fill a segment. Delayed ACK is the receiver-side optimisation: instead of ACKing every segment immediately, it waits briefly, typically up to forty milliseconds on Linux, hoping to send the ACK along with response data. When an application writes a request in two small pieces, for example a header then a body, the second write waits for an ACK, and the receiver waits to send that ACK, producing a forty millisecond stall on every request. The cure is to write the whole message in one call, or to disable Nagle on the socket with TCP_NODELAY, which most latency-sensitive clients and servers, including many HTTP libraries and databases, do by default. Packet captures show the pause.

Likely follow-ups

- Why does it matter for request-response protocols?
- When is Nagle still useful?

Nagle holds back small writes until outstanding data is acknowledged, to reduce tiny packets. Delayed ACK makes the receiver wait up to about 40 ms to ACK, hoping to piggyback. Together they can stall request-response protocols. Set TCP_NODELAY and write complete messages at once.

```
ss -ti | grep -i nodelay
```

```
setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &one, sizeof(one))
```

```
sudo tcpdump -ni eth0 'tcp port 8080' -tttt
```

Interview tip: The tell-tale sign is a consistent 40 ms (or 200 ms) pause between a small write and the next, even on a fast LAN.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q130 **What is the bandwidth-delay product, and why do window scaling and buffer sizes limit throughput on high-latency links?** TCP/UDP

🎙 Spoken answer · ~61 s

To use a link fully, a sender must keep enough unacknowledged data in flight to cover the time it takes for the first byte to be acknowledged. That amount is the bandwidth multiplied by the round-trip time. For a one gigabit link with a hundred millisecond RTT, it is about twelve and a half megabytes. The classic TCP window is limited to sixty-four kilobytes, so the window scaling option multiplies it, allowing much larger windows. If socket buffers or the receive window stay small, the throughput ceiling is window divided by RTT, so a long fat network between regions performs badly even though the link is empty. I check that window scaling is enabled, raise the maximum values in the tcp read and write memory settings, let auto-tuning work, use multiple parallel streams for bulk transfers, and consider BBR congestion control. Measuring with iperf3 shows the effect.

Likely follow-ups

- How does BBR help on long links?
- Why do parallel streams help?

BDP is bandwidth multiplied by round-trip time, the amount of data that must be in flight to fill the pipe. If the TCP window or socket buffers are smaller than the BDP, throughput is capped at window divided by RTT. Window scaling and tuned buffers fix it.

```
sysctl net.ipv4.tcp_window_scaling net.ipv4.tcp_rmem net.ipv4.tcp_wmem
```

```
iperf3 -c host -P 4 -t 20
```

```
ss -ti dst host | grep -E 'wscale|cwnd|rtt'
```

Interview tip: Example to quote: 1 Gbps at 100 ms RTT needs about 12.5 MB in flight, and a 64 KB window gives only around 5 Mbps.

📚 Official docs

- Linux kernel docs: sysctl

certs.eknathalabs.com

Q131 **What is QUIC, and how does it differ from TCP with TLS in connection setup, loss recovery and migration?**

TCP/UDP

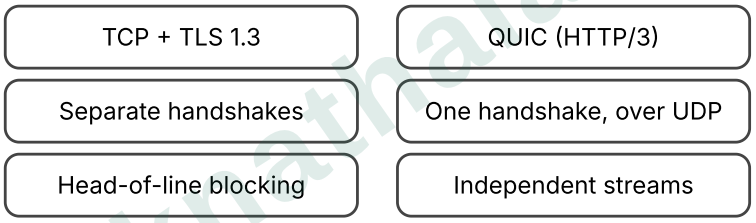
🎙 Spoken answer · ~60 s

TCP plus TLS needs a transport handshake and then a separate cryptographic handshake before application data flows. QUIC folds them together, so a new connection costs one round trip, and a resumed one can send data in the first flight with zero RTT, with replay caveats. QUIC multiplexes independent streams over a single connection, so packet loss on one stream does not block the others, unlike TCP where a lost segment delays everything behind it. It is implemented in user space over UDP, allowing faster iteration than kernel TCP, and its connection identifier, instead of the address tuple, lets a mobile client move from Wi-Fi to cellular without dropping the connection. HTTP/3 is HTTP over QUIC. Operationally, UDP port four forty-three must be allowed, some networks throttle UDP, load balancers must support it, and observability differs because payloads are encrypted, including most transport headers.

Likely follow-ups

- What is 0-RTT risk?
- Why can't middleboxes inspect QUIC easily?

🗺 QUIC vs TCP+TLS



QUIC runs over UDP, integrates TLS 1.3, has multiple independent streams without head-of-line blocking at the transport layer, supports 0-RTT resumption, and connection IDs let connections survive IP changes. It underpins HTTP/3 but is harder to inspect and rate-limit on middleboxes.

```
curl --http3 -I https://example.com
```

```
dig +short HTTPS example.com
```

```
sudo tcpdump -ni eth0 udp port 443
```

Interview tip: Remember to allow UDP 443 through firewalls, otherwise browsers silently fall back to HTTP/2 and you lose the benefit.

📚 Official docs

- RFC 9114: HTTP/3

certs.eknathalabs.com

Q132 **How do OSPF and BGP differ, and where do you meet each in cloud and enterprise networks?**

Routing & subnets

🎙 Spoken answer · ~61 s

OSPF floods link-state information so every router builds the same map of the area and runs shortest path first to compute routes, which gives fast and automatic convergence inside a single administrative domain. It is simple to run in a campus or data centre but does not scale to the internet. BGP is a path-vector protocol where autonomous systems advertise prefixes along with attributes such as AS path, local preference and MED, and policy decides which paths are preferred rather than pure shortest distance. That is why it is the protocol of the internet and of interconnect between organisations. In the cloud I meet BGP everywhere: VPN gateways, Direct Connect, ExpressRoute and Cloud Router exchange routes with on-premises routers over BGP, and Kubernetes CNIs like Calico or Cilium can peer with top-of-rack switches. Within a data centre fabric, many designs now use eBGP throughout instead of OSPF.

Likely follow-ups

- What is the AS path used for?
- iBGP versus eBGP?

OSPF is an interior gateway protocol: link-state, fast convergence, used inside one organisation. BGP is the exterior path-vector protocol, policy-driven, used between organisations and by cloud gateways, with iBGP and eBGP variants. Use OSPF inside a data centre, BGP at the edges.

```
vtysh -c 'show ip ospf neighbor'
```

```
vtysh -c 'show ip bgp summary'
```

```
aws ec2 describe-vpn-connections --query 'VpnConnections[].VgwTelemetry'
```

Interview tip: Cloud VPN, Direct Connect and ExpressRoute all rely on BGP for dynamic route exchange.

📚 Official docs

- RFC 4271: BGP-4
- Cloudflare Learning: What is BGP?

certs.eknathalabs.com

Q133 What is 802.1Q VLAN trunking, and how does inter-VLAN routing work?

Routing & subnets

🎙 Spoken answer · ~62 s

VLANs split a physical switch into multiple logical layer two networks, each its own broadcast domain. When a link has to carry several VLANs, such as between switches or to a hypervisor, it is configured as a trunk, and each frame carries an 802.1Q tag with a VLAN identifier from one to four thousand ninety-four. Access ports carry one VLAN without tags, and the switch adds and removes tags. Because VLANs are separate networks, devices in different VLANs need a router to talk. This is done either with one router interface using sub-interfaces per VLAN, called router on a stick, or more commonly with switch virtual interfaces on a layer three switch, where each VLAN gets a gateway address. Problems come from trunk VLAN lists that omit a VLAN, native VLAN mismatches, and missing gateway configuration. Linux can create VLAN sub-interfaces, and clouds hide this with VPC subnets.

Likely follow-ups

- What is the native VLAN?
- Router-on-a-stick versus SVI?

A trunk port carries multiple VLANs by tagging frames with a 12-bit VLAN ID. Access ports belong to a single untagged VLAN. Hosts in different VLANs communicate only through a layer three device, via router-on-a-stick sub-interfaces or SVIs on a layer three switch.

```
ip -d link show eth0.100
```

```
sudo ip link add link eth0 name eth0.100 type vlan id 100
```

```
bridge vlan show
```

Interview tip: Mention the native VLAN: untagged traffic on a trunk lands there, and a mismatch is a classic misconfiguration and security issue.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q134 **How do VRRP and HSRP provide gateway redundancy, and what goes wrong in a split brain?**

Routing & subnets

🎙 Spoken answer · ~61 s

Hosts are configured with a single default gateway address, so to survive a router failure, VRRP or Cisco's HSRP lets two routers present one virtual IP and virtual MAC. The elected master owns the address, answers ARP and forwards traffic, and sends periodic advertisements. If the backups stop hearing them for a few intervals, the highest priority backup takes over, and sends a gratuitous ARP so switches and hosts update. Keepalived implements VRRP on Linux for load balancers and gateways. The main failure mode is split brain: if the link carrying the advertisements fails while both nodes remain up, each thinks the other is dead and claims the address, producing duplicate IPs, intermittent connectivity and state confusion. To reduce the risk I use redundant heartbeat paths, authenticate advertisements, tune timers, and add tracking scripts that lower priority if the upstream link or the service is unhealthy.

Likely follow-ups

- What does a gratuitous ARP do on failover?
- How would you test failover safely?

Two or more routers share a virtual IP and MAC. The master answers ARP and forwards; backups listen to its advertisements and take over if they stop. If the advertisement path breaks but both routers are alive, both become master (split brain), causing duplicate IPs and flapping.

```
ip addr show | grep -A2 vrrp
```

```
sudo tcpdump -ni eth0 vrrp
```

```
systemctl status keepalived; journalctl -u keepalived -n 30
```

Interview tip: Use a dedicated heartbeat path and consider tracking upstream interfaces so a router with no uplink gives up mastership.

📚 Official docs

- Google SRE Book: Managing Critical State (consensus)

certs.eknathalabs.com

Q135 **What are administrative distance and route summarisation, and how do they influence which route is used?**

Routing & subnets

🎙 **Spoken answer · ~62 s**

A router decides in a fixed order. First, longest prefix match: a more specific route such as a slash twenty-four always beats a slash sixteen, whatever its source. If several sources offer the same prefix, such as a static route, OSPF and BGP, the one with the lowest administrative distance is trusted, for example connected routes are zero, static is one, eBGP twenty and OSPF one hundred ten on typical routers. Only when comparing routes from the same protocol does the metric matter. Understanding this order explains surprises like a floating static route, deliberately given a higher distance so it serves as backup only when the dynamic route disappears. Summarisation combines many specific prefixes into one covering prefix when advertising to other routers. It keeps routing tables small and hides instability inside a region, but it can attract traffic for unreachable addresses, creating black holes, if not handled carefully.

Likely follow-ups

- What is a floating static route?
- How can summarisation create a black hole?

When several protocols offer a route to the same prefix, the lowest administrative distance wins before metrics are compared. Within a protocol, metric chooses. Longest prefix match always beats both. Summarisation advertises one covering prefix to shrink tables and hide flaps.

```
ip route show table all | head
```

```
ip route get 10.20.30.40
```

```
vtysh -c 'show ip route 10.20.0.0/16'
```

Interview tip: Order of decision: longest prefix first, then administrative distance, then metric.

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q136 **What are HSTS, redirects and mixed content, and how do they affect a move to HTTPS?**

HTTP/TLS

🎙 Spoken answer · ~63 s

After enabling TLS I first redirect all HTTP requests to HTTPS with a permanent redirect so existing links work. But that first request is still over plain HTTP and vulnerable to interception. HSTS fixes this: the Strict-Transport-Security response header tells the browser to use HTTPS for the domain for max-age seconds, so later visits skip the insecure step and certificate errors cannot be clicked through. Adding includeSubDomains and the preload flag extends this to all subdomains and to browser built-in lists, but these are hard to reverse, so I start with a short max-age, verify every subdomain supports HTTPS, then raise it. Mixed content occurs when an HTTPS page loads scripts, images or frames over HTTP, which browsers block or flag; I fix it by using relative or HTTPS URLs and a Content-Security-Policy with upgrade-insecure-requests during migration. I also check redirect loops caused by TLS termination at a load balancer.

Likely follow-ups

- What does preload do?
- How can a redirect loop appear behind a load balancer?

HSTS tells browsers to use HTTPS only for a domain for a set period, preventing downgrade. Redirects from HTTP to HTTPS are needed for first visits. Mixed content, HTTP resources on an HTTPS page, is blocked or warned. Roll out HSTS gradually.

```
curl -sI http://example.com | head -5
```

```
curl -sI https://example.com | grep -i strict-transport
```

```
curl -s https://example.com | grep -o 'http://[^"]*' | head
```

Interview tip: Start with a short max-age, then increase. Preload and includeSubDomains are hard to undo.

📚 Official docs

- Mozilla: Server-side TLS configuration
- Mozilla Observatory / HTTP security headers (MDN)

certs.eknathalabs.com

Q137 **WebSockets, Server-Sent Events and long polling: how do they differ, and what does each require from proxies and load balancers?**

HTTP/TLS

🎙 Spoken answer · ~63 s

All three approaches give near real-time updates but have different shapes. Long polling has the client send a request that the server holds until data is ready, then the client immediately asks again, which works everywhere but is inefficient. Server-Sent Events use one long-lived HTTP response with a text event stream, one way from server to browser, with automatic reconnect built into the browser. WebSockets start as an HTTP request with an Upgrade header, then switch to a persistent bidirectional framed protocol, suitable for chat or collaboration. Operationally, each ties up a connection for a long time, so proxies and load balancers must allow the upgrade headers, set long idle and read timeouts, and avoid buffering responses. Heartbeats or pings keep intermediaries from closing idle connections. Scaling needs attention to connection limits, graceful draining on deploys, and a shared pub/sub layer so a message reaches whichever server holds the recipient's connection.

Likely follow-ups

- Why do connections drop at exactly 60 seconds?
- How do you deploy without dropping all sockets?

WebSockets upgrade an HTTP connection to a full-duplex channel. SSE is a one-way server push over a long HTTP response. Long polling repeats held requests. All keep connections open, so proxies need long timeouts, upgrade headers support and sticky or connection-aware balancing.

```
curl -N -H 'Accept: text/event-stream' https://example.com/events
```

```
curl -i -N -H 'Connection: Upgrade' -H 'Upgrade: websocket' -H 'Sec-WebSocket-Version: 13'
-H 'Sec-WebSocket-Key: x3JJHMbDL1EzLkh9GBhXDw==' https://example.com/ws
```

```
nginx -T | grep -E 'proxy_read_timeout|Upgrade'
```

Interview tip: Remember the default 60 second idle timeout in many proxies and load balancers, which is why connections drop at exactly one minute without heartbeats.

📚 Official docs

- Cloudflare Learning: Load balancing

certs.eknathalabs.com

Q138 **What are X-Forwarded-For, Forwarded and the PROXY protocol, and how do you preserve the real client IP behind proxies?**

HTTP/TLS

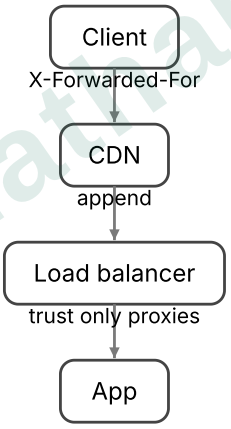
🎙 Spoken answer · ~63 s

When traffic passes through a reverse proxy or load balancer, the application sees the proxy's address as the peer. For HTTP, proxies add an X-Forwarded-For header listing the original client and each hop, plus X-Forwarded-Proto for the scheme, and the standardised Forwarded header does the same. The application or web server must be configured to trust these headers only when the immediate peer is a known proxy, because anyone can send a forged header. The correct approach is to walk the list from the right, skipping trusted proxy addresses, and take the first untrusted one. For non-HTTP protocols, or TLS passthrough where headers can't be added, the PROXY protocol prefixes the connection with a small header carrying source and destination addresses, supported by HAProxy, NGINX and cloud network load balancers, but both sides must be configured for it or connections break. Network load balancers can also preserve the source IP directly.

Likely follow-ups

- Why take the right-most untrusted entry?
- What breaks if only one side enables PROXY protocol?

🗺 Client IP through proxies



Proxies terminate the client connection, so the backend sees the proxy's address. X-Forwarded-For appends client IPs as an HTTP header; PROXY protocol prepends connection info at the TCP level and works for non-HTTP traffic. Only trust these from known proxies, since clients can forge them.

```
curl -H 'X-Forwarded-For: 1.2.3.4' https://example.com/headers
```

```
nginx: set_real_ip_from 10.0.0.0/8; real_ip_header X-Forwarded-For;
```

```
aws elbv2 modify-target-group-attributes --target-group-arn <arn> --attributes
Key=proxy_protocol_v2.enabled,Value=true
```

Interview tip: Take the right-most untrusted entry, not the left-most, otherwise attackers can spoof their address to bypass rate limits.

certs.eknathalabs.com

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

Q139 **How do HTTP compression and content negotiation work, and what are the pitfalls with caching and security?**

HTTP/TLS

🎙 **Spoken answer · ~63 s**

With content negotiation, the client advertises what it can accept, for example gzip, brotli or zstd in Accept-Encoding, and the server picks one and labels the response with Content-Encoding. Text formats like HTML, CSS, JavaScript and JSON shrink by seventy to ninety percent, which reduces latency on slow links, but already compressed formats such as images and video gain nothing. Because the same URL can return different bodies, the server must send Vary: Accept-Encoding so shared caches and CDNs do not serve a compressed body to a client that can't decode it. For static assets I pre-compress at build time with the highest brotli level, for dynamic responses a moderate level balances CPU and size. The security caveat is BREACH, where compression of responses that mix secrets, like CSRF tokens, with attacker-controlled input can reveal the secret through size changes, so I mask tokens or disable compression for sensitive responses.

Likely follow-ups

- What does Vary do?
- What is BREACH?

The client sends Accept-Encoding and the server may reply with Content-Encoding gzip or br. Varying responses require Vary: Accept-Encoding so caches store separate copies. Compressing secrets alongside attacker-controlled input can leak them (BREACH).

```
curl -sI -H 'Accept-Encoding: br,gzip' https://example.com/app.js | grep -i -E 'content-encoding|vary|content-length'
```

```
curl -s --compressed -o /dev/null -w '%{size_download}\n' https://example.com/
```

```
gzip -9 -c app.js | wc -c
```

Interview tip: Pre-compress static assets at build time with brotli, and compress dynamic responses at a moderate level to save CPU.

📚 **Start here (official docs)**

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q140 **IPsec, WireGuard and OpenVPN: how do they differ, and when do you choose each?**

Network security

🎙 Spoken answer · ~65 s

IPsec is the interoperable standard. Key exchange with IKE sets up security associations, and ESP encrypts packets in the kernel, which performs well and is what AWS, Azure and Google VPN gateways and most hardware routers speak, so it is the default for site-to-site. It can be complex to configure, with many parameters and NAT traversal quirks. OpenVPN runs in user space over TLS, on UDP or TCP, is highly configurable and traverses restrictive firewalls on port four forty-three, but has higher overhead. WireGuard has a tiny code base, modern fixed cryptography, a simple peer model based on public keys and allowed IP ranges, very fast handshakes and good roaming, and is in the Linux kernel. It lacks built-in user management and dynamic IP assignment, so tools build on top. I choose IPsec for cloud gateways, WireGuard for new overlays, remote access and Kubernetes node encryption, and OpenVPN where compatibility or TCP fallback is needed.

Likely follow-ups

- What does AllowedIPs do in WireGuard?
- Why does OpenVPN over TCP perform poorly?

IPsec is a standard suite, native in cloud VPN gateways and routers, with IKE negotiation and kernel performance. OpenVPN uses TLS over UDP or TCP in user space, flexible but slower. WireGuard is a small, fast, kernel-level design with static keys and a simple configuration.

```
sudo wg show
```

```
sudo ipsec statusall
```

```
ip xfrm state | head
```

```
wg genkey | tee priv | wg pubkey > pub
```

Interview tip: For site-to-site into a cloud, IPsec is usually the only option the provider offers. For remote access and overlays, WireGuard is often the simplest.

📚 Official docs

- Cloudflare Learning: What is a VPN?
- WireGuard documentation

certs.eknathalabs.com

Q141 **What is perfect forward secrecy and how do cipher suites and key exchange relate to it?**

Network security

🎙 Spoken answer · ~61 s

In older TLS configurations, the client encrypted a pre-master secret with the server's long-term RSA public key. Anyone who recorded the traffic and later obtained the private key could decrypt every past session. Perfect forward secrecy removes that risk by using ephemeral Diffie-Hellman exchanges: for each connection, both sides generate temporary key pairs, derive a shared secret, and discard the private values afterwards, while the long-term certificate key only authenticates the handshake. Compromise of the server key later does not reveal earlier session keys. In cipher suite names, ECDHE indicates this. TLS 1.3 removed static RSA key exchange entirely and supports only ephemeral key exchange with a small set of strong AEAD ciphers, which is one reason to enable it and disable TLS 1.0 and 1.1. I verify the configuration with scanners and keep session ticket keys rotated, since long-lived ticket keys can undermine forward secrecy.

Likely follow-ups

- Why do session ticket keys matter?
- What did TLS 1.3 remove?

With ephemeral Diffie-Hellman key exchange (ECDHE), each session derives a fresh key that is never stored. Recording traffic and later stealing the server's private key cannot decrypt past sessions. TLS 1.3 requires forward secrecy; static RSA key exchange does not provide it.

```
openssl s_client -connect example.com:443 -tls1_3 </dev/null 2>/dev/null | grep -E
'Protocol|Cipher'
```

```
nmap --script ssl-enum-ciphers -p 443 example.com
```

```
testssl.sh example.com
```

Interview tip: If asked how to check, say scan with testssl.sh or SSL Labs and look for ECDHE-only suites and no RSA key exchange.

📚 Official docs

- Mozilla: Server-side TLS configuration

certs.eknathalabs.com

Q142 **How do certificate revocation, OCSP and OCSP stapling work, and what happens when the check fails?**

Network security

🎙 Spoken answer · ~63 s

A certificate can be valid by date but still untrustworthy if its private key leaked. Revocation lets the CA say so. Certificate Revocation Lists are signed files listing revoked serial numbers, which can be large and slow to download. The Online Certificate Status Protocol lets a client query a responder for one certificate, but this adds latency and tells the CA which sites a user visits. OCSP stapling solves both problems: the server periodically fetches a signed response from the CA and sends it during the handshake, so the client gets a fresh status without contacting the CA. An extension called must-staple makes it mandatory. In practice, many browsers soft-fail when they can't check revocation, which weakens the protection, so the industry is moving toward short-lived certificates with automated renewal. When debugging, I check that the server can reach the responder, stapling is enabled and the response is not expired.

Likely follow-ups

- What does must-staple do?
- Why do browsers soft-fail?

CRLs list revoked certificates; OCSP asks a responder about one certificate. Stapling lets the server fetch and attach a recent signed OCSP response to the handshake, avoiding client lookups and privacy leaks. Browsers often soft-fail when revocation checks are unreachable.

```
openssl s_client -connect example.com:443 -status </dev/null 2>/dev/null | grep -A5 'OCSP response'
```

```
openssl x509 -in cert.pem -noout -ocsp_uri
```

```
nginx: ssl_stapling on; ssl_stapling_verify on;
```

Interview tip: Short-lived certificates reduce reliance on revocation, which is the direction many CAs are heading.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q143 **What is a DMZ and how does network segmentation reduce blast radius in modern architectures?**

Network security

🎙 Spoken answer · ~58 s

The idea behind a demilitarised zone is that anything exposed to the internet will eventually be attacked, so it should not sit on the same flat network as sensitive systems. Public-facing servers or load balancers live in a separate segment, with firewall rules that allow inbound traffic from the internet only to specific ports, and outbound traffic from the DMZ to the internal network only to the minimum needed, such as the application port to the app tier. The same principle applies in the cloud: public subnets for load balancers, private subnets for application servers, isolated subnets for databases, and security groups that reference each other rather than address ranges. In Kubernetes, network policies segment namespaces, and a service mesh adds identity-based rules. Combined with egress filtering, logging and regular testing, segmentation limits lateral movement after a compromise.

Likely follow-ups

- Why filter egress as well as ingress?
- How do security groups referencing groups help?

A DMZ is a segment for internet-facing services, isolated from internal networks by firewalls so a compromised web server cannot freely reach databases. Modern designs use VPC subnets, security groups, network policies and identity-aware proxies for finer segmentation.

```
aws ec2 describe-security-groups --group-ids sg-web --query
'SecurityGroups[].IpPermissions'
```

```
kubectl get networkpolicy -A
```

```
nmap -sT -p- 10.0.2.0/24   # verify from the web tier, authorised testing only
```

Interview tip: Test segmentation from inside: attempt connections from the web tier to the database tier and confirm only the intended ports work.

📚 Official docs

- AWS: Disaster recovery & availability concepts (Fault isolation boundaries)
- AWS Builders' Library: Workload isolation using shuffle sharding

certs.eknathalabs.com

Q144 **GSLB versus anycast: how do global load balancing approaches differ?** Load balancing

🎙 Spoken answer · ~59 s

Global server load balancing typically works through DNS. The authority answers with different addresses depending on the user's location, load, or site health, which allows rich policies such as weighted traffic and active-passive failover. Its weaknesses are DNS caching, which delays failover by the TTL and resolver behaviour, and that it sees the resolver rather than the user unless client subnet information is available. Anycast announces the same IP prefix from multiple points of presence, and internet routing delivers each user to the topologically nearest one. If a site fails and stops announcing, traffic shifts as BGP converges, usually within seconds, with no DNS caching issue. It suits stateless or short-lived protocols and edge services such as DNS and CDNs, but long-lived TCP sessions can break if routes shift mid-connection. Many platforms combine anycast edges for entry with application-aware load balancing inside.

Likely follow-ups

- Why can anycast break long TCP sessions?
- How do TTLs affect GSLB failover?

GSLB uses DNS or application-aware control to return the best site per user, with health checks and policies, but is limited by DNS caching. Anycast announces the same IP from many locations and relies on BGP to deliver users to the nearest, giving fast failover at the routing level.

```
dig +short www.example.com @8.8.8.8
```

```
traceroute -n 1.1.1.1
```

```
aws route53 get-health-check-status --health-check-id <id>
```

Interview tip: Most global platforms combine both: anycast for the edge, then application-aware routing behind it.

📚 Official docs

- Cloudflare Learning: Load balancing
- Cloudflare Learning: Anycast

certs.eknathalabs.com

Q145 **How does connection reuse (keep-alive and pooling) between a load balancer and its backends change performance and failure modes?**

Load balancing

🎙 Spoken answer · ~62 s

Opening a new backend connection for each request costs a TCP handshake and possibly TLS, so proxies keep a pool of idle connections to the upstream and reuse them. This reduces latency, backend CPU and ephemeral port usage. The failure mode is a race: if the backend's idle timeout is shorter than the proxy's, the backend closes a pooled connection at the same moment the proxy sends a request on it, and the client gets a 502 or a reset even though both are healthy. This typically appears as rare, intermittent errors on requests that come after a quiet period. The cure is to configure the proxy's keep-alive idle timeout to be shorter than the backend's, enable retries for idempotent requests, and watch metrics for connection resets. Similarly, between client and proxy, the load balancer's idle timeout should be longer than the client's expectations or clients should send heartbeats.

Likely follow-ups

- Why does the race produce 502 errors?
- Which side should close idle connections first?

Reusing backend connections saves handshakes and reduces latency and CPU. But if a backend closes an idle connection just as the proxy sends a request on it, the request fails with a reset. Align idle timeouts so the proxy's is shorter than the backend's.

```
nginx: upstream app { server 10.0.0.5:8080; keepalive 64; keepalive_timeout 55s; }
```

```
curl -w '%{time_connect} %{time_appconnect} %{time_starttransfer}\n' -o /dev/null -s
https://example.com
```

```
ss -tan state established '( dport = :8080 )' | wc -l
```

Interview tip: The rule of thumb: the party that closes first should be the one that knows the connection is idle, usually the proxy, not the backend.

📚 Official docs

- Cloudflare Learning: Load balancing

certs.eknathalabs.com

Q146 **Calico, Flannel and Cilium: how do their data planes differ and what are the trade-offs?** Cloud networking

🎙 Spoken answer · ~63 s

Flannel is the simplest: it gives each node a pod subnet and carries traffic between nodes inside a VXLAN overlay, so it works almost anywhere, but it does not enforce network policies on its own and adds encapsulation overhead. Calico routes pod traffic as plain IP, using BGP to advertise pod CIDRs to the fabric or between nodes, which avoids encapsulation and makes pod addresses visible to the network, with an option for VXLAN or IP-in-IP where the underlay does not allow it, and it implements rich network policy using iptables or an eBPF data plane. Cilium builds on eBPF programs in the kernel to forward packets, enforce identity-based policy including layer seven rules, replace kube-proxy with efficient service load balancing, and provide Hubble flow visibility, but requires a recent kernel. The trade-offs are complexity, kernel version requirements, how well each integrates with the cloud provider's native networking, and operational familiarity.

Likely follow-ups

- What does Hubble provide?
- When would you prefer BGP routing over an overlay?

Flannel is a simple overlay (VXLAN) with no network policy. Calico can run unencapsulated with BGP routing or use IP-in-IP/VXLAN and enforces policy with iptables or eBPF. Cilium uses eBPF for networking, policy, observability and kube-proxy replacement.

```
kubectl get pods -n kube-system -l k8s-app=calico-node
```

```
cilium status
```

```
calicoctl node status
```

```
kubectl exec -n kube-system ds/cilium -- cilium bpf lb list | head
```

Interview tip: Pick based on needs: Flannel for simplicity, Calico for BGP and policy maturity, Cilium for eBPF features and visibility.

📚 Official docs

- Cilium: Introduction
- Cilium documentation
- Calico documentation

certs.eknathalabs.com

Q147 **What are enhanced networking, SR-IOV and jumbo frames, and when do they matter in the cloud?**

Cloud networking

🎙 Spoken answer · ~65 s

On modern cloud instances, enhanced networking gives the VM direct access to a virtual function of the physical network card using SR-IOV, bypassing much of the hypervisor's software switching. On AWS this is the Elastic Network Adapter, on Azure Accelerated Networking, and on Google the gVNIC. The benefits are higher bandwidth, higher packets per second, lower latency and less jitter, which matters for databases, trading, storage and HPC. Jumbo frames raise the MTU from fifteen hundred to nine thousand, so each packet carries more payload and CPU overhead drops, helping bulk transfers inside a VPC or between peered VPCs in the same region. However, the path must support the larger size end to end, and traffic leaving through the internet gateway, VPN or some peering paths is limited to a smaller MTU, so path MTU discovery must work or MSS must be clamped. I verify the driver, enable the features, and test with don't-fragment pings.

Likely follow-ups

- Why do packets per second limits matter?
- How would you validate jumbo frame support?

Enhanced networking gives instances direct hardware access to the NIC (ENA, SR-IOV or Accelerated Networking), cutting latency and CPU overhead and raising packets-per-second. Jumbo frames (MTU 9001) increase throughput inside a VPC but not across the internet or many VPNs.

```
ethtool -i eth0 | grep driver
```

```
ip link show eth0 | grep mtu
```

```
ping -M do -s 8973 10.0.1.20
```

```
aws ec2 describe-instance-types --instance-types c6in.large --query
'InstanceTypes[].NetworkInfo'
```

Interview tip: Mixed MTU paths (9001 inside the VPC, 1500 at the internet or peering edge) are a source of black-hole issues. Test the path.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q148 **How do you give workloads a fixed outbound IP for partner allowlists, and what are the options and limits?**

Cloud networking

🎙 Spoken answer · ~64 s

Partners often restrict access to a list of source IP addresses. In a private network, outbound internet traffic is source-NATed anyway, so I control the address by controlling the NAT. In AWS, a NAT gateway with an Elastic IP in each availability zone gives stable addresses, in Google Cloud a Cloud NAT with reserved static addresses, and in Azure a NAT gateway with static public IPs. For Kubernetes, pod traffic leaves with the node address or the NAT address, so I can restrict egress to nodes in a specific subnet, or use an egress gateway feature in Cilium or Calico to route selected pods through a few gateway nodes with fixed IPs. Considerations are availability, since one gateway is a single point of failure, port exhaustion, the cost of NAT data processing, and documenting the IP set for partners. An alternative is a managed egress proxy or private connectivity with PrivateLink style endpoints.

Likely follow-ups

- Why does each AZ need its own NAT?
- What is an egress gateway in Cilium?

Route egress through a NAT gateway or egress proxy with an elastic or reserved static IP. In Kubernetes, use a dedicated egress gateway or node pool with static IPs. Remember high availability needs several IPs and the partner must allowlist all of them.

```
aws ec2 allocate-address --domain vpc
```

```
aws ec2 create-nat-gateway --subnet-id subnet-1 --allocation-id eipalloc-1
```

```
gcloud compute addresses create egress-ip --region=europe-west1
```

```
kubectl get egressgatewaypolicy -A   # Cilium
```

Interview tip: One NAT gateway per availability zone means one IP per zone. Give the partner the full set, not just one.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q149 **SERVFAIL appears for one domain on your resolver but works on a public resolver. What do you check?** Troubleshooting

🎙 Spoken answer · ~66 s

SERVFAIL means the resolver could not obtain a valid answer. Since another resolver succeeds, I look for differences. First I run dig with trace to follow delegation from the root and see whether the parent and child name servers agree and whether all are reachable. A lame or unreachable authoritative server may be tolerated by one resolver and not another. Then DNSSEC: if the zone is signed and has an expired signature or a mismatched DS record at the parent, validating resolvers return SERVFAIL, while non-validating ones answer normally. Testing with the checking disabled flag reveals this, and delv shows the validation chain. Other causes are firewalls dropping fragmented UDP responses or EDNS packets, so the resolver should fall back to TCP, and resolver cache or rate limits. The fix depends on the cause: repair the delegation or DNSSEC at the zone owner, allow TCP port fifty-three and large packets, or temporarily add a negative trust anchor while waiting.

Likely follow-ups

- What is a DS record?
- What is a negative trust anchor?

Different resolvers behave differently on broken delegation, DNSSEC validation failures, large responses over UDP, EDNS issues, or unreachable authoritative servers. Run dig +trace, check DNSSEC with +cd and delv, and compare resolvers.

```
dig example.com @10.0.0.2
```

```
dig example.com @8.8.8.8
```

```
dig +cd example.com @10.0.0.2
```

```
delv example.com
```

```
dig +trace example.com
```

Interview tip: If dig +cd (checking disabled) works but normal fails, DNSSEC validation is the cause.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q150 **POST requests fail with 502 occasionally after periods of inactivity, while GETs seem fine. What is the likely cause?**

Troubleshooting

🎙 Spoken answer · ~63 s

The pattern of errors after a quiet spell, affecting only some requests, strongly suggests the keep-alive race. The proxy keeps a pool of idle connections to the backend. If the backend, or a firewall or NAT in between, closes one that has been idle longer than its timeout, the proxy may not notice yet, sends a POST on the dead connection, and gets a reset. Idempotent requests such as GET are usually retried automatically on a new connection, so users see nothing, but proxies do not retry POST by default because it might duplicate side effects, so the 502 is returned. I confirm with the proxy error log messages and a packet capture showing the reset on a reused connection. The fix is to make the proxy's idle timeout shorter than the backend's and any network device's, enable TCP keepalives, and for APIs designed with idempotency keys, enable retries for POST safely.

Likely follow-ups

- How do idempotency keys make retries safe?
- What does a NAT idle timeout do?

A stale pooled connection: the backend or an intermediate device closed an idle keep-alive connection and the proxy or client reuses it. Non-idempotent POSTs aren't automatically retried, so the failure surfaces. Align idle timeouts and enable safe retries.

```
curl -v -X POST https://example.com/api -d '{}'
```

```
grep -E 'upstream prematurely closed|recv\(\) failed|Connection reset'
/var/log/nginx/error.log
```

```
sudo tcpdump -ni any 'tcp[tcpflags] & (tcp-rst) != 0' and port 8080
```

Interview tip: Look in the proxy error log for 'upstream prematurely closed connection', the signature of the idle timeout race.

📚 Official docs

- RFC 9110: HTTP Semantics (status codes, methods)

certs.eknathalabs.com

Q151 **Users on one ISP cannot reach your site while everyone else can. How do you investigate?**

Troubleshooting

🎙 Spoken answer · ~66 s

I start by getting precise information: what error do they see, since a timeout, a certificate error and a wrong page all point to different layers, which ISP and resolver they use, and whether it affects IPv4, IPv6 or both. DNS is the first check, because an ISP resolver may return stale or wrong answers or fail on DNSSEC. Next, I test both address families, since a broken IPv6 route with an AAAA record can cause long delays or failures while IPv4 works. I look at path MTU, because tunnels and PPPoE links reduce MTU and black holes only affect larger packets, such as the TLS server hello. A routing problem, like a BGP leak or a missing route from that provider's network, shows in traceroutes from their side, so I use public looking glasses and distributed probing services to test from within that network. Depending on findings, I fix DNS, MTU or address announcements, or engage the provider.

Likely follow-ups

- How does a broken AAAA record appear to users?
- What is a looking glass?

Collect specifics from the affected users: error type, IPv4 or IPv6, resolver, and traceroute. Check DNS answers from their resolver, IPv6 reachability, MTU and BGP route visibility, then use looking glasses and RIPE Atlas probes to test from their network.

```
dig +short www.example.com @<isp-resolver>
```

```
curl -4 -sI https://www.example.com; curl -6 -sI https://www.example.com
```

```
mtr -rwz -c 50 www.example.com
```

```
ping -M do -s 1400 www.example.com
```

Interview tip: Broken IPv6 paths and MTU black holes on specific ISPs are common reasons for partial outages.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q152 **You suspect a SYN flood against a public service. How do you confirm it and mitigate it?**

Troubleshooting

🎙 Spoken answer · ~60 s

A SYN flood sends many connection requests without completing the handshake, filling the server's queue of half-open connections so legitimate clients can't connect. I confirm by counting sockets in SYN_RECV state, watching listen overflow and SYN cookie counters in nstat, and sampling packets to see a high rate of SYNs, often from spoofed or widely distributed addresses, with few completed connections. Mitigations at the host are enabling SYN cookies, which let the server encode state in the sequence number and avoid allocating memory until the client replies with a valid ACK, raising the backlog and somaxconn, and shortening SYN-ACK retries. At the network edge I rate-limit SYNs per source, use cloud DDoS protection, a CDN or a load balancer that proxies TCP and absorbs the handshake, and filter obviously spoofed ranges. Afterwards I review logs, tune alerts on SYN_RECV counts, and rehearse the response.

Likely follow-ups

- How do SYN cookies work?
- Why is spoofing hard to filter?

Look for many half-open connections in SYN_RECV, listen queue overflows, and a flood of SYNs from many or spoofed sources. Enable SYN cookies, tune the backlog, rate-limit at the edge, and use upstream DDoS protection or a CDN.

```
ss -ant state syn-recv | wc -l
```

```
nstat -az TcpExtTCPReqQFullDrop TcpExtListenDrops TcpExtSyncookiesSent
```

```
sysctl net.ipv4.tcp_syncookies net.ipv4.tcp_max_syn_backlog
```

```
sudo tcpdump -ni eth0 'tcp[tcpflags] == tcp-syn' -c 20
```

Interview tip: Say the key point: SYN cookies avoid allocating state until the final ACK, so the backlog cannot be exhausted.

📚 Official docs

- RFC 9293: TCP
- Cloudflare Learning: DDoS attacks

certs.eknathalabs.com

Q153 **Two hosts intermittently lose connectivity and the logs mention a duplicate IP address. How do you find and fix it?**

Troubleshooting

🎙 Spoken answer · ~65 s

When two hosts use the same IP, ARP replies conflict, so other machines keep switching which MAC address they send to, and connections work for a while and then reset. Operating systems often log a duplicate address warning. To confirm, I use arping in duplicate detection mode, look at the neighbour table to see whether the MAC address for that IP changes, and capture ARP traffic to see which two MACs answer. On a managed switch I look up those MACs in the forwarding table to find which ports they are on, or in virtual environments query the hypervisor. Typical causes are a DHCP scope overlapping with manually assigned static addresses, a cloned virtual machine template keeping its static configuration, or a failover pair both active. The fix is correcting the address assignment, excluding static ranges from DHCP, and using IP address management to prevent recurrence. Port security or ARP inspection can alert on conflicts.

Likely follow-ups

- What does arping -D do?
- What is dynamic ARP inspection?

Two devices claim the same IP, so ARP answers alternate and traffic flips between them. Detect with arping -D, ARP tables and switch MAC tables, find the second MAC and its port, then fix DHCP overlap, static assignments or cloned VMs.

```
arping -D -I eth0 10.0.1.50
```

```
ip neigh show 10.0.1.50
```

```
arp-scan --localnet | sort | uniq -d
```

```
sudo tcpdump -eni eth0 arp and host 10.0.1.50
```

Interview tip: Cloud-cloned VMs with static IPs and DHCP pools overlapping static ranges are the usual culprits.

📚 Official docs

- RFC 826: ARP

certs.eknathalabs.com

Q154 **A VPN tunnel shows as up but traffic only flows in one direction. What are the usual causes?** Troubleshooting

🎙 Spoken answer · ~64 s

An established tunnel just means both ends negotiated keys. Data flow depends on several other things. First, routing: each side needs a route for the remote subnet pointing into the tunnel, and the return path must come back through the tunnel too. If return traffic takes a different route, such as the default route out to the internet, the remote end never sees replies. Second, for policy-based IPsec, the traffic selectors, which define the local and remote subnets that are encrypted, must match on both ends exactly, otherwise packets in one direction match and the other do not. Third, firewalls and security groups must allow traffic in both directions, including stateful tracking, and NAT rules may rewrite source addresses so the other side doesn't recognise them. Overlapping address ranges and MTU or fragmentation issues also cause partial flows. I capture traffic on both sides to see where packets disappear, then correct routing or selectors.

Likely follow-ups

- What are traffic selectors?
- Policy-based versus route-based VPN?

Check routes and traffic selectors on both ends, security groups and firewalls for return traffic, asymmetric routing, NAT rules that rewrite addresses, overlapping subnets, and MTU. Capture on both sides to see where packets disappear.

```
ip route get 10.20.0.5
```

```
sudo ip xfrm policy; sudo ip xfrm state
```

```
sudo tcpdump -ni any host 10.20.0.5
```

```
aws ec2 describe-route-tables --filters Name=route.gateway-id,Values=vgw-1
```

Interview tip: Remember that a tunnel being up only proves the key exchange worked. It says nothing about routes, selectors or firewalls.

📚 Official docs

- Cloudflare Learning: What is a VPN?

certs.eknathalabs.com

Q155 **Large file transfers between two regions are slow even though the link is rated at 10 Gbps. What do you investigate?** Troubleshooting

🎙 Spoken answer · ~62 s

A fast link does not guarantee a fast single connection. Throughput per TCP flow is bounded by the window size divided by round-trip time, so on a hundred millisecond path with a default buffer a single stream may reach only tens of megabits. I measure with iperf3, first with one stream and then with several in parallel. If parallel streams scale well, the bottleneck is per-connection: window scaling disabled, socket buffers too small, or slow congestion control recovering from packet loss. I check the connection's RTT, congestion window and retransmission count with ss. High loss on a long path hurts classic loss-based algorithms severely, so I may enable BBR and increase the maximum buffer sizes. For real workloads, I use tools that open many parallel streams or chunk uploads, such as multipart transfers to object storage, and check that cloud limits like per-flow throughput caps are not the issue.

Likely follow-ups

- Why is cloud per-flow throughput limited?
- What does BBR change?

Single-stream TCP is limited by window size divided by RTT, plus packet loss and congestion control. Measure with iperf3 using one and several streams, check window scaling and buffers, loss and retransmits, and consider BBR, parallel streams or a transfer tool built for long fat networks.

```
iperf3 -c remote -t 30
```

```
iperf3 -c remote -P 8 -t 30
```

```
ss -ti dst remote | grep -E 'rtt|cwnd|retrans'
```

```
sysctl net.ipv4.tcp_congestion_control net.core.rmem_max
```

Interview tip: If eight streams are much faster than one, the limit is per-flow window or loss, not the link capacity.

📚 Official docs

- Git LFS

certs.eknathalabs.com

Q156 **Behind a load balancer, the application logs show the load balancer's IP for every request and rate limiting blocks everyone. How do you fix it?**

Troubleshooting

🎙 Spoken answer · ~63 s

When a proxy terminates the client connection, the backend's socket peer is the proxy, so any feature that uses the remote address, logging, geo-location, allowlists and rate limiting, sees one IP for all users and may block everyone after a few requests. The proxy passes the original address in X-Forwarded-For or via the PROXY protocol. I configure the web server or framework to read it, but only when the direct peer is within the proxy's address ranges, and to take the right-most address that is not itself a trusted proxy. I then rate-limit based on that real client IP. I check what the load balancer actually sends by echoing headers, and make sure multiple proxy layers such as a CDN and a load balancer are all included in the trusted list. Finally, I test spoofing: sending a forged header directly should not change the identified client, otherwise attackers can evade limits.

Likely follow-ups

- Why only trust from known proxy ranges?
- How do multiple proxy layers affect the header?

🗺 Preserving client IP



The app is using the peer address instead of the forwarded client address. Configure the app or web server to trust the proxy's forwarded headers (or PROXY protocol) from known proxy ranges only, and rate-limit on the real client IP.

```
curl -s https://app.example.com/debug/headers | jq .
```

```
nginx: set_real_ip_from 10.0.0.0/16; real_ip_header X-Forwarded-For; real_ip_recursive on;
```

```
aws elbv2 describe-target-group-attributes --target-group-arn <arn>
```

Interview tip: After the fix, test that a forged X-Forwarded-For header from the internet does not change the logged IP.

📚 Official docs

- Cloudflare Learning: Load balancing
- RFC 6585: HTTP 429 Too Many Requests
- Microsoft: Cloud design pattern – Throttling

certs.eknathalabs.com

Q157 **A browser app calls your API and fails with a CORS error, although curl works. How do you diagnose and fix it properly?**

Troubleshooting

🎙 Spoken answer · ~63 s

curl works because CORS is a browser-enforced policy, so the server may be responding perfectly well but the browser refuses to expose the response to the page's JavaScript. For a cross-origin request with non-simple methods or headers, the browser first sends a preflight OPTIONS request with the origin, method and headers it wants to use. The server must respond with the matching Access-Control-Allow-Origin, Allow-Methods and Allow-Headers headers and a success status. If the request includes cookies or authorisation, the response must also include Allow-Credentials true and a specific origin, not a wildcard. I reproduce the preflight with curl to see exactly what headers come back, and check that proxies, API gateways or authentication layers aren't blocking OPTIONS before it reaches the app. The fix is configuring an explicit allowlist of origins, caching preflights with Max-Age, and ensuring error responses also carry the CORS headers so the browser shows the real error.

Likely follow-ups

- Why does a wildcard origin fail with credentials?
- What is a simple request?

CORS is enforced by browsers, not servers. For cross-origin calls the server must return Access-Control-Allow-Origin, handle the preflight OPTIONS request with allowed methods and headers, and send credentials flags correctly. Check the preflight response in the network tab.

```
curl -i -X OPTIONS https://api.example.com/items -H 'Origin: https://app.example.com' -H 'Access-Control-Request-Method: POST' -H 'Access-Control-Request-Headers: content-type,authorization'
```

```
curl -i https://api.example.com/items -H 'Origin: https://app.example.com'
```

Interview tip: Never answer with a wildcard origin when credentials are involved. Reflect only allowed origins from an allowlist.

📚 Official docs

- Mozilla Observatory / HTTP security headers (MDN)
- curl documentation

certs.eknathalabs.com

Q158 **WebSocket connections drop exactly every 60 seconds. What is happening and how do you fix it?** Troubleshooting

🎙 Spoken answer · ~62 s

A drop at a precise round number such as sixty seconds is a timer, not randomness. Many proxies and cloud load balancers close connections that have had no data for sixty seconds by default, and WebSockets can stay quiet while waiting for user events. I identify which hop closes it by checking each layer's idle timeout: the CDN, the cloud load balancer, a reverse proxy like NGINX with its proxy read timeout, an ingress controller, and NAT gateways with their own connection tracking timeouts. The fix is twofold. First, send heartbeats, either WebSocket ping frames or small application messages, at an interval shorter than the shortest timeout, perhaps every twenty to thirty seconds, which also detects dead peers. Second, raise the timeouts where appropriate and ensure the upgrade headers are passed. The client should also reconnect with exponential backoff and resume state, because connections will eventually drop during deployments.

Likely follow-ups

- Why are ping frames better than app messages?
- How should clients reconnect?

An intermediary, usually a load balancer or reverse proxy, closes connections it considers idle after a default 60 second timeout. Send application-level pings or heartbeats more often than the timeout and raise the idle and read timeouts on each hop.

```
nginx: proxy_read_timeout 3600s; proxy_send_timeout 3600s; proxy_set_header Upgrade
$http_upgrade; proxy_set_header Connection $connection_upgrade;
```

```
aws elbv2 modify-load-balancer-attributes --load-balancer-arn <arn> --attributes
Key=idle_timeout.timeout_seconds,Value=3600
```

```
wscat -c wss://example.com/ws
```

Interview tip: Every hop has its own timer: CDN, load balancer, proxy, NAT. The shortest one wins.

📚 Official docs

- Cloudflare Learning: Load balancing

certs.eknathalabs.com

Q159 **After scaling out a gRPC service, one pod still receives most of the traffic. Why, and how do you fix the balancing?** Troubleshooting

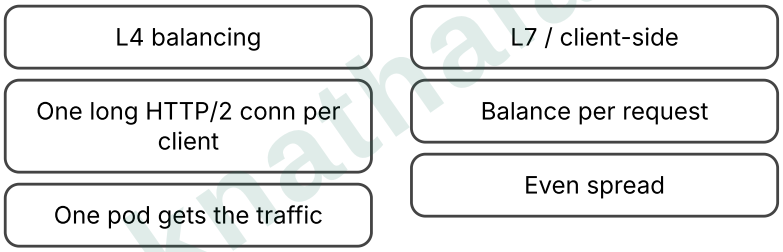
🎙 Spoken answer · ~62 s

Kubernetes Services using kube-proxy balance at the connection level. A gRPC client opens one HTTP/2 connection and multiplexes all its requests over it, so after the first connection is established to one pod, new pods added later receive nothing from that client, and a few busy clients can overload a single pod. The fix is to balance at the request level. Options include putting an L7 proxy such as Envoy or a service mesh in the data path, which understands HTTP/2 streams and spreads them across backends, using client-side load balancing with a headless Service so the client resolves all pod IPs and uses a policy like round robin, or an ingress controller with gRPC support. I can also configure the server with a maximum connection age and grace period, so clients reconnect periodically and rediscover new pods. Health checking and graceful shutdown remain important in either approach.

Likely follow-ups

- What is a headless Service?
- Why does MaxConnectionAge help?

🗺 gRPC imbalance



gRPC uses long-lived HTTP/2 connections that multiplex many requests. A layer 4 load balancer balances connections, not requests, so a client keeps using one connection to one pod. Use an L7 proxy, a service mesh, or client-side load balancing with headless service discovery.

```
kubectl get svc payments -o wide   # check clusterIP: None for headless
```

```
kubectl exec deploy/client -- grpcurl -plaintext payments:50051 list
```

```
kubectl top pod -l app=payments
```

Interview tip: Setting MaxConnectionAge on the server forces periodic reconnects and helps rebalance after scale-out.

📚 Official docs

- gRPC documentation

certs.eknathalabs.com

Q160 **A downstream API returns 429 Too Many Requests and your service's retries make it worse. How do you design client behaviour?**

Troubleshooting

🎙 Spoken answer · ~66 s

A 429 is a request to slow down, so blind immediate retries make the overload worse and can turn a limit into an outage through a retry storm. The client should read the Retry-After header or rate-limit headers and wait at least that long. Where there is no hint, use exponential backoff with jitter, so many clients don't retry in lockstep, plus a maximum number of attempts and a total deadline. Only idempotent operations should be retried automatically, or those protected by idempotency keys. Better still is prevention: a client-side token bucket that keeps request rates under the known limit, request coalescing and caching, and a circuit breaker that stops calling the dependency for a while when failures are high. Retries should be implemented at only one layer of the call chain, otherwise they multiply. I monitor the 429 rate, queue depth and retry counts, and negotiate higher limits or use batch endpoints when legitimate demand exceeds quota.

Likely follow-ups

- Why add jitter?
- What is retry amplification?

Respect Retry-After, use exponential backoff with jitter, cap retries and total time, add client-side rate limiting and circuit breakers, retry only idempotent operations, and avoid retry amplification across layers.

```
curl -si https://api.partner.com/v1/items | grep -iE 'HTTP/|retry-after|x-ratelimit'
```

```
kubectl logs deploy/worker | grep -c 429
```

```
rate(http_client_requests_total{code="429"}[5m])
```

Interview tip: If three layers each retry three times, one failure becomes twenty-seven calls. Retry at one layer only.

📚 Official docs

- RFC 6585: HTTP 429 Too Many Requests
- Microsoft: Cloud design pattern – Retry

certs.eknathalabs.com

Q161 **Some older clients fail TLS handshakes after you tightened the cipher and protocol configuration. How do you find who is affected and decide what to do?**

Troubleshooting

🎙 Spoken answer · ~63 s

First I measure the impact instead of guessing. I enable logging of the negotiated TLS version and cipher with the user agent for successful connections, and count handshake failures in the load balancer or proxy error logs, which show alerts such as protocol version or no shared cipher. That reveals which client populations are affected, whether browsers, mobile apps, monitoring probes or embedded devices. I reproduce with openssl s_client forcing specific protocol versions and ciphers. Then I decide: modern guidance is TLS 1.2 minimum with strong AEAD ciphers and TLS 1.3 enabled, so if the failures come from a handful of old integrations, I ask the owners to upgrade, with a deadline. If a business-critical legacy client must be supported, I isolate it on a separate hostname or listener with a weaker policy and compensating controls, rather than weakening the main endpoint. I document the policy and review it regularly.

Likely follow-ups

- Why isolate legacy clients on a separate endpoint?
- What does 'no shared cipher' mean?

Log the negotiated protocol and cipher plus handshake failures, identify the client types, and weigh supporting them versus requiring upgrades. Test with openssl s_client forcing old versions. Keep legacy support isolated where possible rather than weakening everything.

```
openssl s_client -connect example.com:443 -tls1_1 </dev/null
```

```
nmap --script ssl-enum-ciphers -p 443 example.com
```

```
nginx: log_format tls '$remote_addr $ssl_protocol $ssl_cipher $http_user_agent';
```

```
grep -c 'SSL_do_handshake() failed' /var/log/nginx/error.log
```

Interview tip: Collect data before and after the change. Often the affected clients are a few outdated scripts or devices, not real users.

📚 Official docs

- RFC 8446: TLS 1.3
- Cloudflare Learning: TLS handshake

certs.eknathalabs.com

Q162 **Visiting the site by IP or an alias hostname shows the wrong certificate or another site's content. How does SNI explain this?**

Troubleshooting

🎙 Spoken answer · ~65 s

With HTTPS, the server must present its certificate before the HTTP request is sent, so it can't use the Host header to choose it. Server Name Indication solves this by having the client include the hostname in the TLS ClientHello. The server or load balancer selects the matching certificate, and then routes the HTTP request by Host header. If I connect by IP address, or use a name not configured on the server, there is no matching SNI, so the server falls back to its default certificate and default virtual host, which is often another site. To debug I use openssl s_client with and without the servername option and compare, and curl with the resolve option to connect to a specific IP while sending the correct name. I then check the default server configuration, the certificate list on the load balancer listener, wildcard coverage, and whether a CDN origin is configured to send the right SNI.

Likely follow-ups

- What happens without SNI?
- Why does curl --resolve help?

A server hosting many TLS sites chooses the certificate using the SNI hostname sent in the ClientHello. Without SNI, or with an unmatched name, it returns a default certificate or site. Check the Host header, SNI value, default server block and load balancer listener rules.

```
openssl s_client -connect 203.0.113.10:443 -servername www.example.com </dev/null | openssl x509 -noout -subject -ext subjectAltName
```

```
curl --resolve www.example.com:443:203.0.113.10 https://www.example.com/ -v
```

```
nginx -T | grep -E 'server_name|listen 443|default_server'
```

Interview tip: curl --resolve is the cleanest way to test a specific IP with the correct SNI and Host header.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q163 **An attacker is hammering your login endpoint with credential stuffing. What network and application layers do you use to respond?**

Troubleshooting

🎙 Spoken answer · ~64 s

Credential stuffing uses leaked username and password pairs from other breaches, spread across many IP addresses to evade simple limits. I start by confirming the pattern in logs: a high volume of login attempts, a large share of failures, many different usernames per source and often a user agent or timing pattern. At the edge I use a WAF with rate-based rules scoped to the login path, managed bot control or CAPTCHA challenges, and temporary blocks on abusive networks or countries that are not part of our users, balancing false positives. In the application I throttle by account as well as by address, add progressive delays, require multi-factor authentication or step-up verification for risky logins, check passwords against breached lists, and avoid revealing whether a username exists. I also protect with alerts on failure ratios, notify affected users, and confirm the capacity of downstream systems, such as the identity database, during an attack.

Likely follow-ups

- Why is per-IP limiting not enough?
- What is step-up authentication?

Combine edge controls (WAF rules, rate limiting, bot detection, geo or ASN blocks, CAPTCHA) with application defences (per-account and per-IP throttling, lockout or step-up MFA, breached password checks) and monitoring. Block in layers, not only by IP.

```
awk '{print $1}' access.log | sort | uniq -c | sort -nr | head
```

```
aws wafv2 create-rate-based-rule ...
```

```
grep 'POST /login' access.log | awk '{print $9}' | sort | uniq -c
```

Interview tip: Attackers rotate IPs, so key limits on account, device fingerprint and behaviour as well as source address.

📚 Official docs

- GNU coreutils manual

certs.eknathalabs.com

Q164 **A server with two network interfaces replies to some clients but not others. How does asymmetric routing explain it, and how do you fix it with policy routing?**

Troubleshooting

🎙 Spoken answer · ~63 s

With a single routing table and a single default gateway, a host answers every request through the default route's interface. If a request arrives on the second interface, the reply goes out the first, with a source address belonging to the second network. A stateful firewall or the upstream router sees a response to a connection it never saw, or a spoofed-looking source, and drops it. The server's own reverse path filter may also drop incoming packets whose best return path is another interface. The fix is policy routing: create a separate routing table for each interface, with its own default gateway, and add rules that select the table based on the source address, so replies leave the way requests came in. Persist it in the network configuration, relax rp_filter to loose mode if needed, and verify with ip route get using the from option and with packet captures on both interfaces.

Likely follow-ups

- What does rp_filter do?
- How does ip rule differ from ip route?

The reply leaves via the default route's interface, not the one the request came in on. Reverse-path filtering or the upstream stateful firewall then drops it. Add source-based routing tables and rules so replies use the interface that owns the source address.

```
ip route show; ip rule show
```

```
sudo ip route add default via 10.0.2.1 dev eth1 table 200
```

```
sudo ip rule add from 10.0.2.15/32 table 200
```

```
sysctl net.ipv4.conf.all.rp_filter
```

Interview tip: rp_filter strict mode silently drops packets whose reverse path would use a different interface, a very common hidden cause.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q165 **Docker or Kubernetes workloads on a host cannot reach corporate network ranges because the default bridge subnet overlaps. How do you resolve it?**

Troubleshooting

🎙 Spoken answer · ~64 s

If a corporate network uses part of the range that Docker assigns to its default bridge or to user-defined networks, the host's routing table sends packets for those addresses to the local bridge instead of the real network, so containers and sometimes the host itself cannot reach those destinations. I inspect the routing table to find the conflicting route and the Docker network configuration. For Docker, I set the bridge IP and default address pools in the daemon configuration to unused ranges and recreate the networks, which requires restarting containers. For Kubernetes, the pod CIDR and service CIDR are set at cluster creation and are difficult to change, so the right approach is prevention: coordinate with the network team, choose ranges that do not overlap with on-premises, VPN partners or other clusters, record them in an address management system, and consider using non-routable ranges such as the carrier grade NAT block for pods.

Likely follow-ups

- Can you change a Kubernetes pod CIDR later?
- Why record ranges in IPAM?

The container runtime's default bridge (172.17.0.0/16) or Kubernetes pod and service CIDRs collide with real networks, so the host routes that traffic locally. Change the bridge or CIDR ranges to unused space, and plan address allocation centrally.

```
ip route | grep 172.17
```

```
docker network inspect bridge | jq '.[0].IPAM.Config'
```

```
cat /etc/docker/daemon.json   # {"bip":"10.200.0.1/24","default-address-pools":
[{"base":"10.201.0.0/16","size":24}]}
```

```
kubectl cluster-info dump | grep -m2 -E 'service-cluster-ip-range|cluster-cidr'
```

Interview tip: Kubernetes pod and service CIDRs are hard to change later, so choose them to avoid all current and planned networks.

📚 Official docs

- Docker documentation

certs.eknathalabs.com

Q166 **Cross-AZ latency and data transfer costs are higher than expected for a chatty microservice setup. How do you measure and reduce them?**

Troubleshooting

🎙 Spoken answer · ~61 s

I start by measuring: VPC flow logs or cloud network metrics aggregated by source and destination zone, and request latency broken down by calling zone, show which service pairs generate cross-zone traffic and how much it costs. Typical sources are random load balancing across all zones, replicas not evenly spread, database replicas, and chatty protocols. To reduce it, I enable topology-aware routing for Kubernetes services so traffic prefers endpoints in the caller's zone, use zone-aware load balancing in the mesh or ingress with locality settings, place tightly coupled services together, cache frequently read data, batch calls and compress payloads. I must keep enough capacity in each zone, because strict affinity can overload a small zone, and keep failover to other zones when local endpoints are unhealthy. After changes I recheck both cost and latency, and confirm the availability design still tolerates the loss of a zone.

Likely follow-ups

- How does topology-aware routing decide?
- What are the risks of zone affinity?

Measure inter-zone traffic with flow logs and service metrics, then keep chatty services in the same zone using topology-aware routing, zone-aware load balancing and caching. Balance this against availability, because zone affinity must still fail over.

```
kubectl get endpointslices -l kubernetes.io/service-name=orders -o yaml | grep -E
'zone|hints'
```

```
kubectl annotate svc orders service.kubernetes.io/topology-mode=Auto
```

```
aws ec2 describe-flow-logs
```

```
fields @timestamp, srcaddr, dstaddr, bytes | stats sum(bytes) by srcaddr, dstaddr | sort
sum desc
```

Interview tip: Zone-local routing can overload a zone with fewer pods, so keep replicas balanced per zone and allow spill-over.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q167 **Packet captures show many retransmissions and 'TCP ZeroWindow' from the receiver. What does that tell you and what do you do?**

Troubleshooting

🎙 Spoken answer · ~63 s

In TCP, each side advertises how much more data it can accept, the receive window. A zero window announcement means the receiver's socket buffer is full because the application has not read the data yet, so the sender must stop and send only occasional probes. Since the network is not the bottleneck here, tuning the network will not help. I look at the receiving host: the Recv-Q column in ss shows bytes waiting in the buffer for a particular connection. Then I examine why the application is slow to read: a saturated CPU or thread pool, a blocking call such as a slow database or disk write in the read loop, garbage collection pauses, lock contention, or a single-threaded design. Increasing socket buffers can smooth short bursts but only hides a persistent slow consumer. The real fix is making the consumer faster, adding parallelism or backpressure, or reducing the data rate.

Likely follow-ups

- What is Recv-Q?
- What are window probes?

ZeroWindow means the receiver's buffer is full: the application is not reading fast enough. The problem is at the receiving host or application, not the network. Look at the process's CPU, blocking calls, thread pool or downstream slowness, and socket Recv-Q.

```
ss -tnp | awk '$2>0'
```

```
tshark -r cap.pcap -Y 'tcp.analysis.zero_window' | head
```

```
top -H -p $(pidof app)
```

```
sysctl net.ipv4.tcp_rmem
```

Interview tip: Contrast: loss and retransmissions without zero windows point to the network. Zero window points to a slow consumer.

📚 Official docs

- RFC 9293: TCP
- tcpdump manual

certs.eknathalabs.com

Q168 **A Kubernetes service cannot reach an external API only from some nodes. How do you find the difference?** Cloud networking

🎙 Spoken answer · ~63 s

If only some nodes fail, something differs between them. I run the same debug container pinned to a working node and a failing node and compare results, checking DNS resolution first and then the TCP connection. Differences usually come from placement: nodes in different subnets or availability zones may use different route tables, NAT gateways or network ACLs, and the partner may have allowlisted only some NAT addresses. Security groups attached to node groups can differ, a node might have a stale or missing SNAT rule in the CNI configuration, or a host firewall may block it. I check the route taken from the node, the outbound IP seen by an external echo service, and node labels for zone and group. Once I identify the difference, I fix the routing or allowlist, and prevent drift by building node groups from the same templates and testing egress as part of cluster validation.

Likely follow-ups

- How can you see the node's outbound IP?
- Why would a CNI SNAT rule go missing?

Compare the failing and working nodes: egress route and NAT gateway or IP, security groups and network ACLs, DNS resolution on the node, CNI SNAT behaviour, and node-level firewalls. Test from a debug pod pinned to each node.

```
kubectl run dbg --image=nicolaka/netshoot --overrides='{"spec":{"nodeName":"node-a"}}' -it
--rm -- curl -sv https://api.partner.com
```

```
kubectl get nodes -o wide
```

```
ip route get 203.0.113.50; iptables -t nat -S POSTROUTING | head
```

Interview tip: Look at which subnet each node is in. Different subnets often use different route tables and NAT gateways.

📚 Official docs

- RFC 3022: Traditional NAT

certs.eknathalabs.com

Q169 **An application intermittently gets 'Name or service not known' during DNS-heavy workloads, and the resolver logs show timeouts. What do you investigate?**

Troubleshooting

🎙 Spoken answer · ~61 s

Intermittent resolution failures under load often come from limits rather than from broken configuration. In the cloud, the VPC resolver is rate limited per network interface, for example around one thousand packets per second in AWS, and bursts of lookups, made worse by search domain expansion where one name generates several queries, exceed it and queries are dropped. UDP drops from conntrack table exhaustion or a congested NIC have the same symptom, as does an overloaded in-cluster resolver. I measure query rates and timeouts, test lookups repeatedly against the resolver, and check drop counters. Fixes include deploying a node-local caching resolver, which absorbs repeated queries, lowering ndots or using fully qualified names, increasing cache TTLs and using HTTP connection reuse so applications resolve less often, spreading load across multiple resolvers, and tuning resolver timeout and attempts settings. Monitoring DNS error rates and latency gives early warning.

Likely follow-ups

- What is search domain expansion?
- What does a node-local DNS cache do?

Check resolver capacity and rate limits (cloud VPC DNS has per-ENI packet limits), UDP packet drops and conntrack, search domain amplification, caching, and retry settings. Add a local cache, reduce lookups, and use connection reuse.

```
dig +tries=1 +time=1 api.example.com @169.254.169.253
```

```
conntrack -S | grep -i drop
```

```
cat /etc/resolv.conf
```

```
aws cloudwatch get-metric-statistics --namespace AWS/EC2 --metric-name
NetworkPacketsDropped ...
```

Interview tip: AWS limits traffic to the VPC resolver to about 1,024 packets per second per network interface, a classic cause of intermittent DNS failures.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com

Q170 **Security asks you to prove that production databases cannot be reached from the internet or from the web tier on unintended ports. How do you validate and document it?**

Network security

🎙 Spoken answer · ~64 s

I use two complementary approaches. Static analysis checks the configuration: list security groups, network ACLs, route tables, public IP assignments and, for Kubernetes, network policies, and flag anything allowing zero dot zero dot zero dot zero slash zero or broad ranges to database ports. Infrastructure as code scanners run in the pipeline to prevent such rules from ever being deployed. Reachability tools such as AWS Reachability Analyzer, Azure Network Watcher or Google Connectivity Tests evaluate whether a path exists between a source and destination for a given port, and produce evidence of which component allows or blocks it. Active testing then confirms reality: authorised port scans from the internet and from the web tier show which ports respond, and I compare them with the intended matrix. I document the expected flows, the evidence, test dates and owners, schedule the checks to repeat, and alert on drift. This turns a one-time audit into continuous assurance.

Likely follow-ups

- Why combine static and active testing?
- How would you keep this evidence current?

Combine configuration review with active testing: enumerate security groups, route tables and policies as code, run reachability analysis tools and authorised port scans from the web tier and from outside, and keep automated checks in CI so regressions are caught.

```
aws ec2 describe-security-groups --filters Name=ip-permission.cidr,Values=0.0.0.0/0
```

```
aws ec2 create-network-insights-path --source i-web --destination i-db --protocol tcp --
destination-port 5432
```

```
nmap -Pn -p- 10.0.3.10   # from web tier, authorised only
```

```
checkov -d terraform/ --check CKV_AWS_24,CKV_AWS_260
```

Interview tip: Reachability Analyzer and similar tools prove intent against the actual configuration, without sending traffic.

📚 Start here (official docs)

- Cloudflare Learning Center: networking

certs.eknathalabs.com